

Predicting Success of Television Series

Web Scraping Constants

- Set to False to start with already constructed files..

```
In [145... SKIP_TO_MODELING = True

# for webscraping ratings and downloading sheets from google doc (costly in time)
DOWNLOAD_SHEETS = False

# for webscraping ratings and downloading sheets from google doc (costly in time)
BUILD_RATINGS_FROM_SHEETS = False

# download html ratings for scraping
DOWNLOAD_RATINGS_SCRAPE_FODDER = False

# for webscraping other data
BUILD_RATINGS_FROM_SCRAPING = False

# download episode pages from imdb for scraping
GET_EPISODE_PAGES = False

# download content advisories from imdb
BUILD_CONTENT_FROM_SCRAPING = False

GET_SERIES_RUNS = False

CREATE_MISSING_SERIES_DATAFRAME = False
```

Install Libraries peculiar to this app.

```
In [ ]: !pip install imdbpy

!pip install cinemagoer

!pip install tmdbv3api

!pip install matplotlib seaborn statsmodels

!pip install python-Levenshtein
```

```
In [ ]: !pip install nbconvert[webpdf]
```

```
In [148... # Standard Library imports
import json
import re
import random

# debugging etc
import logging

# files and directories
import os
from os.path import basename, exists
from io import StringIO

# File and directory handling
import glob

# Web access and HTML parsing
import requests
import urllib
from urllib.request import urlretrieve
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
from bs4 import BeautifulSoup # Beautiful Soup for HTML parsing

# Data manipulation and analysis
import pandas as pd
import numpy as np

# Scientific computing Library
import scipy
from statsmodels.regression.mixed_linear_model import MixedLM
import statsmodels.api as sm

# Machine Learning and Data Science tools
from sklearn.model_selection import (train_test_split, GridSearchCV,
                                     StratifiedKFold, cross_val_score)
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.ensemble import (RandomForestRegressor, GradientBoostingRegressor,
                             RandomForestClassifier, ExtraTreesClassifier,
                             AdaBoostClassifier, AdaBoostRegressor,
                             ExtraTreesRegressor, GradientBoostingClassifier)
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             roc_curve, auc, mean_absolute_error, mean_squared_error,
                             r2_score, precision_score, recall_score, f1_score)
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVR
```

```

from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.linear_model import (LogisticRegression, LinearRegression, Ridge,
                                  Lasso, ElasticNet)
from sklearn.inspection import permutation_importance
from sklearn.exceptions import ConvergenceWarning
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier, XGBRegressor
import xgboost as xgb

# Visualization tools
import seaborn as sns
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from statsmodels.graphics.regressionplots import plot_leverage_resid2, plot_ccpr

# API access
import tmdbv3api
from imdb import IMDb
from tmdbv3api import TMDb, Movie, TV, Search
from imdb import IMDbDataAccessError

# Fuzzy matching
from fuzzywuzzy import process, fuzz

# Pretty printing tables
from tabulate import tabulate

# Time
import time
from datetime import datetime, timedelta

# need to merge
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, precision_score, recall_score
import xgboost as xgb

```

In [149... *# For testing: Make warnings fatal.*

```

import warnings
warnings.filterwarnings("error")

```

In [150... *# Create Data Directory Structure.*

```

FINAL_DATA_DIRECTORY = "final/data"
RATINGS_DATA_DIRECTORY = FINAL_DATA_DIRECTORY + "/ratings"
RATING_SHEETS_DATA_DIRECTORY = FINAL_DATA_DIRECTORY + "/ratings/downloads"
SVOD_SHEETS_DATA_DIRECTORY = FINAL_DATA_DIRECTORY + "/ratings/svod"
SVOD_GOOGLEDOC_DATA_DIRECTORY = FINAL_DATA_DIRECTORY + "/ratings/svod/google"
SVOD_DAYS_DATA_DIRECTORY = FINAL_DATA_DIRECTORY + "/ratings/svod/google/days"
SVOD_PROGRAM_DATA_DIRECTORY = FINAL_DATA_DIRECTORY + "/ratings/svod/google/programs"
TRENDS_DATA_DIRECTORY = FINAL_DATA_DIRECTORY + "/trends"
CONTENT_DATA_DIRECTORY = FINAL_DATA_DIRECTORY + "/content"
TV_SERIES_PAGE_DIRECTORY = FINAL_DATA_DIRECTORY + "/ratings/series"

# directories
directories = [
    FINAL_DATA_DIRECTORY,
    RATINGS_DATA_DIRECTORY,
    RATING_SHEETS_DATA_DIRECTORY,
    SVOD_SHEETS_DATA_DIRECTORY,
    SVOD_GOOGLEDOC_DATA_DIRECTORY,
    SVOD_DAYS_DATA_DIRECTORY,
    SVOD_PROGRAM_DATA_DIRECTORY,
    TRENDS_DATA_DIRECTORY,
    CONTENT_DATA_DIRECTORY,
    TV_SERIES_PAGE_DIRECTORY
]

# Check and create each directory if it doesn't exist
for directory in directories:
    if not os.path.exists(directory):
        os.makedirs(directory)
        print(f"Directory '{directory}' created.")

```

In [151... *def get_google_drive_download_link(share_link_url):*

```

    # parse id
    id = share_link_url.split("/d/")[1].split("/")[0]
    # format link
    return f"https://drive.google.com/uc?id={id}&export=download"

```

```

# modeling urls

FILTERED_MAIN_SERIES_FOR_MODELING_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1v-0xsXmmt7CX0xOy6owP4ch8r3bo3PcL/view?usp=drive_link")
UNFILTERED_MAIN_SERIES_FOR_MODELING_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1i6jTpFRfk6mFVVCsyIINK0rDkVmZf5Fu/view?usp=drive_link")
SVOD_DATA_FOR_MODELING_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/12A30S5e8Lvi2m_LwsVc2ceJiZxdtpVl3/view?usp=drive_link")
EPISODE_NIELSEN_FOR_MODELING_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/15b95e1x37t8Toq0lGgme4z9ScyN9cIQ5/view?usp=drive_link")
EPISODES_FOR_MODELING_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/14eGdTdg-1Mab7jeOKRDC5DF7khtctep-/view?usp=drive_link")

```

```

# for modeling

FILTERED_MAIN_SERIES_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/tv_shows_main_with_networks_and_genres.csv"
UNFILTERED_MAIN_SERIES_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/tv_shows_main.csv"
SVOD_DATA_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/svod_merged_data.csv"
EPISODE_NIELSEN_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/episode_nielsen_data.csv"
EPISODES_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/episode_data_with_imdb_ratings_too.csv"

# intermediate files URL

SERIES_RATINGS_JUMP_TABLE_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/10DkAK90psz6HAtAJkSjukDXZSLZeBzU/view?usp=drive_link")
NIELSEN_RATINGS_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1-I9L16_g4E4PCQb8CkKjRRVj5DiL_dU-/view?usp=drive_link")
SVOD_MEASURE_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1f065k2utd192T3G1NDRVj_3UZWCpF_DQ/view?usp=drive_link")
ARCHIVE_RATINGS_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1z0kFGeqnhgTiAmmHTB6JmXkwCrzi34w/view?usp=drive_link")
ORIGINAL_TMDB_DATASET_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1huMGSMBefrcB0yG5jS1NwdjRTaI5eKYX/view?usp=drive_link")
VALID_ID_TMDB_SERIES_MAIN_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1R9MgE3cE7_90h45rWuL98TUoFn6Umai5/view?usp=drive_link")
MERGED_EPISODE_DATA_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1Vx-FC_YQEhfokFB6eJ9-00aYndHSS8DK/view?usp=drive_link")
SERIES_RUNS_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1QkINKV03m5x8VeNeLguad-YpT-XSBaA/view?usp=drive_link")
PREWEBSCRAPING_GOOGLE_DOCS_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1Z2HZ6Le-mKzv-awK9a-ro694reXCwO-r/view?usp=drive_link")
POSTWEBSCRAPING_GOOGLE_DOCS_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1SG3hHfKcGx_NUN1_3y24iWnUpbkGoieY/view?usp=drive_link")
COMPLETE_SERIES_LIST_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1g-U4f0U3R2GjVtUjy3ZT6jKc_thBz-o/view?usp=drive_link")
TV_SERIES_WITH_NETWORK_AND_GENRES_URL = \
    get_google_drive_download_link("https://drive.google.com/file/d/1Hv2b6obxPLtch9fbBdTrVmADxVnO3T82/view?usp=drive_link")

RATINGS_LINK_FILE = RATINGS_DATA_DIRECTORY + "/sheet.html"
GOOGLEDOC_LINK_FILE = RATINGS_DATA_DIRECTORY + "/jump_table.html"

# nielsen ratings (24000 records) of episodes from 2017-2023
NIELSEN_RATINGS_FILE = RATINGS_DATA_DIRECTORY + "/tv_ratings2.csv"
# streaming video on demand (current metrics)
SVOD_MEASURE_FILE = RATINGS_DATA_DIRECTORY + "/svod_ratings2.csv"
# ratings of almost 16000 shows from 2010-2020
ARCHIVE_RATINGS_FILE = RATINGS_DATA_DIRECTORY + "/programs_ratings.csv"
# television series data base
TMDB_SERIES_MAIN_FILE_V3 = RATINGS_DATA_DIRECTORY + "/TMDB_tv_dataset_v3.csv"
# After creating unique id
VALID_ID_TMDB_SERIES_MAIN_FILE = RATINGS_DATA_DIRECTORY + "/TMDB_tv_valid_id_dataset.csv"
# merged episode info
MERGED_EPISODE_DATA_FILE = FINAL_DATA_DIRECTORY + "/merged_episode_data.csv"
# cancelled or not?
SERIES_RUNS_FILE = RATINGS_DATA_DIRECTORY + "/tv_series_runs.csv"
# includes jump table for google links
PREWEBSCRAPING_GOOGLE_DOCS_FILE = RATINGS_DATA_DIRECTORY + "/tv_shows2.csv"
# post google doc scraping
POSTWEBSCRAPING_GOOGLE_DOCS_FILE = RATINGS_DATA_DIRECTORY + "/tv_shows3.csv"
# main series list before unique id
COMPLETE_SERIES_LIST_FILE = FINAL_DATA_DIRECTORY + "/main_series_list.csv"
# main series after adding networks and genres
TV_SERIES_WITH_NETWORK_AND_GENRES_URL = FINAL_DATA_DIRECTORY + "/tv_shows_main_with_network_and_genres.csv"

```

Good citizen web download function.

```

In [152..
# User-Agent header might be required for some websites to allow access
user_agent_headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.150 Safari/537.36"
}

# create a random user agent header.
def get_user_agent_headers():
    # Specify a fallback user agent
    fall_back_ua = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.0.0 Safari/537.36"
    # Instantiate the UserAgent class with popularity filter
    ua = UserAgent(fallback=fall_back_ua)
    # Get a random user agent
    random_ua = ua.random
    return {'User-Agent': random_ua}

# Good citizen web download function
# Respects status 429 and exponential backoff.
def fetch_url_with_retry(url, retries=10, backoff_factor=0.3):
    session = requests.Session()
    retry_strategy = Retry(
        total=retries,
        read=retries,
        connect=retries,
        backoff_factor=backoff_factor,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["HEAD", "GET", "OPTIONS"]
    )
    adapter = HTTPAdapter(max_retries=retry_strategy)
    session.mount('http://', adapter)
    session.mount('https://', adapter)

```

```

try:
    response = session.get(url, headers=user_agent_headers)
    response.raise_for_status()
    return response
except requests.exceptions.HTTPError as he:
    if response.status_code == 429:
        retry_after = int(response.headers.get("Retry-After", 10))
        print(f"Rate limit exceeded. Retrying in {retry_after} seconds...")
        # wait twice as long as requested.
        time.sleep(retry_after * 2)
        return fetch_url_with_retry(url, retries-1, backoff_factor) # Recursive call to retry
    print(f"HTTP error occurred: {he}")
except requests.exceptions.ConnectionError as ce:
    print(f"Connection error occurred: {ce}")
except requests.exceptions.Timeout as te:
    print(f"Timeout occurred: {te}")
except requests.exceptions.RequestException as re:
    print(f"Error during requests to {url}: {re}")
return None

def download_file(url, filename):
    # Check if file already exists
    if not os.path.exists(filename):
        print("Downloading file from url ", url, " to ", filename)
        response = fetch_url_with_retry(url)
        if response:
            with open(filename, "w", encoding='utf-8') as output_file:
                print("writing " + filename)
                output_file.write(response.text)
    else:
        print(f"Already downloaded {filename}")

```

Generally Useful Functions

In [153...

```

def pretty_print_df(df, rows=None, interesting_columns=None, headers='keys'):
    if rows is not None:
        df = df.head(rows) # rows parameter Limits the DataFrame to the specified number of rows
    if interesting_columns:
        # Filter DataFrame to include only specified columns
        df = df[interesting_columns]
    # Use Tabulate pretty printing.
    print(tabulate(df, headers=headers, tablefmt='pretty', showindex=False))

def formatTestStat(value, dec=6):
    format_string = "{:. "+str(dec)+"f}"
    return format_string.format(value)

def pretty_print_df_info_with_nans(df):
    # Show the column names, data types, & count of NaN values.
    columns_info = pd.DataFrame({
        'Column': df.columns,
        'Data Type': df.dtypes,
        'NaN Count': df.isnull().sum()
    }).reset_index(drop=True)

    # Convert 'NaN Count' column to int for better display
    columns_info['NaN Count'] = columns_info['NaN Count'].astype(int)

    table = tabulate(columns_info, headers='keys', tablefmt='pretty', showindex="always")
    print(f"Dataframe Information:\n{table}")

def show_df_info(df, title):
    print(f"\nDataframe information for {title}:\n")
    pretty_print_df_info_with_nans(df)
    print(f"\nThere are {df.shape[0]} rows and {df.shape[1]} columns of information.\n")

```

Web Scrapping

Download TV Ratings for Scrapping

1. Get TV Show Page from <https://tvseriesfinale.com>
 - This site consolidates Nielsen Ratings and Demographics Ratings.
 - It also keeps lists of cancelled series.

Original Sheet with series names is no longer available: TV%20Show%20Ratings%20-%20canceled%20+%20renewed%20TV%20shows, %20ratings%20-%20TV%20Series%20Finale_files/sheet.html

"https://tvseriesfinale.com/TV>Show Ratings - canceled + renewed TV shows, ratings - TV Series Finale_Files/sheet.html"

In [154...

```

if not SKIP_TO_MODELING:

    if DOWNLOAD_SHEETS:

        TV_SERIES_HEAD_URL = "https://tvseriesfinale.com"
        download_file(TV_SERIES_HEAD_URL, RATINGS_LINK_FILE)

        # Read the HTML from the file
        with open(RATINGS_LINK_FILE, 'r', encoding='utf-8') as rankings_file:
            html_content = rankings_file.read()

        # Parse HTML
        soup = BeautifulSoup(html_content, 'html.parser')

```



```

# Pattern to match ul ids for TV shows
ul_pattern = re.compile(r'p2-tv-show-list-\w+')

tv_shows = []

# Find all ul elements matching the pattern
uls = soup.find_all('ul', id=ul_pattern)

# Iterate over matching ul elements
for ul in uls:
    # Find all li elements within the current ul
    lis = ul.find_all('li')
    # Extract href attributes from li elements
    for li in lis:
        href = li.find('a', href=True)
        if href:
            tv_shows.append(href['href'])
            html_filename = href['href'].replace("https://tvseriesfinale.com/tv-show/", "")
            html_filename = html_filename.replace("/", "_")
            download_file(href['href'], TV_SERIES_PAGE_DIRECTORY + "/" + html_filename + ".html")

```

In [155...

```

def write_html_to_file(html_content, filename):
    with open(filename, 'w', encoding='utf-8') as file:
        file.write(html_content)

def find_element_with_debugging(soup, tag, **kwargs):
    element = soup.find(tag, **kwargs)
    # if element:
    #     print(f"Found {tag}: {element}")
    # else:
    #     print(f"Element {tag} not found!")
    return element

def find_next_sibling_with_debugging(element):
    sibling = element.find_next_sibling()
    # if sibling:
    #     print(f"Next sibling: {sibling}")
    # else:
    #     print("No next sibling found!")
    return sibling

def next_sibling_with_debugging(element):
    sibling = element.next_sibling
    # if sibling:
    #     print(f"Next sibling: {sibling}")
    # else:
    #     print("No next sibling found!")
    return sibling

def get_text_from_label(element):
    if not element:
        return None
    # Look for any direct text immediately following the element, if it's a navigable string
    next_node = element.next_sibling
    if next_node and isinstance(next_node, str):
        trimmed_text = next_node.strip()
        if len(trimmed_text) > 0: # Only return if there's meaningful content
            return trimmed_text
    # If no direct text or only whitespace, look for text within the next sibling element,
    # including tags like <a>
    next_element = element.find_next_sibling()
    if next_element:
        return next_element.get_text(strip=True)
    return None

if not SKIP_TO_MODELING:

    if GET_SERIES_RUNS:
        show_names = []
        networks = []
        season_counts = []
        episode_counts = []
        statuses = []

        tv_columns = ['Show Name', 'Network', 'Seasons', 'Episodes', 'Status']
        tv_series_runs_df = pd.DataFrame(columns=tv_columns)

        # Extracting data from HTML files
        for filename in os.listdir(TV_SERIES_PAGE_DIRECTORY):
            if filename.endswith('.html'):
                with open(os.path.join(TV_SERIES_PAGE_DIRECTORY, filename), 'r', encoding='utf-8') as series_file:
                    html_content = series_file.read()

                # filename is show_name.html
                # get show_name
                show_name = filename.split('.')[0]

                # Parse HTML
                soup = BeautifulSoup(html_content, 'html.parser')

                # Write entry_content_div to file for debugging
                entry_content_div = soup.find('div', class_="entry-content")
                write_html_to_file(str(entry_content_div), RATINGS_DATA_DIRECTORY+'junk.html')

                # Debugging - find_element_with_debugging function
                network_element = find_element_with_debugging(entry_content_div, 'strong', string='Network:')
                season_element = find_element_with_debugging(entry_content_div, 'strong', string='Seasons:')

```

```

episode_element = find_element_with_debugging(entry_content_div, 'strong', string='Episodes:')
status_element = find_element_with_debugging(entry_content_div, 'strong', string='Series status:')

# Extracting data
network = get_text_from_label(network_element) if network_element else None
season_count = get_text_from_label(season_element) if season_element else None
episode_count = get_text_from_label(episode_element) if episode_element else None
status = get_text_from_label(status_element) if status_element else None

show_names.append(show_name)
networks.append(network)
season_counts.append(season_count)
episode_counts.append(episode_count)
statuses.append(status)

# Add the extracted data to the DataFrame
tv_series_runs_df['Show Name'] = show_names
tv_series_runs_df['Network'] = networks
tv_series_runs_df['Seasons'] = season_counts
tv_series_runs_df['Episodes'] = episode_counts
tv_series_runs_df['Status'] = statuses

# Write extracted data to a CSV file
tv_series_runs_df.to_csv(SERIES_RUNS_FILE, index=False)
else:
    download_file(SERIES_RUNS_URL, SERIES_RUNS_FILE)
    tv_series_runs_df = pd.read_csv(SERIES_RUNS_FILE)

```

Build ratings for each season for shows.

The ratings data are stored as Google Docs. I first create a dataframe of jump tables. Note: This is fast.

```

In [156... if not SKIP_TO_MODELING:

    if DOWNLOAD_SHEETS:

        download_file(SERIES_RATINGS_JUMP_TABLE_URL, GOOGLEDOC_LINK_FILE)

        # Workaround weird character conversion issue that started recently.
        encoding = "utf-8"
        # Read the HTML from the URL. Force "utf-8" encoding to workaround issue.
        with open(GOOGLEDOC_LINK_FILE, 'r', encoding = encoding) as rankings_file:
            # Read the contents of the file into a buffer
            html_content = rankings_file.read()

        # print(html_content)

        soup = BeautifulSoup(html_content, 'html.parser')

        # Extracting href attributes
        hrefs = [link['href'] for link in soup.find_all('a', href=True)]

        tv_shows = []
        # https://tvseriesfinale.com/tv-show/americas-got-talent-season-13-ratings/
        # Pattern to match hrefs
        pattern = re.compile(r'https://tvseriesfinale.com/tv-show/(.*)-season-(.*)-ratings/')

        # Extracting TV show names and seasons from hrefs
        tv_shows = []
        for href in hrefs:
            match = pattern.search(href)
            if match:
                show_name = match.group(1).replace('-', ' ')
                season = "season " + match.group(2)
                tv_shows.append((show_name, season, href))

        # Creating DataFrame
        tv_shows_df = pd.DataFrame(tv_shows, columns=['Show Name', 'Season', 'Href'])

        # Printing extracted data
        tv_shows_df.to_csv(RATINGS_DATA_DIRECTORY + "/tv_shows.csv", index=False)

```

1. Find the HREF to the Google Sheet Data

This updates the tv_shows.csv to add a column that links to the google doc with ratings info. It is written to tv_shows2.csv.

```

In [157... # convert href field.
# https://www.google.com/url?q=https://tvseriesfinale.com/tv-show/1000000-pyramid-season-one-ratings/&sa=D&source
# =editors&ust=1710525776202777&usg=AOvVaw3U8iARriqaPW_WeGdQxyg0
# to https://tvseriesfinale.com/tv-show/1000000-pyramid-season-one-ratings/

# Function to extract the correct URL from the href field
def extract_correct_url(href):
    # Match the pattern and extract the URL
    match = re.search(r'(?<=q=)(https?:/[^\&]+)', href)
    if match:
        return match.group(0)
    else:
        return None

if not SKIP_TO_MODELING:
    if DOWNLOAD_SHEETS:
        tv_shows_df = pd.read_csv(RATINGS_DATA_DIRECTORY + "/tv_shows.csv")
        # Apply the function to all rows in the DataFrame
        tv_shows_df['Href'] = tv_shows_df['Href'].apply(extract_correct_url)

```

```
# Printing extracted data
tv_shows_df.to_csv(PREWEBSCRAPING_GOOGLE_DOCS_FILE, index=False)
else:
    download_file(PREWEBSCRAPING_GOOGLE_DOCS_URL, PREWEBSCRAPING_GOOGLE_DOCS_FILE)
    tv_shows_df = pd.read_csv(PREWEBSCRAPING_GOOGLE_DOCS_FILE)
```

1. Get the Google Doc

Note: This takes a while even after the google docs are downloaded.

```
In [158... # seriously consider not running this step.
DOWNLOAD_SHEETS = False

def download_season_html_file(show_name, season, google_docs_link):
    html_output_filename = os.path.join(RATING_SHEETS_DATA_DIRECTORY, f"{show_name}_{season}_sheet.html")
    # Check if file already exists
    if not os.path.exists(html_output_filename):
        url = google_docs_link['href']
        response = fetch_url_with_retry(url)
        if response:
            with open(html_output_filename, "w") as html_output_file:
                print("writing " + html_output_filename)
                html_output_file.write(response.text)
        else:
            print(f"Already downloaded {html_output_filename}")

if not SKIP_TO_MODELING:
    if DOWNLOAD_SHEETS:

        tv_shows_df = pd.read_csv(PREWEBSCRAPING_GOOGLE_DOCS_FILE)

        # Extracting Google Docs Links
        google_docs = []
        for index, tv_show_rec in tv_shows_df.iterrows():
            url = tv_show_rec['Href']
            print("2. Show Name: ", tv_show_rec['Show Name'], ". Season: ", tv_show_rec['Season'])
            response = fetch_url_with_retry(url)

            if response:
                show_name = tv_show_rec['Show Name']
                season = tv_show_rec['Season']
                soup = BeautifulSoup(response.text, 'html.parser')
                google_docs_link = soup.find('a', href=re.compile(r'google.com/spreadsheets'))
                print("Link: ", google_docs_link)
                if google_docs_link:
                    google_docs.append(google_docs_link['href'])
                    download_season_html_file(show_name, season, google_docs_link)
                else:
                    google_docs.append(None)
            else:
                google_docs.append(None)

        tv_shows_df['Google Docs'] = google_docs

        # Printing extracted data
        tv_shows_df.to_csv(POSTWEBSCRAPING_GOOGLE_DOCS_FILE, index=False)
    else:
        download_file(POSTWEBSCRAPING_GOOGLE_DOCS_URL, POSTWEBSCRAPING_GOOGLE_DOCS_FILE)
        tv_shows_df = pd.read_csv(POSTWEBSCRAPING_GOOGLE_DOCS_FILE)
```

Scrape Web for Nielsen's Data (in Google Docs)

Read sheets to build TV Ratings database.

This runs on top of the google docs just downloaded. They have a lot of inconsistencies in the format of their tables

```
In [159... if not SKIP_TO_MODELING:
    if BUILD_RATINGS_FROM_SHEETS:

        # Initialize Lists to store extracted data

        day_of_weeks = []
        air_dates = []
        episodes = []
        demo_18_49 = []
        demo_change = []
        viewers_millions = []
        viewers_change = []
        show_names = []
        seasons = []

        tv_columns = ['Show Name', 'Season', 'Day of Week', 'Air Date', 'Episode', '18-49 Demo', \
                      'Demo Change', 'Viewers (millions)', 'Viewers Change']
        tv_ratings_df = pd.DataFrame(columns=tv_columns)

        # Extracting ratings data from Google Docs
        for file in os.listdir(RATING_SHEETS_DATA_DIRECTORY):
            if file.endswith('sheet.html'):
                print(file)
                with open(os.path.join(RATING_SHEETS_DATA_DIRECTORY, file), 'r', encoding='utf-8') as ratings_file:
                    show_name = file.split("_")[0]
                    rest_of_name = file.split("_")[1]
                    season = rest_of_name.split("_")[0]
```

```

ratings_html = ratings_file.read()
soup = BeautifulSoup(ratings_html, 'html.parser')
print(f"Processing {season} of {show_name}")
for row in soup.find_all('tr'):
    cols = row.find_all('td')
    if len(cols) == 8:
        # print("padded show: ", show_name, " for season ", season)
        if cols[7].getText().strip()!="":
            seasons.append(season)
            show_names.append(show_name)
            day_of_weeks.append(cols[0].get_text())
            air_dates.append(cols[1].get_text())
            if cols[2].getText()=="":
                # check if column 3 is in episode format ##-##
                if len(cols[3].get_text().split("-")) == 2:
                    episodes.append(cols[3].get_text())
                    demo_18_49.append(cols[4].get_text())
                    demo_change.append("")
                    viewers_millions.append(cols[6].get_text())
                    viewers_change.append("")
                else:
                    episodes.append("")
                    demo_18_49.append(cols[3].get_text())
                    demo_change.append(cols[4].get_text())
                    viewers_millions.append(cols[5].get_text())
                    viewers_change.append(cols[6].get_text())
            elif cols[2].getText().strip()!="":
                episodes.append(cols[2].get_text())
                demo_18_49.append(cols[3].get_text())
                demo_change.append(cols[4].get_text())
                viewers_millions.append(cols[5].get_text())
                viewers_change.append(cols[6].get_text())
            elif cols[2].getText().strip()=="":
                seasons.append(season)
                show_names.append(show_name)
                day_of_weeks.append(cols[0].get_text())
                air_dates.append(cols[1].get_text())
                # cols[2] is empty
                episodes.append(cols[3].get_text())
                demo_18_49.append(cols[4].get_text())
                demo_change.append(cols[5].get_text())
                viewers_millions.append(cols[6].get_text())
                viewers_change.append(cols[7].get_text())
            else:
                print("**** ERROR! ****", " for padded show: ", show_name, " for season ", season)
        elif len(cols) == 7:
            seasons.append(season)
            show_names.append(show_name)
            # print("normal format line for show: ", show_name, " for season ", season)
            day_of_weeks.append(cols[0].get_text())
            air_dates.append(cols[1].get_text())
            if cols[2].getText().strip()=="":
                # check if column 3 is in episode format ##-##
                if len(cols[3].get_text().split("-")) == 2:
                    print("column 3 has ##-## format for for show: ", show_name, " for season ", season, ". ", cols[3].getText())
                    episodes.append(cols[3].get_text())
                    demo_18_49.append(cols[4].get_text())
                    demo_change.append("")
                    viewers_millions.append(cols[6].get_text())
                    viewers_change.append("")
                else:
                    # missing episode name (60 minutes, big brother)
                    print("column 3 does not have ##-## format for for show: ", show_name, " for season ", season)
                    episodes.append("")
                    demo_18_49.append(cols[3].get_text())
                    demo_change.append(cols[4].get_text())
                    viewers_millions.append(cols[5].get_text())
                    viewers_change.append(cols[6].get_text())
            else:
                episodes.append(cols[2].get_text())
                demo_18_49.append(cols[3].get_text())
                demo_change.append(cols[4].get_text())
                viewers_millions.append(cols[5].get_text())
                viewers_change.append(cols[6].get_text())

# Add the extracted data to the DataFrame
tv_ratings_df['Show Name'] = show_names
tv_ratings_df['Season'] = seasons
tv_ratings_df['Day of Week'] = day_of_weeks
tv_ratings_df['Air Date'] = air_dates
tv_ratings_df['Episode'] = episodes
tv_ratings_df['18-49 Demo'] = demo_18_49
tv_ratings_df['Demo Change'] = demo_change
tv_ratings_df['Viewers (millions)'] = viewers_millions
tv_ratings_df['Viewers Change'] = viewers_change
# write extracted data
tv_ratings_df.to_csv(RATINGS_DATA_DIRECTORY + "/tv_ratings.csv", index=False)

```

Clean Up the Data

```

In [160]: # Mapping dictionary for word representation of numbers to integers
word_to_number = {
    "one": 1,
    "two": 2,
    "three": 3,

```

```

"four": 4,
"five": 5,
"six": 6,
"seven": 7,
"eight": 8,
"nine": 9,
"ten": 10,
}

# Function to extract numeric part from strings like "season one", "season two", etc.
def extract_season_number(season_string):
    if season_string.startswith("season "):
        # Remove "season " from the string
        season_number = season_string.replace("season ", "")
        # Try converting to integer
        try:
            return int(season_number)
        except ValueError:
            # If it's not a numeric value, try to map it to a number
            return word_to_number.get(season_number.lower(), None)
    else:
        return None

if not SKIP_TO_MODELING:

    if BUILD_RATINGS_FROM_SHEETS:
        # Read the CSV file into DataFrame
        tv_ratings_df = pd.read_csv(RATINGS_DATA_DIRECTORY + "/tv_ratings.csv")

        # T1: Drop rows where 'Air Date' is NaN
        tv_ratings_df = tv_ratings_df.dropna(subset=['Air Date'])

        # T2: Drop 'Demo Change' and 'Viewers Change' columns
        tv_ratings_df = tv_ratings_df.drop(columns=['Demo Change', 'Viewers Change', 'Unnamed: 0'])

        # T3: Drop rows with 'Air Date' not in mm/dd/yyyy format
        tv_ratings_df['Air Date'] = pd.to_datetime(tv_ratings_df['Air Date'], errors='coerce')
        tv_ratings_df = tv_ratings_df.dropna(subset=['Air Date'])

        # Drop rows with NaN values in 'Viewers (millions)' column
        tv_ratings_df = tv_ratings_df.dropna(subset=['Viewers (millions)'])

        # Apply the function to 'Season' column
        tv_ratings_df['Season'] = tv_ratings_df['Season'].apply(extract_season_number)

        # Set the 'Season' field of NaN values to 9
        tv_ratings_df['Season'].fillna(9, inplace=True)

        # Convert 'Season' column to integer type
        tv_ratings_df['Season'] = tv_ratings_df['Season'].astype(int)

        # Show DataFrame information
        show_df_info(tv_ratings_df, "ratings")

        # Write extracted data to new CSV file
        tv_ratings_df.to_csv(NIELSEN_RATINGS_FILE, index=False)

    else:
        download_file(NIELSEN_RATINGS_URL, NIELSEN_RATINGS_FILE)
        tv_ratings_df = pd.read_csv(NIELSEN_RATINGS_FILE)

```

These provide by channel Nielsen ratings for series:

<https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-disney-originals.html>
<https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-amazon-prime-video.html>
<https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-hulu-originals.html>
<https://www.ratingsryan.com/2023/03/nielsen-svod-ratings-apple.html>
<https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-netflix-originals.html>
https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-netflix-originals_30.html
<https://www.ratingsryan.com/2022/01/nielsen-svod-ratings-netflix-originals.html>
<https://www.ratingsryan.com/2023/01/nielsen-svod-ratings-netflix-originals.html>
<https://www.ratingsryan.com/2024/02/nielsen-svod-ratings-netflix-originals.html>
<https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-acquired-programs.html>

These provide by channel Nielsen ratings for movies:

<https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-disney-originals.html>
<https://www.ratingsryan.com/2022/07/nielsen-svod-ratings-movies-archive.html>
https://www.ratingsryan.com/2022/07/nielsen-svod-ratings-movies-archive_3.html
<https://www.ratingsryan.com/2023/01/nielsen-svod-ratings-movies-archive.html>

<https://www.ratingsryan.com/2024/02/nielsen-svod-ratings-movies-archive.html>

Here are ratings archives for earier years:

<https://www.ratingsryan.com/p/ratings-archive.html>

By channel:

<https://www.ratingsryan.com/p/a-ratings.html>

<https://www.ratingsryan.com/p/abc-family-ratings.html>

<https://www.ratingsryan.com/p/amc-ratings.html>

<https://www.ratingsryan.com/p/bbca-ratings.html>

<https://www.ratingsryan.com/p/bravo-ratings.html>

<https://www.ratingsryan.com/p/cinemax-ratings.html>

<https://www.ratingsryan.com/p/esquire-ratings.html>

<https://www.ratingsryan.com/p/fx-ratings.html>

<https://www.ratingsryan.com/p/fxx-ratings.html>

<https://www.ratingsryan.com/p/hallmark-ratings.html>

<https://www.ratingsryan.com/p/hbo-ratings.html>

<https://www.ratingsryan.com/p/ifc-ratings.html>

<https://www.ratingsryan.com/p/lifetime.html>

<https://www.ratingsryan.com/p/miscellaneous-shows.html>

https://www.ratingsryan.com/p/mtv-ratings_28.html

<https://www.ratingsryan.com/p/pop-tvg-ratings.html>

<https://www.ratingsryan.com/p/showtime-ratings.html>

<https://www.ratingsryan.com/p/starz-ratings.html>

<https://www.ratingsryan.com/p/sundance-tv-ratings.html>

<https://www.ratingsryan.com/p/syfy-ratings.html>

<https://www.ratingsryan.com/p/tnt-ratings.html>

<https://www.ratingsryan.com/p/tv-land-ratings.html>

<https://www.ratingsryan.com/p/usa-ratings.html>

<https://www.ratingsryan.com/p/vh1.html>

<https://www.ratingsryan.com/p/wgna-ratings.html>

By season:

<https://www.ratingsryan.com/p/2021-22-tv-season.html>

<https://www.ratingsryan.com/p/2020-21-tv-season.html>

<https://www.ratingsryan.com/p/summer-2020.html>

<https://www.ratingsryan.com/p/fall-2018.html>

https://www.ratingsryan.com/p/2017-18-tv-season_14.html

<https://www.ratingsryan.com/p/2016-17-tv-season.html>

<https://www.ratingsryan.com/p/2015-16-tv-season.html>

<https://www.ratingsryan.com/p/2014-15-tv-season.html>

<https://www.ratingsryan.com/p/2013-14-tv-season.html>

Same Idea:

- Download the Pages
- Get link to Ratings Spreadsheet
- Download sheet
- Build Dataframe

In [161... `if not SKIP_TO_MODELING:`

```

pages_to_download = {

    # svod series - link on top for each show and a subpage for each.

    "https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-disney-originals.html",
    "https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-amazon-prime-video.html",
    "https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-hulu-originals.html",
    "https://www.ratingsryan.com/2023/03/nielsen-svod-ratings-apple.html",
    "https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-netflix-originals.html",
    "https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-netflix-originals_30.html",
    "https://www.ratingsryan.com/2022/01/nielsen-svod-ratings-netflix-originals.html",
    "https://www.ratingsryan.com/2023/01/nielsen-svod-ratings-netflix-originals.html",
    "https://www.ratingsryan.com/2024/02/nielsen-svod-ratings-netflix-originals.html",
    "https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-acquired-programs.html",
    "https://www.ratingsryan.com/2021/12/nielsen-svod-ratings-disney-originals.html",

    # svod films

    "https://www.ratingsryan.com/2022/07/nielsen-svod-ratings-movies-archive.html",
    "https://www.ratingsryan.com/2022/07/nielsen-svod-ratings-movies-archive_3.html",
    "https://www.ratingsryan.com/2023/01/nielsen-svod-ratings-movies-archive.html",
    "https://www.ratingsryan.com/2024/02/nielsen-svod-ratings-movies-archive.html",

    # archive links
    "https://www.ratingsryan.com/p/ratings-archive.html",

    # past archives by channel

    "https://www.ratingsryan.com/p/a-ratings.html",
    "https://www.ratingsryan.com/p/abc-family-ratings.html",
    "https://www.ratingsryan.com/p/amc-ratings.html",
    "https://www.ratingsryan.com/p/bbca-ratings.html",
    "https://www.ratingsryan.com/p/bravo-ratings.html",
    "https://www.ratingsryan.com/p/cinemax-ratings.html",
    "https://www.ratingsryan.com/p/esquire-ratings.html",
    "https://www.ratingsryan.com/p/fx-ratings.html",
    "https://www.ratingsryan.com/p/fxx-ratings.html",
    "https://www.ratingsryan.com/p/hallmark-ratings.html",
    "https://www.ratingsryan.com/p/hbo-ratings.html",
    "https://www.ratingsryan.com/p/ifc-ratings.html",
    "https://www.ratingsryan.com/p/lifetime.html",
    "https://www.ratingsryan.com/p/miscellaneous-shows.html",
    "https://www.ratingsryan.com/p/mtv-ratings_28.html",
    "https://www.ratingsryan.com/p/pop-tvg-n-ratings.html",
    "https://www.ratingsryan.com/p/showtime-ratings.html",
    "https://www.ratingsryan.com/p/starz-ratings.html",
    "https://www.ratingsryan.com/p/sundance-tv-ratings.html",
    "https://www.ratingsryan.com/p/syfy-ratings.html",
    "https://www.ratingsryan.com/p/tnt-ratings.html",
    "https://www.ratingsryan.com/p/tv-land-ratings.html",
    "https://www.ratingsryan.com/p/usa-ratings.html",
    "https://www.ratingsryan.com/p/vh1.html",
    "https://www.ratingsryan.com/p/wgna-ratings.html",

    # past archives by season

    "https://www.ratingsryan.com/p/2021-22-tv-season.html",
    "https://www.ratingsryan.com/p/2020-21-tv-season.html",
    "https://www.ratingsryan.com/p/summer-2020.html",

```

```

"https://www.ratingsryan.com/p/fall-2018.html",
"https://www.ratingsryan.com/p/2017-18-tv-season_14.html",
"https://www.ratingsryan.com/p/2016-17-tv-season.html",
"https://www.ratingsryan.com/p/2015-16-tv-season.html",
"https://www.ratingsryan.com/p/2014-15-tv-season.html",
"https://www.ratingsryan.com/p/2013-14-tv-season.html"
}

if DOWNLOAD_RATINGS_SCRAPE_FODDER:
    for page_to_download in pages_to_download:
        # print("Downloading ", page_to_download)
        local_name = page_to_download.split("https://www.ratingsryan.com/")[1]
        local_name = local_name.replace("/", "_")
        # print(name)
        downloadFile(page_to_download, SVOD_SHEETS_DATA_DIRECTORY + "/" + local_name)

```

Get the link for the Ratings Data and Download

```

In [162... def extract_href_links_with_week_ending(html_content):
    soup = BeautifulSoup(html_content, 'html.parser')
    href_links = []
    program_links = []
    # Find all tables
    for table in soup.find_all('table'):
        # Check if the table contains a 'th' tag with the text 'Week Ending'
        th_tags = table.find_all('th')
        if any(th.text.strip() == 'Week Ending' for th in th_tags):
            # If the table contains the header, find all 'a' tags within it
            for a_tag in table.find_all('a'):
                href = a_tag.get('href')
                if href:
                    href_links.append(href)
        if any(th.text.strip() == 'Program' for th in th_tags):
            # If the table contains the header, find all 'a' tags within it
            for a_tag in table.find_all('a'):
                href = a_tag.get('href')
                if href:
                    program_links.append(href)

    return href_links, program_links

if not SKIP_TO_MODELING:

    if DOWNLOAD_RATINGS_SCRAPE_FODDER:

        # Extracting ratings data from Google Docs
        # Iterate through HTML files just downloaded.
        all_href_links = []
        all_program_links = []

        print("Reading HTML from ", SVOD_SHEETS_DATA_DIRECTORY)

        for file in os.listdir(SVOD_SHEETS_DATA_DIRECTORY):
            if file.endswith('.html'):
                print(file)
                with open(os.path.join(SVOD_SHEETS_DATA_DIRECTORY, file), 'r', encoding='utf-8') as svod_file:
                    svod_html = svod_file.read()
                    soup = BeautifulSoup(svod_html, 'html.parser')
                    # Find all iframe tags
                    iframe_tags = soup.find_all('iframe')
                    # Iterate through iframe tags
                    for iframe in iframe_tags:
                        # Extract the value of src attribute
                        src_value = iframe.get('src')
                        if src_value:
                            print("Found google doc in ", file)
                            # Download the Google Doc using the src URL
                            download_file(src_value, SVOD_GOOGLEDOC_DATA_DIRECTORY + "/" + file.split(".")[0] + ".doc.html")
                    # get all the hrefs in the week ending tables
                    href_links, program_links = extract_href_links_with_week_ending(svod_html)
                    if len(href_links) > 0:
                        print("Found day links in ", file)
                        all_href_links.append(href_links)
                    if len(program_links) > 0:
                        print("Found ", len(program_links), " show links in ", file)
                        all_program_links.append(program_links)

        # Flatten the List of Lists
        all_href_links = [link for sublist in all_href_links for link in sublist]
        for href_link in all_href_links:
            downloadFile(href_link, SVOD_DAYS_DATA_DIRECTORY + "/" + href_link.split("/")[-1])

        all_program_links = [link for sublist in all_program_links for link in sublist]
        for href_link in all_program_links:
            download_file(href_link, SVOD_PROGRAM_DATA_DIRECTORY + "/" + href_link.split("/")[-1])

```

Scrape SVOD Data from Google Doc Sheets & Save Data Frame


```

In [163... if not SKIP_TO_MODELING:

    if BUILD_RATINGS_FROM_SCRAPING:

        # List to hold table data
        table_data = []

        # Expected field names
        expected_field_names = ['Show', 'Platform', 'Week Ending', 'Minutes (Millions)']

        for file in os.listdir(SVOD_GOOGLEDOC_DATA_DIRECTORY):
            if file.endswith('.html'):
                print(file)
                with open(os.path.join(SVOD_GOOGLEDOC_DATA_DIRECTORY, file), 'r', encoding='utf-8') as svod_file:
                    html_content = svod_file.read()

                # Parse HTML content
                soup = BeautifulSoup(html_content, 'html.parser')

                # Find all tables with class "waffle"
                tables = soup.find_all('table', class_='waffle')

                print("found ", len(tables), " tables.")

                # Iterate over each table
                for table in tables:
                    # Navigate to tbody
                    tbody = table.find('tbody')
                    if tbody:
                        # Check if the first row of tbody contains the expected column names
                        header_row = tbody.find('tr')
                        if header_row:
                            column_names = [td.text.strip() for td in header_row.find_all('td')]
                            if column_names == expected_field_names:
                                # Read subsequent rows and extract the data
                                data_rows = tbody.find_all('tr')[1:]
                                for row in data_rows:
                                    # Extract only the cells with class "s1" (data)
                                    row_data = [td.text.strip() for td in row.find_all('td', class_='s1')]
                                    table_data.append(row_data)

                            # Convert table data to DataFrame
                            svod_df = pd.DataFrame(table_data, columns=expected_field_names)

                            # T1: Drop rows where 'Show' is NaN
                            svod_df = svod_df.dropna(subset=['Show'])
                            # Write extracted data to new CSV file
                            svod_df.to_csv(SVOD_MEASURE_FILE, index=False)
                        else:
                            download_file(SVOD_MEASURE_URL, SVOD_MEASURE_FILE)
                            svod_df = pd.read_csv(SVOD_MEASURE_FILE)

                show_df_info(svod_df, "svod ratings")

```

Web scrape Ratings Data from Internet Sites

```

In [164... if not SKIP_TO_MODELING:

    # List to hold table data
    table_data = []

    # Expected field names
    expected_field_names = ['Episode Date', 'Viewers (millions)', '18-49 rating']
    all_columns = ['Show', 'Season', 'Episode Date', 'Viewers (millions)', '18-49 rating']

    if BUILD_RATINGS_FROM_SCRAPING:

        for file in os.listdir(SVOD_PROGRAM_DATA_DIRECTORY):
            if file.endswith('.html'):
                print(file)
                with open(os.path.join(SVOD_PROGRAM_DATA_DIRECTORY, file), 'r', encoding='utf-8') as svod_file:
                    html_content = svod_file.read()

                # Parse HTML content
                soup = BeautifulSoup(html_content, 'html.parser')

                # Find all tables with class "waffle"
                tables = soup.find_all('table')

                # print("found ", len(tables), " tables.")

                # Iterate over each table
                for table in tables:
                    # Navigate to tbody
                    tbody = table.find('tbody')
                    good_table = True
                    if tbody:
                        # Extract table name from the first row text
                        first_row_text = tbody.find('tr').text.strip()
                        if len(first_row_text) != 0:
                            season_index = first_row_text.find("Season")
                            show_name = first_row_text[:season_index].strip() if season_index != -1 else first_row_text.strip()
                            if season_index != -1:
                                # season_year: Season 1 Ratings (2016)
                                # print("season: ==> ", first_row_text)
                                season_year = first_row_text.split(' Ratings')[0]

```

```

        season_year = season_year.split(' ')[-1]
        # print("season_year => ", season_year)
        season = season_year
    else:
        try:
            season_year = show_name.split('(')[1]
            season_year = season_year.split('-')[0]
            show_name = show_name.split(' Ratings')[0].strip()
            season = 1
        except IndexError:
            good_table = False
            # print("Invalid string: ", show_name)

    if good_table:
        # if season_year:
        #     print("show: ", show_name)
        #     print("season_index: ", season_index)
        #     print("season_year: ", season_year)

        # Read subsequent rows and extract the data
        data_rows = tbody.find_all('tr')[2:] # Skip the first two rows (headers and season info)
        for row in data_rows:
            row_data = [show_name, season] + [td.text.strip() for td in row.find_all('td', recursive=False)]
            # skip summaries
            if row_data[2] != 'Average':
                if len(row_data)>5:
                    print("Number of elements in row_data:", len(row_data))
                    print(row_data)
                    table_data.append(row_data[:5]) # Append only the first 5 columns
                elif len(row_data)==5:
                    # print(row_data)
                    table_data.append(row_data)

        # Convert table data to DataFrame
        programs_ratings_df = pd.DataFrame(table_data, columns=all_columns)

        # Show DataFrame information
        show_df_info(programs_ratings_df, "svod ratings")

        # Write extracted data to new CSV file
        programs_ratings_df.to_csv(RATINGS_DATA_DIRECTORY + "/programs_ratings.csv", index=False)

    else:

        programs_ratings_df = pd.read_csv(RATINGS_DATA_DIRECTORY + "/programs_ratings.csv")

```

Databases for Preparation

```

In [165... if not SKIP_TO_MODELING:

    # if files are not present (like if you skipped all the webscraping), download them from google drive

    download_file(NIELSEN_RATINGS_URL, NIELSEN_RATINGS_FILE)
    download_file(SVOD_RATINGS_URL, SVOD_MEASURE_FILE)
    download_file(ARCHIVE_RATINGS_URL, ARCHIVE_RATINGS_FILE)
    download_file(ORIGINAL_TMDB_DATASET_URL, TMDB_SERIES_MAIN_FILE_V3)

```

```

In [166... if not SKIP_TO_MODELING:

    nielsen_ratings_df = pd.read_csv(NIELSEN_RATINGS_FILE)
    svod_ratings_df = pd.read_csv(SVOD_MEASURE_FILE)
    archive_ratings_df = pd.read_csv(ARCHIVE_RATINGS_FILE)
    tmdb_tv_df = pd.read_csv(TMDB_SERIES_MAIN_FILE_V3)

```

Show Description and 10 Rows of the Contents

```

In [167... if not SKIP_TO_MODELING:
    show_df_info(nielsen_ratings_df, "Nielsen Viewers")
    show_df_info(svod_ratings_df, "Video on Demand")
    show_df_info(archive_ratings_df, "Archive Program Ratings")
    show_df_info(tmdb_tv_df, "TMDB") # tmdb_tv_dataset_v3

    pretty_print_df(nielsen_ratings_df, rows=10)
    pretty_print_df(svod_ratings_df, rows=10)
    pretty_print_df(archive_ratings_df, rows=10)
    # just first 4 columns
    pretty_print_df(tmdb_tv_df.iloc[:, :4], rows=10)

```

SVOD Data Transformations

- SVOD is newest measure system for platforms.
- Some series are not in the newest data base on Kaggle. Go directly to TMDB API to get this data.
- Also, many entries in the SVOD table are films, not series. They need to be removed.
- New entries for series (not in tmdb_tv_df) are added to main data frame.

Get SVOD Data

```
if not SKIP_TO_MODELING: svod_series_df = pd.read_csv(SVOD_MEASURE_FILE) show_df_info(svod_series_df, "SVOD Data")
```

Rename Show Column to Title

```
In [168... if not SKIP_TO_MODELING:

svod_series_df = svod_series_df.rename(columns={'Show': 'Title'})
```

Convert Platform Field to find network fields.

```
In [169... if not SKIP_TO_MODELING:

platforms_list = ['DISNEY+', 'HULU/DISNEY+', 'NETFLIX', 'AMAZON', 'APPLE TV+', 'MAX/NET', 'MAX',
                  'HULU/NET', 'HULU', 'PEACOCK', 'PARAMOUNT+', 'PARA/NET', 'HULU/AMA', 'PEA/NET',
                  'HULU/PARA+', 'HBO MAX', 'AMAZON/HULU', 'PEA/HULU', 'HULU/MAX', 'AMA/NET',
                  'PARA/AMA', 'MAX/DISNEY+', 'NET/PARA+']

network_replacement_list = ['Disney+', 'Hulu, Disney+', 'Netflix', 'Prime Video', 'Apple TV+', 'HBO Max, Netflix', 'HBO Max',
                             'Hulu, Netflix', 'Hulu', 'Peacock', 'Paramount+', 'Paramount+, Netflix', 'Hulu, Prime Video', 'Peacock, Netflix',
                             'Hulu, Paramount+', 'HBO Max', 'Prime Video, Hulu', 'Peacock, Hulu', 'Hulu, HBO Max', 'Prime Video, Netflix',
                             'Paramount+, Prime Video', 'HBO Max, Disney+', 'Netflix, Paramount+']

# Create boolean columns for each network
# They are not quite the same so make a new field "Networks"
# Create a mapping from platforms_list to network_replacement_list using zip.
platform_to_network = dict(zip(platforms_list, network_replacement_list))

# Map the 'Platform' column to the 'networks' column using the mapping
svod_series_df['networks'] = svod_series_df['Platform'].map(platform_to_network)

show_df_info(svod_series_df, "SVOD Data")
```

Build List of Keys for TMDB Search

- TMDB API Access is Metered.

```
In [170... if not SKIP_TO_MODELING:

titles_to_check = svod_series_df['Title'].unique()

print(len(titles_to_check))
```

For TMDB Use TMDBV3API

<https://github.com/AnthonyBloomer/tmdbv3api>

```
In [171... if not SKIP_TO_MODELING:

tmdb = TMDB()
tmdb.api_key = "de237f1a1a9211ebbbe2e4213faa265"
tmdb.read_access_token = "eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJkZTlIzN2YxYTFhOTIxMmVlYmJlZTJlNDIxM2ZyYTI2NSIsIn" + \
                          "N1YiOi6jY2MYZyZyYmNmEzMDBiMDE3ZTMzMGMjMGJkNCIsInNjb3B1cyI6WyJhcGlfcmlhZCJlZCJ2ZCZxZW9uIjoxfQ.NcNdEoY5xb2P" + \
                          "iF06DaelBjEblm0y-NTF7sPKZ7BT61U"
```

Fill in Missing Rows in Series

- version of tmdb for series on Kaggle is too old for 2024.
- use api to fill in rows.

Translate Genres - helper function.

Genres are numerically encoded in the api, but lists of string literals in the data base.

```
In [172... if not SKIP_TO_MODELING:

tmdb_genre_encoding = [
    {"id": 10759, "name": "Action & Adventure"},
    {"id": 16, "name": "Animation"},
    {"id": 35, "name": "Comedy"},
    {"id": 80, "name": "Crime"},
    {"id": 99, "name": "Documentary"},
    {"id": 18, "name": "Drama"},
    {"id": 10751, "name": "Family"},
    {"id": 10762, "name": "Kids"},
    {"id": 9648, "name": "Mystery"},
    {"id": 10763, "name": "News"},
    {"id": 10764, "name": "Reality"},
    {"id": 10765, "name": "Sci-Fi & Fantasy"},
    {"id": 10766, "name": "Soap"},
    {"id": 10767, "name": "Talk"},
    {"id": 10768, "name": "War & Politics"},
    {"id": 37, "name": "Western"}
]

def get_genre_names(genre_ids):
    genre_names = []
    for genre_id in genre_ids:
        for genre in tmdb_genre_encoding:
            if genre["id"] == genre_id:
                genre_names.append(genre["name"])
                break
    return ", ".join(genre_names)

# Example usage
```

```
genre_ids = [18, 80]
print(get_genre_names(genre_ids)) # Output: Drama, Crime
```

Find Series in SVOD Not Present in TMDb Main File.

```
In [173... if not SKIP_TO_MODELING:

    if CREATE_MISSING_SERIES_DATAFRAME:
        search = Search()
        tv_details_fetcher = TV()
        movie_details_fetcher = Movie()

        # Initialize an empty list to store API results
        tmdb_results = []
        possible_results = []
        best_movie_results = []

        # Loop through each title
        for title in titles_to_check:

            print(f"Accessing TMDb records for {title}")
            # Use the TMDb API to search for the TV show by title
            matches_dict = search.tv_shows(title)

            # Extract the results list from the dictionary
            matches = matches_dict.get('results', [])

            if matches:
                save_page = None
                page_to_use = None
                series_vote_count = 0
                for page in matches:
                    if 'vote_count' in page and isinstance(page['vote_count'], int):
                        test_vote_count = page['vote_count']
                        if page_to_use is None or test_vote_count > series_vote_count:
                            page_to_use = page
                            series_vote_count = test_vote_count

                # make sure it's not a film in disguise
                movie_matches = search.movies(title)
                movie_vote_count = 0
                movie_result_to_use = None
                for movie_match in movie_matches:
                    if 'vote_count' in movie_match and isinstance(movie_match['vote_count'], int):
                        test_vote_count = movie_match['vote_count']
                        if test_vote_count > movie_vote_count:
                            movie_vote_count = test_vote_count
                            movie_result_to_use = movie_match

                # put in two lists (Likely and improbable, which has frequent movie mismatches)
                if page_to_use and series_vote_count > movie_vote_count:
                    print(f"processing tmdb page for {title}")
                    detailed_info = tv_details_fetcher.details(page_to_use['id']) # Fetch the detailed info using the ID
                    # build a list of SVOD shows (includes movies and series)
                    tmdb_results.append([title, page_to_use['id'], detailed_info])
                elif page_to_use:
                    print(f"higher voted film for {title}")
                    detailed_info = tv_details_fetcher.details(page_to_use['id']) # Fetch the detailed info using the ID
                    # build a list of SVOD shows (includes movies and series)
                    possible_results.append([title, page_to_use['id'], detailed_info])
                    # also build a list of films for later comparison.
                    if movie_result_to_use:
                        detailed_info = movie_details_fetcher.details(movie_result_to_use['id'])
                        best_movie_results.append([title, movie_result_to_use['id'], detailed_info])

            else:
                # Log an error or handle the case where no matches are found
                print(f"No matches found for {title}")
```

Create Data Frame with Missing Series Rows

```
In [174... def create_missing_series_dataframe(results_to_test):
    ids = []
    titles = []
    number_of_seasons_list = []
    number_of_episodes_list = []
    original_languages = []
    vote_counts = []
    vote_averages = []
    overviews = []
    adults = []
    backdrop_paths = []
    first_air_dates = []
    last_air_dates = []
    homepages = []
    in_productions = []
    original_names = []
    popularities = []
    poster_paths = []
    types = []
    status_list = []
    taglines = []
    genres_list = []
    created_bys = []
    languages_list = []
    networks_list = []
```

```

origin_countrys = []
spoken_languages_list = []
production_companies_list = []
production_countries_list = []
episode_run_times = []

for tmdb_result in results_to_test:
    title, tmdb_id, detailed_info = tmdb_result

    ids.append(tmdb_id)
    titles.append(title)

    # Extract data from detailed_info, ensuring to handle missing data and data types appropriately
    number_of_seasons_list.append(detailed_info.get('number_of_seasons', None))
    number_of_episodes_list.append(detailed_info.get('number_of_episodes', None))
    original_languages.append(detailed_info.get('original_language', None))
    vote_counts.append(detailed_info.get('vote_count', None))
    vote_averages.append(detailed_info.get('vote_average', None))
    overviews.append(detailed_info.get('overview', None))
    adults.append(detailed_info.get('adult', None))
    backdrop_paths.append(detailed_info.get('backdrop_path', None))
    first_air_dates.append(detailed_info.get('first_air_date', None))
    last_air_dates.append(detailed_info.get('last_air_date', None))
    homepages.append(detailed_info.get('homepage', None))
    in_productions.append(detailed_info.get('in_production', None))
    original_names.append(detailed_info.get('original_name', None))
    popularities.append(detailed_info.get('popularity', None))
    poster_paths.append(detailed_info.get('poster_path', None))
    types.append(detailed_info.get('type', None)) # May need correction based on actual JSON key
    status_list.append(detailed_info.get('status', None))
    taglines.append(detailed_info.get('tagline', None))

    # Join genres into a comma-separated string
    genres_list.append(", ".join([genre['name'] for genre in detailed_info.get('genres', [])]))

    # Join creators into a comma-separated string
    created_by.append(", ".join([creator['name'] for creator in detailed_info.get('created_by', [])]))

    # Join Languages into a comma-separated string
    languages_list.append(", ".join(detailed_info.get('languages', [])))

    # Join networks into a comma-separated string
    networks_list.append(", ".join([network['name'] for network in detailed_info.get('networks', [])]))

    # Join origin countries into a comma-separated string
    origin_countrys.append(", ".join(detailed_info.get('origin_country', [])))

    # Join spoken Languages into a comma-separated string
    spoken_languages_list.append(", ".join([lang['english_name'] for lang in detailed_info.get('spoken_languages', [])]))

    # Join production companies into a comma-separated string
    production_companies_list.append(", ".join([company['name'] for company in detailed_info.get('production_companies', [])]))

    # Join production countries into a comma-separated string
    production_countries_list.append(", ".join([country['name'] for country in detailed_info.get('production_countries', [])]))

    episode_run_times.append(detailed_info.get('episode_run_time', None))

column_names = ["ids", "titles", "number_of_seasons_list", "number_of_episodes_list", "original_languages",
                "vote_counts", "vote_averages", "overviews", "adults", "backdrop_paths", "first_air_dates",
                "last_air_dates", "homepages", "in_productions", "original_names", "popularities", "poster_paths",
                "types", "status_list", "taglines", "genres_list", "created_by", "languages_list", "networks_list",
                "origin_countrys", "spoken_languages_list", "production_companies_list", "production_countries_list",
                "episode_run_times"]

data_lists = [ids, titles, number_of_seasons_list, number_of_episodes_list, original_languages, vote_counts,
              vote_averages, overviews, adults, backdrop_paths, first_air_dates, last_air_dates, homepages,
              in_productions, original_names, popularities, poster_paths, types, status_list, taglines,
              genres_list, created_by, languages_list, networks_list, origin_countrys, spoken_languages_list,
              production_companies_list, production_countries_list, episode_run_times]

# Print the Length of each List to see if they all have the same number of elements
for name, data_list in zip(column_names, data_lists):
    print(f'Length of {name}: {len(data_list)}')

# Combine all the data into a DataFrame
series_data = {
    "id" : ids,
    "Title" : titles,
    "number_of_seasons" : number_of_seasons_list,
    "number_of_episodes" : number_of_episodes_list,
    "original_language" : original_languages,
    "vote_count" : vote_counts,
    "vote_average" : vote_averages,
    "overview" : overviews,
    "adult" : adults,
    "backdrop_path" : backdrop_paths,
    "first_air_date" : first_air_dates,
    "last_air_date" : last_air_dates,
    "homepage" : homepages,
    "in_production" : in_productions,
    "original_name" : original_names,
    "popularity" : popularities,
    "poster_path" : poster_paths,
    "type" : types,
    "status" : status_list,

```

```

        "tagline" : taglines,
        "genres" : genres_list,
        "created_by" : created_bys,
        "languages" : languages_list,
        "networks" : networks_list,
        "origin_country" : origin_countrys,
        "spoken_languages" : spoken_languages_list,
        "production_companies" : production_companies_list,
        "production_countries" : production_countries_list,
        "episode_run_time" : episode_run_times
    }

    return pd.DataFrame(series_data)

if not SKIP_TO_MODELING:

    CREATE_MISSING_SERIES_DATAFRAME = False

    if CREATE_MISSING_SERIES_DATAFRAME:
        streaming_series_df = create_missing_series_dataframe(tmdb_results)
        possible_series_df = create_missing_series_dataframe(possible_results)

        streaming_series_df.to_csv(FINAL_DATA_DIRECTORY + "/missing_series_complete.csv", index=False)
        possible_series_df.to_csv(FINAL_DATA_DIRECTORY + "/possible_series_complete.csv", index=False)
    else:
        streaming_series_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/missing_series_complete.csv")
        possible_series_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/possible_series_complete.csv")

```

Compare the Two Files

- They have the same columns.

```

In [175... if not SKIP_TO_MODELING:

    tmdb_tv_df = pd.read_csv(TMDB_SERIES_MAIN_FILE_V3)

    show_df_info(tmdb_tv_df, "Previous Series")
    show_df_info(streaming_series_df, "New Series")

```

```

In [176... if not SKIP_TO_MODELING:

    tmdb_tv_df = tmdb_tv_df.rename(columns={'name': 'Title'})

```

Add Missing Rows to Main Series DF

- before merge review and resolve obvious errors
- if series, then add row.

Here are the problem rows:

- 46952 - The Blacklist (was 95845)
- 85922 - SPACE FORCE (was 2328)
- 57706 - Family Switch

```

In [177... if not SKIP_TO_MODELING:

    streaming_series_df = streaming_series_df[~streaming_series_df['id'].isin([95845, 2328, 57706])]

```

For probable series, I will exclude all before 1-1-2023.

- The others are already in the main data frame!
- This minimizes the possibility of matching a film's svod data for Black Panther with a cartoon series of the same name.

```

In [178... if not SKIP_TO_MODELING:

    # Convert 'last_air_date' column to datetime type
    possible_series_df['last_air_date'] = pd.to_datetime(possible_series_df['last_air_date'])

    # Filter the rows based on the condition
    filtered_possible_series_df = possible_series_df[possible_series_df['last_air_date'] > '2023-01-01'].copy()

    # Now filtered_probable_series_df contains only the rows with 'last_air_date' Later than January 1, 2023
    pretty_print_df(filtered_possible_series_df, interesting_columns = ['id', 'Title', 'number_of_seasons', 'number_of_episodes',
                                                                    'type', 'status', 'last_air_date', 'vote_count'])

    show_df_info(filtered_possible_series_df, "Filtered by Date")

```

```

In [179... if not SKIP_TO_MODELING:

    merged_streaming_series_df = pd.concat([streaming_series_df, filtered_possible_series_df])

```

```

In [180... if not SKIP_TO_MODELING:

    merged_streaming_series_df.to_csv(FINAL_DATA_DIRECTORY + "/missing_series_complete_and_merged.csv", index=False)

```

Full Outer Join

- merged_streaming_series_df new rows added
- merged_streaming_series_df columns of matching rows take precedence

```
In [181... if not SKIP_TO_MODELING:

    merged_streaming_series_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/missing_series_complete_and_merged.csv")

    # Full outer join on 'id'
    merged_df = tmdb_tv_df.merge(merged_streaming_series_df, on='id', how='outer', suffixes=('_x', '_y'), indicator=True)

    # Preserve Title
    # Ensure 'Title' uses tmdb_tv_df's 'Title' unless it is NaN
    merged_df['Title'] = merged_df['Title_x'].combine_first(merged_df['Title_y'])

    # Prioritize merged_streaming_series_df's data when there's an overlap
    for column in merged_streaming_series_df.columns:
        if column in tmdb_tv_df.columns and column != 'id': # Exclude 'Title' from being overwritten
            merged_df[column] = np.where(merged_df[column + '_y'].isnull(), merged_df[column + '_x'], merged_df[column + '_y'])

    # Clean up the DataFrame
    merged_df.drop(columns=[col for col in merged_df if col.endswith('_x') or col.endswith('_y')], inplace=True)

    # Check the resulting DataFrame
    print(merged_df.head())

In [182... if not SKIP_TO_MODELING:
    # drop the merge column
    merged_df.drop(columns=["_merge"], inplace=True)

    show_df_info(merged_df, "All series!")

In [183... if not SKIP_TO_MODELING:
    if CREATE_MISSING_SERIES_DATAFRAME:
        merged_df.to_csv(COMPLETE_SERIES_LIST_FILE, index=False)
    else:
        download_file(COMPLETE_SERIES_LIST_URL, COMPLETE_SERIES_LIST_FILE)
    merged_df = pd.read_csv(COMPLETE_SERIES_LIST_FILE)
```

Generate a Unique Identifier Across All Series Data Sets

TV SERIES - Unique ID

First attempt to get unique ID from IMDb.

```
In [184... if not SKIP_TO_MODELING:

    # read spreadsheets
    nielsen_ratings_df = pd.read_csv(NIELSEN_RATINGS_FILE)
    svod_ratings_df = pd.read_csv(SVOD_MEASURE_FILE)
    archive_ratings_df = pd.read_csv(ARCHIVE_RATINGS_FILE)
    tmdb_tv_df = merged_df.copy()

    show_df_info(nielsen_ratings_df, "Nielsen Viewers")
    show_df_info(svod_ratings_df, "Video on Demand")
    show_df_info(archive_ratings_df, "Archive Program Ratings")
    show_df_info(tmdb_tv_df, "TMDB tv database")

    pretty_print_df(nielsen_ratings_df, rows=10)
    pretty_print_df(svod_ratings_df, rows=10)
    pretty_print_df(archive_ratings_df, rows=10)
    pretty_print_df(tmdb_tv_df, rows=25, interesting_columns = ['id', 'Title', 'number_of_seasons', 'number_of_episodes', 'vote_count',
        'vote_average', 'first_air_date', 'last_air_date', 'status', 'type'])

In [185... def find_tv_series(title):
    encoded_title = urllib.parse.quote(title.lower().strip())
    page_to_download = f"https://www.imdb.com/find?q={encoded_title}&s=tt&ttype=tv&ref=fn_tv"
    response = fetch_url_with_retry(page_to_download)
    return response

def extract_id_from_html(title, html_content):
    # Load and parse the HTML file
    soup = BeautifulSoup(html_content, 'html.parser')

    # Find the <script> tag with the specific ID
    script_tag = soup.find('script', {'id': '__NEXT_DATA__'})
    if script_tag:
        # Parse the JSON content within the <script> tag
        data = json.loads(script_tag.string)

        # Extract the results from the JSON structure
        results = data.get('props', {}).get('pageProps', {}).get('titleResults', {}).get('results', [])
        if results:
            for result in results:
                title_type = result.get('titleTypeText')
                # Check if the result is a TV series
                if title_type == "TV Series":
                    result_id = result.get('id')
                    # print("TV Series ID:", result_id)
                    return result_id # Stops after finding the first TV series ID
    print("No TV series found in the results for ", title)
```

```

        else:
            print("No results found for ", title)
    else:
        print("Script tag with the ID '__NEXT_DATA__' not found for ", title)
    return None

def get_series_imdb_id(title):
    imdb_html_response = find_tv_series(title)
    if imdb_html_response:
        series_id = extract_id_from_html(title, imdb_html_response.content)
        return series_id
    else:
        return None

# imdb_id_cache = {} # Initialize a cache
# cache_counter = [0] # Use a list to hold the counter so it can be updated inside the function

def get_series_imdb_id_with_cache(title):
    # Normalize the title for cache lookup
    title_key = title.lower().strip()

    # If the title is in the cache, return the cached IMDb ID
    if title_key in imdb_id_cache:
        return imdb_id_cache[title_key]

    # If not in cache, use the original function to fetch IMDb ID
    imdb_id = get_series_imdb_id(title)

    # Store the result in the cache before returning
    imdb_id_cache[title_key] = imdb_id

    # Update and check the counter for progress reporting
    cache_counter[0] += 1
    if cache_counter[0] % 10 == 0:
        print(f"Cache updated with 10 more entries, total entries: {cache_counter[0]}")

    return imdb_id

# series_id = get_series_imdb_id_with_cache("BrEaKing BaD ")
# print("series: ", series_id)

```

Normalize Column Names for Title

```

In [186... if not SKIP_TO_MODELING:

    tmdb_tv_df = tmdb_tv_df.rename(columns={'name': 'Title'})
    nielsen_ratings_df = nielsen_ratings_df.rename(columns={'Show Name': 'Title'})
    svod_ratings_df = svod_ratings_df.rename(columns={'Show': 'Title'})
    archive_ratings_df = archive_ratings_df.rename(columns={'Show': 'Title'})

```

More Cleanup

- Drop Rows without Homepage
- Drop duplicates
- Prepare Title Field with " instead of NaN

```

In [187... if not SKIP_TO_MODELING:

    # Dropping rows where the 'homepage' column is NaN or an empty string
    tmdb_tv_df.dropna(subset=['homepage'], inplace=True)
    tmdb_tv_df = tmdb_tv_df[tmdb_tv_df['homepage'].str.strip() != '']
    tmdb_tv_df.drop_duplicates(subset=['id'], inplace=True)

    # Now, update the 'Title' column for any potential NaN values to avoid errors during the IMDb ID fetching process
    tmdb_tv_df['Title'] = tmdb_tv_df['Title'].fillna('')

```

Must Have 1 Vote or More

```

In [188... if not SKIP_TO_MODELING:

    # Filter out rows where 'vote_count' is less than 1
    tmdb_tv_df = tmdb_tv_df[tmdb_tv_df['vote_count'] >= 1]

```

```

In [189... if not SKIP_TO_MODELING:

    show_df_info(tmdb_tv_df, "tv series after filtering")

```

Make a Copy of Data Set Before Adding ID

- Also, use vote_count >= 5 rather than >=1 to reduce data set.

```

In [190... if not SKIP_TO_MODELING:

    tmdb_tv_df_vt_ge_5 = tmdb_tv_df.copy()
    tmdb_tv_df_vt_ge_5 = tmdb_tv_df_vt_ge_5[tmdb_tv_df_vt_ge_5['vote_count'] >= 5]
    show_df_info(tmdb_tv_df_vt_ge_5, "tv series >= 5 after filtering")

```

```

In [191... if not SKIP_TO_MODELING:

    pretty_print_df(tmdb_tv_df_vt_ge_5, rows=50, interesting_columns = ['id', 'Title', 'number_of_seasons', 'number_of_episodes',
        'vote_count', 'vote_average', 'first_air_date', 'last_air_date', 'type'])

```


Use a Cache to Avoid Unnecessary Beating on IMDB Server.

Save Cache

- It ran for hours so I must save to temp disk file so it doesn't get lost.

Make New Cache (With Dictionary Entries)

- allows storing more than just Imdb ID.

```
In [192... def seed_new_cache(previous_cache):
    new_cache = {}
    for title_key, imdb_id in previous_cache.items():
        new_cache[title_key] = {'IMDb Series ID': imdb_id, 'Title Type': "TV Series"}
    return new_cache

if not SKIP_TO_MODELING:
    RECOVERING_CACHE = False
    if RECOVERING_CACHE:
        imdb_info_cache = seed_new_cache(imdb_id_cache)
```

```
In [193... SAVING_CACHE_BEFORE_CATASTROPHE = False

def write_cache_to_json(cache, file_path):
    with open(file_path, 'w') as file:
        json.dump(cache, file, indent=4)

def read_cache_from_json(file_path):
    with open(file_path, 'r') as file:
        cache = json.load(file)
    return cache

if not SKIP_TO_MODELING:
    if SAVING_CACHE_BEFORE_CATASTROPHE:
        write_cache_to_json(imdb_info_cache, RATINGS_DATA_DIRECTORY + '/imdb_cache_now.json')
```

Verification Function

```
In [194... def verify_cache_contents(cache):
    non_dict_entries = 0
    for key, value in cache.items():
        if not isinstance(value, dict):
            non_dict_entries += 1
        # print(f"Non-dictionary entry found: Key={key}, Value={value}")
    if non_dict_entries == 0:
        print("All entries in the cache are dictionaries.")
    else:
        print(f"There are {non_dict_entries} non-dictionary entries in the cache.")
```

New Algorithm for IMDb Page Fetch

- returns id and title type in dictionary entries
- The title type from tmdb tv is a better choice, so this column is later dropped.
- caches

```
In [195... if not SKIP_TO_MODELING:
    REBUILD_CACHE = False

    cache_counter = [0] # Use a List to hold the counter so it can be updated inside the function

    if REBUILD_CACHE:
        imdb_info_cache = {}
    else:
        imdb_info_cache = read_cache_from_json(RATINGS_DATA_DIRECTORY + '/imdb_cache.json')
        cache_counter[0] = len(imdb_info_cache)

def find_tv_series(title):
    encoded_title = urllib.parse.quote(title.lower().strip())
    page_to_download = f"https://www.imdb.com/find?q={encoded_title}&s=tt&ttype=tv&ref_=fn_tv"
    response = fetch_url_with_retry(page_to_download)
    return response

def extract_tv_title_info_from_html(title, html_content):
    soup = BeautifulSoup(html_content, 'html.parser')
    script_tag = soup.find('script', {'id': '__NEXT_DATA__'})
    if script_tag:
        data = json.loads(script_tag.string)
        results = data.get('props', {}).get('pageProps', {}).get('titleResults', {}).get('results', [])
        if results:
            return {'IMDb Series ID': results[0].get('id'), 'Title Type': results[0].get('titleTypeText')}
        else:
            print("No results found for", title)
    else:
        print("Script tag with the ID '__NEXT_DATA__' not found for", title)
    return {'IMDb Series ID': None, 'Title Type': None}

def get_series_imdb_info(title):
    imdb_html_response = find_tv_series(title)
    if imdb_html_response:
        series_info = extract_tv_title_info_from_html(title, imdb_html_response.content)
        if series_info is None:
```

```

        return {'IMDb Series ID': None, 'Title Type': None}
    return series_info
else:
    return {'IMDb Series ID': None, 'Title Type': None}

def get_series_imdb_info_with_cache(title):
    title_key = title.lower().strip()
    if title_key in imdb_info_cache:
        return imdb_info_cache[title_key]
    else:
        print("not found ", title)
        imdb_info = get_series_imdb_info(title)
        imdb_info_cache[title_key] = imdb_info
        cache_counter[0] += 1
        if cache_counter[0] % 100 == 0:
            print(f"Cache updated with 100 more entries, total entries: {cache_counter[0]}")
        return imdb_info

def apply_imdb_info(row):
    result = get_series_imdb_info_with_cache(row['Title'])
    return pd.Series([result['IMDb Series ID'], result['Title Type']])

```

Add New Columns to DF

```

In [196... if not SKIP_TO_MODELING:
    print(len(imdb_info_cache))

In [197... if not SKIP_TO_MODELING:
    GENERATE_IMDB_ID = True

    # Apply the caching function to DataFrame and create new columns
    if GENERATE_IMDB_ID:
        tmdb_tv_df_vt_ge_5[['IMDb Series ID', 'Title Type']] = tmdb_tv_df_vt_ge_5.apply(apply_imdb_info, axis=1)
    else:
        imdb_info_cache = read_cache_from_json(RATINGS_DATA_DIRECTORY + '/imdb_cache.json')
        verify_cache_contents(imdb_info_cache)

In [198... if not SKIP_TO_MODELING:
    backup_imdb_info_cache = imdb_info_cache.copy()

```

Write CSV File - Intermediate Step

```

In [199... if not SKIP_TO_MODELING:
    tmdb_tv_updated_dataset_file = FINAL_DATA_DIRECTORY + "/TMDB_tv_dataset_unique_id_main.csv"
    tmdb_tv_df_vt_ge_5.to_csv(tmdb_tv_updated_dataset_file, index=False)

```

Fix Missing IDs

```

In [200... if not SKIP_TO_MODELING:
    # Filter rows where 'IMDb Series ID' is either NaN, None, or an empty string
    missing_id_df = tmdb_tv_df_vt_ge_5[tmdb_tv_df_vt_ge_5['IMDb Series ID'].isna() | (tmdb_tv_df_vt_ge_5['IMDb Series ID'] == '')]

    # Show the DataFrame with missing 'IMDb Series ID'
    pretty_print_df(missing_id_df, rows=200, interesting_columns = ['id', 'IMDb Series ID', 'Title', 'number_of_seasons',
                                                                    'number_of_episodes', 'vote_count', 'type'])

```

Remove From Cache to Force Refetch

```

In [201... if not SKIP_TO_MODELING:
    # Loop through each title with a missing IMDb Series ID
    for title in missing_id_df['Title']:
        # Normalize the title to the format used as keys in the cache
        title_key = title.lower().strip()

        # Remove the entry from the cache if it exists
        if title_key in imdb_info_cache:
            del imdb_info_cache[title_key]
            print(f"Removed cache entry for title: {title}")

```

Check Results

```

In [202... if not SKIP_TO_MODELING:
    # Filter rows where 'IMDb Series ID' is either NaN, None, or an empty string
    missing_id_df = tmdb_tv_df_vt_ge_5[tmdb_tv_df_vt_ge_5['IMDb Series ID'].isna() | (tmdb_tv_df_vt_ge_5['IMDb Series ID'] == '')]

    # Show the DataFrame with missing 'IMDb Series ID'
    print(missing_id_df)

    # Titles of the series with missing IDs
    print(missing_id_df['Title'])

```

Analysis

Only 163 Records not found out of 35000+

Remove these rows.

```

In [203... if not SKIP_TO_MODELING:
    # Filter out rows with missing 'IMDb Series ID'

```

```
valid_id_df = tmdb_tv_df_vt_ge_5[tmdb_tv_df_vt_ge_5['IMDb Series ID'].notna() & (tmdb_tv_df_vt_ge_5['IMDb Series ID'] != '')]
```

Write Intermediate Step

```
In [204... if not SKIP_TO_MODELING:
    valid_id_df.to_csv(VALID_ID_TMDB_SERIES_MAIN_FILE, index=False)
```

We Have a Unique ID - Time to Merge It

```
In [205... if not SKIP_TO_MODELING:
    # download from Internet
    download_file(VALID_ID_TMDB_SERIES_MAIN_URL, VALID_ID_TMDB_SERIES_MAIN_FILE)
    valid_id_df = pd.read_csv(VALID_ID_TMDB_SERIES_MAIN_FILE)
```

Pre-Merge - Let's See Data Before

```
In [206... if not SKIP_TO_MODELING:
    show_df_info(valid_id_df, "Validated TV Series DataFrame Filtered from TMDB")
    headers = ['ID', 'Title', 'Seasons', 'Episodes', 'Votes', 'Average', 'First Aired', 'Last Aired', 'Status', 'IMDb Series ID', 'Type']
    pretty_print_df(valid_id_df, rows=10,
                    interesting_columns = ['id', 'Title', 'number_of_seasons', 'number_of_episodes', 'vote_count', 'vote_average',
                                          'first_air_date', 'last_air_date', 'status', 'IMDb Series ID', 'type'],
                    headers = headers)
```

```
In [207... if not SKIP_TO_MODELING:
    show_df_info(nielsen_ratings_df, "Nielsen Viewers")
    show_df_info(svod_ratings_df, "Video on Demand")
    show_df_info(archive_ratings_df, "Archive Program Ratings")
```

```
In [208... if not SKIP_TO_MODELING:
    # Drop rows where 'Title' is NaN in the program_ratings_df
    archive_ratings_df.dropna(subset=['Title'], inplace=True)

    # Optionally, verify the operation by checking for NaN values again
    print(archive_ratings_df['Title'].isna().sum()) # This should print 0, indicating no NaN values
```

```
In [209... if not SKIP_TO_MODELING:
    valid_id_copy_df = valid_id_df.copy()
    nielsen_ratings_copy_df = nielsen_ratings_df.copy()
    svod_ratings_copy_df = svod_ratings_df.copy()
    archive_ratings_copy_df = archive_ratings_df.copy()
```

Use FuzzyWuzzy on Titles to Assign Unique ID

- This only takes a minute or so.

```
In [210... def find_best_match(title, main_df):
    # Normalize the incoming title
    norm_title = title.strip().lower()

    # Exact match first
    main_df['normalized_title'] = main_df['Title'].str.lower().str.strip()
    if norm_title in main_df['normalized_title'].values:
        return main_df.loc[main_df['normalized_title'] == norm_title, 'IMDb Series ID'].iloc[0]

    # Avoid substring matches if there are fewer than two words
    if len(norm_title.split()) >= 2:
        # Substring matches: Use literal string comparison instead of regex
        try:
            matches = main_df[main_df['normalized_title'].str.contains(re.escape(norm_title), na=False, regex=True)]
            if not matches.empty:
                return matches['IMDb Series ID'].iloc[0]

            # Reverse substring match: checking if any title in main_df is a substring of the transaction title
            reverse_matches = main_df[main_df['normalized_title'].apply(lambda x: norm_title in x)]
            if not reverse_matches.empty:
                return reverse_matches['IMDb Series ID'].iloc[0]
        except re.error:
            print(f"Regex error with title: {title}")

    # Fuzzy matching as a fallback
    best_match = process.extractOne(norm_title, main_df['normalized_title'].tolist(), scorer=fuzz.ratio)
    if best_match and best_match[1] > 80: # setting a threshold of 80%
        matched_title = best_match[0]
        return main_df.loc[main_df['normalized_title'] == matched_title, 'IMDb Series ID'].iloc[0]

    return None

def apply_imdb_id_to_df(transaction_df, main_df):
    # Mapping for title to IMDb ID
    title_to_imdb_id = {title: find_best_match(title, main_df) for title in transaction_df['Title'].unique()}

    # Apply the IMDb ID to each row based on the title
    transaction_df['IMDb Series ID'] = transaction_df['Title'].map(title_to_imdb_id)

if not SKIP_TO_MODELING:
    # Exact match first
    valid_id_copy_df['normalized_title'] = valid_id_copy_df['Title'].str.lower().str.strip()

    # Apply to each transaction dataframe
    for df in [nielsen_ratings_copy_df, svod_ratings_copy_df, archive_ratings_copy_df]:
        apply_imdb_id_to_df(df, valid_id_copy_df)
```

```
valid_id_copy_df.drop('normalized_title', axis=1, inplace=True) # Cleanup after the operation
```

```
In [211... if not SKIP_TO_MODELING:
    tv_shows_main_file = FINAL_DATA_DIRECTORY + "/tv_shows_main.csv"
    tv_shows_nielsen_file = FINAL_DATA_DIRECTORY + "/tv_shows_nielsen_ratings.csv"
    tv_shows_svod_file = FINAL_DATA_DIRECTORY + "/tv_shows_svod_data.csv"
    tv_shows_archive_file = FINAL_DATA_DIRECTORY + "/tv_shows_archive_ratings.csv"

    nielsen_ratings_copy_df.to_csv(tv_shows_nielsen_file, index=False)
    svod_ratings_copy_df.to_csv(tv_shows_svod_file, index=False)
    archive_ratings_copy_df.to_csv(tv_shows_archive_file, index=False)
    valid_id_copy_df.to_csv(tv_shows_main_file, index=False)

In [212... if not SKIP_TO_MODELING:
    tv_shows_main_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_main.csv")
    tv_shows_nielsen_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_nielsen_ratings.csv")
    tv_shows_svod_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_svod_data.csv")
    tv_shows_archive_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_archive_ratings.csv")

    unique_titles = []

    def normalize_title(title):
        # Replace '&' with 'and', convert to lowercase because tmdb is inconsistent
        return title.replace('&', 'and').lower()

    # Collect unique titles from each ratings DataFrame and apply normalization
    for df in [tv_shows_nielsen_df, tv_shows_svod_df, tv_shows_archive_df]:
        normalized_titles = df['Title'].apply(normalize_title).unique()
        unique_titles.extend(normalized_titles)

    # Convert the list to a set to get unique titles
    unique_titles = set(unique_titles)

    # Normalize the titles in the main DataFrame and filter rows
    tv_shows_main_df['normalized_title'] = tv_shows_main_df['Title'].apply(normalize_title)
    tv_shows_main_df = tv_shows_main_df[tv_shows_main_df['normalized_title'].isin(unique_titles)]

    # Optionally, remove the 'normalized_title' column if no longer needed
    tv_shows_main_df.drop('normalized_title', axis=1, inplace=True)

    show_df_info(tv_shows_main_df, "Filtered TV Series Main Data")

    headers = ['ID', 'Title', 'Seasons', 'Episodes', 'Votes', 'Average', 'Genres', 'Status', 'IMDb Series ID', 'Type']
    pretty_print_df(tv_shows_main_df, rows=20,
                    interesting_columns = ['id', 'Title', 'number_of_seasons', 'number_of_episodes', 'vote_count', 'vote_average',
                                          'genres', 'status', 'IMDb Series ID', 'type'],
                    headers = headers)
```

```
In [213... if not SKIP_TO_MODELING:
    tv_shows_filtered_main_file = FINAL_DATA_DIRECTORY + "/tv_shows_filtered_main.csv"

    tv_shows_main_df.to_csv(tv_shows_filtered_main_file, index=False)
```

```
In [214... if not SKIP_TO_MODELING:
    tv_shows_filtered_main_file = FINAL_DATA_DIRECTORY + "/tv_shows_filtered_main.csv"

    tv_shows_main_df = pd.read_csv(tv_shows_filtered_main_file)
    tv_shows_nielsen_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_nielsen_ratings.csv")
    tv_shows_svod_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_svod_data.csv")
    tv_shows_archive_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_archive_ratings.csv")
```

See rating information for a title.

```
In [215... if not SKIP_TO_MODELING:
    show_df_info(tv_shows_nielsen_df, "Nielsen Data")
    show_df_info(tv_shows_svod_df, "SVOD Data")
    show_df_info(tv_shows_archive_df, "Archive Ratings Data")
    show_df_info(tv_shows_main_df, "After filtering series list")

def show_rating_info(imdb_series_id):
    nielsen_subset_df = tv_shows_nielsen_df[tv_shows_nielsen_df['IMDb Series ID'] == imdb_series_id]
    svod_subset_df = tv_shows_svod_df[tv_shows_svod_df['IMDb Series ID'] == imdb_series_id]
    archive_ratings_subset_df = tv_shows_archive_df[tv_shows_archive_df['IMDb Series ID'] == imdb_series_id]

    return nielsen_subset_df, svod_subset_df, archive_ratings_subset_df

if not SKIP_TO_MODELING:
    nielsen_subset_df, svod_subset_df, archive_ratings_subset_df = show_rating_info('tt0944947')

    show_df_info(nielsen_subset_df, "Nielsen Data")
    show_df_info(svod_subset_df, "SVOD Data")
    show_df_info(archive_ratings_subset_df, "Archive Ratings Data")

    pretty_print_df(nielsen_subset_df)
    pretty_print_df(svod_subset_df)
    pretty_print_df(archive_ratings_subset_df)
```

Potential Problem

Let's see the extent.

```
In [216... if not SKIP_TO_MODELING:
    # Group by 'IMDb Series ID' and aggregate unique titles
```

```

title_group = nielsen_subset_df.groupby('IMDb Series ID')['Title'].unique()

# Convert unique titles to a DataFrame with a count of titles per IMDb Series ID
title_count = title_group.apply(lambda titles: len(titles)).reset_index(name='Unique Title Count')

# Filter to find IMDb IDs associated with more than one unique title
multiple_titles = title_count[title_count['Unique Title Count'] > 1]

# Display the results
print(multiple_titles)

# Optionally, to see the specific titles for these problematic IDs
problematic_ids = multiple_titles['IMDb Series ID']
detailed_view = nielsen_subset_df[nielsen_subset_df['IMDb Series ID'].isin(problematic_ids)]
print(detailed_view[['IMDb Series ID', 'Title']].drop_duplicates())

```

Fortunately only one of the fuzzy wuzzy match failed!

```

In [217... if not SKIP_TO_MODELING:
# Update the 'IMDb Series ID' for rows with 'Title' exactly matching 'game on'
nielsen_ratings_copy_df.loc[nielsen_ratings_copy_df['Title'].str.lower() == 'game on', 'IMDb Series ID'] = 'tt0111976'

# Verify the update
print(nielsen_ratings_copy_df[nielsen_ratings_copy_df['Title'].str.lower() == 'game on'][['Title', 'IMDb Series ID']])

```

Fixed. Rewrite it.

```

In [218... if not SKIP_TO_MODELING:
nielsen_ratings_copy_df.to_csv(tv_shows_nielsen_file, index=False)

In [219... if not SKIP_TO_MODELING:
# Check SVOD for Problems. We still need to remove films.

# Group by 'IMDb Series ID' and aggregate unique titles
title_group = svod_subset_df.groupby('IMDb Series ID')['Title'].unique()

# Convert unique titles to a DataFrame with a count of titles per IMDb Series ID
title_count = title_group.apply(lambda titles: len(titles)).reset_index(name='Unique Title Count')

# Filter to find IMDb IDs associated with more than one unique title
multiple_titles = title_count[title_count['Unique Title Count'] > 1]

# Display the results
print(multiple_titles)

# Optionally, to see the specific titles for these problematic IDs
problematic_ids = multiple_titles['IMDb Series ID']
detailed_view = svod_subset_df[svod_subset_df['IMDb Series ID'].isin(problematic_ids)]
print(detailed_view[['IMDb Series ID', 'Title']].drop_duplicates())

```

```

In [220... if not SKIP_TO_MODELING:
# Archived Ratings?

# Group by 'IMDb Series ID' and aggregate unique titles
title_group = archive_ratings_subset_df.groupby('IMDb Series ID')['Title'].unique()

# Convert unique titles to a DataFrame with a count of titles per IMDb Series ID
title_count = title_group.apply(lambda titles: len(titles)).reset_index(name='Unique Title Count')

# Filter to find IMDb IDs associated with more than one unique title
multiple_titles = title_count[title_count['Unique Title Count'] > 1]

# Display the results
print(multiple_titles)

# Optionally, to see the specific titles for these problematic IDs
problematic_ids = multiple_titles['IMDb Series ID']
detailed_view = archive_ratings_subset_df[archive_ratings_subset_df['IMDb Series ID'].isin(problematic_ids)]
print(detailed_view[['IMDb Series ID', 'Title']].drop_duplicates())

```

All inconsistencies Resolved!

Now get more series data from IMDb.

- main imdb page: https://www.imdb.com/title/tt0903747/?ref=fn_tt_tt_1
- reviews: https://www.imdb.com/title/tt0903747/reviews/?ref=tt_ql_2
- ratings: https://www.imdb.com/title/tt0903747/ratings/?ref=tturv_ql_4
- episodes: https://www.imdb.com/title/tt0903747/episodes/?season=1&ref=ttep_ep_sn_nx
https://www.imdb.com/title/tt0903747/episodes/?season=2&ref=ttep_ep_sn_nx
https://www.imdb.com/title/tt0903747/episodes/?season=3&ref=ttep_ep_sn_nx
https://www.imdb.com/title/tt0903747/episodes/?season=4&ref=ttep_ep_sn_nx
https://www.imdb.com/title/tt0903747/episodes/?season=5&ref=ttep_ep_sn_nx
- external reviews: https://www.imdb.com/title/tt0903747/externalreviews/?ref=ttcrv_ql_op_4

```

In [221... def find_tv_series_episode_page(imdb_id, season=1):
    page_to_request = f"https://www.imdb.com/title/{imdb_id}/episodes/?season={season}&ref_=ttep_ep_sn_nx"
    response = fetch_url_with_retry(page_to_request)
    return response.content if response else None

def get_tv_series_episodes_pages(imdb_id, series_title):
    season_count = 1
    episodes_data = []

    print("Processing Series: ", series_title)

    while True:
        html_content = find_tv_series_episode_page(imdb_id, season_count)
        if not html_content:
            print(f"No content returned for {series_title} Season {season_count}")
            break

        soup = BeautifulSoup(html_content, 'html.parser')
        script_tag = soup.find('script', {'id': '__NEXT_DATA__'})
        if not script_tag:
            print(f"No relevant script tag found for {series_title} Season {season_count}")
            break # Stop if no relevant script tag is found

        data = json.loads(script_tag.string)

        # Check if each key exists in the JSON structure
        if 'props' in data and 'pageProps' in data['props'] and 'contentData' in data['props']['pageProps']:
            section = data['props']['pageProps']['contentData'].get('section')
            if section:
                season_list = section.get('seasons', [])
                episode_list = section.get('episodes', {}).get('items', [])
            else:
                print(f"No 'section' found in JSON data for {series_title} Season {season_count}")
                break
        else:
            print(f"JSON structure does not contain expected keys for {series_title} Season {season_count}")
            break

        if not episode_list:
            print(f"No episodes found for {series_title} Season {season_count}")
        else:
            for episode in episode_list:
                release_date = episode.get('releaseDate')
                if release_date:
                    formatted_release_date = "{}-{}-{}".format(release_date.get('year'), release_date.get('month'), release_date.get('day'))
                else:
                    formatted_release_date = None # Use None if release date is missing

                episodes_data.append({
                    "IMDb Series ID": imdb_id,
                    "IMDb Episode ID": episode['id'],
                    "type": episode['type'],
                    "season": season_count,
                    "episode": episode['episode'],
                    "title": episode['titleText'],
                    "releaseDate": formatted_release_date,
                    "releaseYear": episode.get('releaseYear'),
                    "rating": episode.get('aggregateRating', None),
                    "voteCount": episode.get('voteCount', 0) # Default to 0 if no votes
                })

            season_count += 1
            if season_count > len(season_list):
                break

    return pd.DataFrame(episodes_data)

if not SKIP_TO_MODELING:
    # Example use case
    imdb_id = 'tt0903747' # Example IMDb ID for Breaking Bad
    episode_df = get_tv_series_episodes_pages(imdb_id, "Breaking Bad")

    show_df_info(episode_df, "Episodes")

```

```

In [222... def concatenate_episodes(dataframes, titles):
    # Standardize columns and data types
    columns = ["IMDb Series ID", "IMDb Episode ID", "type", "season", "episode",
               "title", "releaseDate", "releaseYear", "rating", "voteCount"]
    dtypes = {
        "IMDb Series ID": "string",
        "IMDb Episode ID": "string",
        "type": "string",
        "season": "Int64", # Use nullable integer type
        "episode": "Int64", # Use nullable integer type
        "title": "string",
        "releaseDate": "string",
        "releaseYear": "Int64", # Use nullable integer type
        "rating": "float",
        "voteCount": "Int64" # Use nullable integer type
    }

    standardized_dfs = []
    for df, title in zip(dataframes, titles):
        if not df.empty:
            # Ensure the DataFrame has all the necessary columns with correct dtypes
            for col in columns:

```

```

        if col not in df.columns:
            df[col] = pd.Series(dtype=dtypes[col])
        else:
            df[col] = df[col].astype(dtypes[col])
        standardized_dfs.append(df)
    else:
        print(f"Empty or all-NA DataFrame found for series: {title}")

    if not standardized_dfs:
        return pd.DataFrame(columns=columns)

    # Concatenate all standardized DataFrames into one
    all_episodes_df = pd.concat(standardized_dfs, ignore_index=True)
    return all_episodes_df

```

Skip This Except First Time!

Episode pages are retrieved only when we have some type of ratings data: Nielsen, 18-54 demo, or SVOD.

In [223...

```

if not SKIP_TO_MODELING:
    GET_EPISODE_PAGES = False

    if GET_EPISODE_PAGES:
        # Collecting episodes data
        all_episodes = []
        series_titles = []

        for idx, row in tv_shows_main_df.iterrows():
            series_title = row['Title']
            series_imdb_id = row['IMDb Series ID']
            episode_df = get_tv_series_episodes_pages(series_imdb_id, series_title)
            all_episodes.append(episode_df)
            series_titles.append(series_title)

        # This also takes a while.
        all_episodes_df = concatenate_episodes(all_episodes, series_titles)
        all_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/episodes_before_merge_with_main_df.csv", index=False)

    elif not UPDATE_EPISODE_PAGES:
        all_episodes_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/episode_info.csv")

```

Update Episodes Option

Episode pages are retrieved only when we have some type of ratings data: Nielsen, 18-54 demo, or SVOD.

In [224...

```

def get_missing_episodes(tv_shows_df, episodes_df):
    # Create an empty DataFrame for new episodes
    new_episodes = pd.DataFrame()
    # for update list
    missing_episodes_data_frames = []
    missing_episodes_series_titles = []

    # Check each series in the main DataFrame
    for _, row in tv_shows_df.iterrows():
        series_imdb_id = row['IMDb Series ID']

        # Check if the series is already in the episodes DataFrame
        if not episodes_df[episodes_df['IMDb Series ID'] == series_imdb_id].empty:
            continue

        # If not found, fetch the episodes
        series_title = row['Title']
        series_episodes_df = get_tv_series_episodes_pages(series_imdb_id, series_title)

        # for update list
        missing_episodes_data_frames.append(series_episodes_df)
        missing_episodes_series_titles.append(series_title)

        # Append the new episodes to the new_episodes DataFrame
        new_episodes = pd.concat([new_episodes, series_episodes_df], ignore_index=True)

    return new_episodes, missing_episodes_data_frames, series_titles

if not SKIP_TO_MODELING:
    if UPDATE_EPISODE_PAGES:
        all_episodes_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/episodes_before_merge_with_main_df.csv")
        print(all_episodes_df.shape)

        # Get new episodes that are missing from all_episodes_df
        missing_episodes_df, all_missing_episodes_dataframes, missing_episodes_series_titles = get_missing_episodes(tv_shows_main_df, all_episodes_df)

        # Concatenate new episodes to the existing DataFrame
        all_episodes_df = pd.concat([all_episodes_df, missing_episodes_df], ignore_index=True)
        print(all_episodes_df.shape)
        all_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/episode_info.csv", index=False)
        # patch for next block
        all_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/episodes_before_merge_with_main_df.csv", index=False)

```

Merge Episodes with Main

- eliminate duplicates first

```
In [225... if not SKIP_TO_MODELING:
    # drop original 'type' column
    all_episodes_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/episodes_before_merge_with_main_df.csv")
    tv_shows_main_df = pd.read_csv(tv_shows_filtered_main_file)

    # Drop original 'type' column if it exists
    # 'type' in tmdb main is better.
    if 'type' in all_episodes_df.columns:
        all_episodes_df.drop('type', axis=1, inplace=True)

    # Handle duplicates
    if all_episodes_df.duplicated(subset='IMDb Episode ID').any():
        print("Duplicates found in 'IMDb Episode ID'. Handling duplicates...")
        # Keep the first occurrence
        all_episodes_df = all_episodes_df.drop_duplicates(subset='IMDb Episode ID', keep='first')

    # Check for duplicates in 'IMDb Series ID'
    if tv_shows_main_df.duplicated(subset='IMDb Series ID').any():
        print("Duplicates found in 'IMDb Series ID'. Handling duplicates...")
        # Keep the first occurrence
        tv_shows_main_df = tv_shows_main_df.drop_duplicates(subset='IMDb Series ID', keep='first')

    tv_shows_main_df.to_csv(FINAL_DATA_DIRECTORY + "/tv_shows_main_no_duplicates.csv", index=False)

    # do the same with the larger set
    valid_id_main_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_main.csv")
    if valid_id_main_df.duplicated(subset='IMDb Series ID').any():
        print("Duplicates found in 'IMDb Series ID'. Handling duplicates...")
        # Keep the first occurrence
        valid_id_main_df = valid_id_main_df.drop_duplicates(subset='IMDb Series ID', keep='first')

    valid_id_main_df.to_csv(FINAL_DATA_DIRECTORY + "/valid_id_main_no_duplicates.csv", index=False)

    # Merge while preserving the row count of 'all_episodes_df'
    merged_all_episodes_df = all_episodes_df.merge(tv_shows_main_df[['IMDb Series ID', 'type', 'status', 'genres', 'networks', 'number_of_seasons',
                                                                    'number_of_episodes', 'vote_count', 'vote_average',
                                                                    'in_production', 'popularity']],
                                                  on='IMDb Series ID',
                                                  how='left')

    show_df_info(all_episodes_df, "before merge")
    show_df_info(merged_all_episodes_df, "after merge")
```

```
In [226... if not SKIP_TO_MODELING:
    pretty_print_df(merged_all_episodes_df, rows=10)
    merged_all_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/episodes_merged_with_main_df.csv", index=False)
```

```
In [227... if not SKIP_TO_MODELING:
    all_episodes_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/episodes_merged_with_main_df.csv")

    def get_tv_series_episodes(imdb_id):
        matches_df = all_episodes_df[imdb_id==all_episodes_df['IMDb Series ID']]
        return matches_df

    if not SKIP_TO_MODELING:
        imdb_id = 'tt8772296' # Example IMDb ID for Euphoria
        episode_test_df = get_tv_series_episodes(imdb_id)

        show_df_info(episode_test_df, "Episodes")
        pretty_print_df(episode_test_df)
```

```
In [228... if not SKIP_TO_MODELING:
    all_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/episode_info.csv", index = False)
    tv_shows_main_df.to_csv(FINAL_DATA_DIRECTORY + "/tv_shows_main_no_duplicates.csv", index=False)
```

```
In [229... if not SKIP_TO_MODELING:
    all_episodes_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/episode_info.csv")
    tv_shows_main_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_main_no_duplicates.csv")
    show_df_info(all_episodes_df, "episodes")
    show_df_info(tv_shows_main_df, "series")
```

Handle Genres

```
In [230... if not SKIP_TO_MODELING:
    valid_id_main_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/valid_id_main_no_duplicates.csv")

    # List of genres
    genres_list = [
        "Sci-Fi & Fantasy", "Drama", "Action & Adventure", "Crime", "Mystery",
        "Comedy", "Animation", "Family", "War & Politics", "Soap", "Western",
        "Kids", "Documentary", "Reality", "Romance", "Adult", "Biography",
        "Film-Noir", "Game-Show", "History", "Horror", "Musical", "Music",
        "News", "Short", "Sport", "Talk-Show", "Thriller"
    ]

    if not SKIP_TO_MODELING:
        # Create boolean columns for each genre
        for genre in genres_list:
            tv_shows_main_df[genre] = tv_shows_main_df['genres'].apply(lambda x: genre in x if isinstance(x, str) else False)
```



```
all_episodes_df[genre] = all_episodes_df['genres'].apply(lambda x: genre in x if isinstance(x, str) else False)
valid_id_main_df[genre] = valid_id_main_df['genres'].apply(lambda x: genre in x if isinstance(x, str) else False)
```

Handle Networks

```
In [231... # List of networks
networks_list = [
    "HBO", "Netflix", "AMC", "FOX", "The CW", "ABC", "Disney+", "CBS",
    "Prime Video", "Adult Swim", "NBC", "Hulu", "USA Network", "Cinemax",
    "Comedy Central", "Showtime", "Paramount+", "Paramount Network", "Lifetime", "Starz",
    "Telemundo", "Apple TV+", "HBO Max", "BET", "Hallmark Channel", "Peacock"
]

if not SKIP_TO_MODELING:
    # Create boolean columns for each genre
    for network in networks_list:
        tv_shows_main_df[network] = tv_shows_main_df['networks'].apply(lambda x: network in x if isinstance(x, str) else False)
        all_episodes_df[network] = all_episodes_df['networks'].apply(lambda x: network in x if isinstance(x, str) else False)
        valid_id_main_df[network] = valid_id_main_df['networks'].apply(lambda x: genre in x if isinstance(x, str) else False)
```

```
In [232... if not SKIP_TO_MODELING:
    all_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/episode_info_with_networks_and_genres.csv", index = False)
    tv_shows_main_df.to_csv(FINAL_DATA_DIRECTORY + "/tv_shows_main_with_networks_and_genres.csv", index=False)
    valid_id_main_df.to_csv(FINAL_DATA_DIRECTORY + "/tv_shows_main_with_valid_id_networks_and_genres.csv", index=False)
```

Start Here (before merge)

The merge is only on tv_shows_main, because we have no ratings info for the others.

```
In [233... if not SKIP_TO_MODELING:
    all_episodes_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/episode_info_with_networks_and_genres.csv")
    archived_ratings_episodes_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_archive_ratings.csv")
    nielsen_ratings_episodes_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_nielsen_ratings.csv")
    svod_series_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_svod_data.csv")
    main_series_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_main_with_networks_and_genres.csv")

    show_df_info(all_episodes_df, "Episodes Data")
    show_df_info(archived_ratings_episodes_df, "Archived Ratings")
    show_df_info(nielsen_ratings_episodes_df, "Nielsen Data")
    show_df_info(svod_series_df, "SVOD Data")
    show_df_info(main_series_df, "Main Series Data")

    pretty_print_df(all_episodes_df, rows=10, interesting_columns = ['IMDb Episode ID', 'IMDb Series ID', 'title', 'type', 'voteCount', 'rating'])
    pretty_print_df(archived_ratings_episodes_df, rows=10)
    pretty_print_df(nielsen_ratings_episodes_df, rows=10)
    pretty_print_df(svod_series_df, rows=10)

    headers = ['ID', 'Title', 'Seasons', 'Episodes', 'Votes', 'Average', 'First Aired', 'Last Aired', 'Status', 'IMDb Series ID', 'Type']
    pretty_print_df(main_series_df, rows=20,
                    interesting_columns = ['id', 'Title', 'number_of_seasons', 'number_of_episodes', 'vote_count', 'vote_average', 'first_air_date',
                                          'last_air_date', 'status', 'IMDb Series ID', 'type'],
                    headers = headers)
```

Fix the 'type' field in all_episodes_df

Clean up 'Season' Field in archived_ratings_episodes_df

if season is '6A' change to '6' if season is '3A' change to '3' if season is Sand) or Nick-at-Nite or Land drop the records

```
In [234... if not SKIP_TO_MODELING:
    # Step 1: Replace '6A' with '6' and '3A' with '3'
    archived_ratings_episodes_df['Season'] = archived_ratings_episodes_df['Season'].replace({'6A': '6', '3A': '3', '5.1': '5', '5.2': '5'})

    archived_ratings_episodes_df = archived_ratings_episodes_df[archived_ratings_episodes_df['IMDb Series ID'].notna() &
                                                                archived_ratings_episodes_df['IMDb Series ID'].str.strip().ne('')]

    # Step 2: Drop rows with invalid seasons
    invalid_seasons = ['Sand)', 'Nick-at-Nite', 'Land', 'Season']
    archived_ratings_episodes_df = archived_ratings_episodes_df[~archived_ratings_episodes_df['Season'].isin(invalid_seasons)]

    # Check results
    print(archived_ratings_episodes_df['Season'].unique())
```

Clean up 'Episode' Field in nielsen_ratings_episodes_df

```
In [235... if not SKIP_TO_MODELING:
    nielsen_ratings_episodes_df['Episode'] = nielsen_ratings_episodes_df['Episode'].replace({'16.17': '16-17'})

    # Drop rows with NaN values in the 'Episode' column
    nielsen_ratings_episodes_df = nielsen_ratings_episodes_df.dropna(subset=['Episode'])

    print(nielsen_ratings_episodes_df['Episode'].unique())
```

Remove all unrated episodes.

```
In [236... if not SKIP_TO_MODELING:
    all_episodes_df = all_episodes_df[all_episodes_df['rating'].notna()]
    show_df_info(all_episodes_df, "Episodes Data")

    # Check uniqueness in all_episodes_df
    print(all_episodes_df.duplicated(['IMDb Episode ID']).sum()) # Should be 0 for unique
    print(all_episodes_df.duplicated(['IMDb Series ID', 'season', 'episode']).sum()) # Should be 0 for unique
```

Data type corrections and renaming columns

```
In [237... if not SKIP_TO_MODELING:
    all_episodes_df['releaseYear'] = all_episodes_df['releaseYear'].astype('Int64') # Convert releaseYear to integer

    archived_ratings_episodes_df['Season'] = archived_ratings_episodes_df['Season'].astype('int64') # Convert Season to int
    archived_ratings_episodes_df.rename(columns={'18-49 rating': '18-49 Demo'}, inplace=True) # Rename column for consistency
```

```
In [238... if not SKIP_TO_MODELING:
    show_df_info(archived_ratings_episodes_df, "Archived Rating Data")
```

Standardize date formats and rename columns to 'Episode Date'

```
In [239... # Step 2:
if not SKIP_TO_MODELING:
    # Replace 'None' values with '01' These are fake dates.
    all_episodes_df['releaseDate'] = all_episodes_df['releaseDate'].str.replace('None', '01')
    # Now, perform the transformation to datetime
    all_episodes_df['releaseDate'] = pd.to_datetime(all_episodes_df['releaseDate'], errors='coerce')

    show_df_info(archived_ratings_episodes_df, "archived data")

    archived_ratings_episodes_df['Episode Date'] = pd.to_datetime(archived_ratings_episodes_df['Episode Date'], errors='coerce')
    nielsen_ratings_episodes_df['Air Date'] = pd.to_datetime(nielsen_ratings_episodes_df['Air Date'], errors='coerce')

    nielsen_ratings_episodes_df.rename(columns={'Air Date': 'Episode Date'}, inplace=True)
```

Paranoid Backup Before Merge

```
In [240... if not SKIP_TO_MODELING:
    archived_ratings_episodes_copy_df = archived_ratings_episodes_df.copy()
    all_episodes_copy_df = all_episodes_df.copy()
    nielsen_ratings_episodes_copy_df = nielsen_ratings_episodes_df.copy()
```

```
In [241... # Uncomment to recover.

# archived_ratings_episodes_df = archived_ratings_episodes_copy_df.copy()
# all_episodes_df = all_episodes_copy_df.copy()
# nielsen_ratings_episodes_df = nielsen_ratings_episodes_copy_df.copy()
```

```
In [242... if not SKIP_TO_MODELING:
    show_df_info(all_episodes_df, "All Episodes Data")
    show_df_info(archived_ratings_episodes_df, "Archived Rating Episodes Data")
    show_df_info(nielsen_ratings_episodes_df, "Nielsen's Ratings Episode Data")
```

Must Check for Duplicates - Premerge!

```
In [243... if not SKIP_TO_MODELING:
    # Check uniqueness in all_episodes_df
    print(all_episodes_df.duplicated(['IMDb Episode ID']).sum()) # Should be 0 for unique
    print(all_episodes_df.duplicated(['IMDb Series ID', 'season', 'episode']).sum()) # Should be 0 for unique

    # Extract a DataFrame slice where the 'IMDb Episode ID' column has duplicated values
    duplicated_episodes_slice = all_episodes_df[all_episodes_df.duplicated(subset=['IMDb Episode ID'], keep=False)]

    # Sort the DataFrame slice by 'IMDb Episode ID'
    duplicated_episodes_slice_sorted = duplicated_episodes_slice.sort_values(by='IMDb Episode ID')
    # pretty_print_df(duplicated_episodes_slice_sorted)

    # Sort the DataFrame by 'IMDb Series ID', 'IMDb Episode ID', and 'voteCount'
    all_episodes_df_sorted = all_episodes_df.sort_values(by=['IMDb Series ID', 'IMDb Episode ID', 'voteCount'])

    # Keep the Last occurrence when 'IMDb Episode ID' is duplicated
    all_episodes_df_sorted.drop_duplicates(subset=['IMDb Episode ID'], keep='last', inplace=True)

    # Check uniqueness in all_episodes_df
    print(all_episodes_df_sorted.duplicated(['IMDb Episode ID']).sum()) # Should be 0 for unique
    print(all_episodes_df_sorted.duplicated(['IMDb Series ID', 'season', 'episode']).sum()) # Should be 0 for unique
```

```
In [244... # Function to add IMDb Episode ID based on season and episode number
def get_imdb_episode_id(df, all_episodes_df):
    df = df.merge(all_episodes_df_sorted[['IMDb Series ID', 'season', 'episode', 'IMDb Episode ID']],
                  left_on=['IMDb Series ID', 'Season', 'Episode'],
                  right_on=['IMDb Series ID', 'season', 'episode'],
                  how='left')

    return df

if not SKIP_TO_MODELING:
    # Assuming 'Episode' column is correctly created for archived_ratings_episodes_df
    archived_ratings_episodes_df['Episode'] = archived_ratings_episodes_df.groupby(['IMDb Series ID', 'Season']).cumcount() + 1

    archived_ratings_episodes_df = get_imdb_episode_id(archived_ratings_episodes_df, all_episodes_df_sorted)

    # Splitting and preparing Nielsen's data correctly
    nielsen_ratings_episodes_df[['Season', 'Episode']] = nielsen_ratings_episodes_df['Episode'].str.split('-', expand=True).astype(int)
```

```
nielsen_ratings_episodes_df = get_imdb_episode_id(nielsen_ratings_episodes_df, all_episodes_df_sorted)

# Check results
print(archived_ratings_episodes_df.head())
print(nielsen_ratings_episodes_df.head())
```

```
In [245... if not SKIP_TO_MODELING:
    show_df_info(all_episodes_df, "All Episodes Data")
    show_df_info(archived_ratings_episodes_df, "Archived Rating Episodes Data")
    show_df_info(nielsen_ratings_episodes_df, "Nielsen's Ratings Episode Data")
    show_df_info(svod_series_df, "SVOD Data")
```

```
In [246... if not SKIP_TO_MODELING:
    nielsen_ratings_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/final_nielsen_data.csv", index=False)
    archived_ratings_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/final_archived_data.csv", index=False)
```

Drop Rows without Key

- Missing episodes in episode list (very few cases)
- Episodes of shows with fewer than 5 votes in TMDB.

```
In [247... if not SKIP_TO_MODELING:
    nielsen_ratings_episodes_df = nielsen_ratings_episodes_df[nielsen_ratings_episodes_df['IMDb Episode ID'].notna()]
    archived_ratings_episodes_df = archived_ratings_episodes_df[archived_ratings_episodes_df['IMDb Episode ID'].notna()]
```

Drop Redundant Columns

```
In [248... if not SKIP_TO_MODELING:
    nielsen_ratings_episodes_df.drop(columns=['season', 'episode'], inplace=True)
    archived_ratings_episodes_df.drop(columns=['season', 'episode'], inplace=True)
```

Drop Episode Date from archived_ratings_episodes_df

It has bad data and is a copy of good data in releaseDate of all_episodes_df

```
In [249... if not SKIP_TO_MODELING:
    archived_ratings_episodes_df.drop(columns=['Episode Date'], inplace=True)
```

```
In [250... if not SKIP_TO_MODELING:
    show_df_info(all_episodes_df, "All Episodes Data")
    show_df_info(archived_ratings_episodes_df, "Archived Rating Episodes Data")
    show_df_info(nielsen_ratings_episodes_df, "Nielsen's Ratings Episode Data")

    pretty_print_df(all_episodes_df, rows=10, interesting_columns = ['IMDb Series ID', 'IMDb Episode ID', 'type', 'season', 'episode',
                                                                    'title', 'releaseDate', 'releaseYear', 'rating', 'voteCount'])
    pretty_print_df(archived_ratings_episodes_df, rows=10)
    pretty_print_df(nielsen_ratings_episodes_df, rows=10)
```

```
In [251... if not SKIP_TO_MODELING:
    # Step 4: Merge Dataframes
    # Merge columns: '18-49 Demo', 'Viewers (millions)', based on 'IMDb Episode ID'
    final_df = pd.merge(all_episodes_df, archived_ratings_episodes_df[['IMDb Episode ID', '18-49 Demo', 'Viewers (millions)']],
                        on='IMDb Episode ID', how='left')
    final_df = pd.merge(final_df, nielsen_ratings_episodes_df[['IMDb Episode ID', '18-49 Demo', 'Viewers (millions)']],
                        on='IMDb Episode ID', how='left', suffixes=('_archived', '_nielsen'))

    # Now final_df should contain all episodes with ratings data merged from both sources where available.
```

```
In [252... if not SKIP_TO_MODELING:
    show_df_info(final_df, "All Episodes Data")
```

```
In [253... if not SKIP_TO_MODELING:
    final_df.to_csv(FINAL_DATA_DIRECTORY + "/final_episode_data.csv", index = False)
```

```
In [254... if not SKIP_TO_MODELING:
    show_df_info(final_df, "All Episodes Data")
```

Get Best Values in Ratings Fields

```
In [255... if not SKIP_TO_MODELING:
    # Create '18-49 Demo' column
    final_df['18-49 Demo'] = final_df['18-49 Demo_archived'].fillna(final_df['18-49 Demo_nielsen'])

    # Create 'Viewers (millions)' column
    final_df['Viewers (millions)'] = final_df['Viewers (millions)_archived'].fillna(final_df['Viewers (millions)_nielsen'])
```

Clean up merge columns

```
In [256... if not SKIP_TO_MODELING:
    final_df.drop(columns=['18-49 Demo_archived', 'Viewers (millions)_archived', '18-49 Demo_nielsen', 'Viewers (millions)_nielsen'], inplace=True)
```

Save Merged Data

```
In [257... if not SKIP_TO_MODELING:
    final_df.to_csv(FINAL_DATA_DIRECTORY + "/merged_episode_data.csv", index = False)
```

```
In [258... if not SKIP_TO_MODELING:
    show_df_info(final_df, "Episode Info")
```

Download Already Prepared File

```
In [259... if not SKIP_TO_MODELING:
            download_file(MERGED_EPISODE_DATA_URL, MERGED_EPISODE_DATA_FILE)
```

SVOD Data

```
In [260... if not SKIP_TO_MODELING:
            main_series_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_main_with_networks_and_genres.csv")
            svod_series_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/tv_shows_svod_data.csv")

            show_df_info(svod_series_df, "SVOD Data")
```

Use Platform to find network fields.

```
In [261... platforms_list = ['DISNEY+', 'HULU/DISNEY+', 'NETFLIX', 'AMAZON', 'APPLE TV+', 'MAX/NET', 'MAX',
                    'HULU/NET', 'HULU', 'PEACOCK', 'PARAMOUNT+', 'PARA/NET', 'HULU/AMA', 'PEA/NET',
                    'HULU/PARA+', 'HBO MAX', 'AMAZON/HULU', 'PEA/HULU', 'HULU/MAX', 'AMA/NET',
                    'PARA/AMA', 'MAX/DISNEY+', 'NET/PARA+']

network_replacement_list = ['Disney+', 'Hulu, Disney+', 'Netflix', 'Prime Video', 'Apple TV+', 'HBO Max, Netflix', 'HBO Max',
                             'Hulu, Netflix', 'Hulu', 'Peacock', 'Paramount+', 'Paramount+, Netflix', 'Hulu, Prime Video', 'Peacock, Netflix',
                             'Hulu, Paramount+', 'HBO Max', 'Prime Video, Hulu', 'Peacock, Hulu', 'Hulu, HBO Max', 'Prime Video, Netflix',
                             'Paramount+, Prime Video', 'HBO Max, Disney+', 'Netflix, Paramount+']

if not SKIP_TO_MODELING:
    # Create boolean columns for each network
    # They are not quite the same so make a new field "Networks"
    # Create a mapping from platforms_list to network_replacement_list using zip.
    platform_to_network = dict(zip(platforms_list, network_replacement_list))

    # Map the 'Platform' column to the 'networks' column using the mapping
    svod_series_df['networks'] = svod_series_df['Platform'].map(platform_to_network)

    for network in networks_list:
        svod_series_df[network] = svod_series_df['Platform'].apply(
            lambda x: network in x if isinstance(x, str) else False
        )

In [262... if not SKIP_TO_MODELING:
            show_df_info(svod_series_df, "SVOD Data with networks.")
```

Find Rows with no IMDB match

```
In [263... if not SKIP_TO_MODELING:
            nan_rows_df = svod_series_df[svod_series_df[['IMDb Series ID']].isna().any(axis=1)]
            pretty_print_df(nan_rows_df, interesting_columns=['Title', 'networks', 'IMDb Series ID', 'Week Ending', 'Minutes (Millions)'])
```

Analysis

- These are films. I will drop them.

Copy SVOD Before I Start

```
In [264... if not SKIP_TO_MODELING:
            # Step 1: Drop rows where 'IMDb Series ID' is NaN. Films.
            svod_series_df.dropna(subset=['IMDb Series ID'], inplace=True)
```

Add Factors from Main.

```
In [265... if not SKIP_TO_MODELING:
            columns_to_merge = [
                'id', 'number_of_seasons', 'number_of_episodes', 'original_language',
                'vote_count', 'vote_average', 'adult', 'backdrop_path', 'first_air_date',
                'last_air_date', 'in_production', 'popularity', 'type', 'status', 'created_by',
                'languages', 'origin_country', 'spoken_languages', 'production_companies',
                'production_countries', 'episode_run_time', 'IMDb Series ID',
                'Sci-Fi & Fantasy', 'Drama', 'Action & Adventure', 'Crime', 'Mystery', 'Comedy',
                'Animation', 'Family', 'War & Politics', 'Soap', 'Western', 'Kids', 'Documentary',
                'Reality', 'Romance', 'Adult', 'Biography', 'Film-Noir', 'Game-Show', 'History',
                'Horror', 'Musical', 'Music', 'News', 'Short', 'Sport', 'Talk-Show', 'Thriller'
            ]

            # Merge the DataFrames on 'IMDb Series ID'
            merged_df = svod_series_df.merge(main_series_df[columns_to_merge], on='IMDb Series ID', how='left')

In [266... if not SKIP_TO_MODELING:
            # Now, the DataFrame svod_measures_df should have the 'IMDb Series ID' updated as needed
            show_df_info(merged_df, "SVOD Data")
            pretty_print_df(merged_df, rows=30, interesting_columns = ['Title', 'Platform', 'IMDb Series ID', 'vote_count', 'vote_average'])
```

Well, Falcon and the Winter Soldier is not correct, but Echo is there. Might be another TMDb mismatch.

Update Info from Main Series Data Frame

```
In [267... if not SKIP_TO_MODELING:
            merged_df.dropna(subset=['type'], inplace=True)
            merged_df.to_csv(FINAL_DATA_DIRECTORY + "/svod_merged_data.csv", index=False)
```

```
In [268... if not SKIP_TO_MODELING:
# Remove duplicate rows based on 'Week Ending' and 'IMDb Series ID'
svod_df = merged_df.drop_duplicates(subset=['Week Ending', 'IMDb Series ID'], keep='first').copy()

# Display the new DataFrame information to confirm duplicates are removed
show_df_info(svod_df, "SVOD Data after removing duplicates")
```

Episode Data - Ratings

Drop Rows that have no Rating Data

Save Data for Rating

```
In [269... if not SKIP_TO_MODELING:
# Filter the DataFrame to include only rows where either field has a valid value
episode_nielsen_ratings_df = final_df.copy()

episode_nielsen_ratings_df = episode_nielsen_ratings_df[(episode_nielsen_ratings_df['18-49 Demo'].notna()) |
                                                         (episode_nielsen_ratings_df['Viewers (millions)'].notna())]

episode_nielsen_ratings_df.to_csv(FINAL_DATA_DIRECTORY + "/episode_nielsen_data.csv", index=False)
```

```
In [270... if not SKIP_TO_MODELING:
show_df_info(episode_nielsen_ratings_df, "All Episodes Data")
```

Add Content Advisory from IMDB

```
In [271... def parse_imdb_content(imdb_series_id, imdb_episode_id, soup):
# Initialize data structures
certifications = {
    "Argentina": None, "Australia": None, "Brazil": None, "Finland": None, "France": None,
    "Germany": None, "Ireland": None, "Netherlands": None, "New Zealand": None,
    "Philippines": None, "Singapore": None, "United Kingdom": None, "United States": None
}
content_warnings = {
    "Sex": {"None": 0, "Mild": 0, "Moderate": 0, "Severe": 0, "Report Count": 0},
    "Violence": {"None": 0, "Mild": 0, "Moderate": 0, "Severe": 0, "Report Count": 0},
    "Profanity": {"None": 0, "Mild": 0, "Moderate": 0, "Severe": 0, "Report Count": 0},
    "Drugs": {"None": 0, "Mild": 0, "Moderate": 0, "Severe": 0, "Report Count": 0},
    "Intense": {"None": 0, "Mild": 0, "Moderate": 0, "Severe": 0, "Report Count": 0}
}
descriptions = []

# Parse certification ratings
certification_section = soup.find("tr", id="certifications-list")
if certification_section:
    for item in certification_section.find_all("li", class_="ipl-inline-list__item"):
        link = item.find("a")
        if link and ':' in link.text:
            country, rating = link.text.split(':')
            if country in certifications:
                certifications[country] = rating

# Helper function to parse advisory sections
def parse_advisory_section(section_id, content_type):
    section = soup.find("section", id=section_id)
    if section:
        # Vote parsing
        votes = section.find_all("div", class_="advisory-severity-vote__cast-vote-container")
        if votes:
            for vote in votes:
                for vote_type in ["none", "mild", "moderate", "severe"]:
                    button = vote.find("button", value=vote_type)
                    if button:
                        detail_span = button.find_next_sibling("span", class_="ipl-vote-button_details")
                        if detail_span:
                            content_warnings[content_type][vote_type.capitalize()] = int(detail_span.text.strip())

        # Report count
        reports = section.find_all("li", class_="ipl-zebra-list__item")
        content_warnings[content_type]["Report Count"] = len(reports) - 1 # Adjust for header

        # Description parsing
        for report in reports:
            if not report.find("div", class_="advisory-severity-vote"):
                text = report.text.strip()
                descriptions.append([imdb_series_id, imdb_episode_id, content_type[0], text])

# Parse each advisory section
parse_advisory_section("advisory-nudity", "Sex")
parse_advisory_section("advisory-violence", "Violence")
parse_advisory_section("advisory-profanity", "Profanity")
parse_advisory_section("advisory-alcohol", "Drugs")
parse_advisory_section("advisory-frightening", "Intense")

# Build the data frames
warning_data = {
    "IMDb Series ID": imdb_series_id, "IMDb Episode ID": imdb_episode_id,
    **certifications,
    **{f"{key} - {subkey} Votes": value for key, values in content_warnings.items() for subkey, value in values.items() if "Count" not in subkey},
    **{f"{key} - Report Count": values["Report Count"] for key, values in content_warnings.items()}
}
warnings_df = pd.DataFrame([warning_data])
descriptions_df = pd.DataFrame(descriptions, columns=["IMDb Series ID", "IMDb Episode ID", "Type", "Text"])
```

```
return warnings_df, descriptions_df
```

Sample Data - Euphoria

- | tt8772296 | tt8135530 | tvEpisode | 1 | 1 | Pilot | 2019-6-16 | 2019 | 8.1 | 8848 |
- | tt8772296 | tt8806264 | tvEpisode | 1 | 2 | Stuntin' Like My Daddy | 2019-6-23 | 2019 | 8.1 | 7202 |
- | tt8772296 | tt8806262 | tvEpisode | 1 | 3 | Made You Look | 2019-6-30 | 2019 | 8.3 | 6845 |
- | tt8772296 | tt8806266 | tvEpisode | 1 | 4 | Shook Ones Pt II | 2019-7-7 | 2019 | 9.0 | 8325 |
- | tt8772296 | tt8806268 | tvEpisode | 1 | 5 | '03 Bonnie and Clyde | 2019-7-14 | 2019 | 8.0 | 6194 |
- | tt8772296 | tt8806274 | tvEpisode | 1 | 6 | The Next Episode | 2019-7-21 | 2019 | 8.1 | 6170 |
- | tt8772296 | tt8806270 | tvEpisode | 1 | 7 | The Trials and Tribulations of Trying to Pee While Depressed | 2019-7-28 | 2019 | 8.5 | 6522 |
- | tt8772296 | tt8806272 | tvEpisode | 1 | 8 | And Salt the Earth Behind You | 2019-8-4 | 2019 | 8.2 | 7105 |

Get Content from IMDB

- This will take 12 hours or so. **Set the flag to False.**

```
In [272... # CONTENT_LOCAL_BACKUP = "/path-to-external-drive-with-30gb"

def getContentGuidePage(episode_id):
    return f"https://www.imdb.com/title/{episode_id}/parentalguide/?ref=tt_q1_stry_4"

def getEpisodeContentPage(row):
    series_id = row['IMDb Series ID']
    episode_id = row['IMDb Episode ID']
    url = getContentGuidePage(episode_id)
    html_output_filename = f"{CONTENT_LOCAL_BACKUP}/content_{series_id}_{episode_id}.html"
    download_file(url, html_output_filename)
    print(f"Downloaded and saved HTML for Series ID {series_id} and Episode ID {episode_id}")

if not SKIP_TO_MODELING:
    BUILD_CONTENT_FROM_SCRAPING = False
    if BUILD_CONTENT_FROM_SCRAPING:
        episodes_info_df = pd.read_csv(MERGED_EPISODE_DATA_FILE)
        show_df_info(episodes_info_df, "episodes info")
        # get content advisory pages
        episodes_info_df.apply(getEpisodeContentPage, axis=1)
```

Webscrape Content Advisory

- This will take two hours. Set the flag to False.

```
In [273... if not SKIP_TO_MODELING:
    show_df_info(episode_content_df, "episode content warnings")

In [274... ## external drive - 50gb download
# CONTENT_LOCAL_BACKUP = "/Volumes/Tegebba/content"
if not SKIP_TO_MODELING:
    BUILD_CONTENT_FROM_SCRAPING = False

    if BUILD_CONTENT_FROM_SCRAPING:

        def initialize_content_data_frames(episode_content_columns):
            episode_content_df = pd.DataFrame(columns=episode_content_columns)

            episode_content_details_columns = ["IMDb Series ID", "IMDb Episode ID", "Type", "Text"]
            episode_content_details_df = pd.DataFrame(columns=episode_content_details_columns)

            return episode_content_df, episode_content_details_df

        episode_content_columns = [
            "IMDb Series ID", "IMDb Episode ID", "Argentina", "Australia", "Brazil",
            "Finland", "France", "Germany", "Ireland", "Netherlands", "New Zealand",
            "Philippines", "Singapore", "United Kingdom", "United States",
            "Sex - None Votes", "Sex - Mild Votes", "Sex - Moderate Votes", "Sex - Severe Votes",
            "Violence - None Votes", "Violence - Mild Votes", "Violence - Moderate Votes", "Violence - Severe Votes",
            "Profanity - None Votes", "Profanity - Mild Votes", "Profanity - Moderate Votes", "Profanity - Severe Votes",
            "Drugs - None Votes", "Drugs - Mild Votes", "Drugs - Moderate Votes", "Drugs - Severe Votes",
            "Intense - None Votes", "Intense - Mild Votes", "Intense - Moderate Votes", "Intense - Severe Votes",
            "Sex - Report Count", "Violence - Report Count", "Profanity - Report Count",
            "Drugs - Report Count", "Intense - Report Count"
        ]
        # Initialize with 'N/A' for ratings, 0 for countss, and advisory details
        default_content_data = {col: 'N/A' for col in episode_content_columns if not col.endswith("Votes") and not col.endswith("Count")}
        default_content_data.update({col: 0 for col in episode_content_columns if col.endswith("Votes") or col.endswith("Count")})

        episode_content_df, episode_content_details_df = initialize_content_data_frames(episode_content_columns)

        for content_html_filename in os.listdir(CONTENT_LOCAL_BACKUP):
            if content_html_filename.endswith('.html'):
                _, series_id, episode_id = content_html_filename.replace('.html', '').split("_")
                # print(f"Processing episode: {episode_id} of series: {series_id}")
                with open(os.path.join(CONTENT_LOCAL_BACKUP, content_html_filename), 'r', encoding='utf-8') as content_html_file:
                    html_content = content_html_file.read()

                # Parse HTML content
                soup = BeautifulSoup(html_content, 'html.parser')
                warnings_df, descriptions_df = parse_imdb_content(series_id, episode_id, soup)

                # Fill NaNs for missing certification ratings
                warnings_df = warnings_df.fillna(value=default_content_data)
```

```

# Append the parsed data to the respective data frames
episode_content_df = pd.concat([episode_content_df, warnings_df], ignore_index=True)
episode_content_details_df = pd.concat([episode_content_details_df, descriptions_df], ignore_index=True)

# Optional: Save periodically if memory constraints are a concern
if len(episode_content_df) % 100 == 0:
    print(f"Processing record {len(episode_content_df)}")
    episode_content_df.to_csv(CONTENT_LOCAL_BACKUP + "/temp_episode_content.csv", index=False)
    episode_content_details_df.to_csv(CONTENT_LOCAL_BACKUP + "/temp_episode_content_details.csv", index=False)

# Optionally save to CSV
episode_content_df.to_csv(CONTENT_LOCAL_BACKUP + "/episode_content.csv", index=False)
episode_content_details_df.to_csv(CONTENT_LOCAL_BACKUP + "/episode_content_details.csv", index=False)
else:
    episode_content_df = pd.read_csv(CONTENT_DATA_DIRECTORY + "/episode_content.csv")
    episode_content_details_df = pd.read_csv(CONTENT_DATA_DIRECTORY + "/episode_content_details.csv")

```

Update Advisory (faster)

```

In [275... if not SKIP_TO_MODELING:
    UPDATE_ADVISORY_CONTENT = False
    if UPDATE_ADVISORY_CONTENT:
        episode_content_df = pd.read_csv(CONTENT_LOCAL_BACKUP + "/episode_content.csv")
        episode_content_details_df = pd.read_csv(CONTENT_LOCAL_BACKUP + "/episode_content_details.csv")

        for content_html_filename in os.listdir(CONTENT_LOCAL_BACKUP):
            if content_html_filename.endswith('.html'):
                _, series_id, episode_id = content_html_filename.replace('.html', '').split("_")
                match = episode_content_df[episode_content_df['IMDB Episode ID'] == episode_id]

                if match.empty:
                    try:
                        with open(os.path.join(CONTENT_LOCAL_BACKUP, content_html_filename), 'r', encoding='utf-8') as content_html_file:
                            html_content = content_html_file.read()
                    except Exception as e:
                        print(f"Failed to read {content_html_filename}: {e}")
                        continue

                    soup = BeautifulSoup(html_content, 'html.parser')
                    warnings_df, descriptions_df = parse_imdb_content(series_id, episode_id, soup)

                    warnings_df = warnings_df.fillna(value=default_content_data)

                    episode_content_df = pd.concat([episode_content_df, warnings_df], ignore_index=True)
                    episode_content_details_df = pd.concat([episode_content_details_df, descriptions_df], ignore_index=True)

                    if len(episode_content_df) % 100 == 0:
                        print(f"Processing record {len(episode_content_df)}")
                        episode_content_df.to_csv(CONTENT_LOCAL_BACKUP + "/temp_episode_content.csv", index=False)
                        episode_content_details_df.to_csv(CONTENT_LOCAL_BACKUP + "/temp_episode_content_details.csv", index=False)

                    episode_content_df.to_csv(CONTENT_LOCAL_BACKUP + "/episode_content.csv", index=False)
                    episode_content_details_df.to_csv(CONTENT_LOCAL_BACKUP + "/episode_content_details.csv", index=False)

```

Merge Content Advisories with Episode Info

```

In [276... if not SKIP_TO_MODELING:
    episode_content_df = pd.read_csv(CONTENT_DATA_DIRECTORY + "/episode_content.csv")
    episode_content_details_df = pd.read_csv(CONTENT_DATA_DIRECTORY + "/episode_content_details.csv")
    # (merged_episode_data) = This is before dropping episodes without ratings.
    merged_episode_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/merged_episode_data.csv")

```

```

In [277... if not SKIP_TO_MODELING:
    episode_content_df = episode_content_df.fillna("N/A")

```

```

In [278... if not SKIP_TO_MODELING:
    show_df_info(episode_content_df, "Content Warning Info")
    show_df_info(episode_content_details_df, "Content Warning Descriptions")
    show_df_info(merged_episode_df, "Episode Info")

```

```

In [279... if not SKIP_TO_MODELING:
    # Renaming columns
    episode_content_df = episode_content_df.rename(columns={
        "IMDB Episode ID": "IMDb Episode ID",
        "IMDB Series ID": "IMDb Series ID"
    })

    # Merge Data Frame
    all_episodes_df = pd.merge(merged_episode_df, episode_content_df, on="IMDb Episode ID", how="left")

    duplicate_columns = ['IMDb Series ID_y']
    # Drop the duplicate columns
    all_episodes_df = all_episodes_df.drop(columns=duplicate_columns)

    all_episodes_df = all_episodes_df.rename(columns={
        "IMDb Series ID_x": "IMDb Series ID"
    })

    show_df_info(all_episodes_df, "Episodes Info with Content")

```

Fix the Weird NaN that happened in merge.

```
In [280... if not SKIP_TO_MODELING:
# Find rows where 'IMDb Episode Rating' is NaN
nan_rows = all_episodes_df[all_episodes_df['Argentina'].isna()]

# Print these rows to inspect them
print(nan_rows)
```

```
In [281... if not SKIP_TO_MODELING:
all_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/episode_data_with_content_and_ratings.csv", index = False)
```

```
In [282... if not SKIP_TO_MODELING:
show_df_info(all_episodes_df, "all episodes with content")
```

Perfect.

Use Exponential Numeric Factors to Describe Content

```
In [284... if not SKIP_TO_MODELING:
# Step 1: Create content intensity factors
all_episodes_df['sex_factor'] = 64 * all_episodes_df['Sex - Severe Votes'] + 8 * all_episodes_df['Sex - Moderate Votes'] + \
all_episodes_df['Sex - Mild Votes']
all_episodes_df['violence_factor'] = 64 * all_episodes_df['Violence - Severe Votes'] + 8 * all_episodes_df['Violence - Moderate Votes'] + \
all_episodes_df['Violence - Mild Votes']
all_episodes_df['profanity_factor'] = 64 * all_episodes_df['Profanity - Severe Votes'] + 8 * all_episodes_df['Profanity - Moderate Votes'] + \
all_episodes_df['Profanity - Mild Votes']
all_episodes_df['drugs_factor'] = 64 * all_episodes_df['Drugs - Severe Votes'] + 8 * all_episodes_df['Drugs - Moderate Votes'] + \
all_episodes_df['Drugs - Mild Votes']
all_episodes_df['intense_factor'] = 64 * all_episodes_df['Intense - Severe Votes'] + 8 * all_episodes_df['Intense - Moderate Votes'] + \
all_episodes_df['Intense - Mild Votes']
```

Add rating info from IMDB

```
In [285... if not SKIP_TO_MODELING:
subset_all_episodes_df = all_episodes_df.copy()
```

```
In [286... if not SKIP_TO_MODELING:
all_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/backup_episode_data.csv", index = False)
```



```
In [287... # Define data types for each column
episode_dtype_dict = {
    "IMDb Series ID": str,
    "IMDb Episode ID": str,
    "type": str,
    "season": int,
    "episode": int,
    "title": str,
    "releaseDate": str,
    "releaseYear": int,
    "rating": float,
    "voteCount": int,
    "18-49 Demo": str,
    "Viewers (millions)": float,
    "Argentina": str,
    "Australia": str,
    "Brazil": str,
    "Finland": str,
    "France": str,
    "Germany": str,
    "Ireland": str,
    "Netherlands": str,
    "New Zealand": str,
    "Philippines": str,
    "Singapore": str,
    "United Kingdom": str,
    "United States": str,
    "Sex - None Votes": int,
    "Sex - Mild Votes": int,
    "Sex - Moderate Votes": int,
    "Sex - Severe Votes": int,
    "Violence - None Votes": int,
    "Violence - Mild Votes": int,
    "Violence - Moderate Votes": int,
    "Violence - Severe Votes": int,
    "Profanity - None Votes": int,
    "Profanity - Mild Votes": int,
    "Profanity - Moderate Votes": int,
    "Profanity - Severe Votes": int,
    "Drugs - None Votes": int,
    "Drugs - Mild Votes": int,
    "Drugs - Moderate Votes": int,
    "Drugs - Severe Votes": int,
    "Intense - None Votes": int,
    "Intense - Mild Votes": int,
    "Intense - Moderate Votes": int,
    "Intense - Severe Votes": int,
    "Sex - Report Count": int,
    "Violence - Report Count": int,
    "Profanity - Report Count": int,
    "Drugs - Report Count": int,
    "Intense - Report Count": int
}

if not SKIP_TO_MODELING:
    all_episodes_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/backup_episode_data.csv", dtype=episode_dtype_dict)
```

Add IMDB Episode Ratings.

```
In [288... def getEpisodeRatingPage(imdb_episode_id):
    return f"https://www.imdb.com/title/{imdb_episode_id}/ratings/?ref_=tt_ov_rt"

def request_tv_episode_rating_page(imdb_episode_id):
    page_to_request = getEpisodeRatingPage(imdb_episode_id)
    response = fetch_url_with_retry(page_to_request)
    return response.content if response else None
```

```
In [289... def convert_votes(votes):
    if 'K' in votes:
        # Remove the 'K' and convert to float to handle decimals correctly
        number = float(votes.replace('K', ''))
        # Multiply by 1000 to convert K to thousand
        return int(number * 1000)
    return int(votes) # If there's no 'K', convert directly to integer
```

```
In [290... def save_to_json(data, filepath):
    """Append a dictionary to a JSON file."""
    with open(filepath, 'a') as f: # Open in append mode
        json.dump(data, f)
        f.write('\n') # Write a new line for each entry

def read_processed_data(filepath):
    """Read processed data from a JSON file and return as a dictionary."""
    processed_data = {}
    try:
        with open(filepath, 'r') as f:
            for line in f:
                entry = json.loads(line.strip())
                processed_data[entry['imdb_episode_id']] = entry
    except FileNotFoundError:
        pass # If the file doesn't exist, we start fresh
    return processed_data

def extract_imdb_episode_ratings(row, processed_data, filepath):
    imdb_episode_id = row['IMDb Episode ID']
    if imdb_episode_id in processed_data:
```

```

# Retrieve saved data from the dictionary
data = processed_data[imdb_episode_id]
# print(f'Skipping already processed ID: {imdb_episode_id}')
return pd.Series([data['rating'], data['votes']])

season = row['season']
episode = row['episode']
series_id = row['IMDb Series ID']

# otherwise read from internet IMDb.
print(f'Processing season {season} episode {episode} of {series_id}')

html_content = request_tv_episode_rating_page(imdb_episode_id)
if html_content:
    soup = BeautifulSoup(html_content, 'html.parser')
    rating_div = soup.find('div', {'data-testid': 'rating-button__aggregate-rating__score'})
    if rating_div:
        rating = rating_div.text.strip()
        votes_div = rating_div.find_next_sibling()
        if votes_div:
            votes = votes_div.text.strip()
            converted_votes = convert_votes(votes)
            result = {'imdb_episode_id': imdb_episode_id, 'series_id': row['IMDb Series ID'],
                    'rating': rating, 'votes': converted_votes}
            save_to_json(result, filepath)
            return pd.Series([rating, converted_votes])
    return pd.Series([None, None])

def add_ratings_fields(df, filepath):
    processed_data = read_processed_data(filepath)
    results = df.apply(extract_imdb_episode_ratings, args=(processed_data, filepath), axis=1)
    df[['IMDb Episode Rating', 'IMDb Episode Votes']] = results
    return df

if not SKIP_TO_MODELING:
    final_data_directory = FINAL_DATA_DIRECTORY
    json_filepath = final_data_directory + '/intermediate_results.json'

    updated_df = add_ratings_fields(all_episodes_df, json_filepath)
    updated_df.to_csv(FINAL_DATA_DIRECTORY + "/updated_episode_data.csv", index=False)

# updated_df = pd.read_csv(FINAL_DATA_DIRECTORY + "/updated_episode_data.csv", dtype=episode_dtype_dict)

```

```

In [291...] if not SKIP_TO_MODELING:
    # Find rows where 'IMDb Episode Rating' is NaN
    nan_rows = updated_df[updated_df['IMDb Episode Rating'].isna()]

    # Print these rows to inspect them
    print(nan_rows)

    # Dropping rows where there are NaNs in the '18-49 Demo' and 'Viewers (millions)' columns
    updated_df.dropna(subset=['IMDb Episode Rating'], inplace=True)

```

```

In [292...] # Function to clean and convert the '18-49 Demo' field
def clean_ratings(value):
    value = value.split('/')[0]
    return pd.to_numeric(value, errors='coerce')

if not SKIP_TO_MODELING:
    # Apply the function to the '18-49 Demo' column
    updated_df['IMDb Episode Rating'] = updated_df['IMDb Episode Rating'].apply(clean_ratings)

```

```

In [293...] if not SKIP_TO_MODELING:
    updated_df.to_csv(FINAL_DATA_DIRECTORY + "/episode_data_with_imdb_ratings_too.csv", index=False)
    show_df_info(updated_df, "all episodes with content")

```

Prepare for Modeling

- start with these data sets.

```

In [294...] # SKIP_TO_MODELING = True
if SKIP_TO_MODELING:
    download_file(FILTERED_MAIN_SERIES_FOR_MODELING_URL, FILTERED_MAIN_SERIES_FOR_MODELING_DATA_FILE)
    download_file(UNFILTERED_MAIN_SERIES_FOR_MODELING_URL, UNFILTERED_MAIN_SERIES_FOR_MODELING_DATA_FILE)
    download_file(SVOD_DATA_FOR_MODELING_URL, SVOD_DATA_FOR_MODELING_DATA_FILE)
    download_file(EPISEODE_NIELSEN_FOR_MODELING_URL, EPISEODE_NIELSEN_FOR_MODELING_DATA_FILE)
    download_file(EPISEODES_FOR_MODELING_URL, EPISEODES_FOR_MODELING_DATA_FILE)

    FILTERED_MAIN_SERIES_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/tv_shows_main_with_networks_and_genres.csv"
    UNFILTERED_MAIN_SERIES_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/tv_shows_main.csv"
    SVOD_DATA_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/svod_merged_data.csv"
    EPISEODE_NIELSEN_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/episode_nielsen_data.csv"
    EPISEODES_FOR_MODELING_DATA_FILE = FINAL_DATA_DIRECTORY + "/episode_data_with_imdb_ratings_too.csv"

    Already downloaded final/data/tv_shows_main_with_networks_and_genres.csv
    Already downloaded final/data/tv_shows_main.csv
    Already downloaded final/data/svod_merged_data.csv
    Already downloaded final/data/episode_nielsen_data.csv
    Already downloaded final/data/episode_data_with_imdb_ratings_too.csv

```

```

In [295...] filtered_main_series_df = pd.read_csv(FILTERED_MAIN_SERIES_FOR_MODELING_DATA_FILE)
wider_main_series_df = pd.read_csv(UNFILTERED_MAIN_SERIES_FOR_MODELING_DATA_FILE)
svod_df = pd.read_csv(SVOD_DATA_FOR_MODELING_DATA_FILE)
episode_nielsen_ratings_df = pd.read_csv(EPISEODE_NIELSEN_FOR_MODELING_DATA_FILE)
episodes_with_imdb_ratings_df = pd.read_csv(EPISEODES_FOR_MODELING_DATA_FILE, dtype=episode_dtype_dict)

```

```
In [296... show_df_info(epsisodes_with_imdb_ratings_df, "episode info with imdb ratings")
```

Dataframe information for episode info with imdb ratings:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	title	object	0
5	releaseDate	object	0
6	releaseYear	int32	0
7	rating	float64	0
8	voteCount	int32	0
9	type	object	0
10	status	object	0
11	genres	object	109
12	networks	object	0
13	number_of_seasons	float64	0
14	number_of_episodes	float64	0
15	vote_count	float64	0
16	vote_average	float64	0
17	in_production	bool	0
18	popularity	float64	0
19	Sci-Fi & Fantasy	bool	0
20	Drama	bool	0
21	Action & Adventure	bool	0
22	Crime	bool	0
23	Mystery	bool	0
24	Comedy	bool	0
25	Animation	bool	0
26	Family	bool	0
27	War & Politics	bool	0
28	Soap	bool	0
29	Western	bool	0
30	Kids	bool	0
31	Documentary	bool	0
32	Reality	bool	0
33	Romance	bool	0
34	Adult	bool	0
35	Biography	bool	0
36	Film-Noir	bool	0
37	Game-Show	bool	0
38	History	bool	0
39	Horror	bool	0
40	Musical	bool	0
41	Music	bool	0
42	News	bool	0
43	Short	bool	0
44	Sport	bool	0
45	Talk-Show	bool	0
46	Thriller	bool	0
47	HBO	bool	0
48	Netflix	bool	0
49	AMC	bool	0
50	FOX	bool	0
51	The CW	bool	0
52	ABC	bool	0
53	Disney+	bool	0
54	CBS	bool	0
55	Prime Video	bool	0
56	Adult Swim	bool	0
57	NBC	bool	0
58	Hulu	bool	0
59	USA Network	bool	0
60	Cinemax	bool	0
61	Comedy Central	bool	0
62	Showtime	bool	0
63	Paramount+	bool	0
64	Paramount Network	bool	0
65	Lifetime	bool	0
66	Starz	bool	0
67	Telemundo	bool	0
68	Apple TV+	bool	0
69	HBO Max	bool	0
70	BET	bool	0
71	Hallmark Channel	bool	0
72	Peacock	bool	0
73	18-49 Demo	object	28731
74	Viewers (millions)	float64	28731
75	Argentina	object	48126
76	Australia	object	44442
77	Brazil	object	49437
78	Finland	object	51519
79	France	object	53860
80	Germany	object	47701
81	Ireland	object	53144
82	Netherlands	object	44788
83	New Zealand	object	52962
84	Philippines	object	49986
85	Singapore	object	53584
86	United Kingdom	object	42022
87	United States	object	6327
88	Sex - None Votes	int32	0
89	Sex - Mild Votes	int32	0
90	Sex - Moderate Votes	int32	0
91	Sex - Severe Votes	int32	0

92	Violence - None Votes	int32	0
93	Violence - Mild Votes	int32	0
94	Violence - Moderate Votes	int32	0
95	Violence - Severe Votes	int32	0
96	Profanity - None Votes	int32	0
97	Profanity - Mild Votes	int32	0
98	Profanity - Moderate Votes	int32	0
99	Profanity - Severe Votes	int32	0
100	Drugs - None Votes	int32	0
101	Drugs - Mild Votes	int32	0
102	Drugs - Moderate Votes	int32	0
103	Drugs - Severe Votes	int32	0
104	Intense - None Votes	int32	0
105	Intense - Mild Votes	int32	0
106	Intense - Moderate Votes	int32	0
107	Intense - Severe Votes	int32	0
108	Sex - Report Count	int32	0
109	Violence - Report Count	int32	0
110	Profanity - Report Count	int32	0
111	Drugs - Report Count	int32	0
112	Intense - Report Count	int32	0
113	sex_factor	int64	0
114	violence_factor	int64	0
115	profanity_factor	int64	0
116	drugs_factor	int64	0
117	intense_factor	int64	0
118	IMDb Episode Rating	float64	0
119	IMDb Episode Votes	int64	0

There are 54573 rows and 120 columns of information.

```
In [297...] subset_all_episodes_df = episodes_with_imdb_ratings_df.copy()
```

```
In [298...] # Step 2: Drop specified columns
columns_to_drop = ['title', 'releaseDate', 'releaseYear',
                  'Argentina', 'Australia', 'Brazil', 'Finland', 'France', 'Germany',
                  'Ireland', 'Netherlands', 'New Zealand', 'Philippines', 'Singapore', 'United Kingdom',
                  'United States', 'genres', 'networks']
subset_all_episodes_df.drop(columns=columns_to_drop, inplace=True)

# Step 3: Set up dummy variables for United States and type
# This didn't help at all.
# subset_all_episodes_df = pd.get_dummies(subset_all_episodes_df, columns=['United States', 'type'])

# Step 4: Drop all fields related to voting specifics
columns_to_drop_after = [col for col in subset_all_episodes_df.columns if col.endswith('None Votes') or col.endswith('Moderate Votes') or
                        col.endswith('Severe Votes') or col.endswith('Mild Votes') or col.endswith('Report Count')]
subset_all_episodes_df.drop(columns=columns_to_drop_after, inplace=True)

# Step 5: Handle Status and type
status_dummies = pd.get_dummies(subset_all_episodes_df['status'], prefix='status')
type_dummies = pd.get_dummies(subset_all_episodes_df['type'], prefix='type')

# Concatenate the dummy variables with the original DataFrame
subset_all_episodes_df = pd.concat([subset_all_episodes_df, status_dummies], axis=1)
subset_all_episodes_df = pd.concat([subset_all_episodes_df, type_dummies], axis=1)

# drop status variable
subset_all_episodes_df.drop('status', axis=1, inplace=True)
subset_all_episodes_df.drop('type', axis=1, inplace=True)

# Rename some columns to make formula easier
subset_all_episodes_df = subset_all_episodes_df.rename(columns={
    "IMDb Episode Rating": "IMDb_Episode_Rating",
    "IMDb Episode Votes": "IMDb_Episode_Votes",
    "Viewers (millions)": "Viewers_millions",
    "18-49 Demo": "_18_to_49_Demo",
    "voteCount": "Episode_Votes",
    "vote_count": "Series_Votes"
})
```

Fuzzify fuzzy series data

- Note we don't actually use Series_Votes in the model. Even in 4 pockets it is still too dominating.

```
In [299...] def fuzzify_series(episodes_df, fuzziness = 300):
    series_count = len(episodes_df['IMDb Series ID'].unique())
    # print(f"Total Number of Series Rated: {series_count}")
    # fuzziness is how many series are in each pocket
    pocket_count = np.floor(series_count / fuzziness) + 1
    print(f"Ranking {series_count} series in {pocket_count} classification pockets.")
    # rank series specific columns
    episodes_df['ranking_of_Series_Votes'] = episodes_df['Series_Votes'].rank(method='min')
    episodes_df['ranking_of_vote_average'] = episodes_df['vote_average'].rank(method='min')
    episodes_df['ranking_of_popularity'] = episodes_df['popularity'].rank(method='min')
    # fuzzify the series data - note, later we drop fuzzy_Series_Votes, because it is still too dominating.
    episodes_df['fuzzy_Series_Votes'] = ((episodes_df['ranking_of_Series_Votes']-1)/(episodes_df.shape[0]-1)*pocket_count).apply(np.floor)
    episodes_df['fuzzy_vote_average'] = ((episodes_df['ranking_of_vote_average']-1)/(episodes_df.shape[0]-1)*pocket_count).apply(np.floor)
    episodes_df['fuzzy_popularity'] = ((episodes_df['ranking_of_popularity']-1)/(episodes_df.shape[0]-1)*pocket_count).apply(np.floor)
    return episodes_df
```

Fuzzify Series Data so it doesn't dominate stats.

```
In [300... subset_all_episodes_df = fuzzify_series(subset_all_episodes_df)
```

Ranking 1153 series in 4.0 classification pockets.

```
In [301... # Function to clean and convert the '18-49 Demo' field
def clean_demo(value):
    # Check for values in parentheses and extract them
    match = re.search(r'\((\d+\.\d+)\)', str(value))
    if match:
        return float(match.group(1))
    elif '/' in str(value):
        return float(value.split('/')[0])
    else:
        return pd.to_numeric(value, errors='coerce')

# Apply the function to the '18-49 Demo' column
subset_all_episodes_df['_18_to_49_Demo'] = subset_all_episodes_df['_18_to_49_Demo'].apply(clean_demo)
```

```
In [302... show_df_info(subset_all_episodes_df, "all episodes with content")
```

Dataframe information for all episodes with content:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	rating	float64	0
5	Episode_Votes	int32	0
6	number_of_seasons	float64	0
7	number_of_episodes	float64	0
8	Series_Votes	float64	0
9	vote_average	float64	0
10	in_production	bool	0
11	popularity	float64	0
12	Sci-Fi & Fantasy	bool	0
13	Drama	bool	0
14	Action & Adventure	bool	0
15	Crime	bool	0
16	Mystery	bool	0
17	Comedy	bool	0
18	Animation	bool	0
19	Family	bool	0
20	War & Politics	bool	0
21	Soap	bool	0
22	Western	bool	0
23	Kids	bool	0
24	Documentary	bool	0
25	Reality	bool	0
26	Romance	bool	0
27	Adult	bool	0
28	Biography	bool	0
29	Film-Noir	bool	0
30	Game-Show	bool	0
31	History	bool	0
32	Horror	bool	0
33	Musical	bool	0
34	Music	bool	0
35	News	bool	0
36	Short	bool	0
37	Sport	bool	0
38	Talk-Show	bool	0
39	Thriller	bool	0
40	HBO	bool	0
41	Netflix	bool	0
42	AMC	bool	0
43	FOX	bool	0
44	The CW	bool	0
45	ABC	bool	0
46	Disney+	bool	0
47	CBS	bool	0
48	Prime Video	bool	0
49	Adult Swim	bool	0
50	NBC	bool	0
51	Hulu	bool	0
52	USA Network	bool	0
53	Cinemax	bool	0
54	Comedy Central	bool	0
55	Showtime	bool	0
56	Paramount+	bool	0
57	Paramount Network	bool	0
58	Lifetime	bool	0
59	Starz	bool	0
60	Telemundo	bool	0
61	Apple TV+	bool	0
62	HBO Max	bool	0
63	BET	bool	0
64	Hallmark Channel	bool	0
65	Peacock	bool	0
66	_18_to_49_Demo	float64	28835
67	Viewers_millions	float64	28731
68	sex_factor	int64	0
69	violence_factor	int64	0
70	profanity_factor	int64	0
71	drugs_factor	int64	0
72	intense_factor	int64	0
73	IMDb_Episode_Rating	float64	0
74	IMDb_Episode_Votes	int64	0
75	status_Canceled	bool	0
76	status_Ended	bool	0
77	status_Returning Series	bool	0
78	type_Documentary	bool	0
79	type_Miniseries	bool	0
80	type_News	bool	0
81	type_Reality	bool	0
82	type_Scripted	bool	0
83	type_Talk Show	bool	0
84	ranking_of_Series_Votes	float64	0
85	ranking_of_vote_average	float64	0
86	ranking_of_popularity	float64	0
87	fuzzy_Series_Votes	float64	0
88	fuzzy_vote_average	float64	0
89	fuzzy_popularity	float64	0

There are 54573 rows and 90 columns of information.

```
In [303... subset_all_episodes_df.to_csv(FINAL_DATA_DIRECTORY + "/episode_options_info.csv", index=False)
```

Series with Episodes having NaN and non-NaN values fr _18_to_49_Demo

```
In [304... # Identify series with NaN values in '_18_to_49_Demo'
series_with_nan = subset_all_episodes_df[subset_all_episodes_df['_18_to_49_Demo'].isna()][ 'IMDb Series ID'].unique()

# Identify series with non-NaN values in '_18_to_49_Demo'
series_with_non_nan = subset_all_episodes_df[subset_all_episodes_df['_18_to_49_Demo'].notna()][ 'IMDb Series ID'].unique()

# Find series that have both NaN and non-NaN values in '_18_to_49_Demo'
series_with_both = set(series_with_nan).intersection(series_with_non_nan)

# Convert to list
series_with_both_list = list(series_with_both)

# # Print the List of series
# print(series_with_both_list)
```

Data Frame of Episodes With These Series ID

```
In [305... episodes_with_both_nan_and_non_nan_df = subset_all_episodes_df[subset_all_episodes_df['IMDb Series ID'].isin(series_with_both)].copy()

episodes_with_no_nan_df = subset_all_episodes_df[
    ~subset_all_episodes_df['IMDb Series ID'].isin(series_with_both)
].copy()

episodes_with_both_nan_and_non_nan_df.to_csv(FINAL_DATA_DIRECTORY + "/episodes_with_nan_and_not_nan_ratings.csv", index=False)
```

```
In [306... tt6470487_df = episodes_with_both_nan_and_non_nan_df[episodes_with_both_nan_and_non_nan_df['IMDb Series ID']=='tt6470478']
pretty_print_df(tt6470487_df, interesting_columns = ['IMDb Series ID', 'IMDb Episode ID', '_18_to_49_Demo', 'Viewers_millions',
    'fuzzy_Series_Votes', 'fuzzy_vote_average', 'fuzzy_popularity'])
```


IMDb Series ID	IMDb Episode ID	_18_to_49_Demo	Viewers_millions	fuzzy_Series_Votes	fuzzy_vote_average	fuzzy_popularity
tt6470478	tt6886400	2.19	11.216	3.0	3.0	3.0
tt6470478	tt7179006	2.21	10.925	3.0	3.0	3.0
tt6470478	tt7375334	2.0	10.689	3.0	3.0	3.0
tt6470478	tt7375342	2.0	10.595	3.0	3.0	3.0
tt6470478	tt7375346	1.84	10.391	3.0	3.0	3.0
tt6470478	tt7375350	1.94	10.595	3.0	3.0	3.0
tt6470478	tt7521790	1.86	10.144	3.0	3.0	3.0
tt6470478	tt7524390	1.79	9.967	3.0	3.0	3.0
tt6470478	tt7524394	1.69	9.247	3.0	3.0	3.0
tt6470478	tt7536406	1.59	9.034	3.0	3.0	3.0
tt6470478	tt7536382	1.57	8.296	3.0	3.0	3.0
tt6470478	tt7536386	1.72	9.332	3.0	3.0	3.0
tt6470478	tt7557306	1.71	9.611	3.0	3.0	3.0
tt6470478	tt7762884	1.69	9.634	3.0	3.0	3.0
tt6470478	tt7762890	1.32	7.821	3.0	3.0	3.0
tt6470478	tt7762900	1.79	9.883	3.0	3.0	3.0
tt6470478	tt7762904	1.54	9.032	3.0	3.0	3.0
tt6470478	tt7762906	1.65	9.518	3.0	3.0	3.0
tt6470478	tt8242670	1.29	7.348	3.0	3.0	3.0
tt6470478	tt8680760	1.11	7.181	3.0	3.0	3.0
tt6470478	tt8707328	1.18	7.477	3.0	3.0	3.0
tt6470478	tt8970130	1.07	6.681	3.0	3.0	3.0
tt6470478	tt8970138	1.02	6.786	3.0	3.0	3.0
tt6470478	tt8970140	1.01	6.526	3.0	3.0	3.0
tt6470478	tt8970142	0.99	6.531	3.0	3.0	3.0
tt6470478	tt9174064	1.1	6.835	3.0	3.0	3.0
tt6470478	tt9248698	0.99	6.67	3.0	3.0	3.0
tt6470478	tt9272854	1.4	6.971	3.0	3.0	3.0
tt6470478	tt9389430	1.23	6.257	3.0	3.0	3.0
tt6470478	tt9477126	1.16	6.273	3.0	3.0	3.0
tt6470478	tt9477130	1.13	6.577	3.0	3.0	3.0
tt6470478	tt9452780	1.03	5.962	3.0	3.0	3.0
tt6470478	tt9442014	1.06	6.228	3.0	3.0	3.0
tt6470478	tt9660058	1.07	6.355	3.0	3.0	3.0
tt6470478	tt9660060	1.1	6.728	3.0	3.0	3.0
tt6470478	tt9660062	1.3	7.779	3.0	3.0	3.0
tt6470478	tt9739388	0.96	6.262	3.0	3.0	3.0
tt6470478	tt10700400	0.86	6.065	3.0	3.0	3.0
tt6470478	tt10808042	0.86	5.59	3.0	3.0	3.0
tt6470478	tt10685692	0.8	5.728	3.0	3.0	3.0
tt6470478	tt11061846	0.82	5.539	3.0	3.0	3.0
tt6470478	tt10810120	0.69	5.075	3.0	3.0	3.0
tt6470478	tt11199364	0.83	5.93	3.0	3.0	3.0
tt6470478	tt11149240	0.72	5.485	3.0	3.0	3.0
tt6470478	tt11244584	0.86	5.86	3.0	3.0	3.0
tt6470478	tt11089196	0.88	6.108	3.0	3.0	3.0
tt6470478	tt11268752	0.75	5.088	3.0	3.0	3.0
tt6470478	tt11376716	0.87	5.44	3.0	3.0	3.0
tt6470478	tt11565446	0.8	5.691	3.0	3.0	3.0
tt6470478	tt11317188	0.89	5.586	3.0	3.0	3.0
tt6470478	tt11689722	0.91	5.225	3.0	3.0	3.0
tt6470478	tt11689730	0.82	5.626	3.0	3.0	3.0
tt6470478	tt11718904	0.81	5.663	3.0	3.0	3.0
tt6470478	tt11718914	0.87	5.791	3.0	3.0	3.0
tt6470478	tt11914288	1.05	6.811	3.0	3.0	3.0
tt6470478	tt11914308	1.09	7.708	3.0	3.0	3.0
tt6470478	tt12025954	0.66	4.867	3.0	3.0	3.0
tt6470478	tt13351010	0.61	4.755	3.0	3.0	3.0
tt6470478	tt13252322	0.56	4.303	3.0	3.0	3.0
tt6470478	tt13415870	0.6	4.285	3.0	3.0	3.0
tt6470478	tt13475720	0.57	4.016	3.0	3.0	3.0
tt6470478	tt13498152	0.57	4.055	3.0	3.0	3.0
tt6470478	tt13760916	0.61	3.972	3.0	3.0	3.0
tt6470478	tt13801958	0.67	4.301	3.0	3.0	3.0
tt6470478	tt13945416	0.63	4.402	3.0	3.0	3.0
tt6470478	tt13998324	0.56	3.909	3.0	3.0	3.0
tt6470478	tt14048616	0.58	4.242	3.0	3.0	3.0
tt6470478	tt14202588	0.53	4.263	3.0	3.0	3.0
tt6470478	tt14223286	0.59	4.393	3.0	3.0	3.0
tt6470478	tt14402896	0.5	4.148	3.0	3.0	3.0
tt6470478	tt14420834	0.49	3.775	3.0	3.0	3.0
tt6470478	tt14512260	0.5	3.91	3.0	3.0	3.0
tt6470478	tt14551960	0.55	3.797	3.0	3.0	3.0
tt6470478	tt14597068	0.59	4.033	3.0	3.0	3.0
tt6470478	tt14597070	0.45	3.559	3.0	3.0	3.0
tt6470478	tt14597076	0.55	3.994	3.0	3.0	3.0
tt6470478	tt14578730	0.41	3.993	3.0	3.0	3.0
tt6470478	tt15444554	0.42	3.725	3.0	3.0	3.0
tt6470478	tt15500740	0.38	3.741	3.0	3.0	3.0
tt6470478	tt15589012	0.41	4.104	3.0	3.0	3.0
tt6470478	tt15717144	0.43	3.639	3.0	3.0	3.0
tt6470478	tt15739682	0.38	3.694	3.0	3.0	3.0
tt6470478	tt15921160	0.49	4.22	3.0	3.0	3.0
tt6470478	tt18257498	0.38	3.435	3.0	3.0	3.0
tt6470478	tt18272708	0.4	3.478	3.0	3.0	3.0
tt6470478	tt18351060	0.44	3.662	3.0	3.0	3.0
tt6470478	tt18396542	0.38	3.838	3.0	3.0	3.0
tt6470478	tt18569788	0.44	3.862	3.0	3.0	3.0
tt6470478	tt18922558	0.37	3.659	3.0	3.0	3.0
tt6470478	tt19047628	0.39	3.989	3.0	3.0	3.0
tt6470478	tt19389264	nan	nan	3.0	3.0	3.0
tt6470478	tt19712336	0.39	3.772	3.0	3.0	3.0
tt6470478	tt19712338	0.41	3.681	3.0	3.0	3.0
tt6470478	tt19712340	0.35	3.081	3.0	3.0	3.0
tt6470478	tt19267596	0.39	3.543	3.0	3.0	3.0

tt6470478	tt22187748	0.25	3.182	3.0	3.0	3.0
tt6470478	tt22482390	0.34	3.0	3.0	3.0	3.0
tt6470478	tt22529596	0.33	3.196	3.0	3.0	3.0
tt6470478	tt22801798	0.29	3.312	3.0	3.0	3.0
tt6470478	tt23146066	0.32	3.09	3.0	3.0	3.0
tt6470478	tt23320710	0.33	3.317	3.0	3.0	3.0
tt6470478	tt23624658	0.36	3.215	3.0	3.0	3.0
tt6470478	tt23624660	0.3	3.563	3.0	3.0	3.0
tt6470478	tt23727902	0.32	3.286	3.0	3.0	3.0
tt6470478	tt26315874	0.34	3.43	3.0	3.0	3.0
tt6470478	tt26450545	0.28	3.139	3.0	3.0	3.0
tt6470478	tt26542336	0.29	3.087	3.0	3.0	3.0
tt6470478	tt26690687	0.3	3.119	3.0	3.0	3.0
tt6470478	tt26774056	0.32	3.332	3.0	3.0	3.0
tt6470478	tt26026804	0.31	3.412	3.0	3.0	3.0
tt6470478	tt26936081	0.34	3.475	3.0	3.0	3.0
tt6470478	tt27190137	0.29	3.347	3.0	3.0	3.0
tt6470478	tt27190153	0.35	3.403	3.0	3.0	3.0
tt6470478	tt27190173	0.3	3.476	3.0	3.0	3.0
tt6470478	tt27190231	0.34	3.692	3.0	3.0	3.0
tt6470478	tt27190233	0.32	3.873	3.0	3.0	3.0
tt6470478	tt27552077	0.26	2.859	3.0	3.0	3.0
tt6470478	tt30982667	0.18	2.523	3.0	3.0	3.0
tt6470478	tt31183813	0.18	2.148	3.0	3.0	3.0
tt6470478	tt31183816	nan	nan	3.0	3.0	3.0
tt6470478	tt31278704	nan	nan	3.0	3.0	3.0
tt6470478	tt31279055	nan	nan	3.0	3.0	3.0
tt6470478	tt31309218	nan	nan	3.0	3.0	3.0
tt6470478	tt31309222	nan	nan	3.0	3.0	3.0

NaN Strategy

- NaNs are replaced with local averages
- Uses fencepost logic.

Example:

```
In [307...] data = {
    'IMDb Series ID': ['tt6470478'] * 10,
    'IMDb Episode ID': [f'tt{i}' for i in range(1, 11)],
    '18-49 Demo': [None, None, 2.0, 2.0, 0.82, 0.81, 0.87, 1.05, 1.09, 0.66],
    'Viewers (millions)': [11.216, 10.925, None, 10.595, 5.626, 5.663, None, 6.811, 7.708, None]
}
df = pd.DataFrame(data)

# Function to replace NaNs with local averages (fencepost Logic)
def fill_with_local_average(series):
    series = series.copy()
    for i in range(len(series)):
        if pd.isna(series.iloc[i]):
            # Find valid neighbors
            left = right = i

            # Find left neighbor
            while left > 0 and pd.isna(series.iloc[left]):
                left -= 1

            # Find right neighbor
            while right < len(series) - 1 and pd.isna(series.iloc[right]):
                right += 1

            # Replace NaN based on fencepost Logic
            if left == 0 and pd.isna(series.iloc[left]): # No valid left neighbor
                series.iloc[i] = series.iloc[right]
            elif right == len(series) - 1 and pd.isna(series.iloc[right]): # No valid right neighbor
                series.iloc[i] = series.iloc[left]
            else:
                # Calculate the median of left and right neighbors
                series.iloc[i] = np.nanmedian([series.iloc[left], series.iloc[right]])
    return series

# Apply the local average function to the columns
df['18-49 Demo'] = df.groupby('IMDb Series ID')['18-49 Demo'].apply(fill_with_local_average).reset_index(level=0, drop=True)
df['Viewers (millions)'] = df.groupby('IMDb Series ID')['Viewers (millions)'].apply(fill_with_local_average).reset_index(level=0, drop=True)

# Print the DataFrame to check the changes
print(df)
```

	IMDb Series ID	IMDb Episode ID	18-49 Demo	Viewers (millions)
0	tt6470478	tt1	2.00	11.216
1	tt6470478	tt2	2.00	10.925
2	tt6470478	tt3	2.00	10.760
3	tt6470478	tt4	2.00	10.595
4	tt6470478	tt5	0.82	5.626
5	tt6470478	tt6	0.81	5.663
6	tt6470478	tt7	0.87	6.237
7	tt6470478	tt8	1.05	6.811
8	tt6470478	tt9	1.09	7.708
9	tt6470478	tt10	0.66	7.708

Apply Fencepost to Episodes

```
In [309...] show_df_info(episodes_with_both_nan_and_non_nan_df, "Both Nan and no Nan")
episodes_with_both_nan_and_non_nan_df['18_to_49_Demo'] = \
    episodes_with_both_nan_and_non_nan_df.groupby('IMDb Series ID')['18_to_49_Demo'].apply(fill_with_local_average).reset_index(level=0, drop=True)
```

```
episodes_with_both_nan_and_non_nan_df['Viewers_millions'] = \
    episodes_with_both_nan_and_non_nan_df.groupby('IMDb Series ID')['Viewers_millions'].apply(fill_with_local_average).reset_index(level=0, drop=True)
```

Dataframe information for Both Nan and no Nan:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	rating	float64	0
5	Episode_Votes	int32	0
6	number_of_seasons	float64	0
7	number_of_episodes	float64	0
8	Series_Votes	float64	0
9	vote_average	float64	0
10	in_production	bool	0
11	popularity	float64	0
12	Sci-Fi & Fantasy	bool	0
13	Drama	bool	0
14	Action & Adventure	bool	0
15	Crime	bool	0
16	Mystery	bool	0
17	Comedy	bool	0
18	Animation	bool	0
19	Family	bool	0
20	War & Politics	bool	0
21	Soap	bool	0
22	Western	bool	0
23	Kids	bool	0
24	Documentary	bool	0
25	Reality	bool	0
26	Romance	bool	0
27	Adult	bool	0
28	Biography	bool	0
29	Film-Noir	bool	0
30	Game-Show	bool	0
31	History	bool	0
32	Horror	bool	0
33	Musical	bool	0
34	Music	bool	0
35	News	bool	0
36	Short	bool	0
37	Sport	bool	0
38	Talk-Show	bool	0
39	Thriller	bool	0
40	HBO	bool	0
41	Netflix	bool	0
42	AMC	bool	0
43	FOX	bool	0
44	The CW	bool	0
45	ABC	bool	0
46	Disney+	bool	0
47	CBS	bool	0
48	Prime Video	bool	0
49	Adult Swim	bool	0
50	NBC	bool	0
51	Hulu	bool	0
52	USA Network	bool	0
53	Cinemax	bool	0
54	Comedy Central	bool	0
55	Showtime	bool	0
56	Paramount+	bool	0
57	Paramount Network	bool	0
58	Lifetime	bool	0
59	Starz	bool	0
60	Telemundo	bool	0
61	Apple TV+	bool	0
62	HBO Max	bool	0
63	BET	bool	0
64	Hallmark Channel	bool	0
65	Peacock	bool	0
66	_18_to_49_Demo	float64	16352
67	Viewers_millions	float64	16248
68	sex_factor	int64	0
69	violence_factor	int64	0
70	profanity_factor	int64	0
71	drugs_factor	int64	0
72	intense_factor	int64	0
73	IMDb_Episode_Rating	float64	0
74	IMDb_Episode_Votes	int64	0
75	status_Canceled	bool	0
76	status_Ended	bool	0
77	status_Returning Series	bool	0
78	type_Documentary	bool	0
79	type_Miniseries	bool	0
80	type_News	bool	0
81	type_Reality	bool	0
82	type_Scripted	bool	0
83	type_Talk Show	bool	0
84	ranking_of_Series_Votes	float64	0
85	ranking_of_vote_average	float64	0
86	ranking_of_popularity	float64	0
87	fuzzy_Series_Votes	float64	0
88	fuzzy_vote_average	float64	0
89	fuzzy_popularity	float64	0

There are 30778 rows and 90 columns of information.

```
In [310... episodes_with_both_nan_and_non_nan_df.to_csv(FINAL_DATA_DIRECTORY + "/episodes_with_fenceposted_nan_values.csv", index=False)
```

```
In [311... episode_modeling_df = pd.concat([episodes_with_no_nan_df, episodes_with_both_nan_and_non_nan_df])  
show_df_info(episode_modeling_df, "fenceposted episodes")
```

Dataframe information for fenceposted episodes:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	rating	float64	0
5	Episode_Votes	int32	0
6	number_of_seasons	float64	0
7	number_of_episodes	float64	0
8	Series_Votes	float64	0
9	vote_average	float64	0
10	in_production	bool	0
11	popularity	float64	0
12	Sci-Fi & Fantasy	bool	0
13	Drama	bool	0
14	Action & Adventure	bool	0
15	Crime	bool	0
16	Mystery	bool	0
17	Comedy	bool	0
18	Animation	bool	0
19	Family	bool	0
20	War & Politics	bool	0
21	Soap	bool	0
22	Western	bool	0
23	Kids	bool	0
24	Documentary	bool	0
25	Reality	bool	0
26	Romance	bool	0
27	Adult	bool	0
28	Biography	bool	0
29	Film-Noir	bool	0
30	Game-Show	bool	0
31	History	bool	0
32	Horror	bool	0
33	Musical	bool	0
34	Music	bool	0
35	News	bool	0
36	Short	bool	0
37	Sport	bool	0
38	Talk-Show	bool	0
39	Thriller	bool	0
40	HBO	bool	0
41	Netflix	bool	0
42	AMC	bool	0
43	FOX	bool	0
44	The CW	bool	0
45	ABC	bool	0
46	Disney+	bool	0
47	CBS	bool	0
48	Prime Video	bool	0
49	Adult Swim	bool	0
50	NBC	bool	0
51	Hulu	bool	0
52	USA Network	bool	0
53	Cinemax	bool	0
54	Comedy Central	bool	0
55	Showtime	bool	0
56	Paramount+	bool	0
57	Paramount Network	bool	0
58	Lifetime	bool	0
59	Starz	bool	0
60	Telemundo	bool	0
61	Apple TV+	bool	0
62	HBO Max	bool	0
63	BET	bool	0
64	Hallmark Channel	bool	0
65	Peacock	bool	0
66	_18_to_49_Demo	float64	12483
67	Viewers_millions	float64	12483
68	sex_factor	int64	0
69	violence_factor	int64	0
70	profanity_factor	int64	0
71	drugs_factor	int64	0
72	intense_factor	int64	0
73	IMDb_Episode_Rating	float64	0
74	IMDb_Episode_Votes	int64	0
75	status_Canceled	bool	0
76	status_Ended	bool	0
77	status_Returning Series	bool	0
78	type_Documentary	bool	0
79	type_Miniseries	bool	0
80	type_News	bool	0
81	type_Reality	bool	0
82	type_Scripted	bool	0
83	type_Talk Show	bool	0
84	ranking_of_Series_Votes	float64	0
85	ranking_of_vote_average	float64	0
86	ranking_of_popularity	float64	0
87	fuzzy_Series_Votes	float64	0
88	fuzzy_vote_average	float64	0
89	fuzzy_popularity	float64	0

There are 54573 rows and 90 columns of information.

```
In [312... # Drop any remaining NaNs in the '18-49 Demo' after conversion (if necessary)
episode_modeling_df.dropna(subset=['_18_to_49_Demo'], inplace=True)

In [313... episode_modeling_df.to_csv(FINAL_DATA_DIRECTORY + "/episode_modeling_info.csv", index=False)

In [314... show_df_info(episode_modeling_df, "all episodes with content")
```

Dataframe information for all episodes with content:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	rating	float64	0
5	Episode_Votes	int32	0
6	number_of_seasons	float64	0
7	number_of_episodes	float64	0
8	Series_Votes	float64	0
9	vote_average	float64	0
10	in_production	bool	0
11	popularity	float64	0
12	Sci-Fi & Fantasy	bool	0
13	Drama	bool	0
14	Action & Adventure	bool	0
15	Crime	bool	0
16	Mystery	bool	0
17	Comedy	bool	0
18	Animation	bool	0
19	Family	bool	0
20	War & Politics	bool	0
21	Soap	bool	0
22	Western	bool	0
23	Kids	bool	0
24	Documentary	bool	0
25	Reality	bool	0
26	Romance	bool	0
27	Adult	bool	0
28	Biography	bool	0
29	Film-Noir	bool	0
30	Game-Show	bool	0
31	History	bool	0
32	Horror	bool	0
33	Musical	bool	0
34	Music	bool	0
35	News	bool	0
36	Short	bool	0
37	Sport	bool	0
38	Talk-Show	bool	0
39	Thriller	bool	0
40	HBO	bool	0
41	Netflix	bool	0
42	AMC	bool	0
43	FOX	bool	0
44	The CW	bool	0
45	ABC	bool	0
46	Disney+	bool	0
47	CBS	bool	0
48	Prime Video	bool	0
49	Adult Swim	bool	0
50	NBC	bool	0
51	Hulu	bool	0
52	USA Network	bool	0
53	Cinemax	bool	0
54	Comedy Central	bool	0
55	Showtime	bool	0
56	Paramount+	bool	0
57	Paramount Network	bool	0
58	Lifetime	bool	0
59	Starz	bool	0
60	Telemundo	bool	0
61	Apple TV+	bool	0
62	HBO Max	bool	0
63	BET	bool	0
64	Hallmark Channel	bool	0
65	Peacock	bool	0
66	_18_to_49_Demo	float64	0
67	Viewers_millions	float64	0
68	sex_factor	int64	0
69	violence_factor	int64	0
70	profanity_factor	int64	0
71	drugs_factor	int64	0
72	intense_factor	int64	0
73	IMDb_Episode_Rating	float64	0
74	IMDb_Episode_Votes	int64	0
75	status_Canceled	bool	0
76	status_Ended	bool	0
77	status_Returning Series	bool	0
78	type_Documentary	bool	0
79	type_Miniseries	bool	0
80	type_News	bool	0
81	type_Reality	bool	0
82	type_Scripted	bool	0
83	type_Talk Show	bool	0
84	ranking_of_Series_Votes	float64	0
85	ranking_of_vote_average	float64	0
86	ranking_of_popularity	float64	0
87	fuzzy_Series_Votes	float64	0
88	fuzzy_vote_average	float64	0
89	fuzzy_popularity	float64	0

There are 42090 rows and 90 columns of information.

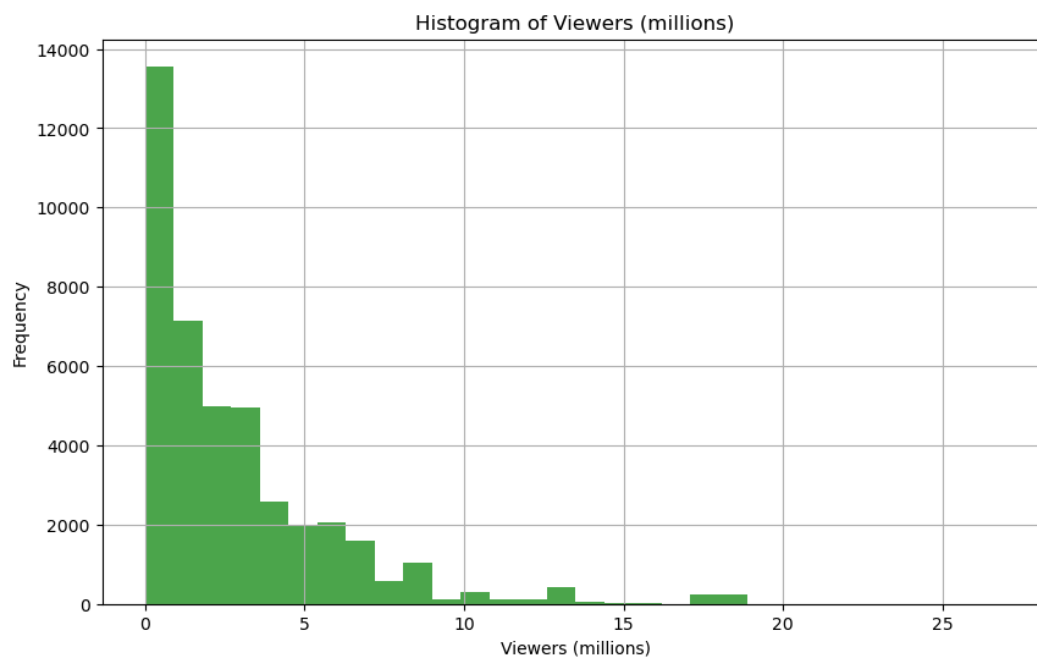
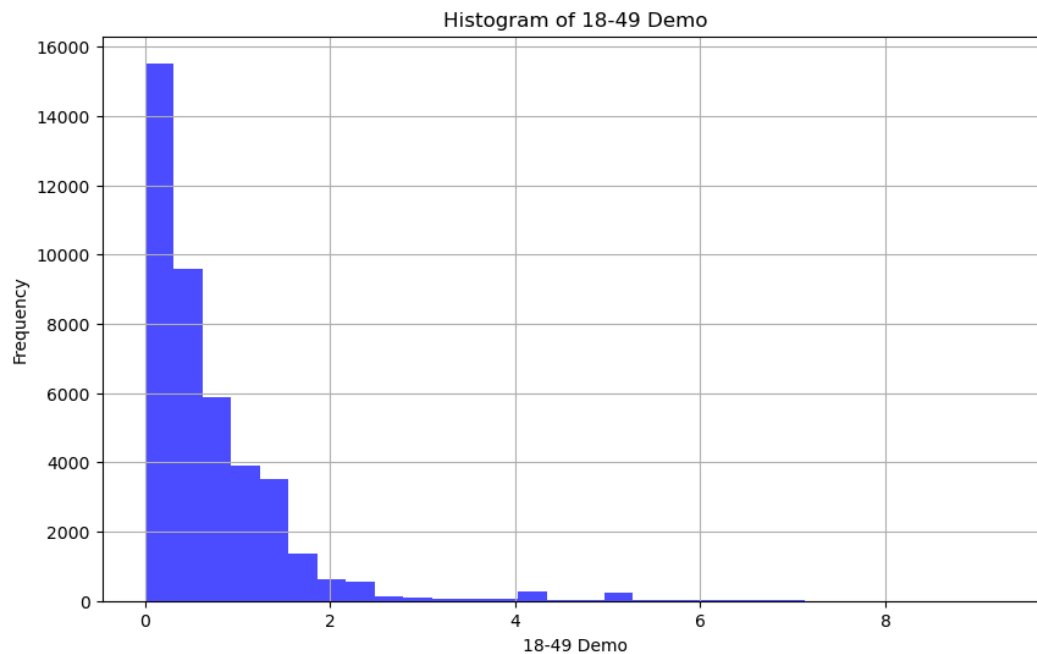
Visualize Target Data

In [315...

```
# Convert '18-49 Demo' and 'Viewers (millions)' columns to numeric types
episode_modeling_df['_18_to_49_Demo'] = pd.to_numeric(episode_modeling_df['_18_to_49_Demo'], errors='coerce')
episode_modeling_df['Viewers_millions'] = pd.to_numeric(episode_modeling_df['Viewers_millions'], errors='coerce')

# Plot histogram of '18-49 Demo'
plt.figure(figsize=(10, 6))
plt.hist(episode_modeling_df['_18_to_49_Demo'].dropna(), bins=30, color='blue', alpha=0.7)
plt.xlabel('18-49 Demo')
plt.ylabel('Frequency')
plt.title('Histogram of 18-49 Demo')
plt.grid(True)
plt.show()

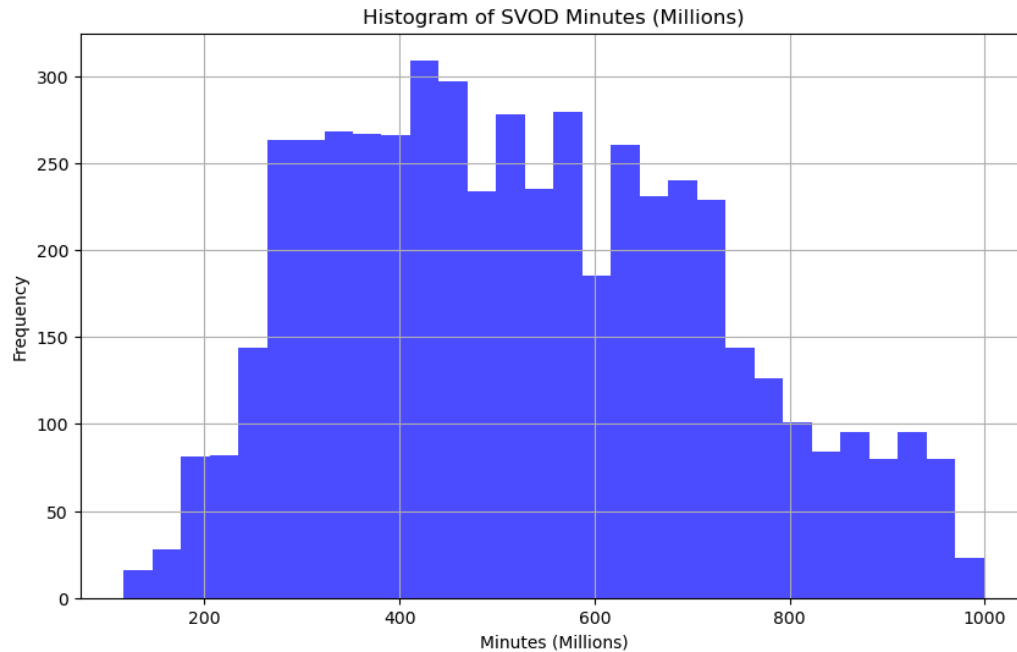
# Plot histogram of 'Viewers (millions)'
plt.figure(figsize=(10, 6))
plt.hist(episode_modeling_df['Viewers_millions'].dropna(), bins=30, color='green', alpha=0.7)
plt.xlabel('Viewers (millions)')
plt.ylabel('Frequency')
plt.title('Histogram of Viewers (millions)')
plt.grid(True)
plt.show()
```



SVOD Data

```
In [316... # Convert 'Minutes (Millions)' column to numeric type
svod_df.loc[:, 'Minutes (Millions)'] = pd.to_numeric(svod_df['Minutes (Millions)'], errors='coerce')

# Plot histogram of '18-49 Demo'
plt.figure(figsize=(10, 6))
plt.hist(svod_df['Minutes (Millions)'].dropna(), bins=30, color='blue', alpha=0.7)
plt.xlabel('Minutes (Millions)')
plt.ylabel('Frequency')
plt.title('Histogram of SVOD Minutes (Millions)')
plt.grid(True)
plt.show()
```



```
In [317... print(svod_df['networks'].unique())

['Disney+' 'Hulu, Disney+' 'Netflix' 'Hulu, Netflix' 'Hulu'
 'Paramount+', 'Netflix' 'HBO Max, Netflix' 'Hulu, Prime Video'
 'Peacock, Netflix' 'Hulu, Paramount+' 'Prime Video' 'HBO Max'
 'Prime Video, Hulu' 'Hulu, HBO Max' 'Prime Video, Netflix'
 'Paramount+', 'Prime Video' 'Peacock' 'Apple TV+' 'Netflix, Paramount+'
 'Paramount+']
```

Histogram by Platform

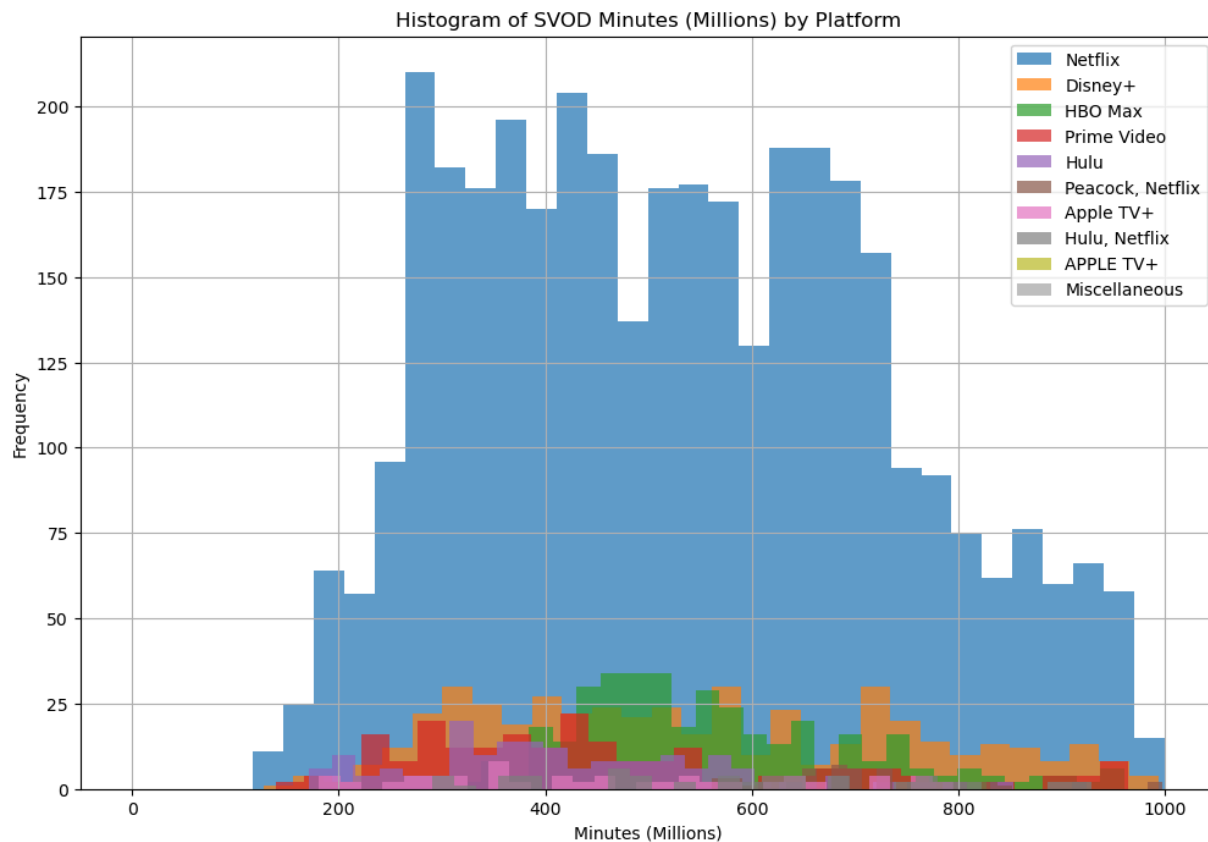
```
In [318... # Select the top 8 platforms by value counts
top_platforms = svod_df['networks'].value_counts().head(8).index
top_platforms = list(top_platforms) + ['APPLE TV+']

# Create a new column 'Platform_Grouped' where all platforms not in the top 8 are Labeled as 'Miscellaneous'
svod_df.loc[:, 'Platform_Grouped'] = svod_df['networks'].where(svod_df['networks'].isin(top_platforms), 'Miscellaneous')

# Plot histogram for each platform's contribution
plt.figure(figsize=(12, 8))
for platform in top_platforms:
    plt.hist(svod_df[svod_df['networks'] == platform]['Minutes (Millions)'].dropna(), bins=30, alpha=0.7, label=platform)

# Plot histogram for Miscellaneous
plt.hist(svod_df[svod_df['networks'] == 'Miscellaneous']['Minutes (Millions)'].dropna(), bins=30, alpha=0.5, color='gray', label='Miscellaneous')

# Set Labels and title
plt.xlabel('Minutes (Millions)')
plt.ylabel('Frequency')
plt.title('Histogram of SVOD Minutes (Millions) by Platform')
plt.legend()
plt.grid(True)
plt.show()
```



Create Subsets

- Eliminate Mini Series (Scheduled for One Season)

['Scripted' 'Miniseries' 'Documentary' 'Reality' 'Talk Show' 'News' 'Video']

- Limited to Series of More Than One Season
- Limited to Series of More Than Two Seasons

```
In [319... # Filter the dataframe based on boolean conditions

series_episode_subset_modeling_df = episode_modeling_df[
    (episode_modeling_df['type_Scripted'] == True) |
    (episode_modeling_df['type_Reality'] == True) |
    (episode_modeling_df['type_Talk Show'] == True)
].copy()
```

GROUPING: Series specific data

Can be used to "salt" the model to group series together to different degrees

- vote_count - TMDB series vote count
- popularity - TMDB popularity
- average_vote - TMDB series average vote (less precise but still identifying)

```
In [320... # this is too absolute for modeling
series_feature_labels = ['rating', 'Series_Votes', 'Episode_Votes', 'vote_average', 'popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']

# goldilocks after dropping fuzzy_Series_Votes (too dominant) -- can be tuned with fuzzify_series.
fuzzy_series_labels = ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']

# this is too random without serie
without_series_feature_labels = ['rating', 'Episode_Votes', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']

# deeper option dive
content_feature_labels = ['sex_factor', 'violence_factor', 'profanity_factor', 'drugs_factor', 'intense_factor']

# svod labels
svod_feature_labels = ['Series_Votes', 'vote_average', 'popularity']

# List of networks
network_feature_labels = [
    "HBO", "Netflix", "AMC", "FOX", "The CW", "ABC", "Disney+", "CBS",
    "Prime Video", "Adult Swim", "NBC", "Hulu", "USA Network", "Cinemax",
    "Comedy Central", "Showtime", "Paramount+", "Paramount Network", "Lifetime", "Starz",
    "Telemundo", "Apple TV+", "HBO Max", "BET", "Hallmark Channel", "Peacock"
]
genre_feature_labels = []
```

```

"Sci-Fi & Fantasy", "Drama", "Action & Adventure", "Crime", "Mystery",
"Comedy", "Animation", "Family", "War & Politics", "Soap", "Western",
"Kids", "Documentary", "Reality", "Romance", "Adult", "Biography",
"Film-Noir", "Game-Show", "History", "Horror", "Musical", "Music",
"News", "Short", "Sport", "Talk-Show", "Thriller"
]

def select_features(main_labels = fuzzy_series_feature_labels, content = False, networks = False, genres = False):
    selected_features = main_labels.copy()
    if content:
        selected_features.extend(content_feature_labels)
    if networks:
        selected_features.extend(network_feature_labels)
    if genres:
        selected_features.extend(genre_feature_labels)
    return selected_features

# print(select_features())
# print(select_features(content=True))
# print(select_features(content=True, networks=True, genres=True))

```

Projection Sets

```
In [321... two_seasons_or_more_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['number_of_seasons']>1].copy()
```

```
In [322... three_seasons_or_more_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['number_of_seasons']>2].copy()
```

```
In [323... show_df_info(two_seasons_or_more_df, "Episodes of series that last at least 2 seasons")
show_df_info(three_seasons_or_more_df, "Episodes of series that last at least 3 seasons")
```

Dataframe information for Episodes of series that last at least 2 seasons:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	rating	float64	0
5	Episode_Votes	int32	0
6	number_of_seasons	float64	0
7	number_of_episodes	float64	0
8	Series_Votes	float64	0
9	vote_average	float64	0
10	in_production	bool	0
11	popularity	float64	0
12	Sci-Fi & Fantasy	bool	0
13	Drama	bool	0
14	Action & Adventure	bool	0
15	Crime	bool	0
16	Mystery	bool	0
17	Comedy	bool	0
18	Animation	bool	0
19	Family	bool	0
20	War & Politics	bool	0
21	Soap	bool	0
22	Western	bool	0
23	Kids	bool	0
24	Documentary	bool	0
25	Reality	bool	0
26	Romance	bool	0
27	Adult	bool	0
28	Biography	bool	0
29	Film-Noir	bool	0
30	Game-Show	bool	0
31	History	bool	0
32	Horror	bool	0
33	Musical	bool	0
34	Music	bool	0
35	News	bool	0
36	Short	bool	0
37	Sport	bool	0
38	Talk-Show	bool	0
39	Thriller	bool	0
40	HBO	bool	0
41	Netflix	bool	0
42	AMC	bool	0
43	FOX	bool	0
44	The CW	bool	0
45	ABC	bool	0
46	Disney+	bool	0
47	CBS	bool	0
48	Prime Video	bool	0
49	Adult Swim	bool	0
50	NBC	bool	0
51	Hulu	bool	0
52	USA Network	bool	0
53	Cinemax	bool	0
54	Comedy Central	bool	0
55	Showtime	bool	0
56	Paramount+	bool	0
57	Paramount Network	bool	0
58	Lifetime	bool	0
59	Starz	bool	0
60	Telemundo	bool	0
61	Apple TV+	bool	0
62	HBO Max	bool	0
63	BET	bool	0
64	Hallmark Channel	bool	0
65	Peacock	bool	0
66	_18_to_49_Demo	float64	0
67	Viewers_millions	float64	0
68	sex_factor	int64	0
69	violence_factor	int64	0
70	profanity_factor	int64	0
71	drugs_factor	int64	0
72	intense_factor	int64	0
73	IMDb_Episode_Rating	float64	0
74	IMDb_Episode_Votes	int64	0
75	status_Canceled	bool	0
76	status_Ended	bool	0
77	status_Returning Series	bool	0
78	type_Documentary	bool	0
79	type_Miniseries	bool	0
80	type_News	bool	0
81	type_Reality	bool	0
82	type_Scripted	bool	0
83	type_Talk Show	bool	0
84	ranking_of_Series_Votes	float64	0
85	ranking_of_vote_average	float64	0
86	ranking_of_popularity	float64	0
87	fuzzy_Series_Votes	float64	0
88	fuzzy_vote_average	float64	0
89	fuzzy_popularity	float64	0

There are 38270 rows and 90 columns of information.

Dataframe information for Episodes of series that last at least 3 seasons:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	rating	float64	0
5	Episode_Votes	int32	0
6	number_of_seasons	float64	0
7	number_of_episodes	float64	0
8	Series_Votes	float64	0
9	vote_average	float64	0
10	in_production	bool	0
11	popularity	float64	0
12	Sci-Fi & Fantasy	bool	0
13	Drama	bool	0
14	Action & Adventure	bool	0
15	Crime	bool	0
16	Mystery	bool	0
17	Comedy	bool	0
18	Animation	bool	0
19	Family	bool	0
20	War & Politics	bool	0
21	Soap	bool	0
22	Western	bool	0
23	Kids	bool	0
24	Documentary	bool	0
25	Reality	bool	0
26	Romance	bool	0
27	Adult	bool	0
28	Biography	bool	0
29	Film-Noir	bool	0
30	Game-Show	bool	0
31	History	bool	0
32	Horror	bool	0
33	Musical	bool	0
34	Music	bool	0
35	News	bool	0
36	Short	bool	0
37	Sport	bool	0
38	Talk-Show	bool	0
39	Thriller	bool	0
40	HBO	bool	0
41	Netflix	bool	0
42	AMC	bool	0
43	FOX	bool	0
44	The CW	bool	0
45	ABC	bool	0
46	Disney+	bool	0
47	CBS	bool	0
48	Prime Video	bool	0
49	Adult Swim	bool	0
50	NBC	bool	0
51	Hulu	bool	0
52	USA Network	bool	0
53	Cinemax	bool	0
54	Comedy Central	bool	0
55	Showtime	bool	0
56	Paramount+	bool	0
57	Paramount Network	bool	0
58	Lifetime	bool	0
59	Starz	bool	0
60	Telemundo	bool	0
61	Apple TV+	bool	0
62	HBO Max	bool	0
63	BET	bool	0
64	Hallmark Channel	bool	0
65	Peacock	bool	0
66	_18_to_49_Demo	float64	0
67	Viewers_millions	float64	0
68	sex_factor	int64	0
69	violence_factor	int64	0
70	profanity_factor	int64	0
71	drugs_factor	int64	0
72	intense_factor	int64	0
73	IMDb_Episode_Rating	float64	0
74	IMDb_Episode_Votes	int64	0
75	status_Canceled	bool	0
76	status_Ended	bool	0
77	status_Returning Series	bool	0
78	type_Documentary	bool	0
79	type_Miniseries	bool	0
80	type_News	bool	0
81	type_Reality	bool	0
82	type_Scripted	bool	0
83	type_Talk Show	bool	0
84	ranking_of_Series_Votes	float64	0
85	ranking_of_vote_average	float64	0
86	ranking_of_popularity	float64	0
87	fuzzy_Series_Votes	float64	0
88	fuzzy_vote_average	float64	0

```
| 89 | fuzzy_popularity | float64 | 0 |
+---+-----+-----+-----+
```

There are 33835 rows and 90 columns of information.

Modeling.

1. Simple Mixed Linear Model - Viewers (Millions)

```
In [324... # Define the model
model = MixedLM.from_formula('Viewers_millions ~ rating + IMDb_Episode_Rating + IMDb_Episode_Votes + fuzzy_popularity + fuzzy_vote_average',
                             groups=two_seasons_or_more_df['IMDb Series ID'],
                             data=two_seasons_or_more_df)

result = model.fit()

print(result.summary())
```

Mixed Linear Model Regression Results						
=====						
Model:	MixedLM	Dependent Variable: Viewers_millions				
No. Observations:	38270	Method:	REML			
No. Groups:	556	Scale:	0.9403			
Min. group size:	1	Log-Likelihood:	-54617.7168			
Max. group size:	765	Converged:	Yes			
Mean group size:	68.8					

	Coef.	Std.Err.	z	P> z	[0.025	0.975]

Intercept	0.430	0.161	2.672	0.008	0.115	0.745
rating	0.514	0.226	2.279	0.023	0.072	0.956
IMDb_Episode_Rating	-0.400	0.226	-1.773	0.076	-0.843	0.042
IMDb_Episode_Votes	0.000	0.000	29.739	0.000	0.000	0.000
fuzzy_popularity	0.781	0.100	7.791	0.000	0.585	0.978
fuzzy_vote_average	-0.125	0.094	-1.338	0.181	-0.308	0.058
Group Var	4.197	0.264				
=====						

Model Summary Interpretation

Dependent Variable: Viewers_millions

Method: Restricted Maximum Likelihood (REML)

Number of Observations: 23,283 (all episodes)

Number of Groups: 556 (all tv series)

Scale: 1.1368, Estimated variance of the residuals (error term).

Group Size: Varies from 1 to 296, with an average of 41.9 per group.

Parameter Estimates

- Intercept (0.368, p=0.008): The base number of million viewers when all predictors are at zero.
- It's statistically significant, indicating that the model intercept is different from zero at the 1% level.
- rating (1.037, p=0.006): For each one-unit increase in the rating, viewership increases by about 1.037 million, on average, holding other variables constant.
- This effect is statistically significant, suggesting that higher ratings are positively associated with higher viewership.
- IMDb_Episode_Rating (-0.859, p=0.023): For each one-point increase in IMDb episode rating, viewership decreases by about 0.859 million.
- This is counterintuitive; I expected higher IMDb ratings to correlate with higher viewership. This negative relationship is statistically significant and warrants further investigation to understand if there are confounding factors or if it reflects viewer preferences or behavior not captured by the model. One possibility is network or genre selection. Highrer but **fewer** ratings.
- IMDb_Episode_Votes (0.000, p<0.001): The coefficient here indicates a positive relationship between the number of IMDb votes and viewership. The precise value isn't shown due to rounding but is significant. This suggests that episodes with more votes, indicative of greater viewer engagement or interest, tend to have higher viewership numbers. This is expected.

Group Variance

- Group Var (3.795): This represents the variance between groups (TV series) after accounting for the effects of the observed variables.
- The value indicates substantial variability in viewership across different series, which is consistent with the expectation that different series will attract different audience sizes. ##### Model Fit and Convergence
 - Log-Likelihood: -35823.9913, which helps in model comparison (higher is better when comparing models). ##### Converged: Yes, indicating that the estimation procedure successfully converged to a solution.

Interpretation:

This model indicates that both ratings and IMDb votes positively impact viewership, while an increase in IMDb episode ratings surprisingly corresponds to a decrease in viewership. The significant group variance suggests that there are substantial differences between series, justifying the use of a mixed-effects model.

Given the counterintuitive finding with IMDb episode ratings, further investigation could include:

- * Examining other factors: Are there external influences or confounding variables that might explain the negative coefficient for IMDb episode ratings?
- * Model diagnostics: Checking for issues with data quality, outliers, or influential points that might affect the IMDb rating

effect.

* Refinement: Adjusting model specifications, exploring interactions between variables, or incorporating additional relevant predictors.

Simple Mixed-Linear Model - 18-49 Demo

```
In [325... # Define the model
model = MixedLM.from_formula('_18_to_49_Demo ~ rating + IMDB_Episode_Rating + IMDB_Episode_Votes + fuzzy_popularity + fuzzy_vote_average',
                             groups=two_seasons_or_more_df['IMDb Series ID'],
                             data=two_seasons_or_more_df)

result = model.fit()

print(result.summary())
```

```

Mixed Linear Model Regression Results
=====
Model:                MixedLM Dependent Variable: _18_to_49_Demo
No. Observations:    38270  Method:                REML
No. Groups:          556    Scale:                0.1352
Min. group size:     1      Log-Likelihood:    -17302.5724
Max. group size:     765    Converged:         Yes
Mean group size:     68.8

-----
                Coef.  Std.Err.  z    P>|z|  [0.025 0.975]
-----
Intercept      -0.071    0.046  -1.553  0.120  -0.160  0.019
rating          0.156    0.085   1.825  0.068  -0.012  0.324
IMDb_Episode_Rating -0.107    0.086  -1.253  0.210  -0.275  0.060
IMDb_Episode_Votes  0.000    0.000  34.409  0.000  0.000  0.000
fuzzy_popularity  0.196    0.026   7.499  0.000  0.145  0.247
fuzzy_vote_average -0.014    0.024  -0.592  0.554  -0.062  0.033
Group Var       0.284    0.047

=====
```

Simple Mixed/Linear for Three Seasons or More

```
In [326... # Define the model
model = MixedLM.from_formula('Viewers_millions ~ rating + IMDB_Episode_Rating + IMDB_Episode_Votes + fuzzy_popularity + fuzzy_vote_average',
                             groups=three_seasons_or_more_df['IMDb Series ID'],
                             data=three_seasons_or_more_df)

result = model.fit()

print(result.summary())
```

```

Mixed Linear Model Regression Results
=====
Model:                MixedLM Dependent Variable: Viewers_millions
No. Observations:    33835  Method:                REML
No. Groups:          385    Scale:                1.0227
Min. group size:     2      Log-Likelihood:    -49513.9212
Max. group size:     765    Converged:         Yes
Mean group size:     87.9

-----
                Coef.  Std.Err.  z    P>|z|  [0.025 0.975]
-----
Intercept       0.486    0.229   2.127  0.033  0.038  0.935
rating          0.565    0.246   2.297  0.022  0.083  1.046
IMDb_Episode_Rating -0.441    0.246  -1.793  0.073  -0.924  0.041
IMDb_Episode_Votes  0.000    0.000  28.457  0.000  0.000  0.000
fuzzy_popularity  0.829    0.128   6.457  0.000  0.578  1.081
fuzzy_vote_average -0.179    0.123  -1.454  0.146  -0.421  0.062
Group Var       5.065    0.365

=====
```

```
In [327... # Define the model
model = MixedLM.from_formula('_18_to_49_Demo ~ rating + IMDB_Episode_Rating + IMDB_Episode_Votes + fuzzy_popularity + fuzzy_vote_average',
                             groups=three_seasons_or_more_df['IMDb Series ID'],
                             data=three_seasons_or_more_df)

result = model.fit()

print(result.summary())
```

```

Mixed Linear Model Regression Results
=====
Model:                MixedLM Dependent Variable: _18_to_49_Demo
No. Observations:    33835  Method:                REML
No. Groups:          385    Scale:                0.1491
Min. group size:     2      Log-Likelihood:    -16805.1013
Max. group size:     765    Converged:         Yes
Mean group size:     87.9

-----
                Coef.  Std.Err.  z    P>|z|  [0.025 0.975]
-----
Intercept      -0.052    0.064  -0.808  0.419  -0.177  0.074
rating          0.171    0.094   1.827  0.068  -0.012  0.355
IMDb_Episode_Rating -0.119    0.094  -1.265  0.206  -0.303  0.065
IMDb_Episode_Votes  0.000    0.000  32.766  0.000  0.000  0.000
fuzzy_popularity  0.200    0.034   5.847  0.000  0.133  0.267
fuzzy_vote_average -0.024    0.033  -0.726  0.468  -0.088  0.041
Group Var       0.359    0.068

=====
```

Interpretation

Another year's data has refined the model's ability to predict demographic trends and has enhanced the statistical significance and strength of the relationships between predictors and the target variable.

- The IMDb Episode Rating Negative effect has a p-value of 0.051, meaning it is almost not statistically significant.
- The Log-Likelihood is now -12958, which is smaller in magnitude (less negative) than the one-year model.

```
In [328... def describe_target_data(df):
    # Descriptive statistics for 'Viewers (millions)'
    viewers_stats = {
        'Mean': df['Viewers_millions'].mean(),
        'Median': df['Viewers_millions'].median(),
        'Mode': df['Viewers_millions'].mode().values, # mode() can return multiple values
        'Variance': df['Viewers_millions'].var(),
        'Standard Deviation': df['Viewers_millions'].std(),
        'Summary': df['Viewers_millions'].describe()
    }

    # Descriptive statistics for '18-49 Demo'
    demo_stats = {
        'Mean': df['_18_to_49_Demo'].mean(),
        'Median': df['_18_to_49_Demo'].median(),
        'Mode': df['_18_to_49_Demo'].mode().values, # mode() can return multiple values
        'Variance': df['_18_to_49_Demo'].var(),
        'Standard Deviation': df['_18_to_49_Demo'].std(),
        'Summary': df['_18_to_49_Demo'].describe()
    }

    # Printing results
    print("Viewers (millions) Statistics:")
    for key, value in viewers_stats.items():
        print(f"{key}: {value}")

    print("\n18-49 Demo Statistics:")
    for key, value in demo_stats.items():
        print(f"{key}: {value}")

def describe_svod_target_data(df):
    # Descriptive statistics for 'Viewers (millions)'
    viewers_stats = {
        'Mean': df['Minutes (Millions)'].mean(),
        'Median': df['Minutes (Millions)'].median(),
        'Mode': df['Minutes (Millions)'].mode().values, # mode() can return multiple values
        'Variance': df['Minutes (Millions)'].var(),
        'Standard Deviation': df['Minutes (Millions)'].std(),
        'Summary': df['Minutes (Millions)'].describe()
    }

    # Printing results
    print("Viewers (millions) Statistics:")
    for key, value in viewers_stats.items():
        print(f"{key}: {value}")
```

In []:

Show and Graph Regression Results

```
In [329... def show_regression_results(title, df, target_variable_name, loading_df, feature_labels, best_estimator, rmse,
                             r2, mae, coefs_df, X, Y, predictions, residuals):

    print(f"\n\n{title}\n")

    # Suppress the DeprecationWarning
    warnings.filterwarnings("ignore", category=DeprecationWarning)

    # Show variance of Data.
    describe_target_data(df)

    if loading_df is not None and not loading_df.empty:
        # Insert the new column with feature labels into loadings_df_viewers
        if 'Feature_Label' not in loading_df.columns:
            loading_df.insert(0, 'Feature_Label', feature_labels)

    # Print all the results
    print(f"Best Estimator for {target_variable_name}: ", best_estimator)
    print("Root mean square error: ", rmse)
    print("r2: ", r2)
    print("Mean average error: ", mae)

    if loading_df is not None and not loading_df.empty:
        pretty_print_df(loading_df)

    if coefs_df is not None and not coefs_df.empty:
        pretty_print_df(coefs_df)

    # Ensure no Inf values
    predictions = np.where(np.isinf(predictions), np.nan, predictions)
    residuals = np.where(np.isinf(residuals), np.nan, residuals)
    Y = np.where(np.isinf(Y), np.nan, Y)

    # Drop NaNs from the data
    valid_mask = ~np.isnan(predictions) & ~np.isnan(Y)
    predictions = predictions[valid_mask]
    Y = Y[valid_mask]
```

```

# Drop NaNs and Infs from residuals as well
residuals = residuals[valid_mask]

# Scatterplot with Fitted Line
plt.figure(figsize=(10, 6))
sns.scatterplot(x=Y, y=predictions)
plt.plot([np.nanmin(Y), np.nanmax(Y)], [np.nanmin(Y), np.nanmax(Y)], color='red')
plt.xlabel('Actual Values', fontsize = 12)
plt.ylabel('Predicted Values', fontsize = 12)
plt.title(f'Scatterplot with Fitted Line of {target_variable_name}')
plt.show()

# Residual Plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x=predictions, y=residuals)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted Values', fontsize = 12)
plt.ylabel('Residuals', fontsize = 12)
plt.title(f'Residual Plot for {target_variable_name}')
plt.show()

# Suppress the specific FutureWarning
with warnings.catch_warnings():
    warnings.filterwarnings("ignore", category=FutureWarning, message=".*use_inf_as_na option is deprecated.*")

# Histogram of Residuals
plt.figure(figsize=(10, 6))
sns.histplot(residuals, kde=True)
plt.title(f'Histogram of Residuals for {target_variable_name}')
plt.xlabel('Residuals', fontsize = 12)
plt.ylabel('Frequency', fontsize = 12)
plt.show()

# Q-Q Plot
fig = plt.figure(figsize=(10, 6))
sm.qqplot(residuals, line='s', ax=plt.gca())
plt.title('Q-Q Plot')
plt.show()

# Predicted vs. Actual Plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x=predictions, y=Y)
plt.plot([np.nanmin(predictions), np.nanmax(predictions)], [np.nanmin(Y), np.nanmax(Y)], color='red')
plt.xlabel('Predicted Values', fontsize = 12)
plt.ylabel('Actual Values', fontsize = 12)
plt.title(f'Predicted vs. Actual Plot for {target_variable_name}')
plt.show()

# Unsuppress the DeprecationWarning
warnings.filterwarnings("error", category=DeprecationWarning)

def show_svod_regression_results(title, df, target_variable_name, loading_df, feature_labels, best_estimator, rmse,
                                r2, mae, coeffs_df, X, Y, predictions, residuals):
    print(f"\n\n{title}\n\n")

    describe_svod_target_data(df)

    # Suppress the DeprecationWarning
    warnings.filterwarnings("ignore", category=DeprecationWarning)

    if loading_df is not None and not loading_df.empty:
        # Insert the new column with feature labels into loadings_df_viewers
        if 'Feature_Label' not in loading_df.columns:
            loading_df.insert(0, 'Feature_Label', feature_labels)

    # Print all the results
    print(f"Best Estimator for {target_variable_name}: ", best_estimator)
    print("Root mean square error: ", rmse)
    print("r2: ", r2)
    print("Mean average error: ", mae)

    if loading_df is not None and not loading_df.empty:
        pretty_print_df(loading_df)

    pretty_print_df(coeffs_df)
    print("\n")

    # Ensure no Inf values
    predictions = np.where(np.isinf(predictions), np.nan, predictions)
    residuals = np.where(np.isinf(residuals), np.nan, residuals)
    Y = np.where(np.isinf(Y), np.nan, Y)

    # Drop NaNs from the data
    valid_mask = ~np.isnan(predictions) & ~np.isnan(Y)
    predictions = predictions[valid_mask]
    Y = Y[valid_mask]

    # Drop NaNs and Infs from residuals as well
    residuals = residuals[valid_mask]

    # Scatterplot with Fitted Line
    plt.figure(figsize=(10, 6))
    sns.scatterplot(x=Y, y=predictions)
    plt.plot([np.nanmin(Y), np.nanmax(Y)], [np.nanmin(Y), np.nanmax(Y)], color='red')
    plt.xlabel('Actual Values')
    plt.ylabel('Predicted Values')
    plt.title(f'Scatterplot with Fitted Line of {target_variable_name}')

```

```

plt.show()

# Residual Plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x=predictions, y=residuals)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.title(f'Residual Plot for {target_variable_name}')
plt.show()

# Suppress the specific FutureWarning
with warnings.catch_warnings():
    warnings.filterwarnings("ignore", category=FutureWarning, message=".*use_inf_as_na option is deprecated.*")

# Histogram of Residuals
plt.figure(figsize=(10, 6))
sns.histplot(residuals, kde=True)
plt.title(f'Histogram of Residuals for {target_variable_name}')
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.show()

# Q-Q Plot
fig = plt.figure(figsize=(10, 6))
sm.qqplot(residuals, line='s', ax=plt.gca())
plt.title('Q-Q Plot')
plt.show()

# Predicted vs. Actual Plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x=predictions, y=Y)
plt.plot([np.nanmin(predictions), np.nanmax(predictions)], [np.nanmin(Y), np.nanmax(Y)], color='red')
plt.xlabel('Predicted Values')
plt.ylabel('Actual Values')
plt.title(f'Predicted vs. Actual Plot for {target_variable_name}')
plt.show()

# Unsuppress the DeprecationWarning
warnings.filterwarnings("error", category=DeprecationWarning)

```

Split Data into Test/Train Depending on Season

```

In [330... def test_train_over_seasons(df, test_seasons, series_grouping = True, fuzziness=100, genres=False, networks=False, content=False):

    # Define features
    if series_grouping:
        fuzzify_series(df, fuzziness)

    test_data = df[df['season'] > test_seasons]
    training_data = df[df['season'] <= test_seasons]

    if fuzziness != 0:
        columns = select_features(genres=genres, networks=networks, content=content)
        X_train = training_data[columns]
    else:
        columns = select_features(main_labels = series_feature_labels, genres=genres, networks=networks, content=content)
        X_train = training_data[columns]

    Y_train = training_data[['_18_to_49_Demo', 'Viewers_millions']]

    if fuzziness:
        X_test = test_data[columns]
    else:
        X_test = test_data[columns]

    Y_test = test_data[['_18_to_49_Demo', 'Viewers_millions']]

    return X_train, X_test, Y_train, Y_test, df[columns], df[['_18_to_49_Demo', 'Viewers_millions']]

```

Simple Extra Tree Regression per Season

```

In [331... def estimate_future_performance_etr(df, test_seasons):
    # each record in the dataframe corresponds to an episode
    # separate the episodes for the first N seasons

    X_train, X_test, Y_train, Y_test, X, Y = test_train_over_seasons(df, test_seasons)

    # Define the parameters for the ExtraTreesRegressor
    params = {
        'max_depth': 30,
        'max_features': None,
        'min_samples_leaf': 1,
        'min_samples_split': 10,
        'n_estimators': 300
    }

    # Initialize the ExtraTreesRegressor with the specified parameters
    extra_trees = ExtraTreesRegressor(**params)

    # Fit the regressor to the training data
    extra_trees.fit(X_train, Y_train)

    # Predictions

```

```

predictions = extra_trees.predict(X_test)

# Splitting the predictions and actual values into separate arrays for each target variable
predictions_18_to_49_Demo = predictions[:, 0]
predictions_Viewers_millions = predictions[:, 1]
actual_18_to_49_Demo = Y_test['_18_to_49_Demo'].values
actual_Viewers_millions = Y_test['Viewers_millions'].values

# Evaluating _18_to_49_Demo
mse_18_to_49_Demo = mean_squared_error(actual_18_to_49_Demo, predictions_18_to_49_Demo)
rmse_18_to_49_Demo = mse_18_to_49_Demo ** 0.5
r2_18_to_49_Demo = r2_score(actual_18_to_49_Demo, predictions_18_to_49_Demo)
mae_18_to_49_Demo = mean_absolute_error(actual_18_to_49_Demo, predictions_18_to_49_Demo)

# Evaluating Viewers_millions
mse_Viewers_millions = mean_squared_error(actual_Viewers_millions, predictions_Viewers_millions)
rmse_Viewers_millions = mse_Viewers_millions ** 0.5
r2_Viewers_millions = r2_score(actual_Viewers_millions, predictions_Viewers_millions)
mae_Viewers_millions = mean_absolute_error(actual_Viewers_millions, predictions_Viewers_millions)

# Printing results for _18_to_49_Demo
print("Results for '_18_to_49_Demo':")
print("Mean Squared Error:", mse_18_to_49_Demo)
print("Root Mean Square Error:", rmse_18_to_49_Demo)
print("R^2 Score:", r2_18_to_49_Demo)
print("Mean Absolute Error:", mae_18_to_49_Demo)

# Printing results for Viewers_millions
print("\nResults for 'Viewers_millions':")
print("Mean Squared Error:", mse_Viewers_millions)
print("Root Mean Square Error:", rmse_Viewers_millions)
print("R^2 Score:", r2_Viewers_millions)
print("Mean Absolute Error:", mae_Viewers_millions)

```

Simple Random Forest Regression by Season

```

In [332... def estimate_future_performance_rfr(df, test_seasons):
# Separate the episodes by season for training and testing
X_train, X_test, Y_train, Y_test, X, Y = test_train_over_seasons(df, test_seasons)

# Define the parameters for the RandomForestRegressor
params = {
    'max_depth': 30,
    'max_features': 'log2',
    'min_samples_leaf': 1,
    'min_samples_split': 10,
    'n_estimators': 300
}

# Initialize the RandomForestRegressor with the specified parameters
random_forest = RandomForestRegressor(**params)

# Fit the model
random_forest.fit(X_train, Y_train)

# Make predictions
predictions = random_forest.predict(X_test)

# Separate predictions for evaluation
predictions_18_to_49_Demo = predictions[:, 0]
predictions_Viewers_millions = predictions[:, 1]
actual_18_to_49_Demo = Y_test['_18_to_49_Demo'].values
actual_Viewers_millions = Y_test['Viewers_millions'].values

# Evaluating _18_to_49_Demo
mse_18_to_49_Demo = mean_squared_error(actual_18_to_49_Demo, predictions_18_to_49_Demo)
rmse_18_to_49_Demo = mse_18_to_49_Demo ** 0.5
r2_18_to_49_Demo = r2_score(actual_18_to_49_Demo, predictions_18_to_49_Demo)
mae_18_to_49_Demo = mean_absolute_error(actual_18_to_49_Demo, predictions_18_to_49_Demo)

# Evaluating Viewers_millions
mse_Viewers_millions = mean_squared_error(actual_Viewers_millions, predictions_Viewers_millions)
rmse_Viewers_millions = mse_Viewers_millions ** 0.5
r2_Viewers_millions = r2_score(actual_Viewers_millions, predictions_Viewers_millions)
mae_Viewers_millions = mean_absolute_error(actual_Viewers_millions, predictions_Viewers_millions)

# Printing results for _18_to_49_Demo
print("Results for '_18_to_49_Demo':")
print("Mean Squared Error:", mse_18_to_49_Demo)
print("Root Mean Square Error:", rmse_18_to_49_Demo)
print("R^2 Score:", r2_18_to_49_Demo)
print("Mean Absolute Error:", mae_18_to_49_Demo)

# Printing results for Viewers_millions
print("\nResults for 'Viewers_millions':")
print("Mean Squared Error:", mse_Viewers_millions)
print("Root Mean Square Error:", rmse_Viewers_millions)
print("R^2 Score:", r2_Viewers_millions)
print("Mean Absolute Error:", mae_Viewers_millions)

```

Samples Records, Fits to MinMaxScaler, and Implements PCA.

Show Variance of 2-3 Year Data

```
In [333... def fit_sample_and_reduce(sample_rate, df, target, n_components=3, series_grouping = True, fuzziness = 300,
                        genres=False, networks=False, content=False):
    # Resetting index for consistency
    # Ensure df is a DataFrame and reset index
    if isinstance(df, pd.DataFrame):
        df = df.reset_index(drop=True)
    else:
        raise ValueError("df must be a pandas DataFrame")

    # Ensure target is a Series and reset index
    if isinstance(target, pd.Series):
        target = target.reset_index(drop=True)
    else:
        raise ValueError("target must be a pandas Series")

    # Define features
    if series_grouping:
        column_names = select_features(genres=genres, networks=networks, content=content)
        df = fuzzify_series(df, fuzziness)
    else:
        column_names = select_features(main_labels = fuzzy_series_feature_labels, genres=genres, networks=networks, content=content)

    features = df[column_names]

    print(f"fit sample at rate {sample_rate} & reduce to {n_components} using pca. Input features: {column_names}")

    scaler = MinMaxScaler()
    scaled_features = scaler.fit_transform(features)
    scaled_features_df = pd.DataFrame(scaled_features, columns=features.columns)

    # Sampling
    if sample_rate > 0:
        sampled_features_df = scaled_features_df.sample(frac=sample_rate, random_state=42)
    else:
        sampled_features_df = scaled_features_df

    # Ensuring target index matches the sampled features
    sampled_target = target.loc[sampled_features_df.index].reset_index(drop=True)
    sampled_features_df = sampled_features_df.reset_index(drop=True)

    if n_components > 0:
        # Applying PCA
        pca = PCA(n_components=n_components)
        reduced_features_sampled = pca.fit_transform(sampled_features_df)
        pca_columns = [f"PC{i+1}" for i in range(n_components)] # Dynamic column names based on number of components

        # Loadings are in pca.components_
        loadings_df = pd.DataFrame(pca.components_.T, columns=pca_columns, index=features.columns)

        reduced_features_sampled_df = pd.DataFrame(reduced_features_sampled, columns=pca_columns)
        return reduced_features_sampled_df, sampled_target, loadings_df, pca
    else:
        return sampled_features_df, sampled_target
```

```
In [334... def fit_data_for_testing(test_df, fitted_pca, series_grouping = True, fuzziness = 300, genres=False, networks=False, content=False):

    # Define features
    if series_grouping:
        column_names = select_features(genres=genres, networks=networks, content=content)
        test_df = fuzzify_series(test_df, fuzziness)
    else:
        column_names = select_features(main_labels = fuzzy_series_feature_labels, genres=genres, networks=networks, content=content)

    features = df[column_names]

    scaler = MinMaxScaler()
    scaled_features = scaler.fit_transform(features)

    if fitted_pca:
        # Applying PCA
        reduced_features = fitted_pca.fit_transform(scaled_features)
        pca_columns = [f"PC{i+1}" for i in range(n_components)] # Dynamic column names based on number of components
        reduced_features_sampled_df = pd.DataFrame(reduced_features_sampled, columns=pca_columns)
        return reduced_features_sampled_df
    else:
        scaled_features_df = pd.DataFrame(scaled_features, columns=features.columns)
        return sampled_features_df
```

```
In [335... def svod_fit_reduce(df, n_components=3, fuzziness=300, genres=False, networks=False, content=False):
    # Resetting index for consistency
    # Ensure df is a DataFrame and reset index
    if isinstance(df, pd.DataFrame):
        df = df.reset_index(drop=True)
    else:
        raise ValueError("df must be a pandas DataFrame")

    # Define features
    column_names = select_features(main_labels=svod_feature_labels, genres=genres, networks=networks, content=content)
    df = fuzzify_series(df, fuzziness)

    features = df[column_names]

    scaler = MinMaxScaler()
    scaled_features = scaler.fit_transform(features)
    scaled_features_df = pd.DataFrame(scaled_features, columns=features.columns)

    if n_components > 0:
```

```
# Applying PCA
pca = PCA(n_components=n_components)
reduced_features = pca.fit_transform(scaled_features_df)
pca_columns = [f"PC{i + 1}" for i in range(n_components)] # Dynamic column names based on number of components

# Loadings are in pca.components_
loadings_df = pd.DataFrame(pca.components_.T, columns=pca_columns, index=features.columns)

reduced_features_sampled_df = pd.DataFrame(reduced_features, columns=pca_columns)
return reduced_features_sampled_df, loadings_df, pca
else:
    return scaled_features_df
```

```
In [336... print("\ndata of series running two seasons or more")
describe_target_data(two_seasons_or_more_df)
print("\ndata of series running three seasons or more")
describe_target_data(three_seasons_or_more_df)
```

```
data of series running two seasons or more
Viewers (millions) Statistics:
Mean: 3.0117604958522115
Median: 1.99
Mode: [0.624]
Variance: 10.211331092712312
Standard Deviation: 3.195517343516119
Summary: count    38270.000000
mean             3.011760
std              3.195517
min              0.006000
25%             0.671000
50%             1.990000
75%             4.093750
max             26.972000
Name: Viewers_millions, dtype: float64
```

```
18-49 Demo Statistics:
Mean: 0.7473094735398338
Median: 0.5
Mode: [0.7]
Variance: 0.7039714481405789
Standard Deviation: 0.8390300639074734
Summary: count    38270.000000
mean             0.747309
std              0.839030
min              0.000000
25%             0.200000
50%             0.500000
75%             1.060000
max             9.300000
Name: _18_to_49_Demo, dtype: float64
```

```
data of series running three seasons or more
Viewers (millions) Statistics:
Mean: 3.2304460719514516
Median: 2.256
Mode: [0.624]
Variance: 10.80646435590437
Standard Deviation: 3.2873187183332817
Summary: count    33835.000000
mean             3.230446
std              3.287319
min              0.008000
25%             0.795000
50%             2.256000
75%             4.540000
max             26.972000
Name: Viewers_millions, dtype: float64
```

```
18-49 Demo Statistics:
Mean: 0.8077153240688486
Median: 0.58
Mode: [0.7]
Variance: 0.7502568457920092
Standard Deviation: 0.8661736810778825
Summary: count    33835.000000
mean             0.807715
std              0.866174
min              0.000000
25%             0.240000
50%             0.580000
75%             1.100000
max             9.300000
Name: _18_to_49_Demo, dtype: float64
```

2. Use Regression to Project Future Viewers and Ratings

- Extra Tree Regression
- 1 Season to project future seasons
- 2 Seasons to project future seasons

```
In [337... print("Estimate Using Only First Season")
estimate_future_performance_etr(two_seasons_or_more_df, 1)

print("\nEstimate Using First Two Seasons")
estimate_future_performance_etr(three_seasons_or_more_df, 2)
```

```

Estimate Using Only First Season
Ranking 556 series in 6.0 classification pockets.
Results for ' _18_to_49_Demo':
Mean Squared Error: 0.6836859364317913
Root Mean Square Error: 0.8268530319420685
R^2 Score: 0.07605962275864708
Mean Absolute Error: 0.5092520954950781

```

```

Results for 'Viewers_millions':
Mean Squared Error: 9.5595248895314
Root Mean Square Error: 3.09184813493991
R^2 Score: 0.09401638122054345
Mean Absolute Error: 2.093296145798781

```

```

Estimate Using First Two Seasons
Ranking 385 series in 4.0 classification pockets.
Results for ' _18_to_49_Demo':
Mean Squared Error: 0.733016368033973
Root Mean Square Error: 0.8561637507124282
R^2 Score: 0.08727952685101481
Mean Absolute Error: 0.5143981151260241

```

```

Results for 'Viewers_millions':
Mean Squared Error: 9.902221783460414
Root Mean Square Error: 3.1467795892722474
R^2 Score: 0.12714655395996854
Mean Absolute Error: 2.0900159489553416

```

```

In [338... print("Estimate Using Only First Season")
estimate_future_performance_rfr(two_seasons_or_more_df, 1)

print("\nEstimate Using First Two Seasons")
estimate_future_performance_rfr(three_seasons_or_more_df, 2)

```

```

Estimate Using Only First Season
Ranking 556 series in 6.0 classification pockets.
Results for ' _18_to_49_Demo':
Mean Squared Error: 0.6667612468580543
Root Mean Square Error: 0.8165544971758188
R^2 Score: 0.09893182655307375
Mean Absolute Error: 0.5095441547180061

```

```

Results for 'Viewers_millions':
Mean Squared Error: 9.227432506781046
Root Mean Square Error: 3.037668926460065
R^2 Score: 0.12548972975722017
Mean Absolute Error: 2.110629560072917

```

```

Estimate Using First Two Seasons
Ranking 385 series in 4.0 classification pockets.
Results for ' _18_to_49_Demo':
Mean Squared Error: 0.710021648167926
Root Mean Square Error: 0.8426278230440329
R^2 Score: 0.11591156361215571
Mean Absolute Error: 0.5121689722323666

```

```

Results for 'Viewers_millions':
Mean Squared Error: 9.8889385961423
Root Mean Square Error: 3.144668280779755
R^2 Score: 0.1283174301611436
Mean Absolute Error: 2.159551488648213

```

Analysis of Simple Regression

- RMSE is within one standard deviation.
- Extra tree regression performs a little better.
- Limited to very basic features (no genre, network, or content)
- Also no sampling.

3. Survival Model using PCA, Feature Engineering, and ExtraTreesClassifier.

Test/Train for Classifier Modeling

- Will a series be cancelled or ended?

```

In [339... series_survival_df = series_episode_subset_modeling_df[(series_episode_subset_modeling_df['status_Canceled']!=True) |
                                                         (series_episode_subset_modeling_df['status_Ended']!=True)].copy()
show_df_info(series_survival_df, "series survival logical modeling data")

```

Dataframe information for series survival logical modeling data:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	rating	float64	0
5	Episode_Votes	int32	0
6	number_of_seasons	float64	0
7	number_of_episodes	float64	0
8	Series_Votes	float64	0
9	vote_average	float64	0
10	in_production	bool	0
11	popularity	float64	0
12	Sci-Fi & Fantasy	bool	0
13	Drama	bool	0
14	Action & Adventure	bool	0
15	Crime	bool	0
16	Mystery	bool	0
17	Comedy	bool	0
18	Animation	bool	0
19	Family	bool	0
20	War & Politics	bool	0
21	Soap	bool	0
22	Western	bool	0
23	Kids	bool	0
24	Documentary	bool	0
25	Reality	bool	0
26	Romance	bool	0
27	Adult	bool	0
28	Biography	bool	0
29	Film-Noir	bool	0
30	Game-Show	bool	0
31	History	bool	0
32	Horror	bool	0
33	Musical	bool	0
34	Music	bool	0
35	News	bool	0
36	Short	bool	0
37	Sport	bool	0
38	Talk-Show	bool	0
39	Thriller	bool	0
40	HBO	bool	0
41	Netflix	bool	0
42	AMC	bool	0
43	FOX	bool	0
44	The CW	bool	0
45	ABC	bool	0
46	Disney+	bool	0
47	CBS	bool	0
48	Prime Video	bool	0
49	Adult Swim	bool	0
50	NBC	bool	0
51	Hulu	bool	0
52	USA Network	bool	0
53	Cinemax	bool	0
54	Comedy Central	bool	0
55	Showtime	bool	0
56	Paramount+	bool	0
57	Paramount Network	bool	0
58	Lifetime	bool	0
59	Starz	bool	0
60	Telemundo	bool	0
61	Apple TV+	bool	0
62	HBO Max	bool	0
63	BET	bool	0
64	Hallmark Channel	bool	0
65	Peacock	bool	0
66	_18_to_49_Demo	float64	0
67	Viewers_millions	float64	0
68	sex_factor	int64	0
69	violence_factor	int64	0
70	profanity_factor	int64	0
71	drugs_factor	int64	0
72	intense_factor	int64	0
73	IMDb_Episode_Rating	float64	0
74	IMDb_Episode_Votes	int64	0
75	status_Canceled	bool	0
76	status_Ended	bool	0
77	status_Returning Series	bool	0
78	type_Documentary	bool	0
79	type_Miniseries	bool	0
80	type_News	bool	0
81	type_Reality	bool	0
82	type_Scripted	bool	0
83	type_Talk Show	bool	0
84	ranking_of_Series_Votes	float64	0
85	ranking_of_vote_average	float64	0
86	ranking_of_popularity	float64	0
87	fuzzy_Series_Votes	float64	0
88	fuzzy_vote_average	float64	0
89	fuzzy_popularity	float64	0

There are 27566 rows and 90 columns of information.

```
In [340... def survival_test(target, pca_components = 5, series_grouping = True, fuzziness = 300, genres=False, networks=False, content=False):
    # Record the start time
    start_time = time.time()

    # Define the parameters for the ExtraTreesClassifier
    params = {
        'max_depth': 30,
        'max_features': None,
        'min_samples_leaf': 1,
        'min_samples_split': 10,
        'n_estimators': 300
    }

    # Initialize the ExtraTreesClassifier with the specified parameters
    extra_trees = ExtraTreesClassifier(**params)

    if pca_components > 0:
        X, Y, loadings_df, pca = fit_sample_and_reduce(0, series_survival_df, target, pca_components, series_grouping,
                                                    fuzziness, genres=genres, networks=networks, content=content)
    else:
        X, Y = fit_sample_and_reduce(0, series_survival_df, target, 0, series_grouping, fuzziness, genres=genres, networks=networks, content=content)

    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2,
                                                        random_state=42)
    X_train, X_valid, Y_train, Y_valid = train_test_split(X_train, Y_train,
                                                         test_size=0.25, random_state=42)

    # Training data in X_train and corresponding labels in y_train
    extra_trees.fit(X_train, Y_train)
    predictions = extra_trees.predict(X_test)

    # Record the end time
    end_time = time.time()

    # X_train is the training dataset
    num_rows, num_columns = X_train.shape

    print("Number of rows in X_train:", num_rows)
    print("Number of columns in X_train:", num_columns)

    # Calculate evaluation metrics
    extra_trees_accuracy = accuracy_score(Y_test, predictions)
    extra_trees_precision = precision_score(Y_test, predictions)
    extra_trees_recall = recall_score(Y_test, predictions)
    f1_score = 2 * (extra_trees_precision * extra_trees_recall) / (extra_trees_recall
                                                                + extra_trees_precision)

    # Calculate confusion matrix
    conf_matrix = confusion_matrix(Y_test, predictions)
    tn, fp, fn, tp = conf_matrix.ravel()
    # Calculate specificity
    specificity = tn / (tn + fp)

    print("Improved Extra Trees Classifier Accuracy:", extra_trees_accuracy)
    print("Improved Extra Trees Classifier Precision:", extra_trees_precision)
    print("Improved Extra Trees Classifier Recall:", extra_trees_recall)
    print("Improved Extra Trees Classifier F1 Score:", f1_score)
    print("Improved Extra Trees Classifier Specificity:", specificity)

    # Calculate the elapsed time
    elapsed_time = end_time - start_time

    feature_importances = extra_trees.feature_importances_
    features = X_train.columns

    # Create a DataFrame for easier visualization of feature importances
    importance_df = pd.DataFrame({
        'Feature': features,
        'Importance': feature_importances
    })
    importance_df = importance_df.sort_values(by='Importance', ascending=False) # Sort by importance

    print("Elapsed time:", elapsed_time, "seconds")
    if pca_components > 0:
        return extra_trees, conf_matrix, extra_trees_accuracy, extra_trees_precision, extra_trees_recall, specificity, importance_df, loadings_df, pca
    else:
        return extra_trees, conf_matrix, extra_trees_accuracy, extra_trees_precision, extra_trees_recall, specificity, importance_df

In [341... # Sample 50 random records with series grouping of 50.
random_df = series_survival_df.sample(n=50, random_state = 42).copy()
fuzzify_series(random_df, fuzziness = 50)
pretty_print_df(random_df, rows=50, interesting_columns=select_features())
```

Ranking 48 series in 1.0 classification pockets.

rating	Episode_Votes	fuzzy_vote_average	fuzzy_popularity	IMDb_Episode_Rating	IMDb_Episode_Votes
7.4	1010.0	0.0	0.0	7.4	1000.0
6.0	81.0	0.0	0.0	6.0	81.0
7.8	287.0	0.0	0.0	7.8	287.0
8.8	761.0	0.0	0.0	8.8	758.0
7.6	223.0	0.0	0.0	7.6	222.0
9.2	9538.0	0.0	0.0	9.2	9500.0
6.2	41.0	0.0	0.0	6.2	41.0
8.6	36.0	0.0	0.0	8.6	35.0
8.5	3455.0	0.0	0.0	8.5	3500.0
7.1	1397.0	0.0	0.0	7.1	1400.0
8.7	3685.0	0.0	0.0	8.7	3700.0
8.5	47.0	0.0	0.0	8.5	47.0
7.8	219.0	0.0	0.0	7.8	219.0
7.1	558.0	0.0	0.0	7.1	558.0
7.1	25.0	0.0	0.0	7.1	25.0
8.0	3459.0	0.0	0.0	8.0	3500.0
7.7	10.0	0.0	0.0	7.7	10.0
6.4	747.0	0.0	0.0	6.4	744.0
7.1	481.0	0.0	0.0	7.1	481.0
5.3	29.0	0.0	0.0	5.3	29.0
7.5	135.0	0.0	0.0	7.5	135.0
7.6	164.0	0.0	0.0	7.6	164.0
6.6	106.0	0.0	0.0	6.6	105.0
8.1	4122.0	0.0	0.0	8.1	4100.0
7.5	141.0	0.0	0.0	7.5	142.0
7.5	710.0	0.0	0.0	7.5	709.0
6.9	259.0	0.0	0.0	6.9	259.0
8.0	128.0	0.0	0.0	8.0	128.0
7.5	196.0	0.0	0.0	7.5	195.0
8.2	190.0	0.0	0.0	8.2	189.0
7.5	6772.0	0.0	0.0	7.5	6800.0
7.9	2971.0	0.0	0.0	7.9	3000.0
7.9	1390.0	0.0	0.0	7.9	1400.0
7.9	102.0	0.0	0.0	7.9	102.0
7.8	378.0	0.0	0.0	7.8	378.0
8.1	456.0	0.0	0.0	8.1	456.0
7.9	99.0	0.0	0.0	7.9	99.0
7.7	822.0	0.0	0.0	7.7	820.0
7.0	752.0	0.0	0.0	7.0	751.0
6.7	46.0	0.0	0.0	6.7	46.0
8.0	46.0	0.0	0.0	8.0	46.0
8.3	416.0	0.0	0.0	8.4	415.0
6.4	247.0	0.0	0.0	6.4	247.0
7.8	1381.0	0.0	0.0	7.8	1400.0
7.5	378.0	0.0	0.0	7.5	377.0
6.7	1128.0	0.0	0.0	6.7	1100.0
7.9	616.0	0.0	0.0	7.9	615.0
8.6	1090.0	0.0	0.0	8.6	1100.0
7.8	410.0	0.0	0.0	7.8	409.0
8.4	282.0	0.0	0.0	8.4	282.0

```

In [342... def survival_run(pca_components, fuzziness = 300):
    print("Attempt to Predict Status Canceled\n")

    if pca_components > 0:
        fitted_classifier_canceled, conf_matrix_canceled, extra_trees_accuracy_canceled, extra_trees_precision_canceled, \
            extra_trees_recall_canceled, specificity_canceled, importance_df_canceled, loadings_df_canceled, pca_canceled = \
            survival_test(series_survival_df['status_Canceled'], pca_components, fuzziness!=0, fuzziness)
    else:
        fitted_classifier_canceled, conf_matrix_canceled, extra_trees_accuracy_canceled, extra_trees_precision_canceled, \
            extra_trees_recall_canceled, specificity_canceled, importance_df_canceled = \
            survival_test(series_survival_df['status_Canceled'], 0, fuzziness!=0, fuzziness)

    print("\n\nAttempt to Predict Status Ended\n")
    if pca_components > 0:
        fitted_classifier_ended, conf_matrix_ended, extra_trees_accuracy_ended, extra_trees_precision_ended, extra_trees_recall_ended, \
            specificity_ended, importance_df_ended, loadings_df_ended, pca_ended = \
            survival_test(series_survival_df['status_Ended'], pca_components, fuzziness!=0, fuzziness)
    else:
        fitted_classifier_ended, conf_matrix_ended, extra_trees_accuracy_ended, extra_trees_precision_ended, extra_trees_recall_ended, \
            specificity_ended, importance_df_ended = survival_test(series_survival_df['status_Ended'], 0, fuzziness!=0, fuzziness)

    if pca_components > 0:
        return fitted_classifier_canceled, fitted_classifier_ended, conf_matrix_canceled, conf_matrix_ended, \
            importance_df_canceled, importance_df_ended, loadings_df_canceled, loadings_df_ended, pca_canceled, pca_ended
    else:
        return fitted_classifier_canceled, fitted_classifier_ended, conf_matrix_canceled, conf_matrix_ended, \
            importance_df_canceled, importance_df_ended

```

```

In [343... def graph_survival_confusion(conf_matrix_canceled, conf_matrix_ended):
    # Define true and predicted labels for each matrix
    true_labels_canceled = ['Actually Not Canceled', 'Actually Canceled']
    predicted_labels_canceled = ['Predicted Not Canceled', 'Predicted Canceled']

    true_labels_ended = ['Actually Not Finished', 'Actually Finished']
    predicted_labels_ended = ['Predicted Not Finished', 'Predicted Finished']

    # Create a figure to hold the subplots
    plt.figure(figsize=(24, 12)) # Adjust the overall size of the figure

    # Plot the first confusion matrix

```

```

plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st subplot
sns.heatmap(conf_matrix_canceled, annot=True, cmap='Reds', fmt='d',
            xticklabels=predicted_labels_canceled, yticklabels=true_labels_canceled,
            annot_kws={"size": 20}) # Adjust annotation text size
plt.xlabel('Predicted labels', fontsize=18)
plt.ylabel('True labels', fontsize=18)
plt.title('Confusion Matrix for Canceled Predictions', fontsize=20)
plt.xticks(fontsize=15)
plt.yticks(fontsize=15)

# Plot the second confusion matrix
plt.subplot(1, 2, 2) # 1 row, 2 columns, 2nd subplot
sns.heatmap(conf_matrix_ended, annot=True, cmap='Greens', fmt='d',
            xticklabels=predicted_labels_ended, yticklabels=true_labels_ended,
            annot_kws={"size": 20}) # Adjust annotation text size
plt.xlabel('Predicted labels', fontsize=18)
plt.ylabel('True labels', fontsize=18)
plt.title('Confusion Matrix for Finished Predictions', fontsize=20)
plt.xticks(fontsize=15)
plt.yticks(fontsize=15)

# Display the plots
plt.show()

```

```

In [344... def graph_survival_importance(importance_df):
    pretty_print_df(importance_df)

    warnings.filterwarnings("ignore", category=DeprecationWarning)

    # Plotting the feature importances for better visual interpretation
    plt.figure(figsize=(10, 6))
    sns.barplot(x='Importance', y='Feature', data=importance_df.head(10))
    plt.title('Feature Importances')
    plt.xlabel('Importance')
    plt.ylabel('Features')
    plt.show()

    warnings.filterwarnings("error", category=DeprecationWarning)

```

```

In [345... def show_pca_heatmap(pca_components, loadings_df):
    if pca_components > 0:
        plt.figure(figsize=(12, 6))
        sns.heatmap(loadings_df, annot=True, cmap='coolwarm')
        plt.title('PCA Component Loadings Heat Map')
        plt.show()

```

```

In [346... pca_components = 5
fuzziness = 300

fitted_classifier_canceled_300, fitted_classifier_ended_300, conf_matrix_canceled_300, conf_matrix_ended_300, \
importance_df_canceled_300, importance_df_ended_300, loadings_df_canceled_300, loadings_df_ended_300, \
pca_canceled_300, pca_ended_300 = survival_run(pca_components, fuzziness)

```

Attempt to Predict Status Canceled

Ranking 585 series in 2.0 classification pockets.
fit sample at rate 0 & reduce to 5 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']
Number of rows in X_train: 16539
Number of columns in X_train: 5
Improved Extra Trees Classifier Accuracy: 0.749183895538629
Improved Extra Trees Classifier Precision: 0.6807489198271723
Improved Extra Trees Classifier Recall: 0.6638576779026217
Improved Extra Trees Classifier F1 Score: 0.6721972031287035
Improved Extra Trees Classifier Specificity: 0.8031379514505624
Elapsed time: 3.4070026874542236 seconds

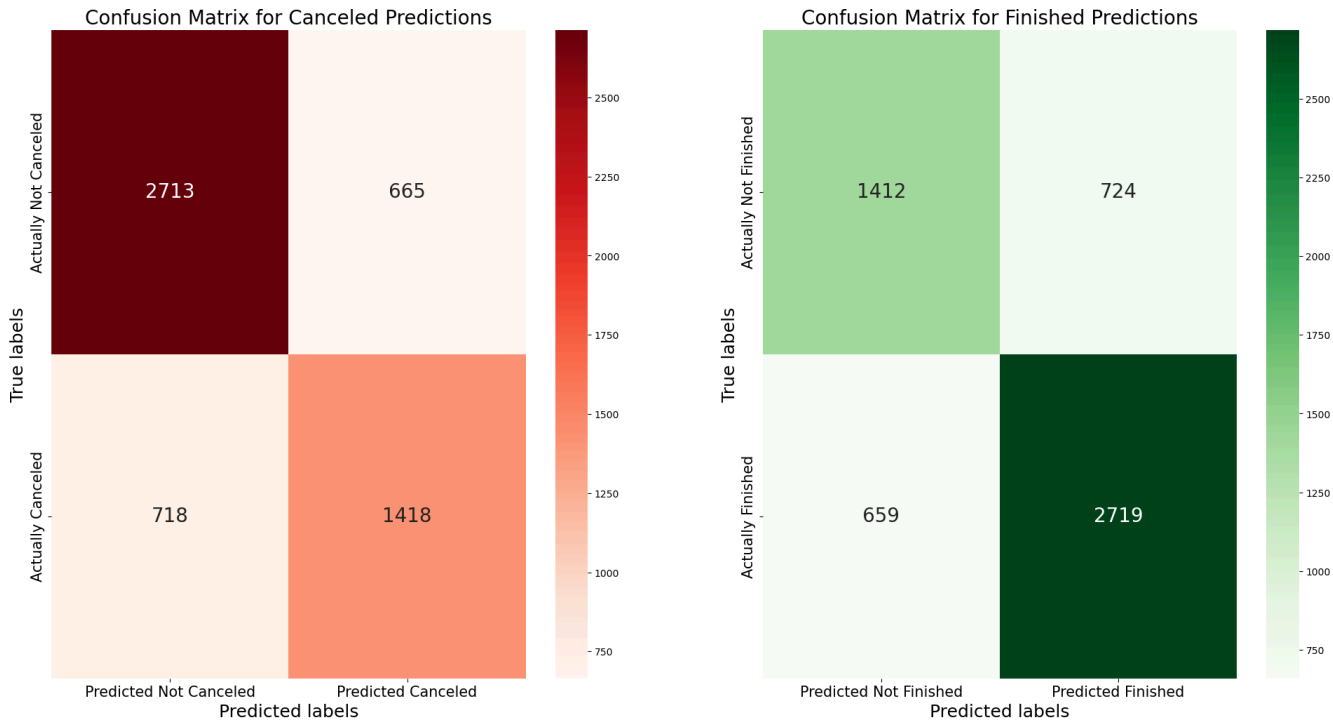
Attempt to Predict Status Ended

Ranking 585 series in 2.0 classification pockets.
fit sample at rate 0 & reduce to 5 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']
Number of rows in X_train: 16539
Number of columns in X_train: 5
Improved Extra Trees Classifier Accuracy: 0.749183895538629
Improved Extra Trees Classifier Precision: 0.7897182689514958
Improved Extra Trees Classifier Recall: 0.8049141503848432
Improved Extra Trees Classifier F1 Score: 0.797243805893564
Improved Extra Trees Classifier Specificity: 0.6610486891385767
Elapsed time: 3.308445692062378 seconds

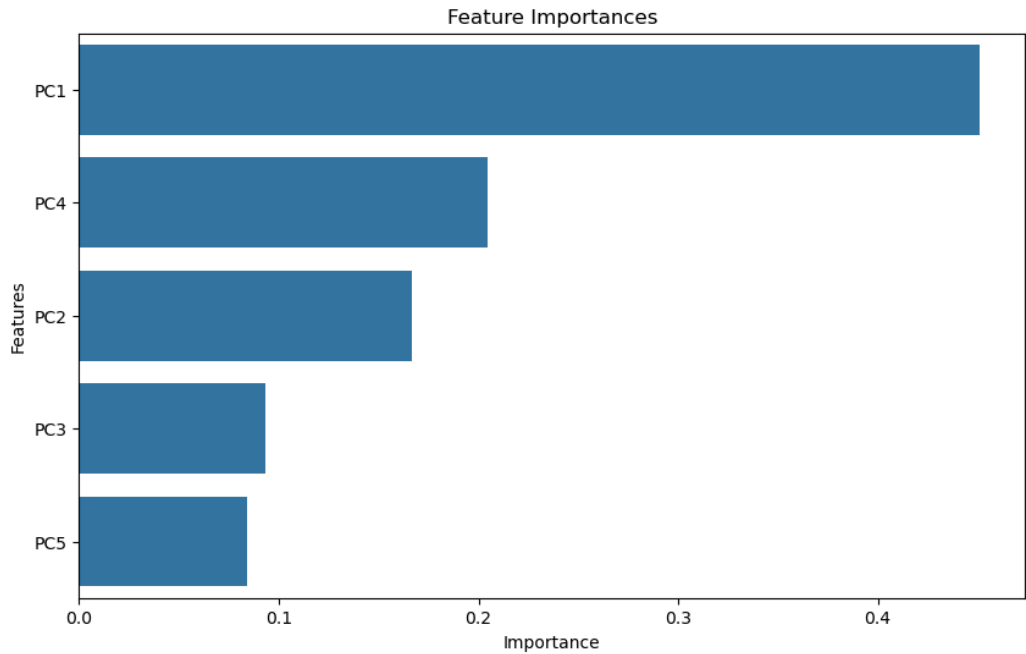
```

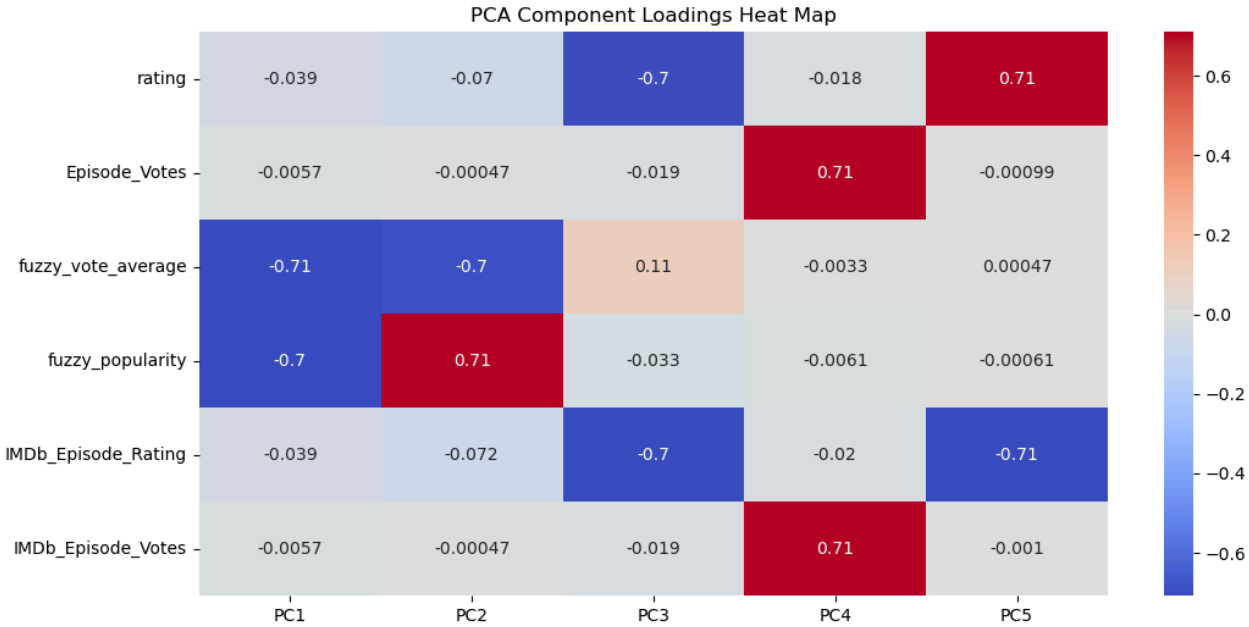
In [347... graph_survival_confusion(conf_matrix_canceled_300, conf_matrix_ended_300)
graph_survival_importance(importance_df_canceled_300)
show_pca_heatmap(pca_components, loadings_df_canceled_300)
graph_survival_importance(importance_df_ended_300)
show_pca_heatmap(pca_components, loadings_df_ended_300)

```

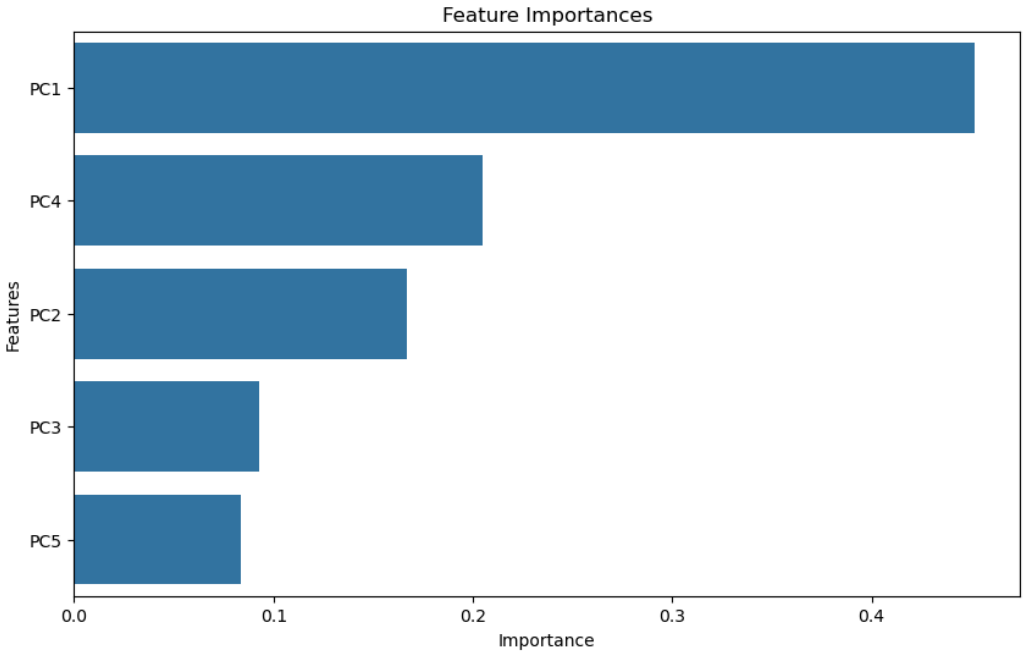


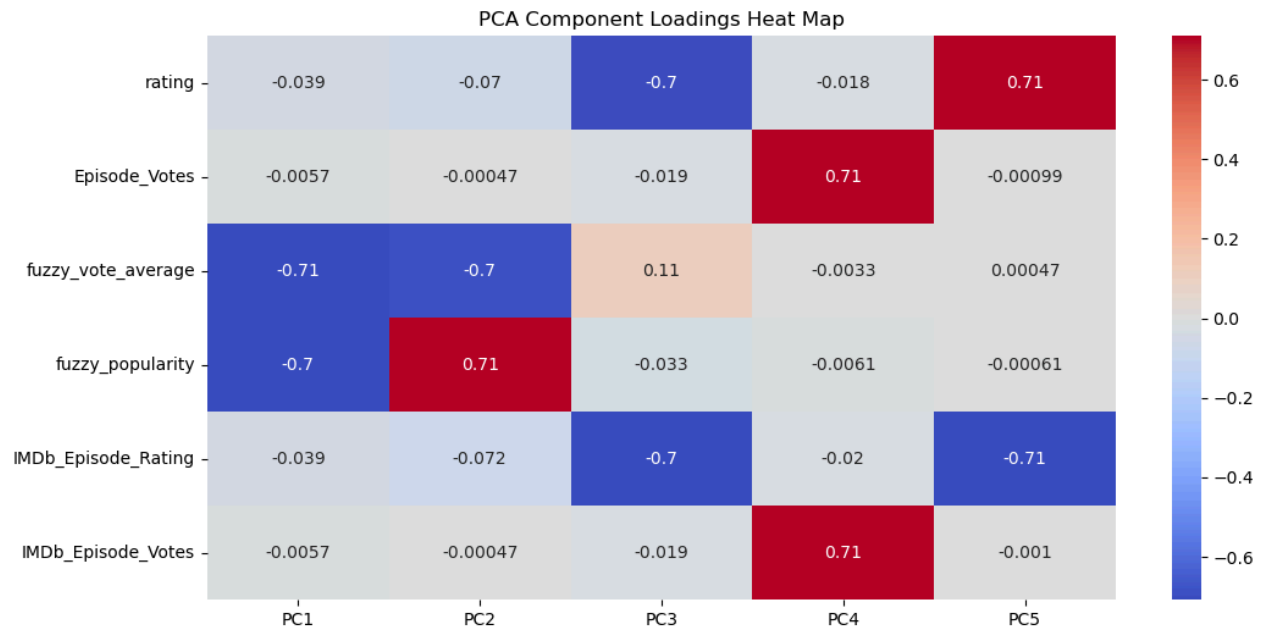
Feature	Importance
PC1	0.4512498885862017
PC4	0.2044784847810241
PC2	0.16688354606512856
PC3	0.0935106434444216
PC5	0.08387743712322408





Feature	Importance
PC1	0.4519542201310644
PC4	0.2047018572763432
PC2	0.16677817456887797
PC3	0.09291078432343358
PC5	0.08365496370028074





In [348...

```
pca_components = 5
fuzziness = 100

fitted_classifier_canceled_100, fitted_classifier_ended_100, conf_matrix_canceled_100, conf_matrix_ended_100, \
importance_df_canceled_100, importance_df_ended_100, loadings_df_canceled_100, loadings_df_ended_100, \
pca_canceled_100, pca_ended_100 = survival_run(pca_components, fuzziness)
```

Attempt to Predict Status Canceled

Ranking 585 series in 6.0 classification pockets.

fit sample at rate 0 & reduce to 5 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']

Number of rows in X_train: 16539

Number of columns in X_train: 5

Improved Extra Trees Classifier Accuracy: 0.8407689517591586

Improved Extra Trees Classifier Precision: 0.7975402081362346

Improved Extra Trees Classifier Recall: 0.7893258426966292

Improved Extra Trees Classifier F1 Score: 0.7934117647058824

Improved Extra Trees Classifier Specificity: 0.8732978093546477

Elapsed time: 3.009153127670288 seconds

Attempt to Predict Status Ended

Ranking 585 series in 6.0 classification pockets.

fit sample at rate 0 & reduce to 5 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']

Number of rows in X_train: 16539

Number of columns in X_train: 5

Improved Extra Trees Classifier Accuracy: 0.8398621690243018

Improved Extra Trees Classifier Precision: 0.8676687297376953

Improved Extra Trees Classifier Recall: 0.8715216104203671

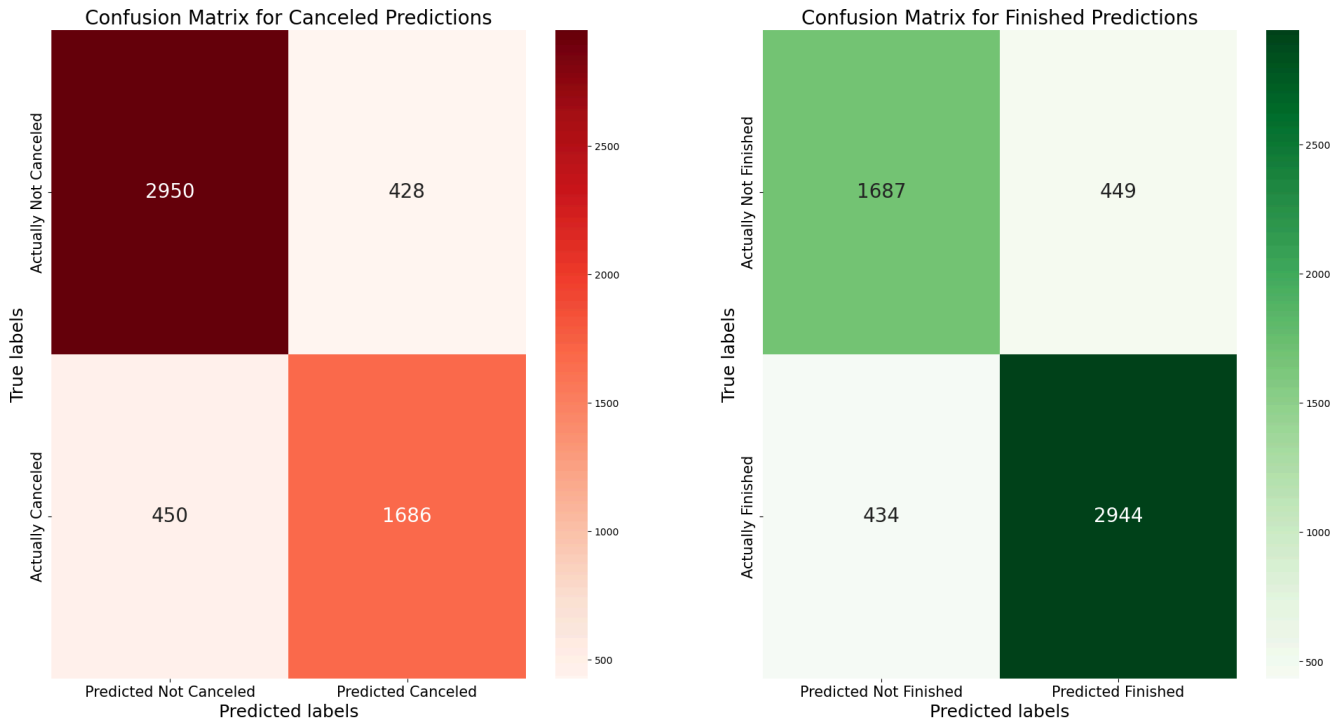
Improved Extra Trees Classifier F1 Score: 0.8695909023777876

Improved Extra Trees Classifier Specificity: 0.7897940074906367

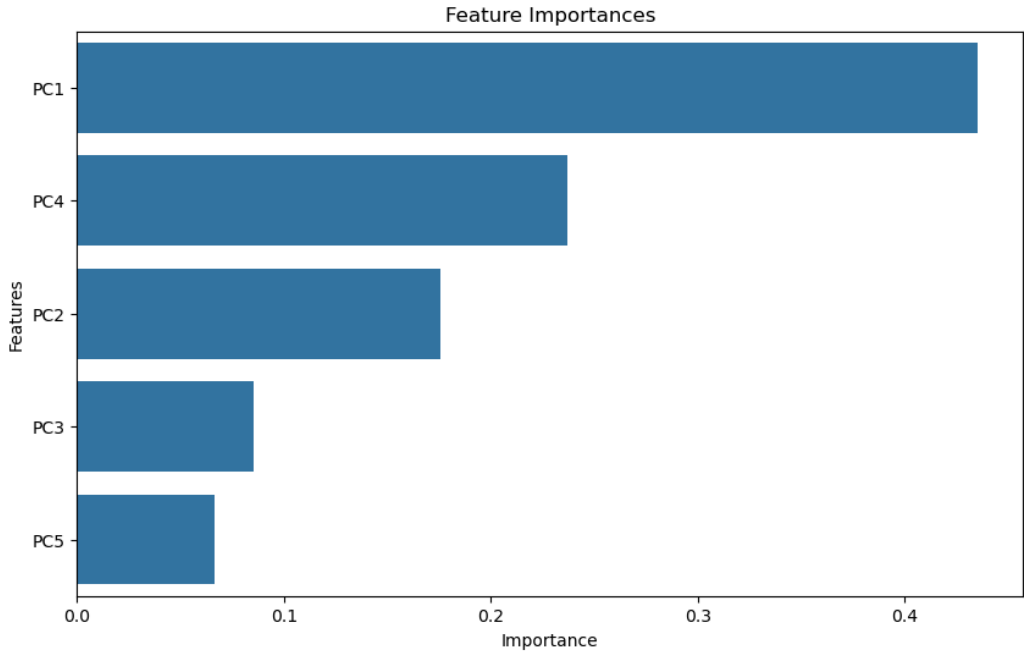
Elapsed time: 2.8948895931243896 seconds

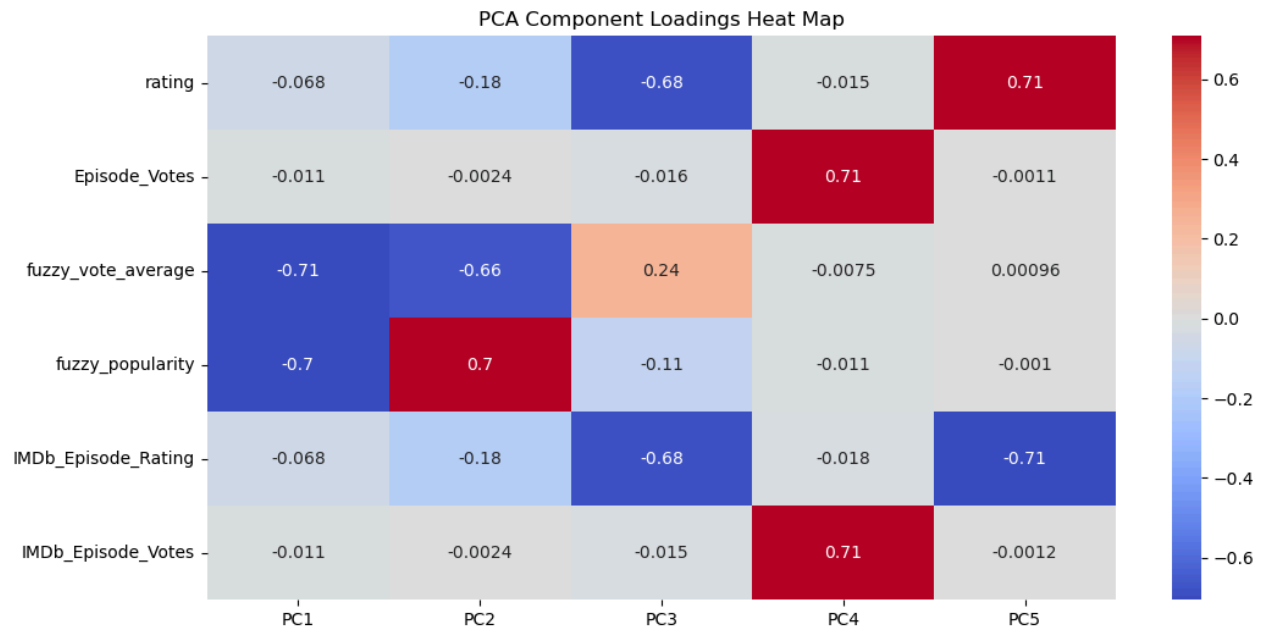
In [349...

```
graph_survival_confusion(conf_matrix_canceled_100, conf_matrix_ended_100)
graph_survival_importance(importance_df_canceled_100)
show_pca_heatmap(pca_components, loadings_df_canceled_100)
```



Feature	Importance
PC1	0.4354051671194713
PC4	0.23699057553934072
PC2	0.17577644603255188
PC3	0.08531669343123897
PC5	0.06651111787739701





```
In [350... pca_components = 5
fuzziness = 50

fitted_classifier_canceled_50, fitted_classifier_ended_50, conf_matrix_canceled_50, conf_matrix_ended_50, \
importance_df_canceled_50, importance_df_ended_50, loadings_df_canceled_50, loadings_df_ended_50, \
pca_canceled_50, pca_ended_50 = survival_run(pca_components, fuzziness)
```

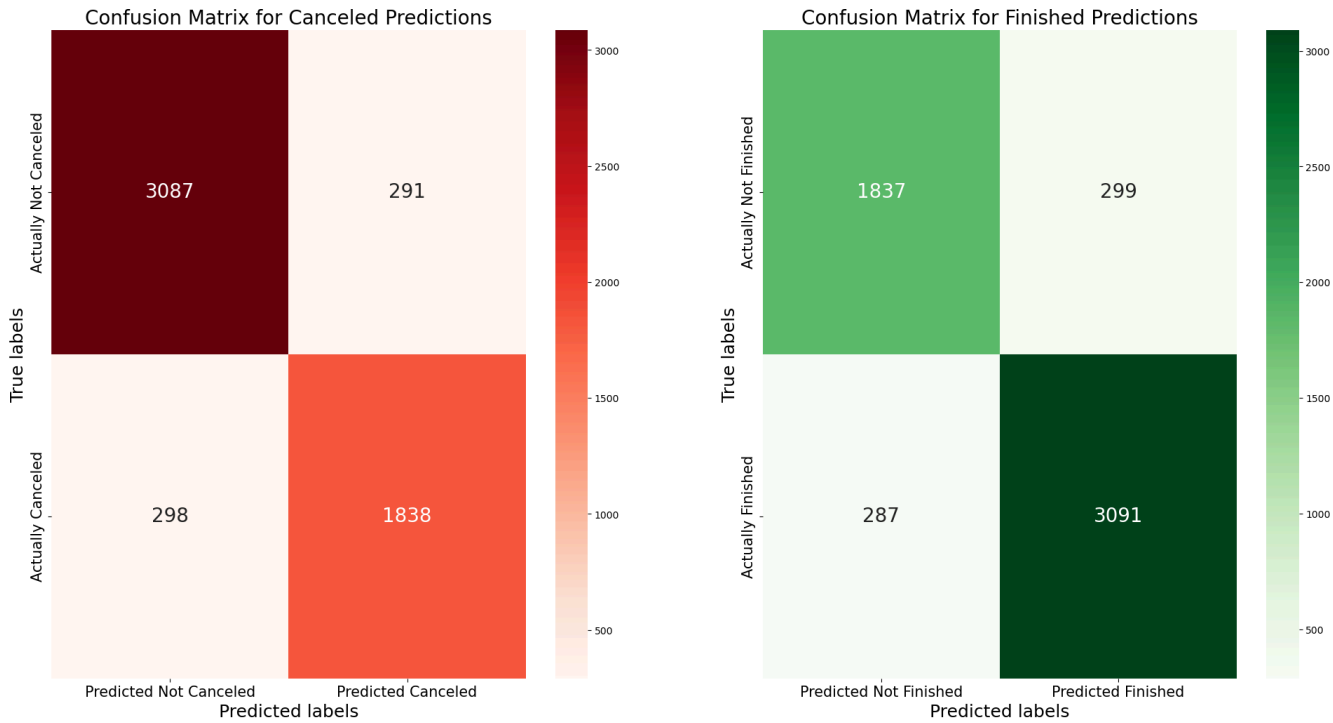
Attempt to Predict Status Canceled

Ranking 585 series in 12.0 classification pockets.
fit sample at rate 0 & reduce to 5 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']
Number of rows in X_train: 16539
Number of columns in X_train: 5
Improved Extra Trees Classifier Accuracy: 0.8931809938338774
Improved Extra Trees Classifier Precision: 0.8633161108501644
Improved Extra Trees Classifier Recall: 0.8604868913857678
Improved Extra Trees Classifier F1 Score: 0.8618991793669403
Improved Extra Trees Classifier Specificity: 0.9138543516873889
Elapsed time: 2.7539448738098145 seconds

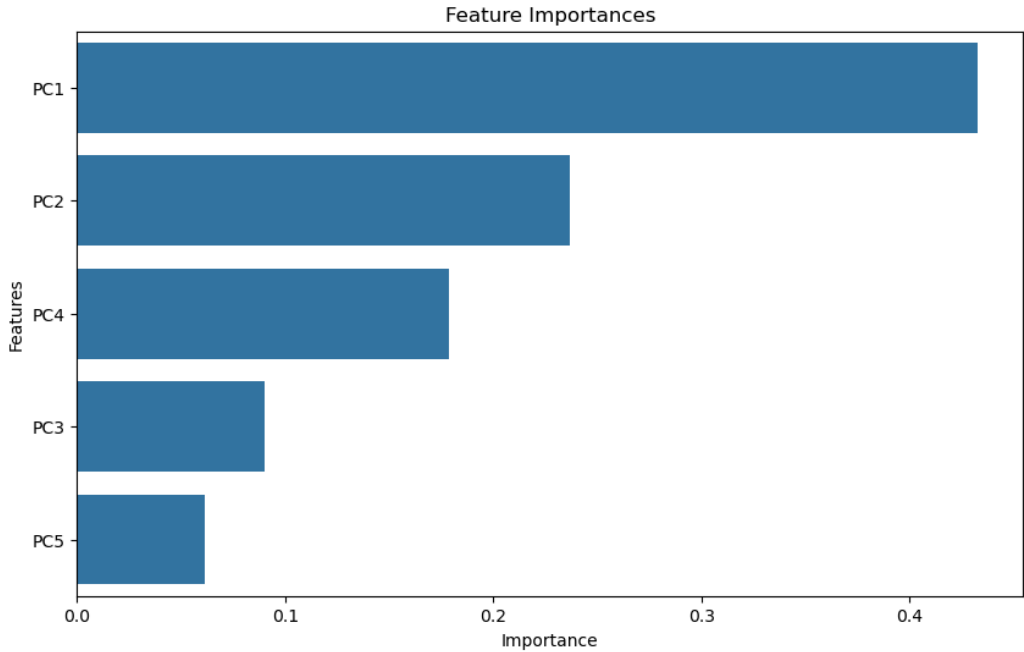
Attempt to Predict Status Ended

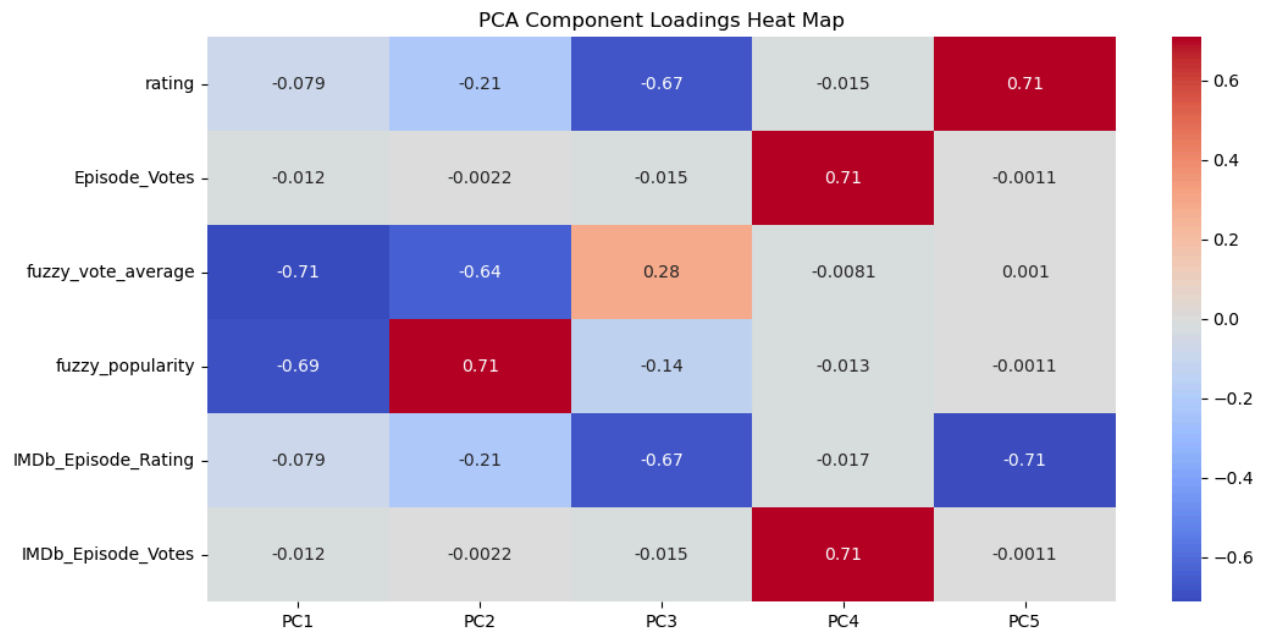
Ranking 585 series in 12.0 classification pockets.
fit sample at rate 0 & reduce to 5 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']
Number of rows in X_train: 16539
Number of columns in X_train: 5
Improved Extra Trees Classifier Accuracy: 0.8937250634747914
Improved Extra Trees Classifier Precision: 0.9117994100294985
Improved Extra Trees Classifier Recall: 0.9150384843102427
Improved Extra Trees Classifier F1 Score: 0.9134160756501182
Improved Extra Trees Classifier Specificity: 0.8600187265917603
Elapsed time: 2.6774375438690186 seconds

```
In [351... graph_survival_confusion(conf_matrix_canceled_50, conf_matrix_ended_50)
graph_survival_importance(importance_df_canceled_50)
show_pca_heatmap(pca_components, loadings_df_canceled_50)
```

Feature	Importance
PC1	0.4328984545256436
PC2	0.2369597235336307
PC4	0.1786747082344861
PC3	0.09031526027109514
PC5	0.061151853435144524





Analysis

- Predicting whether a series will finish at its end or be canceled is relatively straightforward with ExtraTreesClassifier.
- This model allows the amount of series-specific data to be tuned.
- While series-specific data may not be "known" entirely early on in a series, estimates across wide ranges is certainly possible.

Presentation Graph Contrasting Fuzziness

```
In [352... # Data for Series 1: Status Canceled
fuzziness = [50, 100, 300]
accuracy_1 = [0.9490388103010519, 0.8944504896626768, 0.7956111715632934]
precision_1 = [0.9368817710786622, 0.8644465290806754, 0.7459775719161384]
recall_1 = [0.9311797752808989, 0.8628277153558053, 0.7162921348314607]
f1_score_1 = [0.934022070908664, 0.8636363636363636, 0.730833532362073]
specificity_1 = [0.9603315571343991, 0.9144464179988159, 0.8457667258732978]

# Data for Series 2: Status Ended
accuracy_2 = [0.9481320275661952, 0.8931809938338774, 0.7952484584693508]
precision_2 = [0.95550972304066, 0.9117212872748746, 0.823039356506751]
recall_2 = [0.9600355239786856, 0.9141503848431024, 0.8481349911190054]
f1_score_2 = [0.957767277023036, 0.9129342202512933, 0.8353987461729115]
specificity_2 = [0.9293071161048689, 0.8600187265917603, 0.7116104868913857]

# Create bar plots
bar_width = 0.15
index = np.arange(len(fuzziness))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7), sharey=True)

# Bar graph for Series 1: Status Canceled
ax1.bar(index, accuracy_1, bar_width, color='blue', label='Accuracy')
ax1.bar(index + bar_width, precision_1, bar_width, color='green', label='Precision')
ax1.bar(index + 2 * bar_width, recall_1, bar_width, color='red', label='Recall')
ax1.bar(index + 3 * bar_width, f1_score_1, bar_width, color='yellow', label='F1 Score')
ax1.bar(index + 4 * bar_width, specificity_1, bar_width, color='black', label='Specificity')

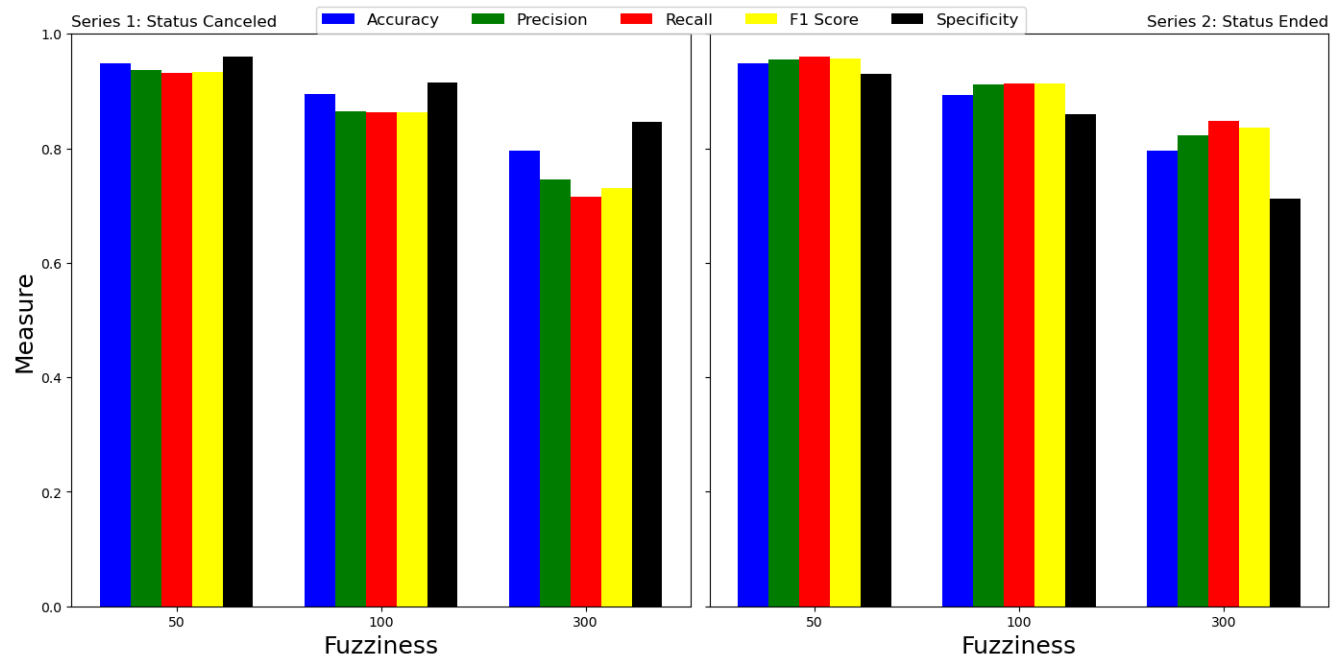
ax1.set_xlabel('Fuzziness', fontsize=18)
ax1.set_ylabel('Measure', fontsize=18)
ax1.set_title('Series 1: Status Canceled', loc='left')
ax1.set_xticks(index + 2 * bar_width)
ax1.set_xticklabels(fuzziness)
ax1.set_ylim(0, 1)

# Bar graph for Series 2: Status Ended
ax2.bar(index, accuracy_2, bar_width, color='blue', label='Accuracy')
ax2.bar(index + bar_width, precision_2, bar_width, color='green', label='Precision')
ax2.bar(index + 2 * bar_width, recall_2, bar_width, color='red', label='Recall')
ax2.bar(index + 3 * bar_width, f1_score_2, bar_width, color='yellow', label='F1 Score')
ax2.bar(index + 4 * bar_width, specificity_2, bar_width, color='black', label='Specificity')

ax2.set_xlabel('Fuzziness', fontsize=18)
ax2.set_title('Series 2: Status Ended', loc='right')
ax2.set_xticks(index + 2 * bar_width)
ax2.set_xticklabels(fuzziness)
ax2.set_ylim(0, 1)

# Add Legend
handles, labels = ax1.get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', ncol=5, fontsize=12)

plt.tight_layout()
plt.show()
```



General Model Over All Data

- RandomForestRegressor without hyperparameter tuning. (default settings below)
- Fits series factors in 12 pockets of 100 series each

Default Settings:

- `n_estimators`: 100 The number of trees in the forest.
- `criterion`: "squared_error" The function to measure the quality of a split. "squared_error" is the default for regression tasks.
- `max_depth`: None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.
- `min_samples_split`: 2 The minimum number of samples required to split an internal node.
- `min_samples_leaf`: 1 The minimum number of samples required to be at a leaf node.
- `min_weight_fraction_leaf`: 0.0 The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.
- `max_features`: "auto" The number of features to consider when looking for the best split; "auto" means that `max_features=sqrt(n_features)`.
- `max_leaf_nodes`: None Grow trees with `max_leaf_nodes` in best-first fashion. If None, then unlimited number of leaf nodes.
- `min_impurity_decrease`: 0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value.
- `bootstrap`: True Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.
- `oob_score`: False Whether to use out-of-bag samples to estimate the R^2 on unseen data.
- `n_jobs`: None The number of jobs to run in parallel. None means 1 unless in a `joblib.parallel_backend` context. -1 means using all processors.
- `random_state`: None Controls both the randomness of the bootstrapping of the samples used when building trees (if `bootstrap=True`) and the sampling of the features to consider when looking for the best split at each node (if `max_features < n_features`).
- `verbose`: 0 Controls the verbosity when fitting and predicting.
- `warm_start`: False When set to True, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just fit a whole new forest.

Create General Model for Predictions

```
In [353... data_for_tuning_df = fuzzify_series(series_episode_subset_modeling_df, 100).copy()

y = data_for_tuning_df[['Viewers_millions', '_18_to_49_Demo']]

selected_features = select_features(content=True, networks=True, genres=True)
print("\nSelected Features: ", selected_features)

X = data_for_tuning_df[selected_features]

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Build the pipeline directly with the RandomForestRegressor
```

```

pipeline = Pipeline(steps=[
    ('random_forest_regressor', RandomForestRegressor(random_state=42))
])

# Fit the model for 'Viewers (millions)'
pipeline.fit(X_train, y_train['Viewers_millions'])

# Predict and evaluate for 'Viewers (millions)'
y_pred_millions = pipeline.predict(X_test)
mse_millions = mean_squared_error(y_test['Viewers_millions'], y_pred_millions)
rmse_millions = mse_millions ** 0.5
r2_millions = r2_score(y_test['Viewers_millions'], y_pred_millions)
mae_millions = mean_absolute_error(y_test['Viewers_millions'], y_pred_millions)
# calculate residuals
residuals_millions = y_test['Viewers_millions'] - y_pred_millions

print(f'\nRMSE for Viewers (millions): {rmse_millions}')

# Fit the model for '18-49 Demo'
pipeline.fit(X_train, y_train['_18_to_49_Demo'])

# Predict and evaluate for '18-49 Demo'
y_pred_demo = pipeline.predict(X_test)
mse_demo = mean_squared_error(y_test['_18_to_49_Demo'], y_pred_demo)
rmse_demo = mse_demo ** 0.5
r2_demo = r2_score(y_test['_18_to_49_Demo'], y_pred_demo)
mae_demo = mean_absolute_error(y_test['_18_to_49_Demo'], y_pred_demo)
# calculate residuals
residuals_demo = y_test['_18_to_49_Demo'] - y_pred_demo

print(f'RMSE for 18-49 Demo: {rmse_demo}')

```

Ranking 743 series in 8.0 classification pockets.

Selected Features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes', 'sex_factor', 'violence_factor', 'profanity_factor', 'drugs_factor', 'intense_factor', 'HBO', 'Netflix', 'AMC', 'FOX', 'The CW', 'ABC', 'Disney+', 'CBS', 'Prime Video', 'Adult Swim', 'NBC', 'Hulu', 'USA Network', 'Cinemax', 'Comedy Central', 'Showtime', 'Paramount+', 'Paramount Network', 'Lifetime', 'Starz', 'Telemundo', 'Apple TV+', 'HBO Max', 'BET', 'Hallmark Channel', 'Peacock', 'Sci-Fi & Fantasy', 'Drama', 'Action & Adventure', 'Crime', 'Mystery', 'Comedy', 'Animation', 'Family', 'War & Politics', 'Soap', 'Western', 'Kids', 'Documentary', 'Reality', 'Romance', 'Adult', 'Biography', 'Film-Noir', 'Game-Show', 'History', 'Horror', 'Musical', 'Music', 'News', 'Short', 'Sport', 'Talk-Show', 'Thriller']

RMSE for Viewers (millions): 0.6941218084130591
 RMSE for 18-49 Demo: 0.28081271678633923

In [354...

```

print("\nResults of Random Forest Regressor")
print("\n\tDomain: All Episodes")
print(f'RMSE for 18-49 Demo: {rmse_demo}')
print(f'R2 for 18-49 Demo: {r2_demo}')
print(f'MAE for 18-49 Demo: {mae_demo}')

print(f'\nRMSE for Viewers: {rmse_millions}')
print(f'R2 for Viewers: {r2_millions}')
print(f'MAE for Viewers: {mae_millions}')

```

Results of Random Forest Regressor

Domain: All Episodes
 RMSE for 18-49 Demo: 0.28081271678633923
 R2 for 18-49 Demo: 0.8813082539987818
 MAE for 18-49 Demo: 0.11073407065190413

RMSE for Viewers: 0.6941218084130591
 R2 for Viewers: 0.948545532117081
 MAE for Viewers: 0.3062364709756377

12 classification pockets of popularity and average series rating.

- RMSE for Viewers (millions): 0.6927846068173983
- RMSE for 18-49 Demo: 0.27001305917352675

Simple RFR Analysis

These are remarkable results that show how effective the technique of adding series specific data (fuzziness = 100 here, so 12 buckets) improves the performance of our regression model. This uses 12 classification pockets of fuzzy series popularity and average series rating. This helps fix the series data to itself, which makes sense, because past experience is a good predictor of the future.

- RMSE for Viewers (millions): 0.6927846068173983
- RMSE for 18-49 Demo: 0.27001305917352675

In [355...

```

# save this data to measure hyperparameters later.
data_for_tuning_df.to_csv(FINAL_DATA_DIRECTORY + "/episodes_for_hyperparameter_tuning.csv", index=False)

```

Feature Importance of General Model

- Episode_Votes is more important now.

In [356...

```

feature_importances = pipeline.named_steps['random_forest_regressor'].feature_importances_
features = X.columns

# Create a DataFrame for easier visualization
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importances
})

```

```
importance_df = importance_df.sort_values(by='Importance', ascending=False) # Sort by importance
pretty_print_df(importance_df)
```

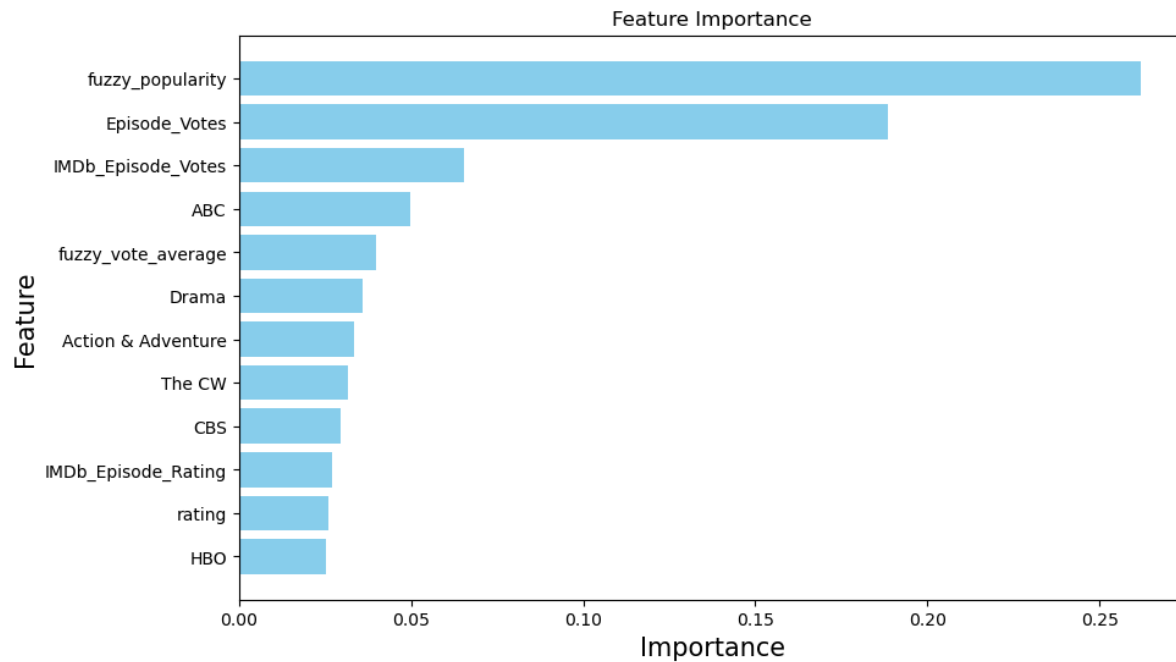
Feature	Importance
fuzzy_popularity	0.2617547911900264
Episode_Votes	0.1885595695410629
IMDb_Episode_Votes	0.0654292462732351
ABC	0.04989801693675365
fuzzy_vote_average	0.039964507162337556
Drama	0.03582631470984319
Action & Adventure	0.03334144635598341
The CW	0.03188220370945975
CBS	0.029758448492253085
IMDb_Episode_Rating	0.027280354136415566
rating	0.026049050454669143
HBO	0.025303244444906964
profanity_factor	0.022392935069733534
Crime	0.020731952147424577
Comedy	0.018936702652147672
NBC	0.01734758382253393
USA Network	0.015015488480040603
Sci-Fi & Fantasy	0.013519460893297254
sex_factor	0.010439436504163388
violence_factor	0.00744746104915111
Reality	0.007036983148379679
Mystery	0.0070352273869406165
FOX	0.0062163872804912685
AMC	0.005154437130250427
intense_factor	0.0043973355207139785
Disney+	0.004390185103972589
drugs_factor	0.004288898244446292
Family	0.004173186786712441
Lifetime	0.004066006631739384
Animation	0.0033927991344783375
War & Politics	0.0016237168068539295
Showtime	0.0016021373463816337
Paramount+	0.0010854576457781655
Cinemax	0.0008844704667227072
Starz	0.0008802253236263379
Prime Video	0.0007977965565620024
Netflix	0.0004923018517638361
Hulu	0.00033535937421811545
BET	0.00027046097803933669
Comedy Central	0.0002377760504331384
HBO Max	0.00018242486479069791
Western	0.00016114055534219724
Adult Swim	0.00014528136292698963
Paramount Network	9.335858572875499e-05
Hallmark Channel	7.954655877757181e-05
Documentary	3.8499099898817055e-05
Soap	3.2240573528833925e-05
Peacock	1.9775598382402886e-05
Romance	8.370006680864935e-06
Talk-Show	0.0
Sport	0.0
Horror	0.0
Short	0.0
News	0.0
Music	0.0
Musical	0.0
Apple TV+	0.0
History	0.0
Game-Show	0.0
Film-Noir	0.0
Biography	0.0
Adult	0.0
Kids	0.0
Telemundo	0.0
Thriller	0.0

View Feature Importance Graphically

In [357...

```
top_features_df = importance_df.head(12)
# Extract feature names and importances
features = top_features_df['Feature']
importances = top_features_df['Importance']

# Plot feature importances
plt.figure(figsize=(10, 6))
plt.barh(features, importances, color='skyblue')
plt.xlabel('Importance', fontsize = 15)
plt.ylabel('Feature', fontsize = 15)
plt.title('Feature Importance')
plt.gca().invert_yaxis() # Invert y-axis to have the most important feature on top
plt.show()
```



```
In [358.. y = data_for_tuning_df[['Viewers_millions', '_18_to_49_Demo']]
```

```
selected_features = select_features(content=True, networks=True, genres=True)
print("\nSelected Features: ", selected_features)
```

```
X = data_for_tuning_df[selected_features]
```

```
show_regression_results("RFR General Model - Viewers (Millions)", data_for_tuning_df, 'Viewers_millions', None, importance_df['Feature'], \
                        pipeline, rmse_millions, r2_millions, mae_millions, None, \
                        X_test, y_test['Viewers_millions'], y_pred_millions, residuals_millions)
```

```
show_regression_results("RFR General Model - Demo (18-49)", data_for_tuning_df, '_18_to_49_Demo', None, importance_df['Feature'], \
                        pipeline, rmse_demo, r2_demo, mae_demo, None, X_test, y_test['_18_to_49_Demo'], y_pred_demo, residuals_demo)
```

```
Selected Features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes', 'sex_factor', 'vi
olence_factor', 'profanity_factor', 'drugs_factor', 'intense_factor', 'HBO', 'Netflix', 'AMC', 'FOX', 'The CW', 'ABC', 'Disney+', 'CBS', 'Prime Video',
'Adult Swim', 'NBC', 'Hulu', 'USA Network', 'Cinemax', 'Comedy Central', 'Showtime', 'Paramount+', 'Paramount Network', 'Lifetime', 'Starz', 'Telemund
o', 'Apple TV+', 'HBO Max', 'BET', 'Hallmark Channel', 'Peacock', 'Sci-Fi & Fantasy', 'Drama', 'Action & Adventure', 'Crime', 'Mystery', 'Comedy', 'Anim
ation', 'Family', 'War & Politics', 'Soap', 'Western', 'Kids', 'Documentary', 'Reality', 'Romance', 'Adult', 'Biography', 'Film-Noir', 'Game-Show', 'His
tory', 'Horror', 'Musical', 'Music', 'News', 'Short', 'Sport', 'Talk-Show', 'Thriller']
```

RFR General Model - Viewers (Millions)

Viewers (millions) Statistics:

Mean: 2.8906696475178064

Median: 1.864

Mode: [0.624]

Variance: 9.70721064047449

Standard Deviation: 3.1156396839933995

Summary: count 41651.00000

mean 2.89067

std 3.11564

min 0.00600

25% 0.64000

50% 1.86400

75% 3.84000

max 26.97200

Name: Viewers_millions, dtype: float64

18-49 Demo Statistics:

Mean: 0.7120083203853316

Median: 0.47

Mode: [0.7]

Variance: 0.6670111299265791

Standard Deviation: 0.8167074934923636

Summary: count 41651.000000

mean 0.712008

std 0.816707

min 0.000000

25% 0.190000

50% 0.470000

75% 1.000000

max 9.300000

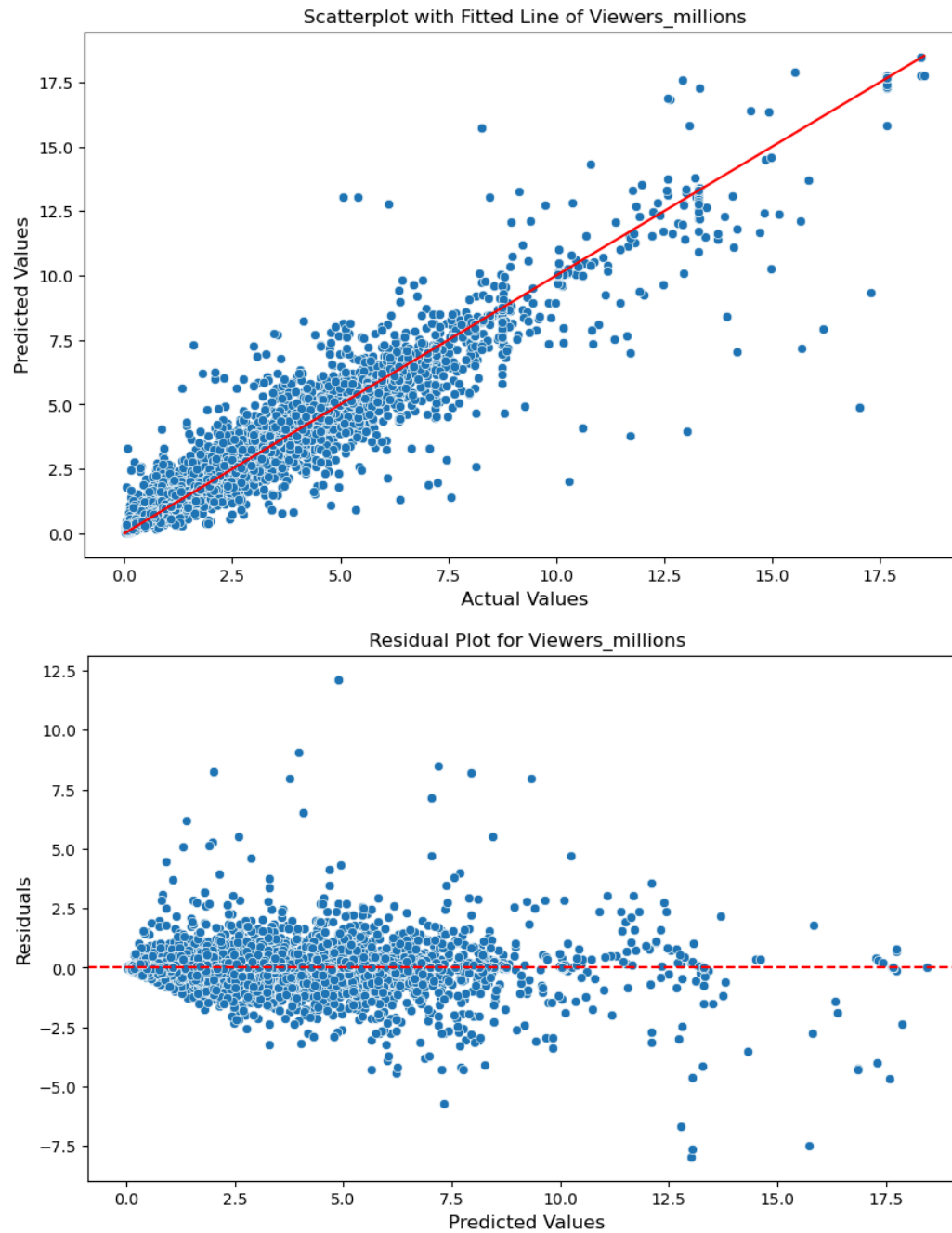
Name: _18_to_49_Demo, dtype: float64

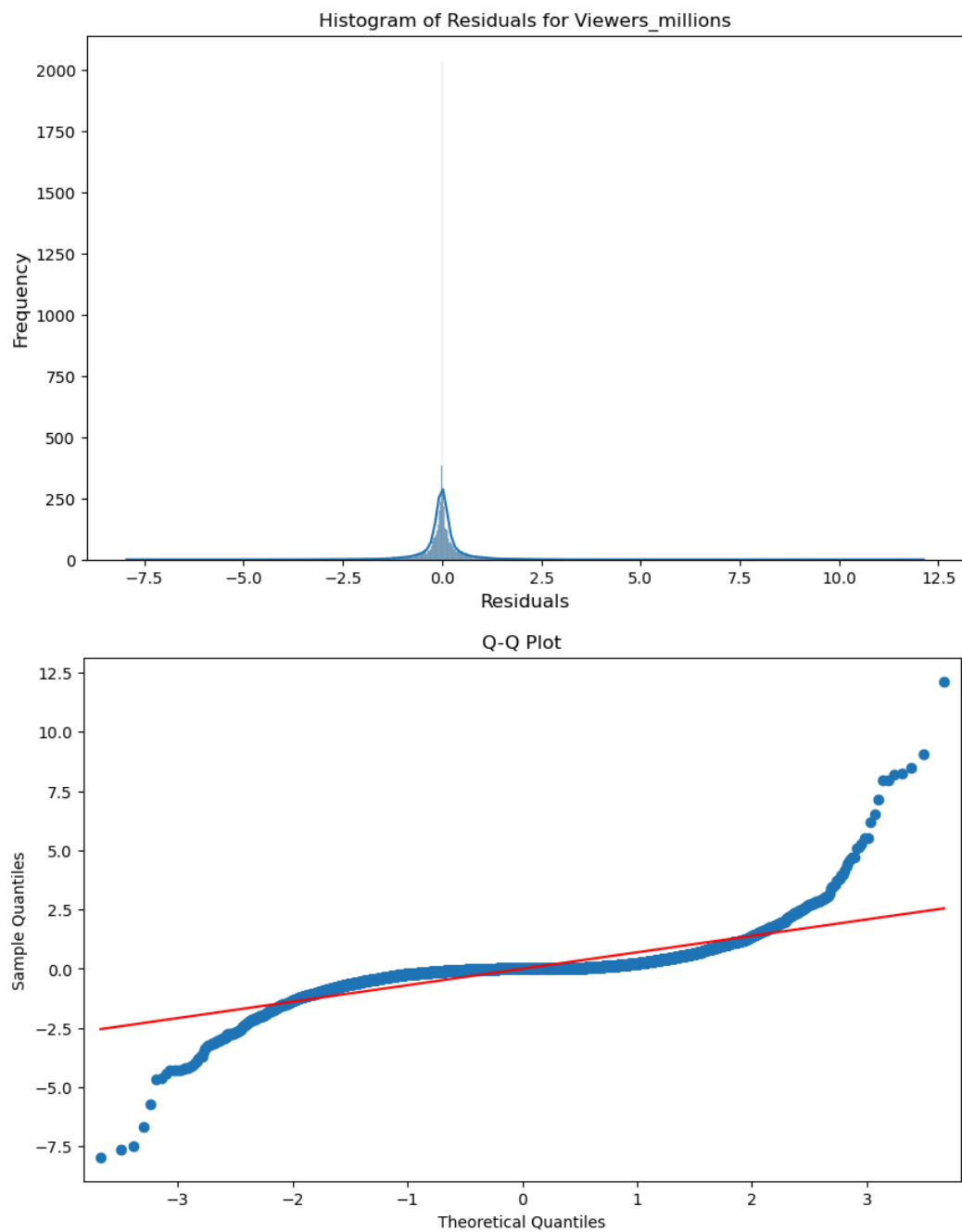
Best Estimator for Viewers_millions: Pipeline(steps=[('random_forest_regressor', \
RandomForestRegressor(random_state=42))])

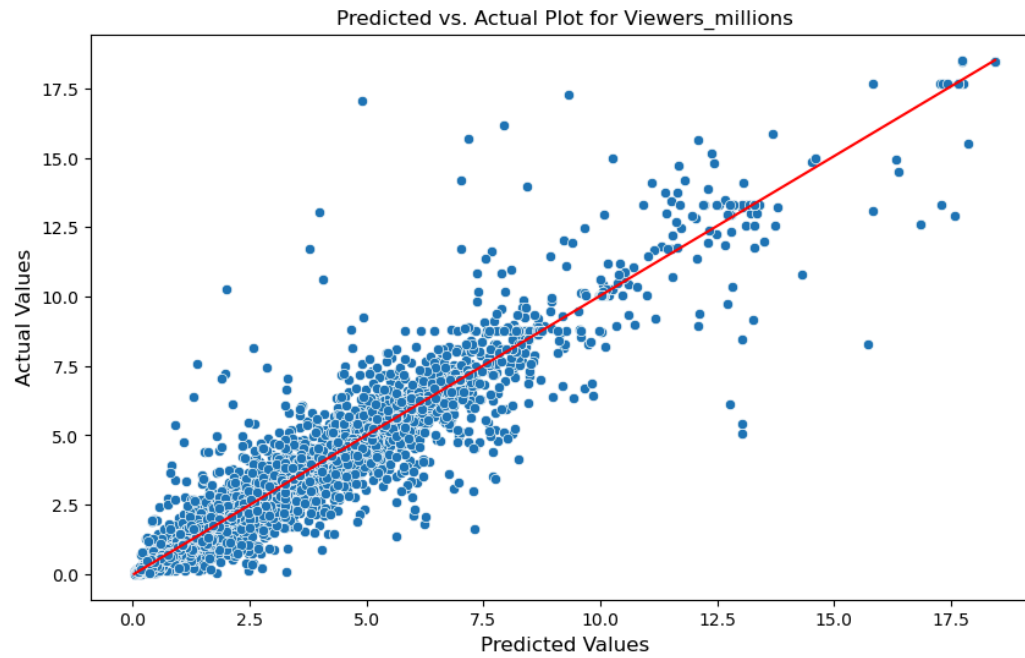
Root mean square error: 0.6941218084130591

r2: 0.948545532117081

Mean average error: 0.3062364709756377







RFR General Model - Demo (18-49)

Viewers (millions) Statistics:

Mean: 2.8906696475178064

Median: 1.864

Mode: [0.624]

Variance: 9.70721064047449

Standard Deviation: 3.1156396839933995

Summary: count 41651.00000

mean 2.89067

std 3.11564

min 0.00000

25% 0.64000

50% 1.86400

75% 3.84000

max 26.97200

Name: Viewers_millions, dtype: float64

18-49 Demo Statistics:

Mean: 0.7120083203853316

Median: 0.47

Mode: [0.7]

Variance: 0.6670111299265791

Standard Deviation: 0.8167074934923636

Summary: count 41651.000000

mean 0.712008

std 0.816707

min 0.000000

25% 0.190000

50% 0.470000

75% 1.000000

max 9.300000

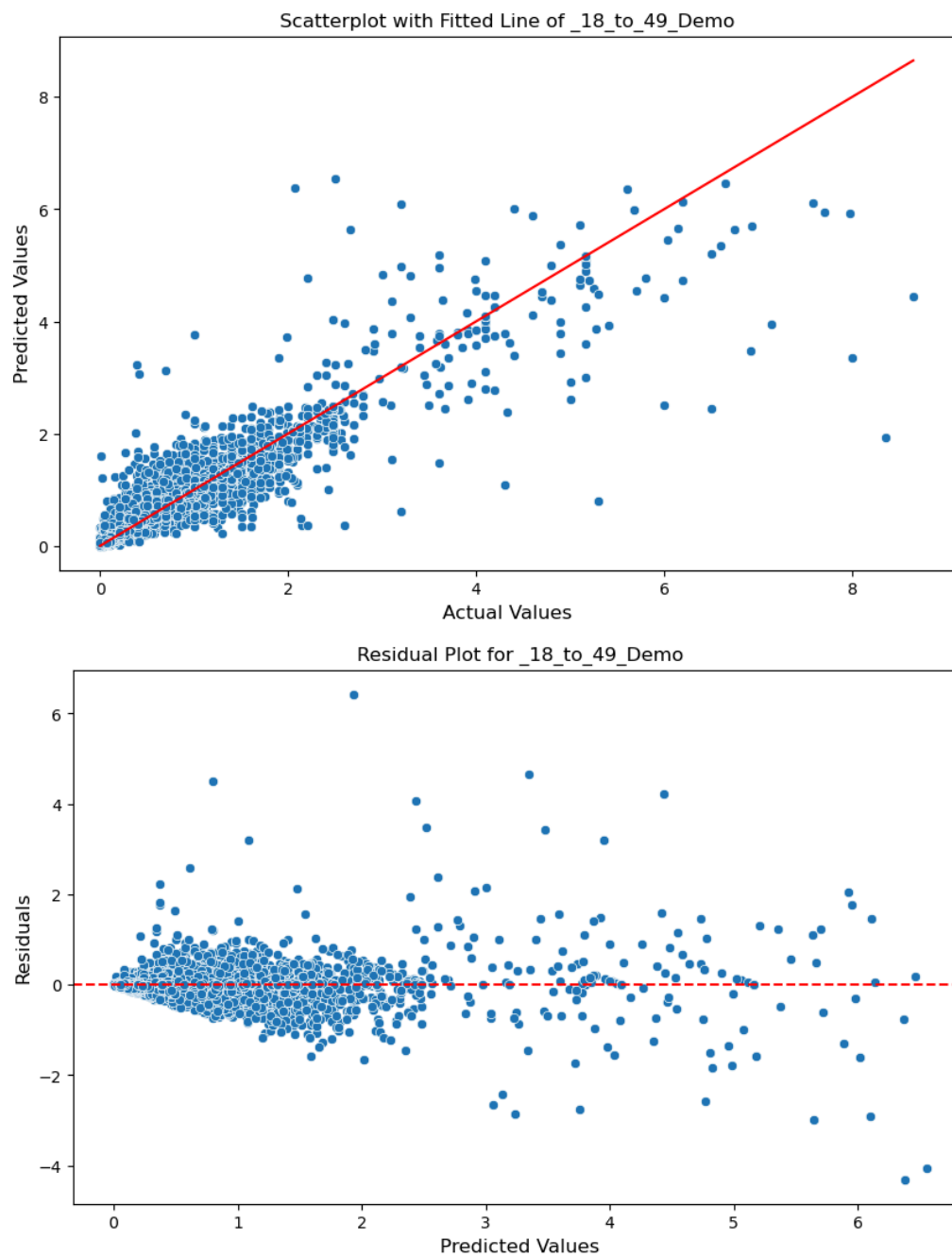
Name: _18_to_49_Demo, dtype: float64

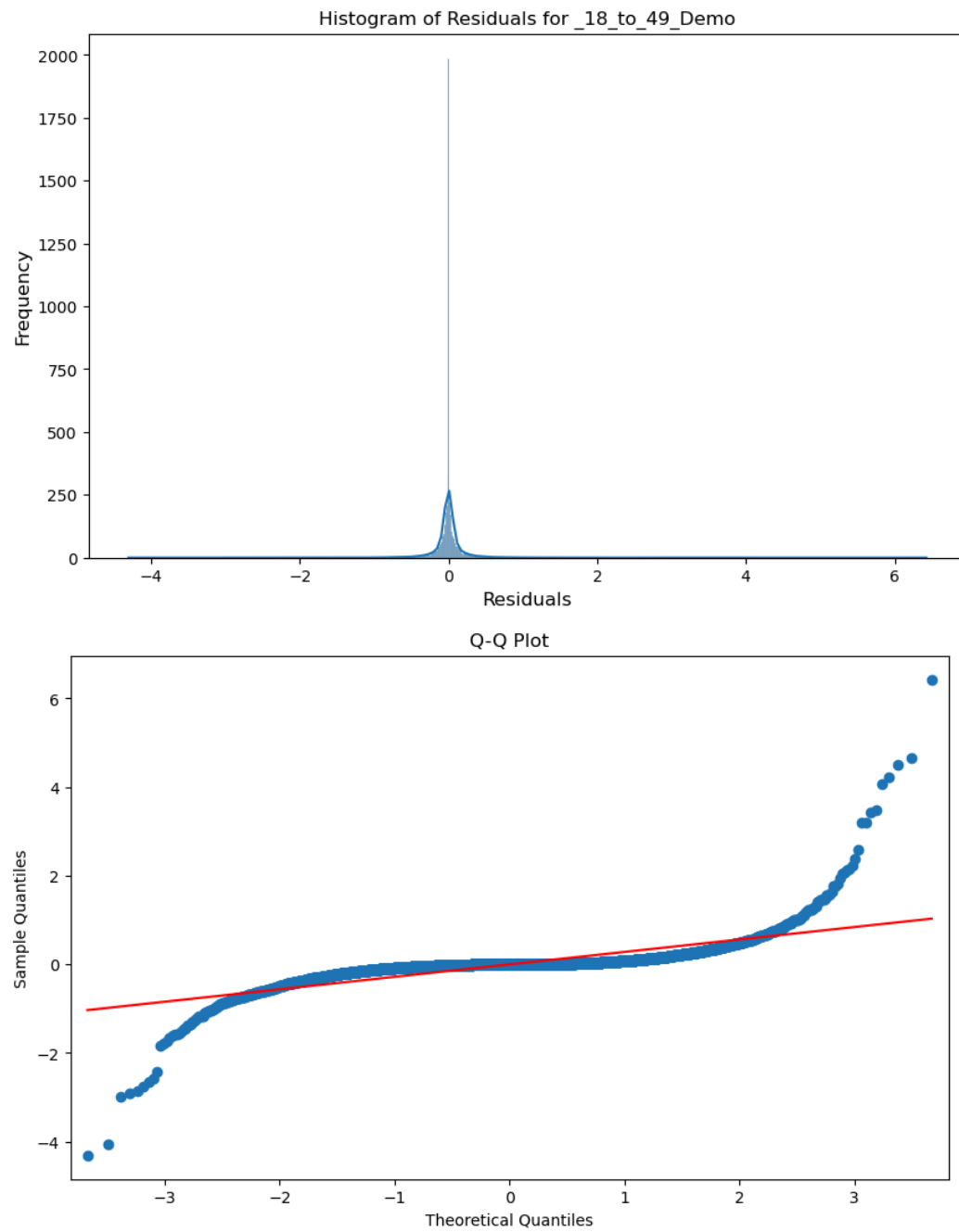
Best Estimator for _18_to_49_Demo: Pipeline(steps=[('random_forest_regressor',
RandomForestRegressor(random_state=42))])

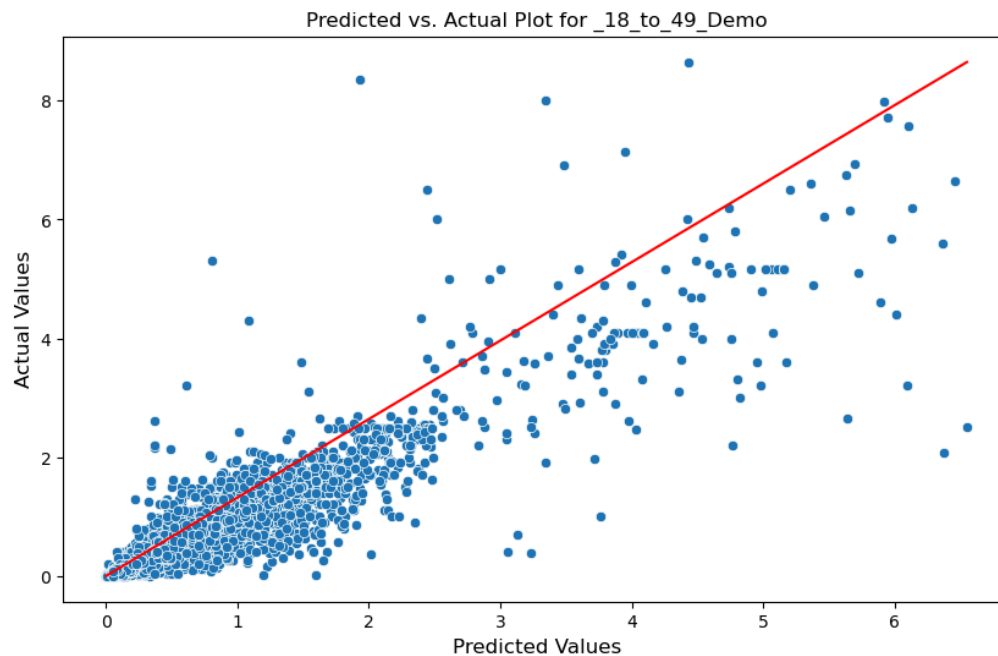
Root mean square error: 0.28081271678633923

r2: 0.8813082539987818

Mean average error: 0.11073407065190413







Baseline

- To compare effects of genre and networks I will use a simple model
- First create a baseline for testing.

```
In [359... # start from saved data
series_episode_subset_modeling_df = data_for_tuning_df.copy()

def RandomForestStandardModel(X, y, pipeline):
    # Split the data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # Fit the model for 'Viewers (millions)'
    pipeline.fit(X_train, y_train['Viewers_millions'])

    # Predict and evaluate for 'Viewers (millions)'
    y_pred_millions = pipeline.predict(X_test)
    mse_millions = mean_squared_error(y_test['Viewers_millions'], y_pred_millions)
    rmse_millions = mse_millions ** 0.5

    print(f'RMSE for Viewers (millions): {rmse_millions}')

    # Fit the model for '18-49 Demo'
    pipeline.fit(X_train, y_train['_18_to_49_Demo'])

    # Predict and evaluate for '18-49 Demo'
    y_pred_demo = pipeline.predict(X_test)
    mse_demo = mean_squared_error(y_test['_18_to_49_Demo'], y_pred_demo)
    rmse_demo = mse_demo ** 0.5

    print(f'RMSE for 18-49 Demo: {rmse_demo}')
```

```
In [360... print("Features selected: ", select_features())

y = series_episode_subset_modeling_df[['Viewers_millions', '_18_to_49_Demo']]
X = series_episode_subset_modeling_df[select_features()]

pipeline = Pipeline(steps=[
    ('new_forest_regressor', RandomForestRegressor(random_state=42))
])

RandomForestStandardModel(X, y, pipeline)

Features selected: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']
RMSE for Viewers (millions): 1.6784840210417142
RMSE for 18-49 Demo: 0.4790697859815602
```

```
In [361... feature_importances = pipeline.named_steps['new_forest_regressor'].feature_importances_
features = X.columns

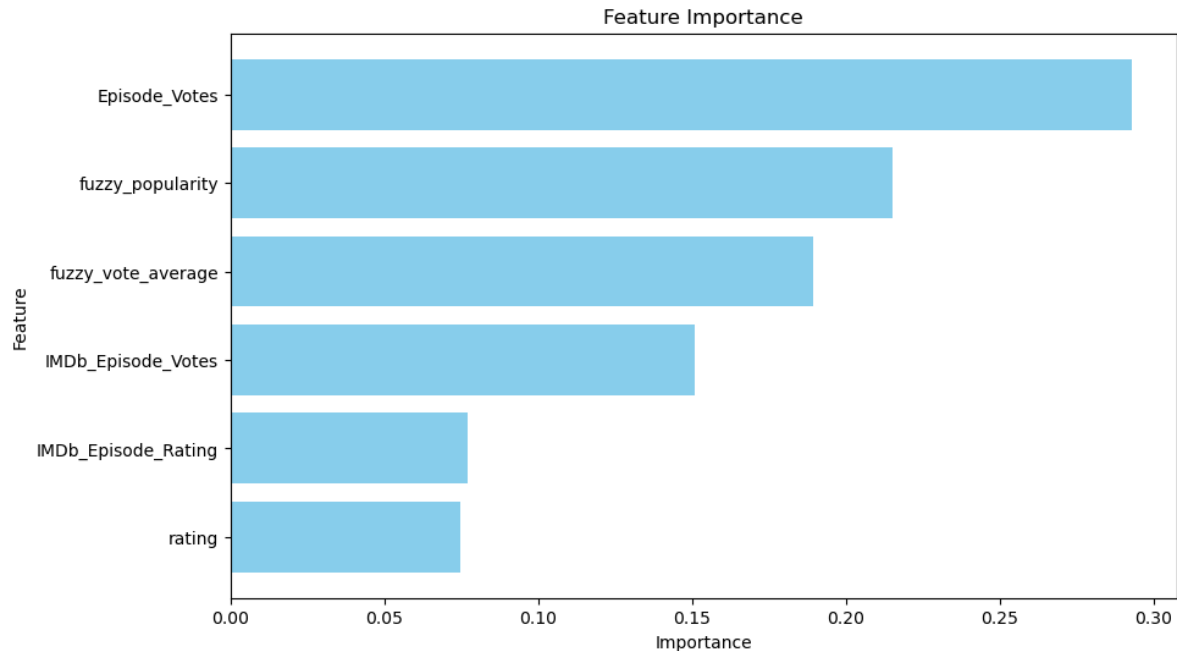
# Create a DataFrame for easier visualization
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importances
})
importance_df = importance_df.sort_values(by='Importance', ascending=False) # Sort by importance

pretty_print_df(importance_df)
```

Feature	Importance
Episode_Votes	0.29269401564357594
fuzzy_popularity	0.2150510843431422
fuzzy_vote_average	0.18943009117811793
IMDb_Episode_Votes	0.15091507162326118
IMDb_Episode_Rating	0.07715629862952371
rating	0.07475343858237904

```
In [362... # Extract feature names and importances
features = importance_df.head(12)['Feature']
importances = importance_df.head(12)['Importance']

# Plot feature importances
plt.figure(figsize=(10, 6))
plt.barh(features, importances, color='skyblue')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.title('Feature Importance')
plt.gca().invert_yaxis() # Invert y-axis to have the most important feature on top
plt.show()
```



Genres

- Create an independent function for use by multiple sets

```
In [363... print("Features selected: ", select_features(genres=True))

y = series_episode_subset_modeling_df[['Viewers_millions', '_18_to_49_Demo']]
X = series_episode_subset_modeling_df[select_features(genres=True)]

pipeline = Pipeline(steps=[
    ('new_forest_regressor', RandomForestRegressor(random_state=42))
])

RandomForestStandardModel(X, y, pipeline)

Features selected: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes', 'Sci-Fi & Fantas
y', 'Drama', 'Action & Adventure', 'Crime', 'Mystery', 'Comedy', 'Animation', 'Family', 'War & Politics', 'Soap', 'Western', 'Kids', 'Documentary', 'Rea
lity', 'Romance', 'Adult', 'Biography', 'Film-Noir', 'Game-Show', 'History', 'Horror', 'Musical', 'Music', 'News', 'Short', 'Sport', 'Talk-Show', 'Thrill
er']
RMSE for Viewers (millions): 0.9098038530984021
RMSE for 18-49 Demo: 0.322200073272271
```

Effects of Genre

```
In [364... feature_importances = pipeline.named_steps['new_forest_regressor'].feature_importances_
features = X.columns

# Create a DataFrame for easier visualization
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importances
})
importance_df = importance_df.sort_values(by='Importance', ascending=False) # Sort by importance
```

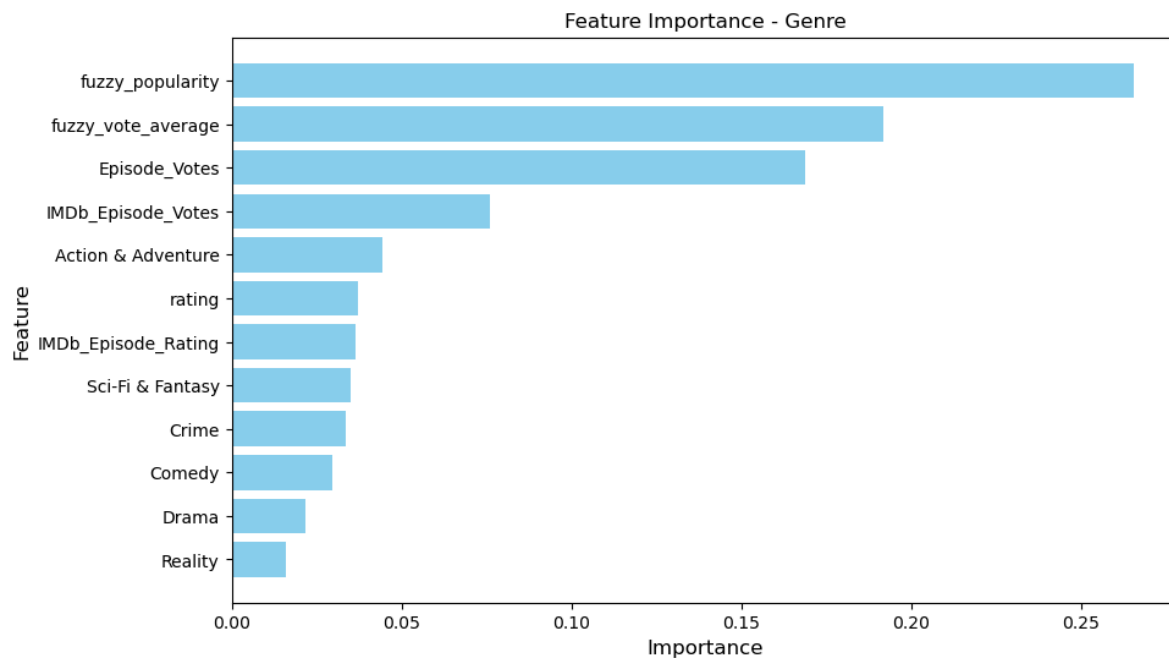
```
pretty_print_df(importance_df)
```

Feature	Importance
fuzzy_popularity	0.2653368389501569
fuzzy_vote_average	0.19171692159737858
Episode_Votes	0.16884943454782306
IMDb_Episode_Votes	0.07612497136617204
Action & Adventure	0.04440187806257892
rating	0.037319416762937145
IMDb_Episode_Rating	0.03639529074980397
Sci-Fi & Fantasy	0.03520880919760356
Crime	0.0335986340345434
Comedy	0.02980745828702388
Drama	0.02191459220028643
Reality	0.015984777335306958
Mystery	0.015091906238384613
Animation	0.008178963158209956
Western	0.007752087281657092
Family	0.005246168845367029
Soap	0.004386795715626554
War & Politics	0.0025850318108630423
Documentary	9.454211719637884e-05
Romance	5.481741080381268e-06
Musical	0.0
Talk-Show	0.0
Sport	0.0
Short	0.0
News	0.0
Music	0.0
Kids	0.0
Horror	0.0
History	0.0
Game-Show	0.0
Film-Noir	0.0
Biography	0.0
Adult	0.0
Thriller	0.0

View Feature Importance Graphically

```
In [365... # Extract feature names and importances
features = importance_df.head(12)['Feature']
importances = importance_df.head(12)['Importance']

# Plot feature importances
plt.figure(figsize=(10, 6))
plt.barh(features, importances, color='skyblue')
plt.xlabel('Importance', fontsize = 12)
plt.ylabel('Feature', fontsize = 12)
plt.title('Feature Importance - Genre')
plt.gca().invert_yaxis() # Invert y-axis to have the most important feature on top
plt.show()
```



Analysis

- Including the genre does change the performance of the model, but not drastically.
- Some genres do have more sway than others in the model (drama especially)

Content

- Create an independent function for use by multiple sets

```
In [366... print("Features selected: ", select_features(content=True))

y = series_episode_subset_modeling_df[['Viewers_millions', '_18_to_49_Demo']]

X = series_episode_subset_modeling_df[select_features(content=True)]

pipeline = Pipeline(steps=[
    ('new_forest_regressor', RandomForestRegressor(random_state=42))
])

RandomForestStandardModel(X, y, pipeline)

Features selected: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes', 'sex_factor', 'violence_factor', 'profanity_factor', 'drugs_factor', 'intense_factor']
RMSE for Viewers (millions): 1.5532698879685323
RMSE for 18-49 Demo: 0.44060239699777515
```

Effects of Content

```
In [367... feature_importances = pipeline.named_steps['new_forest_regressor'].feature_importances_
features = X.columns

# Create a DataFrame for easier visualization
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importances
})
importance_df = importance_df.sort_values(by='Importance', ascending=False) # Sort by importance

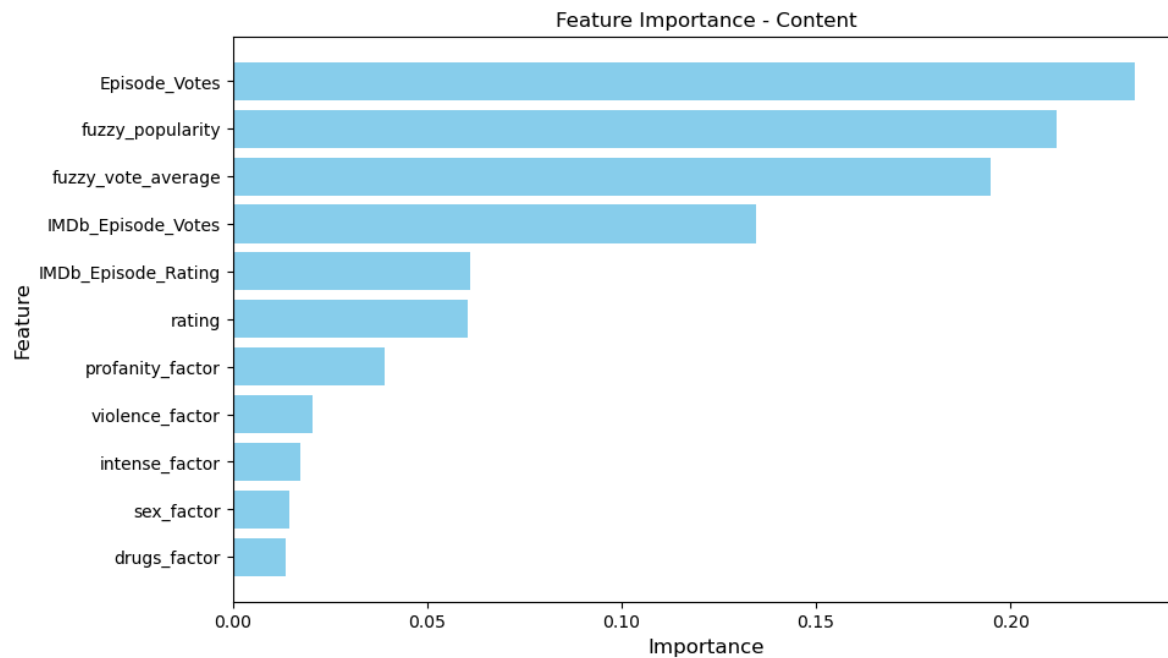
pretty_print_df(importance_df)

+-----+-----+
| Feature | Importance |
+-----+-----+
| Episode_Votes | 0.23186215775989522 |
| fuzzy_popularity | 0.21189892205249442 |
| fuzzy_vote_average | 0.19477971414371387 |
| IMDb_Episode_Votes | 0.13449609349116666 |
| IMDb_Episode_Rating | 0.06106438676221217 |
| rating | 0.06053999012352845 |
| profanity_factor | 0.039119668476836185 |
| violence_factor | 0.020659357225110097 |
| intense_factor | 0.017512744971640678 |
| sex_factor | 0.014542913135371718 |
| drugs_factor | 0.013524051858080553 |
+-----+-----+
```

View Feature Importance Graphically

```
In [368... # Extract feature names and importances
features = importance_df.head(12)['Feature']
importances = importance_df.head(12)['Importance']

# Plot feature importances
plt.figure(figsize=(10, 6))
plt.barh(features, importances, color='skyblue')
plt.xlabel('Importance', fontsize = 12)
plt.ylabel('Feature', fontsize = 12)
plt.title('Feature Importance - Content')
plt.gca().invert_yaxis() # Invert y-axis to have the most important feature on top
plt.show()
```



Analysis

- Including the content warnings section features have very little impact on the model.

Effects of Network (as a factor)

- Surprisingly some networks as factors are impactful, especially HBO

```
In [369... # Build the pipeline directly with the RandomForestRegressor
pipeline = Pipeline(steps=[
    ('network_forest_regressor', RandomForestRegressor(random_state=42))
])

y = series_episode_subset_modeling_df[['Viewers_millions', '_18_to_49_Demo']]

X = series_episode_subset_modeling_df[select_features(networks=True)]

RandomForestStandardModel(X, y, pipeline)

RMSE for Viewers (millions): 0.8785393487476163
RMSE for 18-49 Demo: 0.3263249143497531

In [370... feature_importances = pipeline.named_steps['network_forest_regressor'].feature_importances_
features = X.columns

print("Number of features:", len(features))
print("Number of importances:", len(feature_importances))

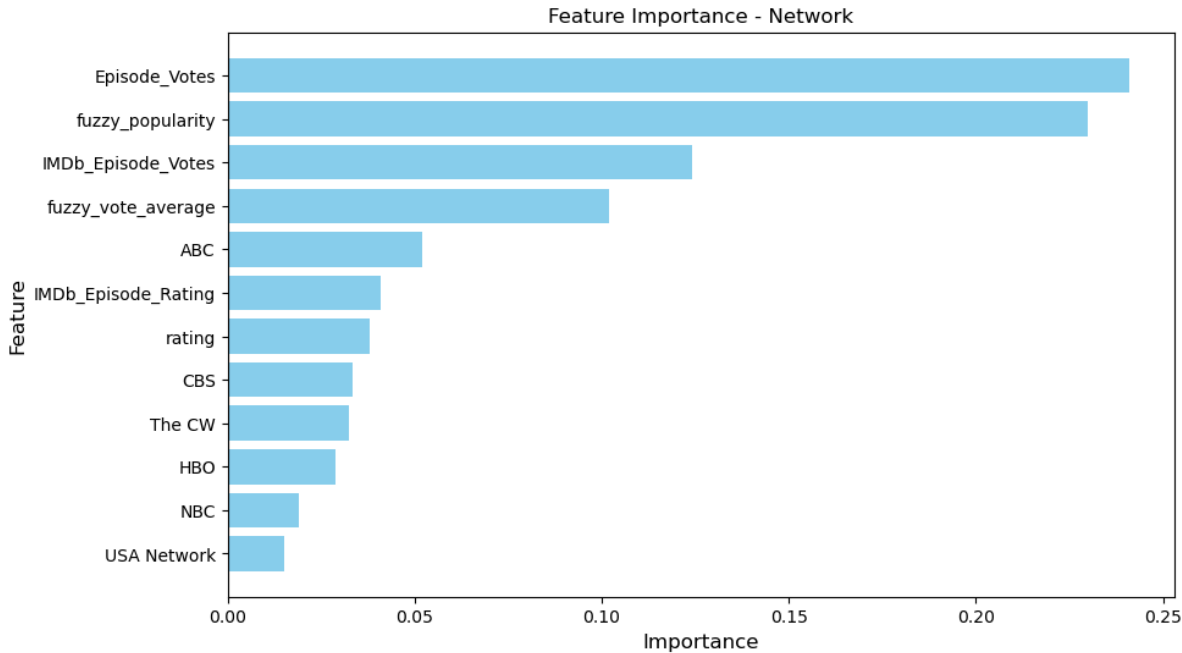
# Create a DataFrame for easier visualization
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importances
})
importance_df = importance_df.sort_values(by='Importance', ascending=False) # Sort by importance
pretty_print_df(importance_df)
```


Number of features: 32
Number of importances: 32

Feature	Importance
Episode_Votes	0.24082753783497515
fuzzy_popularity	0.2299525886887421
IMDb_Episode_Votes	0.12427852390520126
fuzzy_vote_average	0.10205178817781743
ABC	0.051881051105447495
IMDb_Episode_Rating	0.04086116994569659
rating	0.03805767004805455
CBS	0.03337525210889228
The CW	0.03248351714559052
HBO	0.02875343390724442
NBC	0.019070080729690048
USA Network	0.015290573273136918
FOX	0.012103272615702829
AMC	0.007758366047993002
Lifetime	0.0040425197743977365
Showtime	0.0033974734049644515
Starz	0.0033683944189772043
Prime Video	0.0027453883904687976
Disney+	0.002380833852344722
Comedy Central	0.0015766983961708657
Netflix	0.0015463022256742335
Paramount+	0.0012034733934312275
Cinemax	0.0011354855406754533
Hallmark Channel	0.0006019754967538228
Hulu	0.0003573567059683452
HBO Max	0.0003049789392379774
BET	0.00024069569464475282
Paramount Network	0.00021789546783671323
Adult Swim	9.953476823868393e-05
Peacock	3.6167996030341044e-05
Telemundo	0.0
Apple TV+	0.0

```
In [371... # Extract feature names and importances
features = importance_df.head(12)['Feature']
importances = importance_df.head(12)['Importance']

# Plot feature importances
plt.figure(figsize=(10, 6))
plt.barh(features, importances, color='skyblue')
plt.xlabel('Importance', fontsize = 12)
plt.ylabel('Feature', fontsize = 12)
plt.title('Feature Importance - Network')
plt.gca().invert_yaxis() # Invert y-axis to have the most important feature on top
plt.show()
```



Analysis

Adding either the genres or networks fields improves the model:

Comparison

Base: (ratings and popularity)

RMSE for Viewers (millions): 1.4903374678722703 RMSE for 18-49 Demo: 0.4315727098420625

Genres + Base:

RMSE for Viewers (millions): 0.8120123508793952 RMSE for 18-49 Demo: 0.29888043078887533

Content + Base:

RMSE for Viewers (millions): 1.3449122081514344 RMSE for 18-49 Demo: 0.39495695279400417

Network + Base:

RMSE for Viewers (millions): 0.7884000053292562 RMSE for 18-49 Demo: 0.29561069243298943

Filtering

Try Filtering by Crime, Reality, Comedy, Sci-Fi & Fantasy

```
In [372... crime_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['Crime']==True].copy()
sci-fi_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['Sci-Fi & Fantasy']==True].copy()
drama_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['Drama']==True].copy()
comedy_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['Comedy']==True].copy()
action_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['Action & Adventure']==True].copy()
animation_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['Animation']==True].copy()

In [373... print("Features selected: ", select_features())

y = crime_episode_modeling_df[['Viewers_millions', '_18_to_49_Demo']]

X = crime_episode_modeling_df[select_features()]

print("\nCrime Results:")
RandomForestStandardModel(X, y, pipeline)

y = sci-fi_episode_modeling_df[['Viewers_millions', '_18_to_49_Demo']]

X = sci-fi_episode_modeling_df[select_features()]

print("\nSci-Fi & Fantasy Results:")
RandomForestStandardModel(X, y, pipeline)

y = drama_episode_modeling_df[['Viewers_millions', '_18_to_49_Demo']]

X = drama_episode_modeling_df[select_features()]

print("\nDrama Results:")
RandomForestStandardModel(X, y, pipeline)

y = comedy_episode_modeling_df[['Viewers_millions', '_18_to_49_Demo']]

X = comedy_episode_modeling_df[select_features()]

print("\nComedy Results:")
RandomForestStandardModel(X, y, pipeline)

y = action_episode_modeling_df[['Viewers_millions', '_18_to_49_Demo']]

X = action_episode_modeling_df[select_features()]

print("\nAction Results:")
RandomForestStandardModel(X, y, pipeline)

y = animation_episode_modeling_df[['Viewers_millions', '_18_to_49_Demo']]

X = animation_episode_modeling_df[select_features()]

print("\nAnimation Results:")
RandomForestStandardModel(X, y, pipeline)

Features selected:  ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes']

Crime Results:
RMSE for Viewers (millions): 1.2633543632046322
RMSE for 18-49 Demo: 0.32619827701114923

Sci-Fi & Fantasy Results:
RMSE for Viewers (millions): 0.9501357696702952
RMSE for 18-49 Demo: 0.44587810414597323

Drama Results:
RMSE for Viewers (millions): 1.5199092840095854
RMSE for 18-49 Demo: 0.45591043512724105

Comedy Results:
RMSE for Viewers (millions): 1.39677825974694
RMSE for 18-49 Demo: 0.4015679810990522

Action Results:
RMSE for Viewers (millions): 1.2431008972239852
RMSE for 18-49 Demo: 0.4970792125806512

Animation Results:
RMSE for Viewers (millions): 0.44637386671061413
RMSE for 18-49 Demo: 0.18142521963902555
```

Filter by Platform

Filter by Platform to see the effects on the model.

```
In [374... hbo_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['HBO']==True].copy()
netflix_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['Netflix']==True].copy()
disney_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['Disney+']==True].copy()
nbc_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['NBC']==True].copy()
prime_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['Prime Video']==True].copy()
cw_episode_modeling_df = series_episode_subset_modeling_df[series_episode_subset_modeling_df['The CW']==True].copy()
```

```
In [375... def network_modeling(df, name):
    y = df[['Viewers_millions', '_18_to_49_Demo']]
    X = df[select_features()]
    print(f"\nResults for {name}: There are {df.shape[0]} rows and {df.shape[1]} columns of information.\n")
    RandomForestStandardModel(X, y, pipeline)
```

```
network_modeling(hbo_modeling_df, "HBO")
network_modeling(netflix_episode_modeling_df, "Netflix")
network_modeling(disney_episode_modeling_df, "Disney+")
network_modeling(nbc_episode_modeling_df, "NBC")
network_modeling(prime_episode_modeling_df, "Prime Video")
network_modeling(cw_episode_modeling_df, "The CW")
```

Results for HBO: There are 2403 rows and 90 columns of information.

RMSE for Viewers (millions): 1.1834116464759734
RMSE for 18-49 Demo: 0.6574010091792084

Results for Netflix: There are 622 rows and 90 columns of information.

RMSE for Viewers (millions): 0.4841498311649565
RMSE for 18-49 Demo: 0.15176746236961042

Results for Disney+: There are 472 rows and 90 columns of information.

RMSE for Viewers (millions): 0.8147268207107152
RMSE for 18-49 Demo: 0.18811608777649994

Results for NBC: There are 2646 rows and 90 columns of information.

RMSE for Viewers (millions): 1.9243824895764405
RMSE for 18-49 Demo: 0.6012788341161669

Results for Prime Video: There are 221 rows and 90 columns of information.

RMSE for Viewers (millions): 0.3974469956582612
RMSE for 18-49 Demo: 0.11663499845434237

Results for The CW: There are 2114 rows and 90 columns of information.

RMSE for Viewers (millions): 0.17315364967081387
RMSE for 18-49 Demo: 0.05830549685647639

Analysis

Some platforms are more predictable than others, though this may be because there are fewer offerings.

Regression Grid Runner

```
In [376... def regressor_model_run(x_reduced_train, x_reduced_test, y_reduced_train, y_reduced_test, optimized=True):
    # Define the pipeline
    pipeline = Pipeline([
        ('scaler', MinMaxScaler()), # Including scaler
        ('regressor', RandomForestRegressor()) # Default, will be overridden by GridSearchCV
    ])

    # Define a grid search with various regressors
    if optimized:
        grid_search_space = [
            # RandomForestRegressor search space
            {
                'regressor': [RandomForestRegressor()],
                'regressor__n_estimators': [50, 100, 200],
                'regressor__max_depth': [None, 10, 30],
                'regressor__min_samples_split': [2, 5, 10],
                'regressor__min_samples_leaf': [1, 2, 4],
                'regressor__max_features': ['sqrt', 'log2', None]
            },
            # DecisionTreeRegressor search space
            {
                'regressor': [DecisionTreeRegressor()],
                'regressor__max_depth': [None, 10, 20, 30],
                'regressor__min_samples_split': [2, 5],
                'regressor__min_samples_leaf': [1, 2]
            }
        ]
    else:
        grid_search_space = [
            # RandomForestRegressor search space
            {
                'regressor': [RandomForestRegressor()],
                'regressor__n_estimators': [50, 100, 200],
                'regressor__max_depth': [None, 10, 30],
                'regressor__min_samples_split': [2, 5, 10],
                'regressor__min_samples_leaf': [1, 2, 4],
```

```

        'regressor__max_features': ['sqrt', 'log2', None]
    },
    # GradientBoostingRegressor search space
    {
        'regressor': [GradientBoostingRegressor()],
        'regressor__n_estimators': [100, 200, 300],
        'regressor__learning_rate': [0.01, 0.1, 0.2],
        'regressor__max_depth': [3, 5, 7]
    },
    # DecisionTreeRegressor search space
    {
        'regressor': [DecisionTreeRegressor()],
        'regressor__max_depth': [None, 10, 20, 30],
        'regressor__min_samples_split': [2, 5, 10],
        'regressor__min_samples_leaf': [1, 2, 4]
    },
    # Ridge Regression search space
    {
        'regressor': [Ridge(max_iter=10000)], # Increase max_iter
        'regressor__alpha': [0.01, 0.1, 1, 10, 100]
    },
    # Lasso Regression search space
    {
        'regressor': [Lasso(max_iter=10000)], # Increase max_iter
        'regressor__alpha': [0.01, 0.1, 1, 10, 100]
    },
    # ElasticNet search space
    {
        'regressor': [ElasticNet(max_iter=10000)], # Increase max_iter
        'regressor__alpha': [0.01, 0.1, 1, 10, 100],
        'regressor__l1_ratio': [0.1, 0.5, 0.9]
    },
    # XGBoost Regressor search space
    {
        'regressor': [XGBRegressor()],
        'regressor__n_estimators': [100, 200, 300],
        'regressor__learning_rate': [0.01, 0.1, 0.2],
        'regressor__max_depth': [3, 5, 7]
    },
    # AdaBoost Regressor search space
    {
        'regressor': [AdaBoostRegressor()],
        'regressor__n_estimators': [50, 100, 200],
        'regressor__learning_rate': [0.01, 0.1, 1]
    }
]

# Initialize GridSearchCV
grid_search = GridSearchCV(estimator=pipeline, param_grid=grid_search_space, cv=5, n_jobs=-1, verbose=2, scoring='neg_mean_squared_error')

try:
    # Fit the model
    grid_search.fit(x_reduced_train, y_reduced_train)

    # Print the best parameters and regressor
    print("Best parameters found: ", grid_search.best_params_)
    print("Best estimator found: ", grid_search.best_estimator_)

    # Predict and evaluate on test set
    y_pred = grid_search.predict(x_reduced_test)
    mse = mean_squared_error(y_reduced_test, y_pred)
    rmse = mse ** 0.5
    r2 = r2_score(y_reduced_test, y_pred)
    mae = mean_absolute_error(y_reduced_test, y_pred)

    print(f'RMSE: {rmse}, R2: {r2}, MAE: {mae}')

    # Extract and print coefficients or feature importance
    best_regressor = grid_search.best_estimator_.named_steps['regressor']
    if hasattr(best_regressor, 'coef_'):
        coefficients = np.append(best_regressor.intercept_, best_regressor.coef_)
        labels = ["intercept"] + list(x_reduced_train.columns)
        coefs = pd.DataFrame({"variable": labels, "coef": coefficients})
    elif hasattr(best_regressor, "feature_importances_"):
        labels = list(x_reduced_train.columns)
        coefs = pd.DataFrame({"variable": labels, "importance": best_regressor.feature_importances_})

    print(coefs)
    return grid_search.best_estimator_, coefs, rmse, r2, mae

except ConvergenceWarning as cw:
    print("ConvergenceWarning: ", cw)
except Exception as e:
    print("An error occurred: ", e)

return None, None, None, None, None

```

SVOD Data

```

In [377... svod_df = pd.read_csv(SVOD_DATA_FOR_MODELING_DATA_FILE)
show_df_info(svod_df, "Streaming Video on Demand Data")

svod_df = svod_df.rename(columns={
    "vote_count": "Series_Votes"
})

```

Dataframe information for Streaming Video on Demand Data:

Dataframe Information:

	Column	Data Type	NaN Count
0	Title	object	0
1	Platform	object	0
2	Week Ending	object	0
3	Minutes (Millions)	object	0
4	IMDb Series ID	object	0
5	networks	object	0
6	HBO	bool	0
7	Netflix	bool	0
8	AMC	bool	0
9	FOX	bool	0
10	The CW	bool	0
11	ABC	bool	0
12	Disney+	bool	0
13	CBS	bool	0
14	Prime Video	bool	0
15	Adult Swim	bool	0
16	NBC	bool	0
17	Hulu	bool	0
18	USA Network	bool	0
19	Cinemax	bool	0
20	Comedy Central	bool	0
21	Showtime	bool	0
22	Paramount+	bool	0
23	Paramount Network	bool	0
24	Lifetime	bool	0
25	Starz	bool	0
26	Telemundo	bool	0
27	Apple TV+	bool	0
28	HBO Max	bool	0
29	BET	bool	0
30	Hallmark Channel	bool	0
31	Peacock	bool	0
32	id	float64	0
33	number_of_seasons	float64	0
34	number_of_episodes	float64	0
35	original_language	object	0
36	vote_count	float64	0
37	vote_average	float64	0
38	adult	bool	0
39	backdrop_path	object	198
40	first_air_date	object	0
41	last_air_date	object	0
42	in_production	bool	0
43	popularity	float64	0
44	type	object	0
45	status	object	0
46	created_by	object	586
47	languages	object	0
48	origin_country	object	0
49	spoken_languages	object	0
50	production_companies	object	138
51	production_countries	object	259
52	episode_run_time	object	0
53	Sci-Fi & Fantasy	bool	0
54	Drama	bool	0
55	Action & Adventure	bool	0
56	Crime	bool	0
57	Mystery	bool	0
58	Comedy	bool	0
59	Animation	bool	0
60	Family	bool	0
61	War & Politics	bool	0
62	Soap	bool	0
63	Western	bool	0
64	Kids	bool	0
65	Documentary	bool	0
66	Reality	bool	0
67	Romance	bool	0
68	Adult	bool	0
69	Biography	bool	0
70	Film-Noir	bool	0
71	Game-Show	bool	0
72	History	bool	0
73	Horror	bool	0
74	Musical	bool	0
75	Music	bool	0
76	News	bool	0
77	Short	bool	0
78	Sport	bool	0
79	Talk-Show	bool	0
80	Thriller	bool	0

There are 5849 rows and 81 columns of information.

```
In [378... svod_df['Minutes (Millions)'] = svod_df['Minutes (Millions)']
svod_df['Minutes (Millions)'] = pd.to_numeric(svod_df['Minutes (Millions)'].str.replace(',', ''), errors='coerce')
```

```
In [379... # Descriptive statistics for 'Viewers (millions)'
svod_stats = {
    'Mean': svod_df['Minutes (Millions)'].mean(),
```

```

'Median': svod_df['Minutes (Millions)'].median(),
'Mode': svod_df['Minutes (Millions)'].mode().values, # mode() can return multiple values
'Variance': svod_df['Minutes (Millions)'].var(),
'Standard Deviation': svod_df['Minutes (Millions)'].std(),
'Summary': svod_df['Minutes (Millions)'].describe()
}

# Printing results
print("Minutes (millions) Statistics:")
for key, value in svod_stats.items():
    print(f"{key}: {value}")

```

```

Minutes (millions) Statistics:
Mean: 633.9964096426739
Median: 544.0
Mode: [425]
Variance: 201487.61028122465
Standard Deviation: 448.8737130655176
Summary: count      5849.000000
mean      633.996410
std       448.873713
min       117.000000
25%       387.000000
50%       544.000000
75%       724.000000
max       7203.000000
Name: Minutes (Millions), dtype: float64

```

SVOD Modeling

```

In [380... # content is episode specific so must be False here.
def svod_regressor_model_run(df, pca_components = 5, fuzziness=20, genres=True, networks=True, optimized=True):

    Y = df['Minutes (Millions)']

    if pca_components!=0:
        X, loadings_df, pca = svod_fit_reduce(df, n_components=pca_components, fuzziness = fuzziness, genres=genres, networks=networks, content=False)
    else:
        X = svod_fit_reduce(df, n_components=pca_components, fuzziness = fuzziness, genres=genres, networks=networks, content=False)

    # get test and train sets
    x_reduced_train, x_reduced_test, y_reduced_train, y_reduced_test = train_test_split(X, Y, test_size=0.2, random_state=42)

    best_estimator, coefs, rmse, r2, mae = regressor_model_run(x_reduced_train, x_reduced_test, y_reduced_train, y_reduced_test, optimized)

    print("Best SVOD regression estimator: ", best_estimator)

    predictions = best_estimator.predict(X)
    residuals = Y - predictions

    if pca_components!=0:
        return best_estimator, coefs, rmse, r2, mae, X, Y, predictions, residuals, loadings_df, pca
    else:
        return best_estimator, coefs, rmse, r2, mae, X, Y, predictions, residuals,

```

```

In [383... best_estimator, coefs, rmse, r2, mae, X, Y, predictions, residuals, loadings_df, pca = \
    svod_regressor_model_run(svod_df, pca_components = 5, fuzziness=20, genres=True, networks=True, optimized=True)

print("Best estimator: ", best_estimator)
print(f'RMSE for Minutes (millions): {rmse}')
print(f'R2 for Minutes (millions): {r2}')
print(f'MAE for Minutes (millions): {mae}')

Ranking 433 series in 22.0 classification pockets.
Fitting 5 folds for each of 259 candidates, totalling 1295 fits
Best parameters found: {'regressor': RandomForestRegressor(), 'regressor__max_depth': 30, 'regressor__max_features': 'log2', 'regressor__min_samples_le
af': 2, 'regressor__min_samples_split': 5, 'regressor__n_estimators': 50}
Best estimator found: Pipeline(steps=[('scaler', MinMaxScaler()),
    ('regressor',
      RandomForestRegressor(max_depth=30, max_features='log2',
                           min_samples_leaf=2, min_samples_split=5,
                           n_estimators=50))])
RMSE: 369.44558331320513, R2: 0.37595029421340864, MAE: 198.38999348247228
variable importance
0    PC1    0.224112
1    PC2    0.186823
2    PC3    0.181743
3    PC4    0.170852
4    PC5    0.236470
Best SVOD regression estimator: Pipeline(steps=[('scaler', MinMaxScaler()),
    ('regressor',
      RandomForestRegressor(max_depth=30, max_features='log2',
                           min_samples_leaf=2, min_samples_split=5,
                           n_estimators=50))])
Best estimator: Pipeline(steps=[('scaler', MinMaxScaler()),
    ('regressor',
      RandomForestRegressor(max_depth=30, max_features='log2',
                           min_samples_leaf=2, min_samples_split=5,
                           n_estimators=50))])
RMSE for Minutes (millions): 369.44558331320513
R2 for Minutes (millions): 0.37595029421340864
MAE for Minutes (millions): 198.38999348247228

```

```

In [383... show_svod_regression_results("SVOD Regression", svod_df, 'Minutes (Millions)', loadings_df, \
    select_features(main_labels = svod_feature_labels, genres=True, networks=True, content=False), \
    best_estimator, rmse, r2, mae, \
    coefs, X, Y, predictions, residuals)

```

SVOD Regression

Viewers (millions) Statistics:

Mean: 633.9964096426739

Median: 544.0

Mode: [425]

Variance: 201487.61028122465

Standard Deviation: 448.8737130655176

Summary: count 5849.000000

mean 633.996410

std 448.873713

min 117.000000

25% 387.000000

50% 544.000000

75% 724.000000

max 7203.000000

Name: Minutes (Millions), dtype: float64

```
Best Estimator for Minutes (Millions): Pipeline(steps=[('scaler', MinMaxScaler()),
('regressor',
RandomForestRegressor(max_depth=30, max_features='log2',
min_samples_leaf=2, min_samples_split=5,
n_estimators=50))])
```

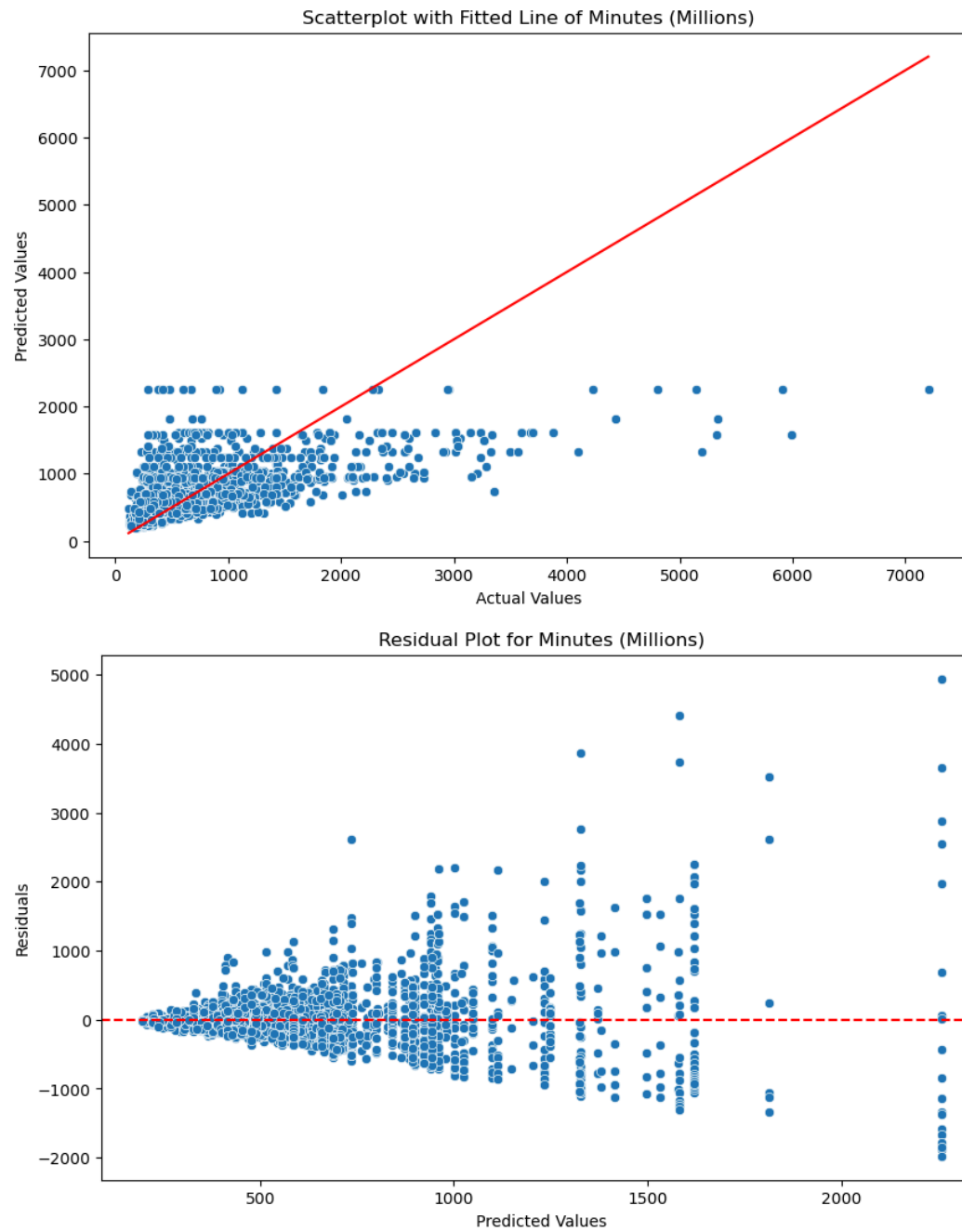
Root mean square error: 369.44558331320513

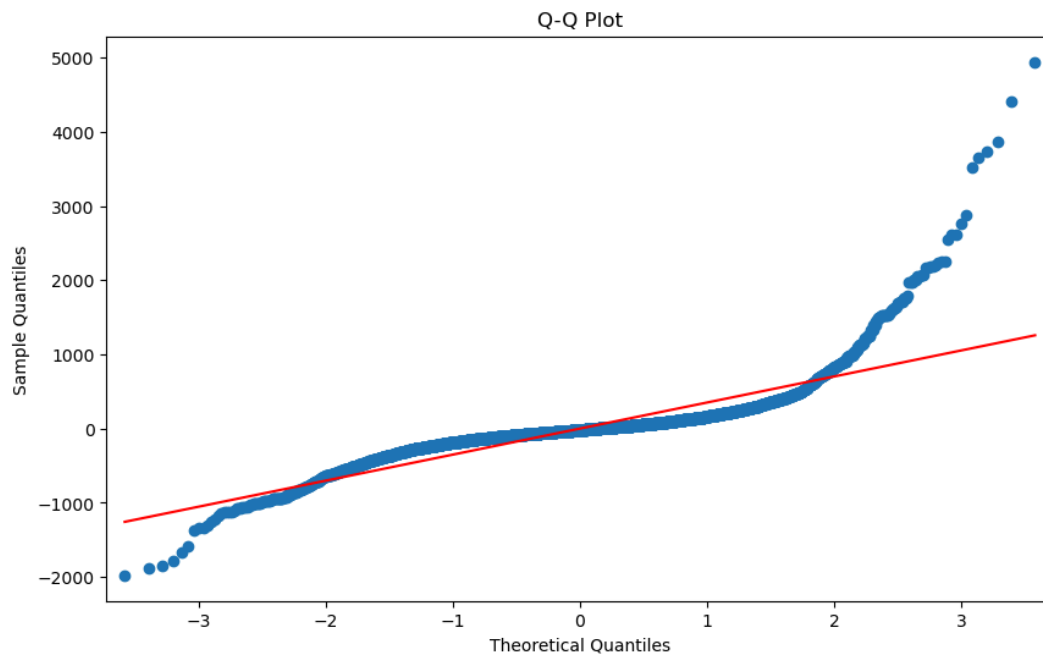
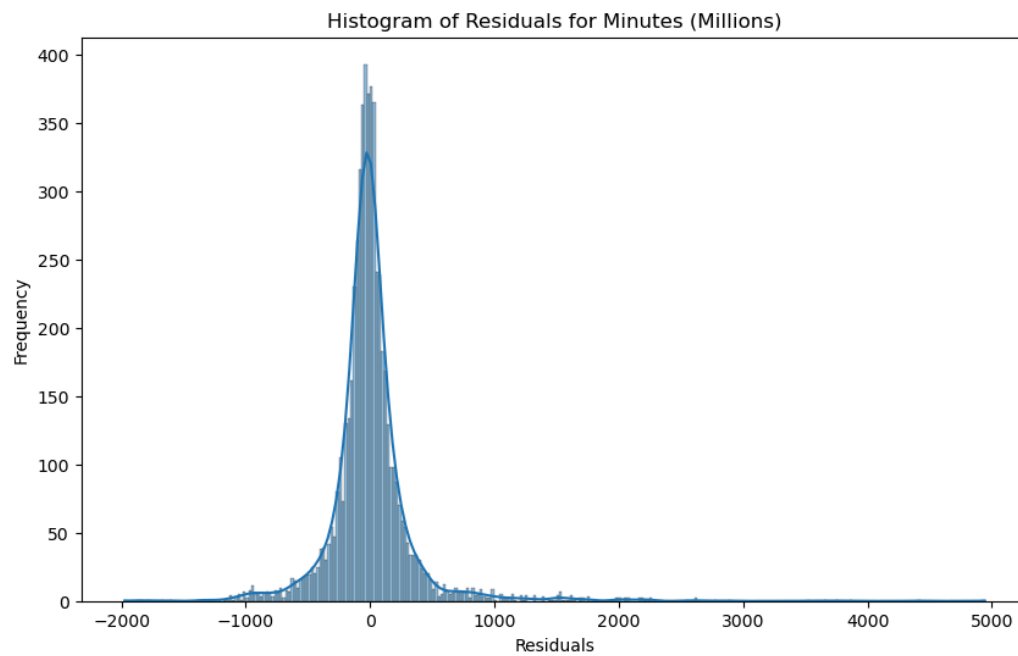
r2: 0.37595029421340864

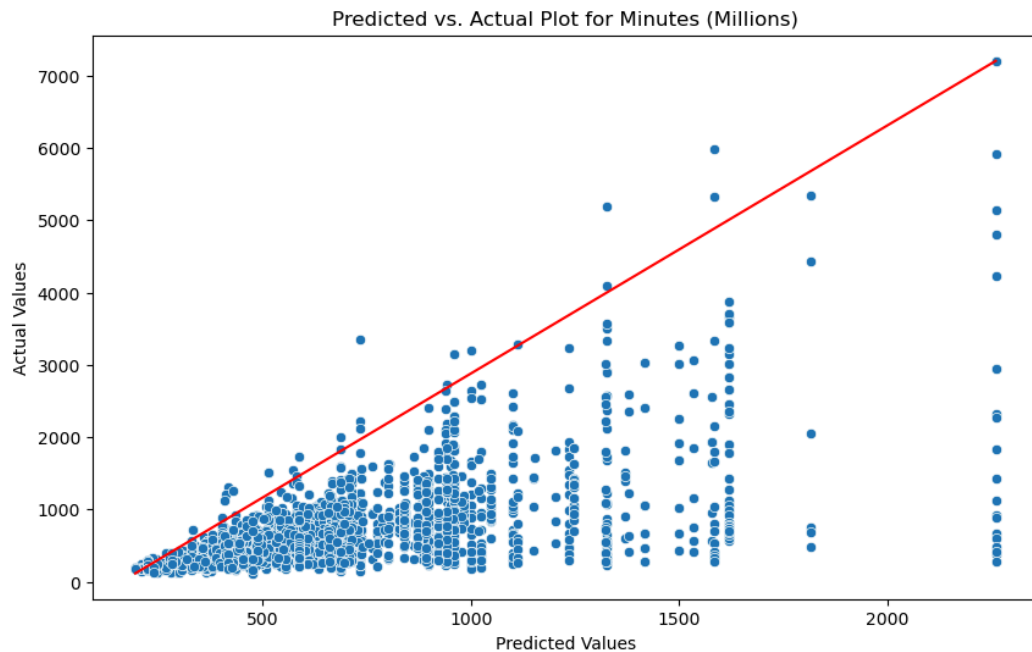
Mean average error: 198.38999348247228

Feature_Label	PC1	PC2	PC3	PC4	PC5
Series_Votes	-0.03954679916280632	0.013578530597164106	-0.13078727710778232	-0.007407745495844531	0.043316592498588394
vote_average	-4.749515268884822e-05	-0.04045293181364934	-0.037529178663649104	0.021584080892869617	0.010616590508029032
popularity	-0.03344340298819627	-0.09296044804457812	-0.14623474059164615	-0.1664261737844787	-0.17310487402186397
HBO	0.0544781608602126	-0.037936941803578365	-0.08653003647060754	0.07844337721871744	0.042468401180325535
Netflix	-6.992108584880751e-23	3.3789133258017796e-20	8.793969453943429e-20	-2.9252040888956305e-19	-9.471747825859056e-19
AMC	-1.7720902358765997e-23	1.731337190075072e-20	5.455128151731066e-20	3.895563326206773e-20	-8.265875841806388e-19
FOX	1.7542931544898343e-24	-5.422219513864687e-23	1.4490341734260542e-21	6.053032538991487e-21	9.151392463534693e-21
The CW	-1.364849149084827e-24	4.0778212595665335e-23	-1.0822698583592375e-21	-4.107740313082684e-21	-7.080674462013772e-21
ABC	-1.1067085231966779e-25	1.931315484392954e-23	-8.488118054161538e-23	-5.071492310131886e-22	-1.4260310575689335e-21
Disney+	5.609987870377044e-26	-2.759788210161745e-23	3.947784564286487e-23	4.506227395359608e-22	1.7431753924262485e-21
CBS	-1.871656301358557e-29	-1.066013207869372e-26	-1.2579839060732385e-26	1.838947328428648e-26	5.324314037686891e-25
Prime Video	-4.744426199357347e-31	-3.887017280596924e-27	-1.8599580905408915e-26	4.965245392102419e-26	9.78132234938783e-25
Adult Swim	-2.8009037053297123e-29	-1.9355454650469515e-26	-1.482520810157965e-27	1.1590147803819558e-25	5.164027158779813e-25
NBC	3.50341047822681e-32	3.4531685821027213e-29	4.2718799414898197e-29	-2.5544119374630456e-28	-2.859406498712368e-27
Hulu	0.0	0.0	-0.0	-0.0	0.0
USA Network	0.0	0.0	-0.0	-0.0	0.0
Cinemax	0.0	0.0	-0.0	-0.0	0.0
Comedy Central	0.0	0.0	-0.0	-0.0	0.0
Showtime	0.0	0.0	-0.0	-0.0	0.0
Paramount+	0.0	0.0	-0.0	-0.0	0.0
Paramount Network	0.0	0.0	-0.0	-0.0	0.0
Lifetime	0.0	0.0	-0.0	-0.0	0.0
Starz	0.0	0.0	-0.0	-0.0	0.0
Telemundo	0.0	0.0	-0.0	-0.0	0.0
Apple TV+	0.0	0.0	-0.0	-0.0	0.0
HBO Max	0.0	0.0	-0.0	-0.0	0.0
BET	0.0	0.0	-0.0	-0.0	0.0
Hallmark Channel	0.0	0.0	-0.0	-0.0	0.0
Peacock	0.0	0.0	-0.0	-0.0	0.0
Sci-Fi & Fantasy	-0.0836244115811598	0.5094088190337351	-0.4028188870011579	0.09695271392525424	0.5301344104040896
Drama	-0.597436660992121	-0.2326791775592236	-0.2561311630856518	-0.16781281589940863	-0.22989413451961876
Action & Adventure	-0.21192378239235551	0.4634907235787516	-0.13684966114234276	0.6328741412338145	-0.3020317313911474
Crime	-0.3573244464901514	-0.14609533380079112	0.5805545177161533	0.46466061373963946	-0.031132917642495567
Mystery	-0.20077623305543413	-0.13758320172218527	0.29930775241951707	0.028006469103375006	0.7078955231947525
Comedy	0.48769981091900816	-0.49263648570610974	-0.2632886961075991	0.5210949145599074	0.08031421108960449
Animation	0.31604574206600633	0.3276267937552763	0.2971468984710152	-0.0629797705587832	-0.10058275728139632
Family	0.07223536648792972	-0.10526670863857487	-0.11698242871696818	0.09748598479177517	-0.014054725440245835
War & Politics	-0.005042476967490744	0.0063397138916259565	-0.005978482303795189	0.001368468641998546	-0.01729717898888527
Soap	-0.0012226565916831581	-0.0015491205707076662	0.00171209710049377	-0.003031804080098439	-0.0005575360884297803
Western	-0.010625682732380631	-0.002292745218091271	0.0073537215095047515	-0.017657288218875355	-0.00829862535645324
Kids	0.2705217347798214	0.23845956181127187	0.3247609320806831	-0.12789559683984117	-0.1466193447688332
Documentary	0.008789388204722496	0.007843188101333977	0.0314459161282264	-0.014930576992253008	0.004578664703775569
Reality	0.034218366537540824	0.03579519387833656	0.039218109847797084	-0.0944980768452528	0.019702149999233226
Romance	-0.00021444207215915	-0.00014983184369812358	-0.00020128086863329262	-0.0009517079525414747	-0.0005045739306085528
Adult	0.0	0.0	-0.0	-0.0	0.0
Biography	0.0	0.0	-0.0	-0.0	0.0
Film-Noir	0.0	0.0	-0.0	-0.0	0.0
Game-Show	0.0	0.0	-0.0	-0.0	0.0
History	0.0	0.0	-0.0	-0.0	0.0
Horror	0.0	0.0	-0.0	-0.0	0.0
Musical	0.0	0.0	-0.0	-0.0	0.0
Music	0.0	0.0	-0.0	-0.0	0.0
News	0.0	0.0	-0.0	-0.0	0.0
Short	0.0	0.0	-0.0	-0.0	0.0
Sport	0.0	0.0	-0.0	-0.0	0.0
Talk-Show	0.0	0.0	-0.0	-0.0	0.0
Thriller	0.0	0.0	-0.0	-0.0	0.0

variable	importance
PC1	0.22411163437623116
PC2	0.18682322994268763
PC3	0.18174308438246967
PC4	0.17085227309060007
PC5	0.2364697782079315







Analysis: Not Great

- Barely within one standard deviation.
- Not much data and high variance.

In [384...

```
param_grid_dt = {
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 10, 20],
    'min_samples_leaf': [1, 5, 10]
}

grid_search_dt = GridSearchCV(DecisionTreeRegressor(random_state=42), param_grid=param_grid_dt, cv=5,
                              scoring='neg_mean_squared_error', verbose=1, n_jobs=-1)
grid_search_dt.fit(X_train, y_train)

best_dt = grid_search_dt.best_estimator_
print("Best parameters for Decision Tree:", grid_search_dt.best_params_)

# Make predictions
y_pred = grid_search_dt.predict(X_test)

# Calculate RMSE
rmse = mean_squared_error(y_test, y_pred)
rmse = rmse ** 0.5
print(f'RMSE: {rmse}')
```

Fitting 5 folds for each of 36 candidates, totalling 180 fits
 Best parameters for Decision Tree: {'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 20}
 RMSE: 0.5845975323451119

Grid Search

- Find best classifier and regressor
- Classifier has data independent of series
- Regressor has blurry series data as a grouping mechanism

Classifier Grid

Optimized Version

In [385...

```
def optimized_classifier_model_run(df, target, sampling_rate, pca_components=3, series_grouping=True, fuzziness=300,
                                  genres=False, networks=False, content=False):

    if pca_components > 0:
        X, Y, loadings_df, pca = fit_sample_and_reduce(sampling_rate, series_survival_df, target,
                                                       pca_components, series_grouping, fuzziness, genres=genres, networks=networks, content=content)
    else:
        X, Y = fit_sample_and_reduce(sampling_rate, series_survival_df, target, 0, series_grouping, fuzziness,
                                     genres=genres, networks=networks, content=content)

    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

    # Define the pipeline
    pipeline = Pipeline([
        ('classifier', RandomForestClassifier())
    ])

    # Define a grid search with various classifiers
```

```

grid_search_space = [
    # RandomForestClassifier search space
    {
        'classifier': [RandomForestClassifier()],
        'classifier__n_estimators': [10, 100],
        'classifier__max_features': [1, 2]
    },
    # ExtraTreesClassifier search space
    {
        'classifier': [ExtraTreesClassifier()],
        'classifier__n_estimators': [100, 200],
        'classifier__max_depth': [None, 10],
        'classifier__min_samples_split': [2, 5],
        'classifier__min_samples_leaf': [1, 2],
        'classifier__max_features': ['sqrt', 'log2']
    },
    # Logistic Regression search space with Liblinear solver
    {
        'classifier': [LogisticRegression(max_iter=1000, solver='liblinear')],
        'classifier__penalty': ['l1', 'l2'],
        'classifier__C': [0.01, 0.1, 1, 10]
    },
    # Gradient Boosting Classifier search space
    {
        'classifier': [GradientBoostingClassifier()],
        'classifier__n_estimators': [100, 200],
        'classifier__learning_rate': [0.01, 0.1],
        'classifier__max_depth': [3, 5]
    },
    # AdaBoost Classifier search space
    {
        'classifier': [AdaBoostClassifier()],
        'classifier__n_estimators': [50, 100],
        'classifier__learning_rate': [0.01, 0.1],
        'classifier__algorithm': ['SAMME']
    },
    # XGBoost Classifier search space
    {
        'classifier': [xgb.XGBClassifier()],
        'classifier__n_estimators': [100, 200],
        'classifier__learning_rate': [0.01, 0.1],
        'classifier__max_depth': [3, 5]
    }
]

# Initialize GridSearchCV with StratifiedKFold
# Added StratifiedKFold cross-validation to validate the model's performance robustly and ensure it generalizes well to unseen data.
stratified_k_fold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid_search = GridSearchCV(estimator=pipeline, param_grid=grid_search_space, cv=stratified_k_fold, n_jobs=-1, verbose=2)

# Fit the model
start_time = time.time()
grid_search.fit(X_train, Y_train)
end_time = time.time()

# Print the best parameters and classifier
print("Best parameters found: ", grid_search.best_params_)
print("Best estimator found: ", grid_search.best_estimator_)

# Cross-Validation Scores
cv_scores = cross_val_score(grid_search, X, Y, cv=stratified_k_fold, scoring='accuracy')
print("Cross-validation scores:", cv_scores)
print("Mean cross-validation score:", cv_scores.mean())

# Predict and evaluate on test set
y_pred = grid_search.predict(X_test)
print(classification_report(Y_test, y_pred))

# Calculate confusion matrix
conf_matrix = confusion_matrix(Y_test, y_pred)
tn, fp, fn, tp = conf_matrix.ravel()
# Calculate specificity
specificity = tn / (tn + fp)

# Print confusion matrix
print("Confusion Matrix:")
print(conf_matrix)

# Calculate additional metrics
accuracy = accuracy_score(Y_test, y_pred)
precision = precision_score(Y_test, y_pred)
recall = recall_score(Y_test, y_pred)
f1 = 2 * (precision * recall) / (precision + recall)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Specificity:", specificity)

# Calculate the elapsed time
elapsed_time = end_time - start_time
print("Elapsed time:", elapsed_time, "seconds")

# Calculate feature importances if the classifier has the attribute
if hasattr(grid_search.best_estimator_['classifier'], 'feature_importances_'):
    feature_importances = grid_search.best_estimator_['classifier'].feature_importances_

```

```

features = X_train.columns

# Create a DataFrame for easier visualization of feature importances
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importances
})
importance_df = importance_df.sort_values(by='Importance', ascending=False) # Sort by importance

print("Feature importances:")
print(importance_df)
else:
    importance_df = None

if pca_components > 0:
    return grid_search.best_estimator_, conf_matrix, accuracy, precision, recall, specificity, importance_df, cv_scores, loadings_df, pca
else:
    return grid_search.best_estimator_, conf_matrix, accuracy, precision, recall, specificity, importance_df, cv_scores

```

Usage

- First run without genre, networks, and content
- No PCA
- Fuzziness of 300 = 4 boxes of grouping

In [386...

```

sample_rate = 0.25
# use pca
pca_components = 0
series_grouping = True
fuzziness = 300

print("Attempt to Predict Status Canceled\n")
canceled_estimator, canceled_conf_matrix, canceled_accuracy, canceled_precision, canceled_recall, \
    canceled_specificity, canceled_importance_df, canceled_cv_scores = \
    optimized_classifier_model_run(series_survival_df, series_survival_df['status_Canceled'], sample_rate, \
        pca_components, series_grouping, fuzziness)

print("\n\nAttempt to Predict Status Ended\n")
ended_estimator, ended_conf_matrix, ended_accuracy, ended_precision, ended_recall, \
    ended_specificity, ended_importance_df, ended_cv_scores = \
    optimized_classifier_model_run(series_survival_df, series_survival_df['status_Ended'], sample_rate, pca_components, series_grouping, fuzziness)

```

```
Attempt to Predict Status Canceled

Ranking 585 series in 2.0 classification pockets.
fit sample at rate 0.25 & reduce to 0 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rat
ing', 'IMDb_Episode_Votes']
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Best parameters found: {'classifier': ExtraTreesClassifier(), 'classifier__max_depth': None, 'classifier__max_features': 'sqrt', 'classifier__min_sampl
es_leaf': 2, 'classifier__min_samples_split': 5, 'classifier__n_estimators': 100}
Best estimator found: Pipeline(steps=[('classifier',
    ExtraTreesClassifier(min_samples_leaf=2,
                        min_samples_split=5))])
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Cross-validation scores: [0.78752719 0.76867295 0.75616836 0.77866473 0.7670537 ]
Mean cross-validation score: 0.7716173874970924
precision    recall    f1-score    support

   False      0.82      0.77      0.80      847
   True       0.67      0.73      0.70      532

 accuracy          0.76      0.76      0.76      1379
 macro avg         0.74      0.75      0.75      1379
weighted avg         0.76      0.76      0.76      1379

Confusion Matrix:
[[655 192]
 [144 388]]
Accuracy: 0.7563451776649747
Precision: 0.6689655172413793
Recall: 0.7293233082706767
F1 Score: 0.697841726618705
Specificity: 0.7733175914994097
Elapsed time: 7.160754919052124 seconds
Feature importances:
Feature Importance
3    fuzzy_popularity    0.429876
1      Episode_Votes    0.157312
5    IMDb_Episode_Votes    0.151848
2    fuzzy_vote_average    0.104599
0         rating         0.081747
4    IMDb_Episode_Rating    0.074617
```

```
Attempt to Predict Status Ended

Ranking 585 series in 2.0 classification pockets.
fit sample at rate 0.25 & reduce to 0 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rat
ing', 'IMDb_Episode_Votes']
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Best parameters found: {'classifier': XGBClassifier(base_score=None, booster=None, callbacks=None,
    colsample_bylevel=None, colsample_bynode=None,
    colsample_bytree=None, device=None, early_stopping_rounds=None,
    enable_categorical=False, eval_metric=None, feature_types=None,
    gamma=None, grow_policy=None, importance_type=None,
    interaction_constraints=None, learning_rate=None, max_bin=None,
    max_cat_threshold=None, max_cat_to_onehot=None,
    max_delta_step=None, max_depth=None, max_leaves=None,
    min_child_weight=None, missing=nan, monotone_constraints=None,
    multi_strategy=None, n_estimators=None, n_jobs=None,
    num_parallel_tree=None, random_state=None, ...), 'classifier__learning_rate': 0.01, 'classifier__max_depth': 5, 'classifier__n_estimator
s': 200}
Best estimator found: Pipeline(steps=[('classifier',
    XGBClassifier(base_score=None, booster=None, callbacks=None,
                  colsample_bylevel=None, colsample_bynode=None,
                  colsample_bytree=None, device=None,
                  early_stopping_rounds=None,
                  enable_categorical=False, eval_metric=None,
                  feature_types=None, gamma=None, grow_policy=None,
                  importance_type=None,
                  interaction_constraints=None, learning_rate=0.01,
                  max_bin=None, max_cat_threshold=None,
                  max_cat_to_onehot=None, max_delta_step=None,
                  max_depth=5, max_leaves=None,
                  min_child_weight=None, missing=nan,
                  monotone_constraints=None, multi_strategy=None,
                  n_estimators=200, n_jobs=None,
                  num_parallel_tree=None, random_state=None, ...))])
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Fitting 5 folds for each of 64 candidates, totalling 320 fits
Cross-validation scores: [0.78752719 0.76867295 0.75399129 0.77866473 0.7670537 ]
Mean cross-validation score: 0.7711819738541317
precision    recall    f1-score    support

   False      0.68      0.73      0.71      532
   True       0.82      0.79      0.81      847

 accuracy          0.77      0.77      0.77      1379
 macro avg         0.75      0.76      0.76      1379
weighted avg         0.77      0.77      0.77      1379

Confusion Matrix:
[[388 144]
```

```
[179 668]]
Accuracy: 0.7657722987672226
Precision: 0.8226600985221675
Recall: 0.7886658795749705
F1 Score: 0.805304400241109
Specificity: 0.7293233082706767
Elapsed time: 8.152930498123169 seconds
Feature importances:
      Feature  Importance
3  fuzzy_popularity  0.894246
1    Episode_Votes  0.034334
4  IMdb_Episode_Rating  0.025057
2  fuzzy_vote_average  0.018900
5  IMdb_Episode_Votes  0.015709
0           rating    0.011755
```

Confusion Matrices of Classifier

```
In [387... # Define true and predicted labels for each matrix
true_labels_canceled = ['Actually Not Canceled', 'Actually Canceled']
predicted_labels_canceled = ['Predicted Not Canceled', 'Predicted Canceled']

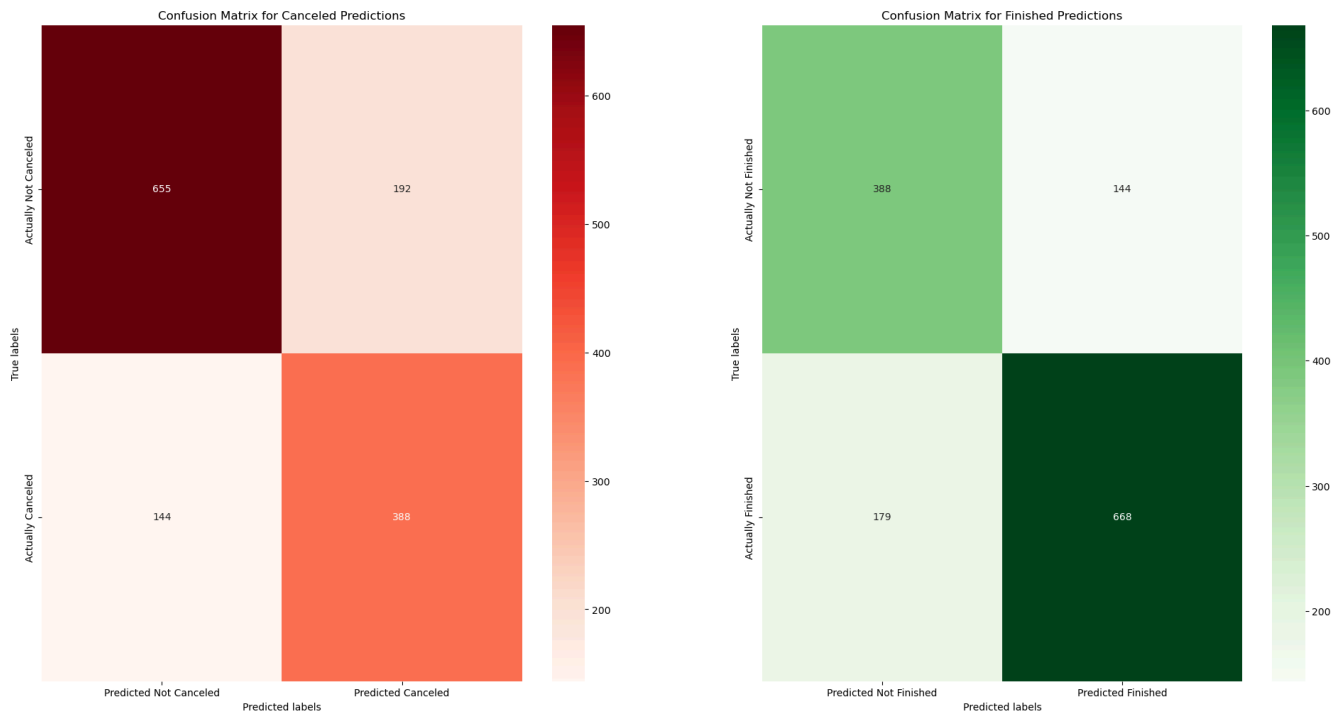
true_labels_ended = ['Actually Not Finished', 'Actually Finished']
predicted_labels_ended = ['Predicted Not Finished', 'Predicted Finished']

# Create a figure to hold the subplots
plt.figure(figsize=(24, 12)) # Adjust the overall size of the figure

# Plot the first confusion matrix
plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st subplot
sns.heatmap(canceled_conf_matrix, annot=True, cmap='Reds', fmt='d', xticklabels=predicted_labels_canceled, yticklabels=true_labels_canceled)
plt.xlabel('Predicted labels')
plt.ylabel('True labels')
plt.title('Confusion Matrix for Canceled Predictions')

# Plot the second confusion matrix
plt.subplot(1, 2, 2) # 1 row, 2 columns, 2nd subplot
sns.heatmap(ended_conf_matrix, annot=True, cmap='Greens', fmt='d', xticklabels=predicted_labels_ended, yticklabels=true_labels_ended)
plt.xlabel('Predicted labels')
plt.ylabel('True labels')
plt.title('Confusion Matrix for Finished Predictions')

# Display the plots
plt.show()
```



Feature Importance - Ended

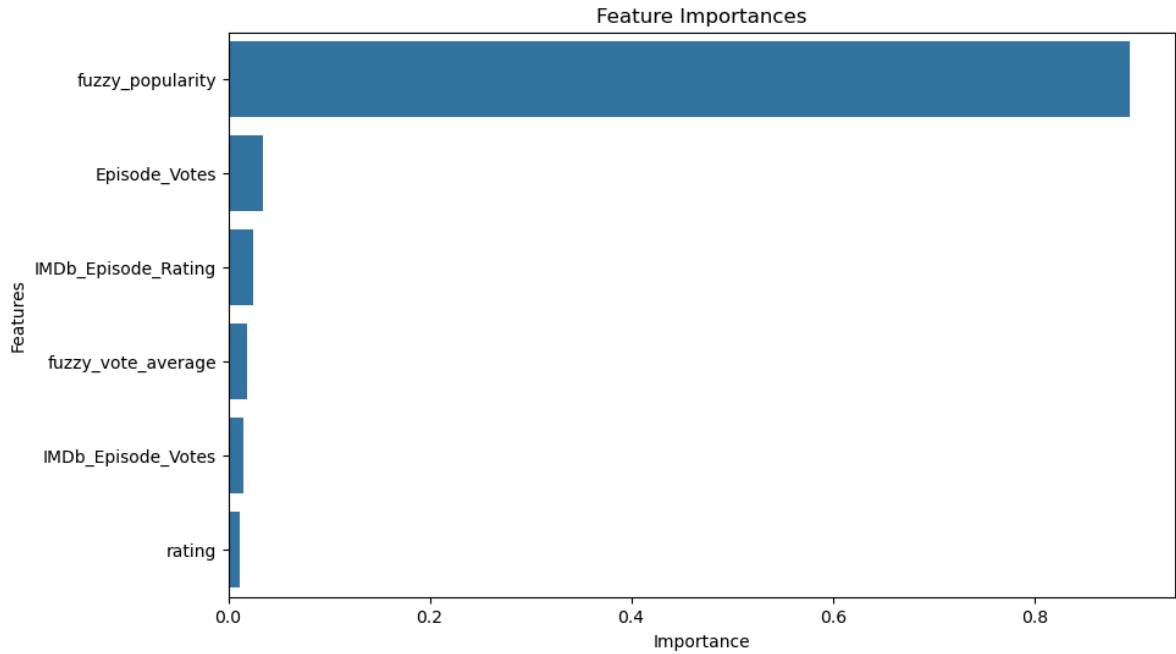
```
In [388... pretty_print_df(ended_importance_df)

warnings.filterwarnings("ignore", category=DeprecationWarning)

# Plotting the feature importances for better visual interpretation
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=ended_importance_df.head(10))
plt.title('Feature Importances')
plt.xlabel('Importance')
plt.ylabel('Features')
plt.show()

warnings.filterwarnings("error", category=DeprecationWarning)
```

Feature	Importance
fuzzy_popularity	0.8942456841468811
Episode_Votes	0.03433409333229065
IMDb_Episode_Rating	0.025056812912225723
fuzzy_vote_average	0.018900031223893166
IMDb_Episode_Votes	0.01570853963494301
rating	0.011754804290831089



Feature Importance - Canceled

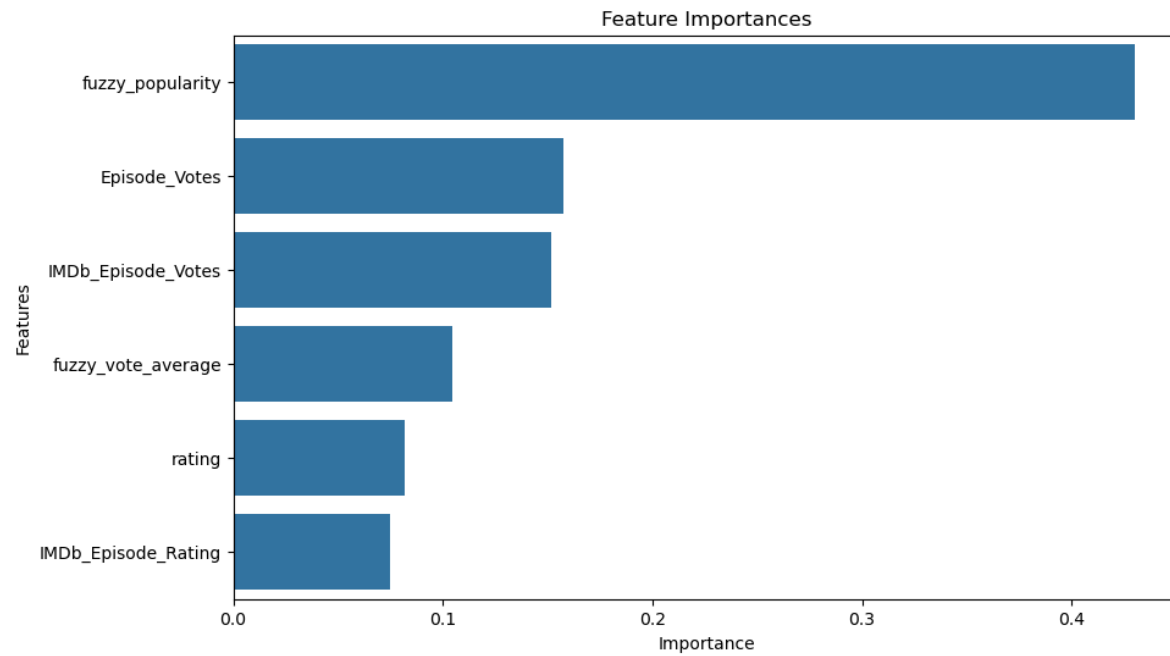
```
In [389... pretty_print_df(canceled_importance_df)

warnings.filterwarnings("ignore", category=DeprecationWarning)

# Plotting the feature importances for better visual interpretation
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=canceled_importance_df.head(10))
plt.title('Feature Importances')
plt.xlabel('Importance')
plt.ylabel('Features')
plt.show()

warnings.filterwarnings("error", category=DeprecationWarning)
```

Feature	Importance
fuzzy_popularity	0.42987632368303735
Episode_Votes	0.15731192493815857
IMDb_Episode_Votes	0.15184840969549573
fuzzy_vote_average	0.1045994355997243
rating	0.08174731555205927
IMDb_Episode_Rating	0.07461659053152495



Show Input Data Frame

```
In [390... def main_set_regressor_model_run(df, target, sampling_rate, pca_components = 5, series_grouping = True, fuzziness=100,
                                genres=True, networks=True, content=True, optimized=True):
    # do feature engineering on features
    # Ensure target is a pandas Series
    if not isinstance(target, pd.Series):
        raise ValueError("target must be a pandas Series")

    if pca_components!=0:
        X, Y, loadings_df, pca = fit_sample_and_reduce(sampling_rate, df, target, pca_components, series_grouping=True,
                                                       fuzziness = fuzziness, genres=genres, networks=networks, content=content)
    else:
        X, Y = fit_sample_and_reduce(sampling_rate, df, target, 0, series_grouping=True, fuzziness = fuzziness,
                                     genres=genres, networks=networks, content=content)

    # get test and train sets
    x_reduced_train, x_reduced_test, y_reduced_train, y_reduced_test = train_test_split(X, Y, test_size=0.2, random_state=42)

    best_estimator, coefs, rmse, r2, mae = regressor_model_run(x_reduced_train, x_reduced_test, y_reduced_train, y_reduced_test, optimized)

    predictions = best_estimator.predict(X)
    residuals = Y - predictions

    if pca_components!=0:
        return best_estimator, coefs, rmse, r2, mae, X, Y, predictions, residuals, loadings_df, pca
    else:
        return best_estimator, coefs, rmse, r2, mae, X, Y, predictions, residuals
```

```
In [391... show_df_info(series_survival_df, "Survival Records")
```


Dataframe information for Survival Records:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	rating	float64	0
5	Episode_Votes	int32	0
6	number_of_seasons	float64	0
7	number_of_episodes	float64	0
8	Series_Votes	float64	0
9	vote_average	float64	0
10	in_production	bool	0
11	popularity	float64	0
12	Sci-Fi & Fantasy	bool	0
13	Drama	bool	0
14	Action & Adventure	bool	0
15	Crime	bool	0
16	Mystery	bool	0
17	Comedy	bool	0
18	Animation	bool	0
19	Family	bool	0
20	War & Politics	bool	0
21	Soap	bool	0
22	Western	bool	0
23	Kids	bool	0
24	Documentary	bool	0
25	Reality	bool	0
26	Romance	bool	0
27	Adult	bool	0
28	Biography	bool	0
29	Film-Noir	bool	0
30	Game-Show	bool	0
31	History	bool	0
32	Horror	bool	0
33	Musical	bool	0
34	Music	bool	0
35	News	bool	0
36	Short	bool	0
37	Sport	bool	0
38	Talk-Show	bool	0
39	Thriller	bool	0
40	HBO	bool	0
41	Netflix	bool	0
42	AMC	bool	0
43	FOX	bool	0
44	The CW	bool	0
45	ABC	bool	0
46	Disney+	bool	0
47	CBS	bool	0
48	Prime Video	bool	0
49	Adult Swim	bool	0
50	NBC	bool	0
51	Hulu	bool	0
52	USA Network	bool	0
53	Cinemax	bool	0
54	Comedy Central	bool	0
55	Showtime	bool	0
56	Paramount+	bool	0
57	Paramount Network	bool	0
58	Lifetime	bool	0
59	Starz	bool	0
60	Telemundo	bool	0
61	Apple TV+	bool	0
62	HBO Max	bool	0
63	BET	bool	0
64	Hallmark Channel	bool	0
65	Peacock	bool	0
66	_18_to_49_Demo	float64	0
67	Viewers_millions	float64	0
68	sex_factor	int64	0
69	violence_factor	int64	0
70	profanity_factor	int64	0
71	drugs_factor	int64	0
72	intense_factor	int64	0
73	IMDb_Episode_Rating	float64	0
74	IMDb_Episode_Votes	int64	0
75	status_Canceled	bool	0
76	status_Ended	bool	0
77	status_Returning Series	bool	0
78	type_Documentary	bool	0
79	type_Miniseries	bool	0
80	type_News	bool	0
81	type_Reality	bool	0
82	type_Scripted	bool	0
83	type_Talk Show	bool	0
84	ranking_of_Series_Votes	float64	0
85	ranking_of_vote_average	float64	0
86	ranking_of_popularity	float64	0
87	fuzzy_Series_Votes	float64	0
88	fuzzy_vote_average	float64	0
89	fuzzy_popularity	float64	0

There are 27566 rows and 90 columns of information.

```
In [392... # didn't use pca
save_canceled_estimator = canceled_estimator
save_ended_estimator = ended_estimator
```

Regressor Grid Model

- regressors use blurry series data for grouping

Example of Use of Regressor Model

- Later, I will sample over a larger set.
- Then I will set optimization true.

```
In [393... sampling_rate = 0
pca_components = 5

series_episode_subset_modeling_df = data_for_tuning_df.copy()

best_estimator_18_to_49, coefs_18_to_49, rmse_18_to_49, r2_18_to_49, mae_18_to_49, X_18_to_49, Y_18_to_49, \
    predictions_18_to_49, residuals_18_to_49, loadings_df_18_to_49, pca_18_to_49 = \
    main_set_regressor_model_run(series_episode_subset_modeling_df, series_episode_subset_modeling_df['_18_to_49_Demo'], \
        sampling_rate, pca_components, optimized=True)

best_estimator_viewers, coefs_viewers, rmse_viewers, r2_viewers, mae_viewers, X_viewers, Y_viewers, \
    predictions_viewers, residuals_viewers, loading_df_viewers, pca_viewers = \
    main_set_regressor_model_run(series_episode_subset_modeling_df, series_episode_subset_modeling_df['Viewers_millions'], \
        sampling_rate, pca_components, optimized=True)
```

Ranking 743 series in 8.0 classification pockets.

fit sample at rate 0 & reduce to 5 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes', 'sex_factor', 'violence_factor', 'profanity_factor', 'drugs_factor', 'intense_factor', 'HBO', 'Netflix', 'AMC', 'FOX', 'The CW', 'ABC', 'Disney+', 'CBS', 'Prime Video', 'Adult Swim', 'NBC', 'Hulu', 'USA Network', 'Cinemax', 'Comedy Central', 'Showtime', 'Paramount+', 'Paramount Network', 'Lifetime', 'Starz', 'Telemundo', 'Apple TV+', 'HBO Max', 'BET', 'Hallmark Channel', 'Peacock', 'Sci-Fi & Fantasy', 'Drama', 'Action & Adventure', 'Crime', 'Mystery', 'Comedy', 'Animation', 'Family', 'War & Politics', 'Soap', 'Western', 'Kids', 'Documentary', 'Reality', 'Romance', 'Adult', 'Biography', 'Film-Noir', 'Game-Show', 'History', 'Horror', 'Musical', 'Music', 'News', 'Short', 'Sport', 'Talk-Show', 'Thriller']

Fitting 5 folds for each of 259 candidates, totalling 1295 fits

Best parameters found: {'regressor': RandomForestRegressor(), 'regressor__max_depth': None, 'regressor__max_features': 'sqrt', 'regressor__min_samples_leaf': 2, 'regressor__min_samples_split': 5, 'regressor__n_estimators': 100}

Best estimator found: Pipeline(steps=[('scaler', MinMaxScaler()),

```
( 'regressor',
    RandomForestRegressor(max_features='sqrt', min_samples_leaf=2,
        min_samples_split=5))])
```

RMSE: 0.30590652344328473, R2: 0.8591475346433551, MAE: 0.13535140363625603

variable	importance
0	PC1 0.153192
1	PC2 0.102952
2	PC3 0.295629
3	PC4 0.205635
4	PC5 0.242591

Ranking 743 series in 8.0 classification pockets.

fit sample at rate 0 & reduce to 5 using pca. Input features: ['rating', 'Episode_Votes', 'fuzzy_vote_average', 'fuzzy_popularity', 'IMDb_Episode_Rating', 'IMDb_Episode_Votes', 'sex_factor', 'violence_factor', 'profanity_factor', 'drugs_factor', 'intense_factor', 'HBO', 'Netflix', 'AMC', 'FOX', 'The CW', 'ABC', 'Disney+', 'CBS', 'Prime Video', 'Adult Swim', 'NBC', 'Hulu', 'USA Network', 'Cinemax', 'Comedy Central', 'Showtime', 'Paramount+', 'Paramount Network', 'Lifetime', 'Starz', 'Telemundo', 'Apple TV+', 'HBO Max', 'BET', 'Hallmark Channel', 'Peacock', 'Sci-Fi & Fantasy', 'Drama', 'Action & Adventure', 'Crime', 'Mystery', 'Comedy', 'Animation', 'Family', 'War & Politics', 'Soap', 'Western', 'Kids', 'Documentary', 'Reality', 'Romance', 'Adult', 'Biography', 'Film-Noir', 'Game-Show', 'History', 'Horror', 'Musical', 'Music', 'News', 'Short', 'Sport', 'Talk-Show', 'Thriller']

Fitting 5 folds for each of 259 candidates, totalling 1295 fits

Best parameters found: {'regressor': RandomForestRegressor(), 'regressor__max_depth': None, 'regressor__max_features': None, 'regressor__min_samples_leaf': 2, 'regressor__min_samples_split': 5, 'regressor__n_estimators': 200}

Best estimator found: Pipeline(steps=[('scaler', MinMaxScaler()),

```
( 'regressor',
    RandomForestRegressor(max_features=None, min_samples_leaf=2,
        min_samples_split=5,
        n_estimators=200))])
```

RMSE: 0.8055253915175268, R2: 0.9307037068853239, MAE: 0.38116900630642053

variable	importance
0	PC1 0.094111
1	PC2 0.062650
2	PC3 0.330622
3	PC4 0.216775
4	PC5 0.295842

```
In [394... save_regression_estimator_18_to_49_demo = best_estimator_18_to_49
save_regression_estimator_viewers = best_estimator_viewers
save_pca_18_to_49_demo = pca_18_to_49
save_pca_viewers = pca_viewers
```

Results

```
In [395... # Insert the new column with feature labels into loadings_df_18_to_49
if 'Feature_Label' not in loadings_df_18_to_49.columns:
    loadings_df_18_to_49.insert(0, 'Feature_Label', select_features(main_labels = fuzzy_series_feature_labels,
        genres=True, networks=True, content=True))

# Print the results
print("Best Estimator for 18 to 49 Demo: ", best_estimator_18_to_49)
print("Root mean square error: ", rmse_18_to_49)
print("r2: ", r2_18_to_49)
print("Mean average error: ", mae_18_to_49)
pretty_print_df(loadings_df_18_to_49)
pretty_print_df(coefs_18_to_49)
```

```
Best Estimator for 18 to 49 Demo: Pipeline(steps=[('scaler', MinMaxScaler()),
('regressor',
RandomForestRegressor(max_features='sqrt', min_samples_leaf=2,
min_samples_split=5))])
```

Root mean square error: 0.30590652344328473

r2: 0.8591475346433551

Mean average error: 0.13535140363625603

Feature_Label	PC1	PC2	PC3	PC4	PC5
rating	-0.05064262016151516	-0.06342916491059726	0.0012947570215445972	-0.0034374570864443474	0.0003680815039197266
Episode_Votes	-0.004074585888950699	-0.002529994492817613	0.004185364046280426	0.005631205511058928	-0.004795386856515494
fuzzy_vote_average	-0.20466750851834925	-0.23449089754813982	0.3425609694834862	0.00295547277442453	-0.3007120613840873
fuzzy_popularity	-0.05667332392891312	0.02360193292824175	0.4864773505970554	0.029161237487896296	-0.4026922826038
IMDb_Episode_Rating	-0.05101724097028444	-0.06395254737910724	0.0008628193164777785	-0.0035955432903270073	0.0005251967308706331
IMDb_Episode_Votes	-0.004062916528934763	-0.002523142013646176	0.004174038336573751	0.0056194826482320725	-0.004780870676897575
sex_factor	-0.0008658934923563049	-0.0005553378968552729	0.0006021645460245545	0.0015199529151362085	-0.0011384947932460744
violence_factor	-0.0011065310209093693	-0.00044921167133270224	0.0005912943076668264	0.0020177012127229697	-0.001353561959942323
profanity_factor	-0.0010833234642656886	-0.0021394553148176153	0.002121760792292045	0.00188654766936126	-0.002366892326519234
drugs_factor	-0.0013204621199558184	-0.0011938502602535815	0.0016780726876306344	0.00042538771400192697	-0.0016301900580357185
intense_factor	-0.0010524842318062053	-0.0004607758150031588	0.000559902139709074	0.001495852403610924	-0.0011074488731988138
HBO	-0.015344134015910107	-0.07611125631582506	-0.02584612812222046	-0.00814170556272988	0.06532561689498179
Netflix	-0.0024420584840150585	-0.00976986076579672	0.003865238683886905	0.022317453337497347	-0.013370686704880674
AMC	-0.02857555049521095	-0.00041176982140835863	0.0010029228519522182	0.039860308719272736	-0.01593448340147336
FOX	0.142761980247612	0.04330790512568089	0.34436051520755856	0.1641337270883536	-0.31200657199898335
The CW	-0.07260433492776643	0.0035412812105143844	-0.008236603817217894	0.14322620092002722	-0.03990487697662374
ABC	0.09497322612071969	0.055458851174749337	-0.3519320925109839	-0.4906392908403235	-0.4951983635006312
Disney+	0.023060549388927334	0.04499276707876444	-0.010750074305013327	-0.036529799932466744	-0.05584385535683514
CBS	-0.014668046685698531	0.14215652426764752	0.3104157761508531	-0.0965534739331619	0.4780682483331555
Prime Video	-0.008440167735541442	0.00014727225573593533	-0.0003910780432355532	0.009498970923804789	-0.0060098945609633565
Adult Swim	0.0033213445264342442	-0.004330708451296324	-0.0014828783099664928	0.006187790123128373	0.004093739029714726
NBC	-0.013311723675696082	-0.015027293273287376	-0.04884169191697482	-0.00045964062653244694	-0.011068712186918591
Hulu	-3.1276772085529655e-05	-0.0032563789288100353	-0.00793260204402068	0.00027947008609099065	-0.0019046965232821176
USA Network	-0.038519911674537624	-0.005213026496190774	-0.00035080659639329104	-0.014678960144064659	0.026734607689233855
Cinemax	-0.0071521232502773785	0.001331731297014362	-0.0017947440239963477	0.005950125105244722	0.004999268463447549
Comedy Central	0.028436770138422825	-0.0438900217151775	0.01889302013474469	0.009789335967593283	0.00552292334965801
Showtime	-0.017759964375281222	-0.05511260049667084	-0.030008311717332058	-0.014075260754689154	0.02631013495112753
Paramount+	-0.02696126225010378	-0.00044860999088117446	0.0470302179557728	-0.03320049221566666	0.04189493085712876
Paramount Network	-0.0011461522543050246	-0.0013752299557036288	-0.000436662627453836	0.00017591163244602752	-0.0016009692336287574
Lifetime	0.00537589974849053	0.020715605679857085	-0.021407992879292317	0.00323282779285875	0.016065707898538107
Starz	-0.01024118085005502	-0.00778929568131736	-0.02341958870892591	0.016181167763082198	0.014135185273389646
Telemundo	-0.0	0.0	0.0	0.0	-0.0
Apple TV+	-0.0	0.0	0.0	0.0	-0.0
HBO Max	0.0053432464814405604	-0.003074622389925401	-0.0037482173276798623	0.001235949873416624	0.009106880552742478
BET	0.00017049089412947465	-0.0019312688359981514	-0.002934575067353295	0.00022974440521386196	0.0023426886242145363
Hallmark Channel	-0.006073692478853398	-0.005889389095220464	-0.007389649443827644	0.008869812333123615	-0.006758854439973927
Peacock	0.013555458282122188	0.037427592374254304	0.01837719000261744	-0.003310756957098572	0.03861010932521633
Sci-Fi & Fantasy	-0.14664963944203205	0.020351730787440084	-0.16262538437862326	0.60322930172699	-0.12496218329054563
Drama	-0.6769210322126961	-0.14846844498549476	-0.2423741521856367	-0.08824834304449893	-0.09038955896662941
Action & Adventure	-0.1525243984942803	0.02636363795670053	0.049469561191418085	0.36851348051453914	0.09804353037461108
Crime	-0.3322098953127639	-0.021750855903549797	0.3985053419199955	-0.37310996452948136	0.19108271791890247
Mystery	-0.18974990413578405	-0.018089692919775163	0.06537926118630691	-0.17742579210324108	0.06521671733669837
Comedy	0.4244284041995145	-0.7104309363810755	0.016850120085542142	-0.057174850119675914	0.14787122995031748
Animation	0.09799864787258731	-0.11879980496676597	0.1882228687165456	0.08362516551693608	-0.14977402492150219
Family	0.09245358096414742	-0.03251840256231355	0.08663595109433106	-0.03935492770167072	-0.2035147609634516
War & Politics	-0.015152845842884686	0.006282527327671735	-0.0035215434295764294	0.0072621104719798124	0.009445391180270551
Soap	-0.010712274934258819	0.0009675674884162799	-0.017792091290370793	-0.00963151438112319	-0.02971025774560537
Western	-0.005633081224602009	0.0009286065426752754	0.0008345852860502238	0.013278309129217722	-0.002076597924136986
Kids	-0.0	0.0	0.0	0.0	-0.0
Documentary	0.0032654650558157565	0.006149134655438112	0.0010520641468318103	9.1062600280779965e-05	0.017243279132234973
Reality	0.2473149731021309	0.5934105782331124	0.068306541256321	-0.07490192130140276	-0.013631057953475873
Romance	-0.0009410484696034003	-0.00018730655895541392	-0.0006700687900625127	4.7015474392297156e-05	-0.00073121406644447505
Adult	-0.0	0.0	0.0	0.0	-0.0
Biography	-0.0	0.0	0.0	0.0	-0.0
Film-Noir	-0.0	0.0	0.0	0.0	-0.0
Game-Show	-0.0	0.0	0.0	0.0	-0.0
History	-0.0	0.0	0.0	0.0	-0.0
Horror	-0.0	0.0	0.0	0.0	-0.0
Musical	-0.0	0.0	0.0	0.0	-0.0
Music	-0.0	0.0	0.0	0.0	-0.0
News	-0.0	0.0	0.0	0.0	-0.0
Short	-0.0	0.0	0.0	0.0	-0.0
Sport	-0.0	0.0	0.0	0.0	-0.0
Talk-Show	-0.0	0.0	0.0	0.0	-0.0
Thriller	-0.0	0.0	0.0	0.0	-0.0

variable	importance
PC1	0.15319232024221863
PC2	0.10295199500656636
PC3	0.2956294289707757
PC4	0.2056335058086157
PC5	0.24259119769428228

```
In [396... # Insert the new column with feature labels into loadings_df_viewers
if 'Feature_Label' not in loadings_df_18_to_49.columns:
    loadings_df_18_to_49.insert(0, 'Feature_Label', select_features(main_labels = fuzzy_series_feature_labels,
                                                                    genres=True, networks=True, content=True))

# Print the results
print("Best Estimator for 18 to 49 Demo): ", best_estimator_18_to_49)
print("Root mean square error: ", rmse_18_to_49)
print("r2: ", r2_18_to_49)
print("Mean average error: ", mae_18_to_49)
pretty_print_df(loadings_df_18_to_49)
pretty_print_df(coefs_18_to_49)
```

```
Best Estimator for 18 to 49 Demo): Pipeline(steps=[('scaler', MinMaxScaler()),
('regressor',
RandomForestRegressor(max_features='sqrt', min_samples_leaf=2,
min_samples_split=5))])
```

Root mean square error: 0.30590652344328473

r2: 0.8591475346433551

Mean average error: 0.13535140363625603

Feature_Label	PC1	PC2	PC3	PC4	PC5
rating	-0.05064262016151516	-0.06342916491059726	0.0012947570215445972	-0.0034374570864443474	0.0003680815039197266
Episode_Votes	-0.004074585888950699	-0.002529994492817613	0.004185364046280426	0.005631205511058928	-0.004795386856515494
fuzzy_vote_average	-0.20466750851834925	-0.23449089754813982	0.3425609694834862	0.00295547277442453	-0.3007120613840873
fuzzy_popularity	-0.05667332392891312	0.02360193292824175	0.4864773505970554	0.029161237487896296	-0.4026922826038
IMDb_Episode_Rating	-0.05101724097028444	-0.06395254737910724	0.0008628193164777785	-0.0035955432903270073	0.0005251967308706331
IMDb_Episode_Votes	-0.004062916528934763	-0.002523142013646176	0.004174038336573751	0.0056194826482320725	-0.004780870676897575
sex_factor	-0.0008658934923563049	-0.0005553378968552729	0.0006021645460245545	0.0015199529151362085	-0.0011384947932460744
violence_factor	-0.0011065310209093693	-0.00044921167133270224	0.0005912943076668264	0.0020177012127229697	-0.001353561959942323
profanity_factor	-0.0010833234642656886	-0.0021394553148176153	0.0021217607692292045	0.00188654766936126	-0.002366892326519234
drugs_factor	-0.0013204621199558184	-0.0011938502602535815	0.0016780726876306344	0.00042538771400192697	-0.0016301900580357185
intense_factor	-0.0010524842318062053	-0.0004607758150031588	0.000559902139709074	0.001495852403610924	-0.0011074488731988138
HBO	-0.015344134015910107	-0.07611125631582506	-0.02584612812222046	-0.00814170556272988	0.06532561689498179
Netflix	-0.0024420584840150585	-0.00976986076579672	0.003865238683886905	0.022317453337497347	-0.013370686704880674
AMC	-0.02857555049521095	-0.00041176982140835863	-0.0010029228159522182	0.039860308719272736	-0.01593448340147336
FOX	0.142761980247612	0.04330790512568089	0.34436051520755856	0.1641337270883536	-0.31200657199898335
The CW	-0.07260433492776643	0.0035412812105143844	-0.008236603817217894	0.14322620092002722	-0.03990487697662374
ABC	0.09497322612071969	0.055458851174749337	-0.3519320925109839	-0.4906392908403235	-0.4951983635006312
Disney+	0.023060549388927334	0.04499276707876444	-0.010750074305013327	-0.036529799932466744	-0.05584385535683514
CBS	-0.014668046685698531	0.14215652426764752	0.3104157761508531	-0.0965534739331619	0.4780682483315555
Prime Video	-0.008440167735541442	0.00014727225573593533	-0.0003910780432355532	0.009498970923804789	-0.0060098945609633565
Adult Swim	0.0033213445264342442	-0.004330708451296324	-0.0014828783099664928	0.006187790123128373	0.004093739029714726
NBC	-0.013311723675696082	-0.015027293273287376	0.04884169191697482	-0.00045964602653244694	-0.011068712186918591
Hulu	-3.1276772085529655e-05	-0.0032563789288100353	-0.00793260204402068	0.00027947008609099065	-0.0019046965232821176
USA Network	-0.038519911674537624	-0.005213026496190774	-0.00035080659639329104	-0.014678960144064659	0.026734607689233855
Cinemax	-0.00715121232502773785	0.001333713297014362	-0.0017947440239963477	0.005950125105244722	0.004999268463447549
Comedy Central	0.028436770138422825	-0.0438900217151775	0.01889302013474469	0.009789335967593283	0.00552292334965801
Showtime	-0.017759964375281222	-0.05511260049667084	-0.030008311717332058	-0.014075260754689154	0.026310134951712753
Paramount+	-0.02696126225010378	-0.00044860999088117446	0.0470302179557728	-0.03320049221566666	0.04189493085712876
Paramount Network	-0.0011461522543050246	-0.0013752299557036288	-0.000436662627453836	0.00017591163244602752	-0.0016009692336287574
Lifetime	0.00537589974849053	0.020715605679857085	-0.021407992879292317	0.00323282779285875	0.016065707898538107
Starz	-0.01024118085005502	-0.00778929568131736	-0.02341958870892591	0.016181167763082198	0.014135185273389646
Telemundo	-0.0	0.0	0.0	0.0	-0.0
Apple TV+	-0.0	0.0	0.0	0.0	-0.0
HBO Max	0.0053432464814405604	-0.003074622389925401	-0.0037482173276798623	0.001235949873416624	0.009106880552742478
BET	0.00017049089412947465	-0.0019312688359981514	-0.00293457595067353295	0.00022974440521386196	0.0023426886242145363
Hallmark Channel	-0.006073692478853398	-0.005889389095220464	-0.007389649443827644	0.008869812333123615	-0.006758854439973927
Peacock	0.013555458282122188	0.037427592374254304	0.01837719000261744	-0.003310756957098572	0.03861010932521633
Sci-Fi & Fantasy	-0.14664963944203205	0.020351730787440084	-0.16262538437862326	0.6032299301726099	-0.12496218329054563
Drama	-0.6769210322126961	-0.1484684498549476	-0.2423741521856367	-0.08824834304449893	-0.09038955896662941
Action & Adventure	-0.1525243984942803	0.02636363795670053	0.049469561191418085	0.36851348051453914	0.09804353037461108
Crime	-0.3322098953127639	-0.021750855903549797	0.3985053419199955	-0.37310996452948136	0.19108271791890247
Mystery	-0.18974990413578405	-0.018089692919775163	0.06537926118630691	-0.17742579210324108	0.06521671733669837
Comedy	0.42444284041995145	-0.7104309363810755	0.016850120085542142	-0.057174850119675914	0.14787122995031748
Animation	0.09799864787258731	-0.11879980496676597	0.1882228687165456	0.08362516551693608	-0.14977402492150219
Family	0.09245358096414742	-0.03251840256231355	0.08663595109433106	-0.03935492770167072	-0.2035147609634516
War & Politics	-0.015152845842884686	0.006282527327671735	-0.0035215434295764294	0.0072621104719798124	0.009445391180270551
Soap	-0.010712274934258819	0.0009675674884162799	-0.017792091290370793	-0.00963151438112319	-0.02971025774560537
Western	-0.005633081224602009	0.0009286065426752754	0.0008345852860502238	0.013278309129217722	-0.002076597924136986
Kids	-0.0	0.0	0.0	0.0	-0.0
Documentary	0.0032654650558157565	0.006149134655438112	0.0010520641468318103	9.106260280779965e-05	0.017243279132234973
Reality	0.24731497310212309	0.5934105782331124	0.068306541256321	-0.07490192130140276	-0.013631057953475873
Romance	-0.0009410484696034003	-0.00018730655895541392	-0.0006700687900625127	4.7015474392297156e-05	-0.00073121406644447505
Adult	-0.0	0.0	0.0	0.0	-0.0
Biography	-0.0	0.0	0.0	0.0	-0.0
Film-Noir	-0.0	0.0	0.0	0.0	-0.0
Game-Show	-0.0	0.0	0.0	0.0	-0.0
History	-0.0	0.0	0.0	0.0	-0.0
Horror	-0.0	0.0	0.0	0.0	-0.0
Musical	-0.0	0.0	0.0	0.0	-0.0
Music	-0.0	0.0	0.0	0.0	-0.0
News	-0.0	0.0	0.0	0.0	-0.0
Short	-0.0	0.0	0.0	0.0	-0.0
Sport	-0.0	0.0	0.0	0.0	-0.0
Talk-Show	-0.0	0.0	0.0	0.0	-0.0
Thriller	-0.0	0.0	0.0	0.0	-0.0

variable	importance
PC1	0.15319232024221863
PC2	0.10295199500656636
PC3	0.2956294289707757
PC4	0.2056335058086157
PC5	0.24259119769428228

Display Results in Graphs

In [397... # Print the results

```
show_regression_results("Viewers Regression", series_episode_subset_modeling_df, 'Viewers_millions', loading_df_viewers, \
select_features(main_labels = fuzzy_series_feature_labels, genres=True, networks=True, content=True), \
best_estimator_viewers, rmse_viewers, r2_viewers, mae_viewers, \
coefs_viewers, X_viewers, Y_viewers, predictions_viewers, residuals_viewers)

show_regression_results("18 to 49 Demo Regression", series_episode_subset_modeling_df, '18 to 49 Demo', loadings_df_18_to_49, \
select_features(main_labels = fuzzy_series_feature_labels, genres=True, networks=True, content=True), \
```

```
best_estimator_18_to_49, rmse_18_to_49, r2_18_to_49, mae_18_to_49, \
coefs_18_to_49, X_18_to_49, Y_18_to_49, predictions_18_to_49, residuals_18_to_49)
```

Viewers Regression

Viewers (millions) Statistics:

Mean: 2.8906696475178064

Median: 1.864

Mode: [0.624]

Variance: 9.70721064047449

Standard Deviation: 3.1156396839933995

Summary: count 41651.00000

mean 2.89067

std 3.11564

min 0.00600

25% 0.64000

50% 1.86400

75% 3.84000

max 26.97200

Name: Viewers_millions, dtype: float64

18-49 Demo Statistics:

Mean: 0.7120083203853316

Median: 0.47

Mode: [0.7]

Variance: 0.6670111299265791

Standard Deviation: 0.8167074934923636

Summary: count 41651.000000

mean 0.712008

std 0.816707

min 0.000000

25% 0.190000

50% 0.470000

75% 1.000000

max 9.300000

Name: _18_to_49_Demo, dtype: float64

Best Estimator for Viewers_millions: Pipeline(steps=[('scaler', MinMaxScaler()),

('regressor',

RandomForestRegressor(max_features=None, min_samples_leaf=2,

min_samples_split=5,

n_estimators=200)])

Root mean square error: 0.8055253915175268

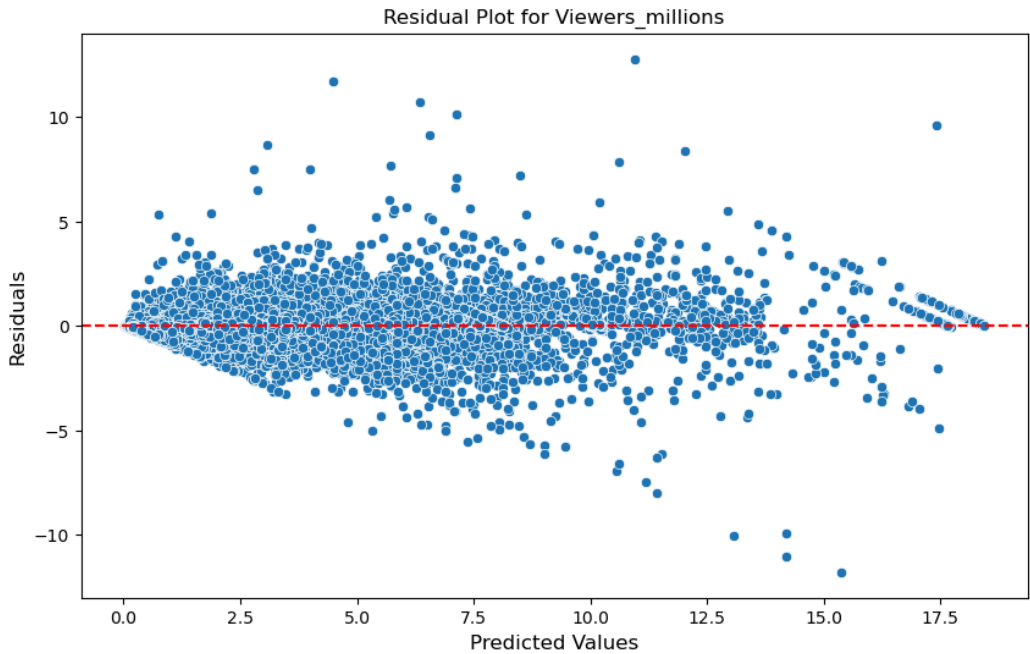
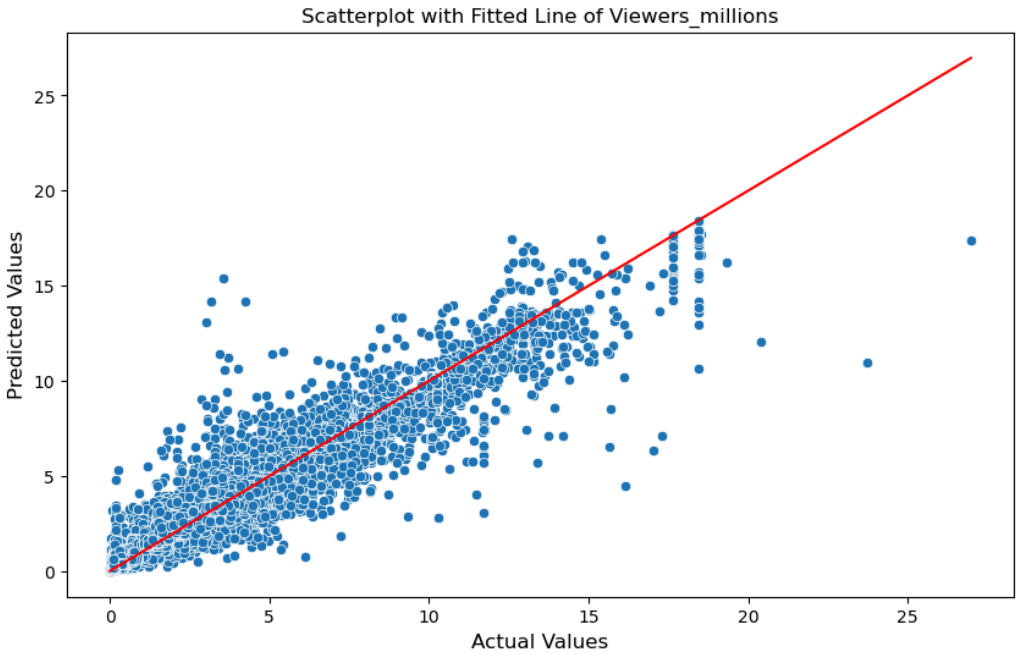
r2: 0.9307037068853239

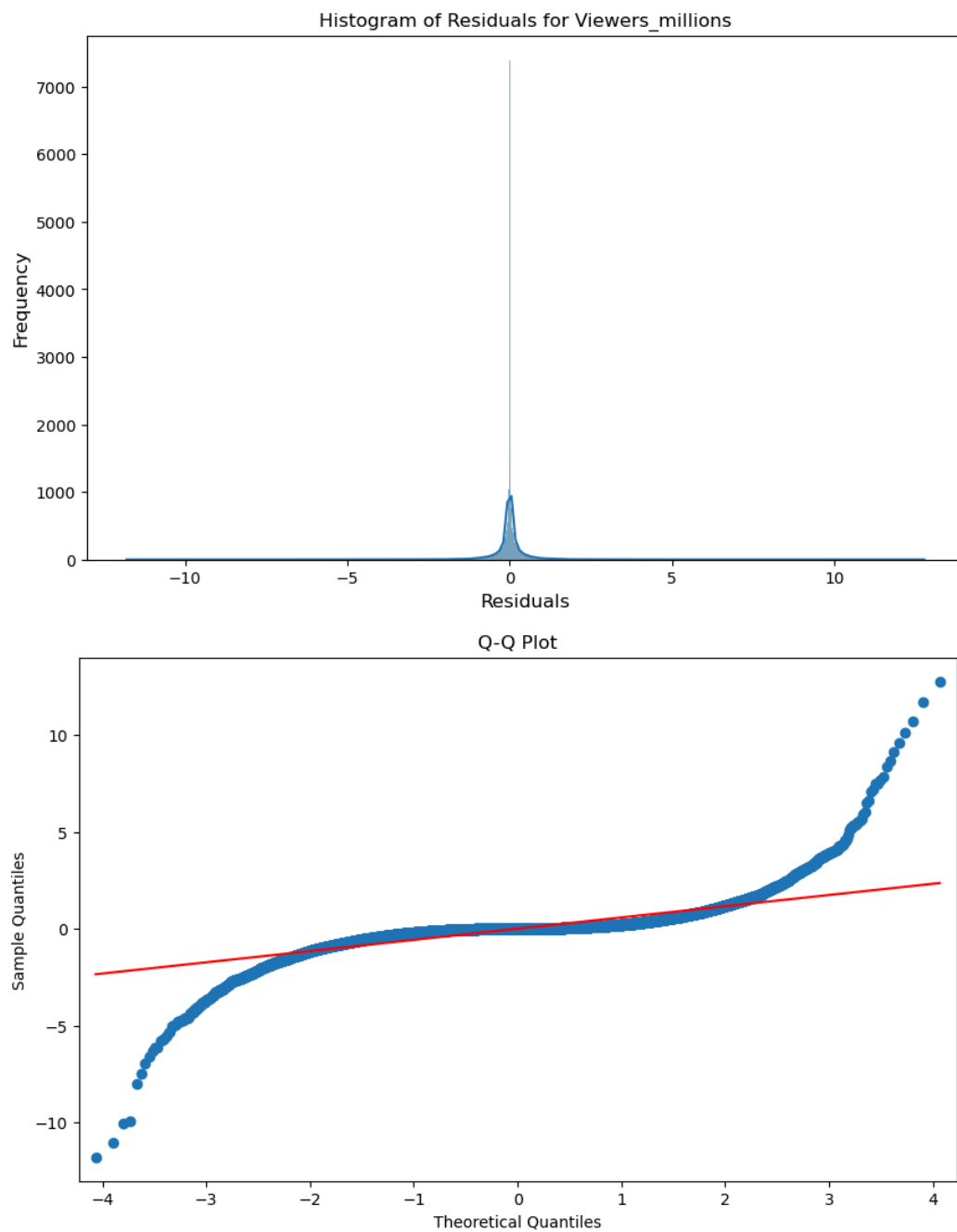
Mean average error: 0.38116900630642053

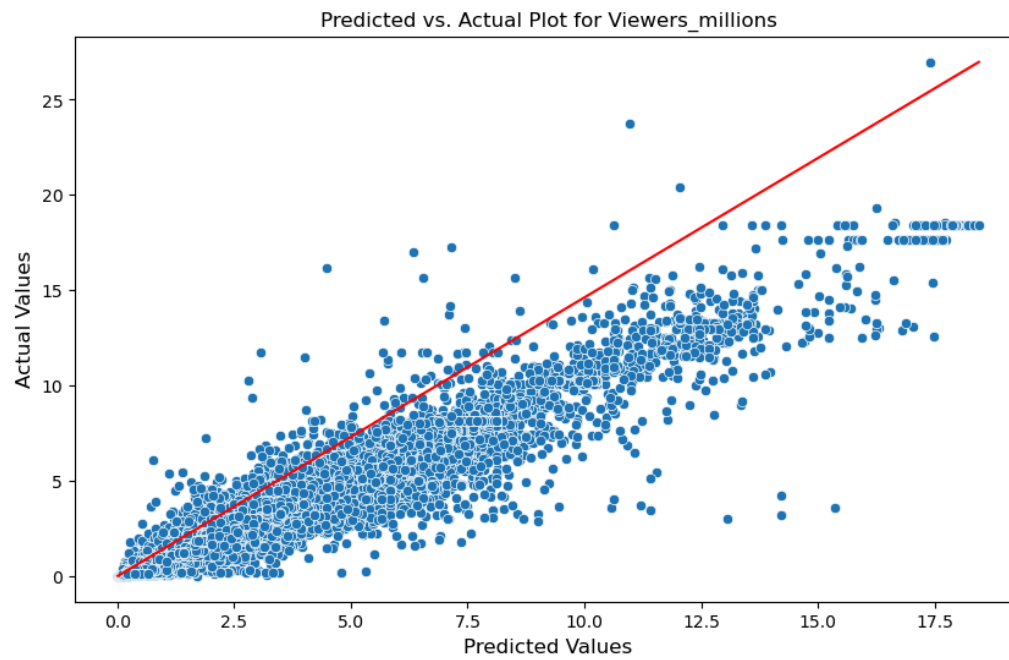
Feature_Label	PC1	PC2	PC3	PC4	PC5
rating	-0.05064262158298302	-0.06342925614716009	0.0012954474551373043	-0.0034369309396194456	0.00037064238684079276
Episode_Votes	-0.00407458590875576	-0.0025299911667548023	0.004185374892784674	0.005631296001467804	-0.004795436397087378
fuzzy_vote_average	-0.20466751614323075	-0.2344915971705098	0.34256471341030564	0.0029540259496700413	-0.3006941500890519
fuzzy_popularity	-0.056673318235856265	0.02360241776419042	0.48647448785016917	0.02916135162214167	-0.4027049632326354
IMDb_Episode_Rating	-0.051017242408109856	-0.06395263962901589	0.0008635175246110455	-0.0035950089289000725	0.0005277878143522629
IMDb_Episode_Votes	-0.004062916548549651	-0.002523138688928012	0.004174049093366653	0.005619572835965401	-0.004780920297350569
sex_factor	-0.0008658934901471368	-0.0005553360586422704	0.0006021650735163627	0.0015199865124645422	-0.0011385254522658405
violence_factor	-0.0011065310384172674	-0.0004492109464922526	0.0005913034810928341	0.002017743354321692	-0.0013535616322270755
profanity_factor	-0.001083323493816356	-0.0021394550759503868	0.00212177586362765	0.0018866097711689653	-0.0023668601635311727
drugs_factor	-0.0013204621510916204	-0.001193850632572562	0.0016780869022630178	0.0004254344694408212	-0.0016301538887893976
intense_factor	-0.0010524842511858213	-0.0004607755339959192	0.0005599116765346363	0.0014958883126154246	-0.0011074399381063788
HBO	-0.015344127147621789	-0.07611076064921744	-0.025849177537653045	-0.00814281363077641	0.0653131021832063
Netflix	-0.002442059318727563	-0.009769927618376868	0.0038656155309450347	0.022317435673898177	-0.013368902898085505
AMC	-0.028575552505348295	-0.00041186235761035586	-0.0010020261889111322	0.03986154354839812	-0.015931618820195646
FOX	0.1427619858367859	0.04330817039391035	0.34435785132670327	0.16412972911888762	-0.31201518609590484
The CW	-0.07260432535946107	0.0035414039418828797	-0.008241398260460693	0.14321337591754335	-0.03991228040464518
ABC	0.09497322929519461	0.055459021110135294	-0.35193362012405566	-0.490641026402971	-0.4952037101703751
Disney+	0.023060548537507213	0.0449928391412794	-0.010749569213861311	-0.03652716207588089	-0.05584487649768625
CBS	-0.014668043332640261	0.14215676521915901	0.3104142453739814	-0.09655405766178012	0.47806153825668396
Prime Video	-0.008440168104902402	0.00014725356229376797	-0.00039090283591698444	0.00949919475580332	-0.006009296199287172
Adult Swim	0.0033213442840579204	-0.0043307241802018216	-0.0014827629667166233	0.006187883870618462	0.0040941853814492404
NBC	-0.013311712784168523	-0.015026330880764197	0.04883675639916282	-0.00045822322409806997	-0.011093148398581327
Hulu	-3.127653341605181e-05	-0.0032563613970795244	-0.007932720191673527	0.0002793982066699931	-0.0019051935358347643
USA Network	-0.038519917831909806	-0.005213715350725619	-0.00034811529615601786	-0.014682538087874165	0.026749988055796294
Cinemax	-0.007152123694115874	0.001333685114244013	-0.0017945330068465423	0.005950293845290971	0.00500090887703809
Comedy Central	0.028436767604857253	-0.04389021792676352	0.018894124566758423	0.009789886740021863	0.005528468346693017
Showtime	-0.017759971091356578	-0.0551126470940174	-0.03000471339313676	-0.014065320281191432	0.0263145207980458
Paramount+	-0.02696126219845786	-0.0004486099564739478	0.04703021020585106	-0.03320058588656055	0.04189493672878775
Paramount Network	-0.001146152288288909	-0.0013752329969709091	-0.00043665158738635427	0.00017590107901160787	-0.0016008880474990964
Lifetime	0.005375898638647693	0.02071553883110439	-0.021407524847238873	0.0032331835072412253	0.016067716888156596
Starz	-0.010241182007247749	-0.007789358761027441	-0.02341906721906241	0.016181733189115255	0.014137080964115024
Telemundo	-0.0	-0.0	-0.0	-0.0	-0.0
Apple TV+	-0.0	-0.0	-0.0	-0.0	-0.0
HBO Max	0.005343246938609818	-0.00307460210643532	-0.0037484286451007153	0.0012356438443121719	0.009106297837486312
BET	0.00017049064650101733	-0.0019312926180603807	-0.0029344580001648794	0.0002296819357167111	0.0023432944549012765
Hallmark Channel	-0.006073692993365031	-0.005889419280271906	-0.007389410988566891	0.008869996635045492	-0.006758006853343415
Peacock	0.01355457844224796	0.03742754780364012	0.018377423949789536	-0.0033109562336657264	0.03861116571657176
Sci-Fi & Fantasy	-0.1466496398006928	0.020351880573696905	-0.1626251238619792	0.0032331933259227	-0.12496543152617567
Drama	-0.6769210280823446	-0.14846810187602164	-0.24237609889128586	-0.08824877245454817	-0.090399089816184
Action & Adventure	-0.15252439851057586	0.026363655347521744	0.049469635338238435	0.3685135421474246	0.09804291025769007
Crime	-0.3322089953352423	-0.02175077790522984	0.3985053122812126	-0.3731086498789932	0.19108075059368804
Mystery	-0.18974990620539126	-0.01808987191627715	0.06538053884993858	-0.1774257912520428	0.06522180583922285
Comedy	0.42442840809718285	-0.7104306748623704	0.016848242629238567	-0.057176858868040886	0.14786316562182877
Animation	0.09790864328313491	-0.1188000362199671	0.1882250338080617	0.08362888906661384	-0.1497662493151895
Family	0.09245357451048607	-0.032518483996776426	0.0866394397181717	-0.03934621707689826	-0.203509546587622
War & Politics	-0.015152846151075351	0.0062825316237498825	-0.003521391341737439	0.00726281003115957	0.009445574223218626
Soap	-0.010712273826602342	0.0009675302589093	-0.01779266963830414	-0.009633889779633413	-0.029710020842892593
Western	-0.0056330815996771325	0.0009285787453213462	0.0008347616002151962	0.01327840175464836	-0.0020758073979899633
Kids	-0.0	0.0	-0.0	-0.0	-0.0
Documentary	0.0032654651754747614	0.006149141098946636	0.0010519977434328975	9.110713078947752e-05	0.017243167843892532
Reality	0.24731497275590159	0.5934105705168804	0.0683067947233938	-0.07490243859817411	-0.013631395040943331
Romance	-0.000941048646722547	-0.00018731012080696254	-0.0006699757204034492	4.723956623011144e-05	-0.0007310448470152892
Adult	-0.0	0.0	-0.0	-0.0	-0.0

Biography	-0.0	0.0	-0.0	-0.0	-0.0
Film-Noir	-0.0	0.0	-0.0	-0.0	-0.0
Game-Show	-0.0	0.0	-0.0	-0.0	-0.0
History	-0.0	0.0	-0.0	-0.0	-0.0
Horror	-0.0	0.0	-0.0	-0.0	-0.0
Musical	-0.0	0.0	-0.0	-0.0	-0.0
Music	-0.0	0.0	-0.0	-0.0	-0.0
News	-0.0	0.0	-0.0	-0.0	-0.0
Short	-0.0	0.0	-0.0	-0.0	-0.0
Sport	-0.0	0.0	-0.0	-0.0	-0.0
Talk-Show	-0.0	0.0	-0.0	-0.0	-0.0
Thriller	-0.0	0.0	-0.0	-0.0	-0.0

variable	importance
PC1	0.09411081799695471
PC2	0.06264978770605241
PC3	0.3306216843167299
PC4	0.21677541843155868
PC5	0.2958422915487043







18 to 49 Demo Regression

Viewers (millions) Statistics:

Mean: 2.8906696475178064

Median: 1.864

Mode: [0.624]

Variance: 9.70721064047449

Standard Deviation: 3.1156396839933995

Summary: count 41651.00000

mean 2.89067

std 3.11564

min 0.00600

25% 0.64000

50% 1.86400

75% 3.84000

max 26.97200

Name: Viewers_millions, dtype: float64

18-49 Demo Statistics:

Mean: 0.7120083203853316

Median: 0.47

Mode: [0.7]

Variance: 0.6670111299265791

Standard Deviation: 0.8167074934923636

Summary: count 41651.000000

mean 0.712008

std 0.816707

min 0.000000

25% 0.190000

50% 0.470000

75% 1.000000

max 9.300000

Name: _18_to_49_Demo, dtype: float64

Best Estimator for _18_to_49_Demo: Pipeline(steps=[('scaler', MinMaxScaler()), ('regressor', RandomForestRegressor(max_features='sqrt', min_samples_leaf=2, min_samples_split=5))])

Root mean square error: 0.30590652344328473

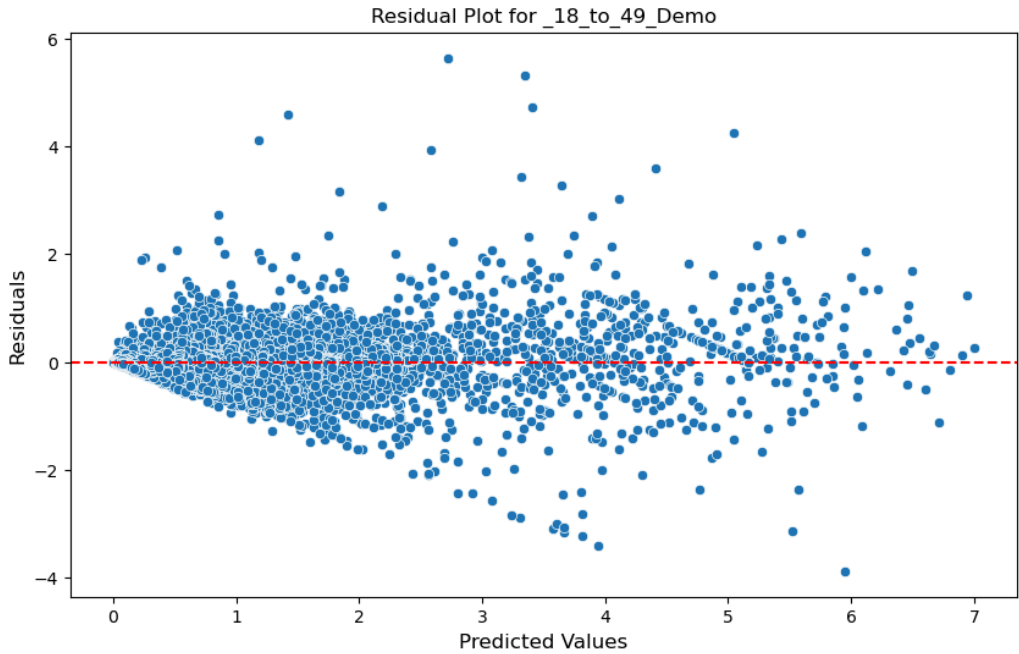
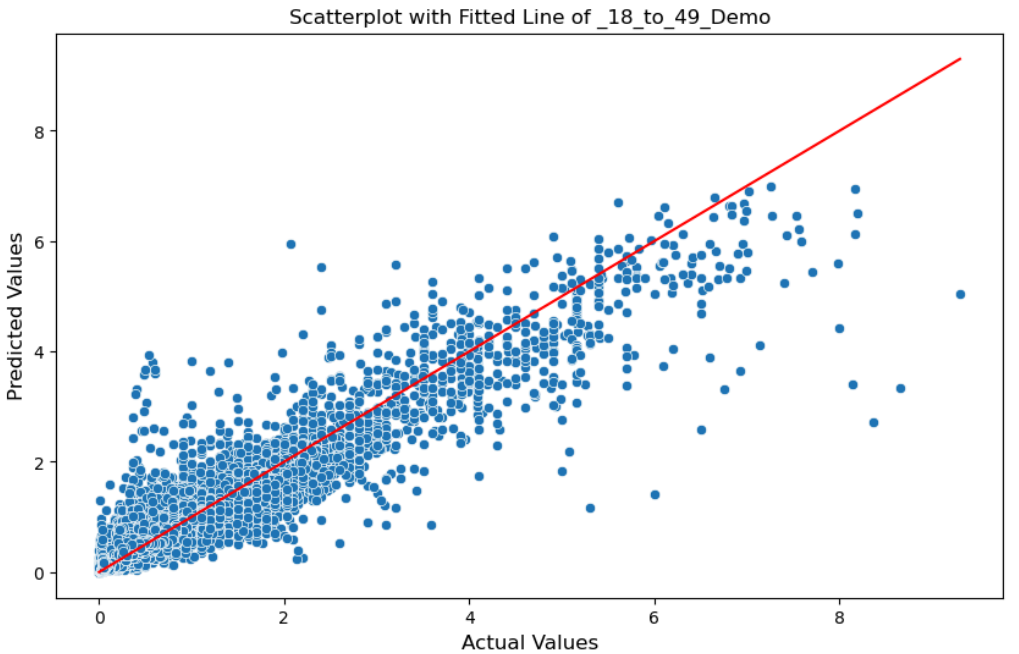
r2: 0.8591475346433551

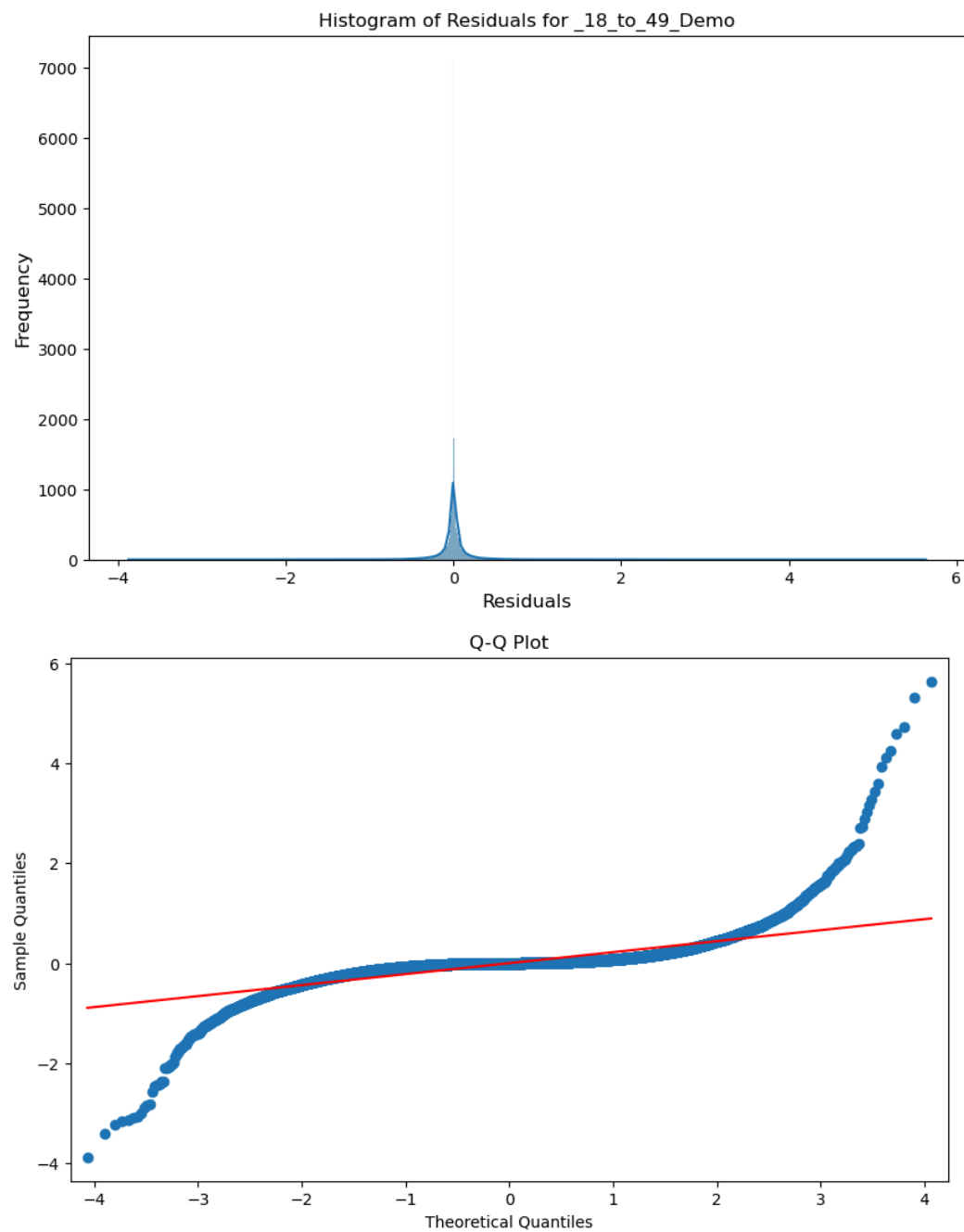
Mean average error: 0.13535140363625603

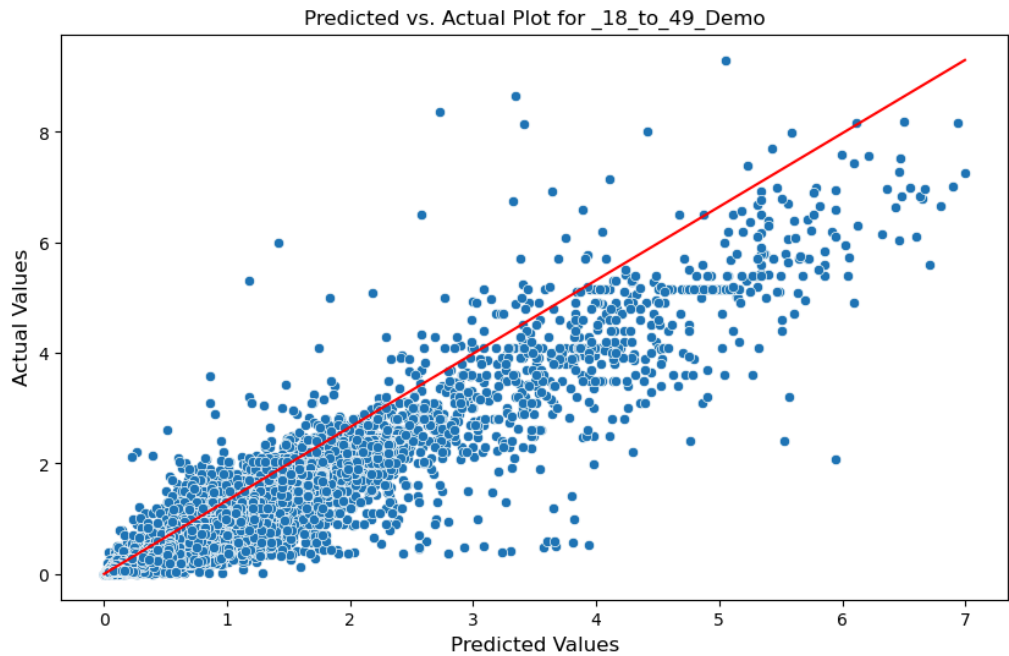
Feature_Label	PC1	PC2	PC3	PC4	PC5
rating	-0.05064262016151516	-0.06342916491059726	0.0012947570215445972	-0.0034374570864443474	0.0003680815039197266
Episode_Votes	-0.004074585888950699	-0.002529994492817613	0.004185364046280426	0.005631205511058928	-0.004795386856515494
fuzzy_vote_average	-0.20466750851834925	-0.23449089754813982	0.3425609694834862	0.002955472774427453	-0.3007120613840873
fuzzy_popularity	-0.05667332392891312	0.02360193292824175	0.4864773505970554	0.029161237487896296	-0.4026922826038
IMDb_Episode_Rating	-0.05101724097028444	-0.06395254737910724	0.0008628193164777785	-0.0035953423903270073	0.0005251967308706331
IMDb_Episode_Votes	-0.004062916528934763	-0.002523142013646176	0.00417408336573751	0.0056194826482320725	-0.004780870676897575
sex_factor	-0.0008658934923563049	-0.0005553378968552729	0.0006021645460245545	0.0015199529151362085	-0.0011384947932460744
violence_factor	-0.0011065310209093693	-0.00044921167133270224	0.0005912943076668264	0.0020177012127229697	-0.001353561959942323
profanity_factor	-0.0010833234642656886	-0.0021394553148176153	0.0021217607692292045	0.00188654766936126	-0.002366892326519234
drugs_factor	-0.0013204621199558184	-0.0011938502602535815	0.0016780726876306344	0.00042538771400192697	-0.0016301900580357185
intense_factor	-0.0010524842318062053	-0.0004607758150031588	0.000559902139709074	0.001495852403610924	-0.0011074488731988138
HBO	-0.015344134015910107	-0.07611125631582506	-0.02584612812222046	-0.00814170556272988	0.06532561689498179
Netflix	-0.0024420584840150585	-0.00976986076579672	0.0038652386838896905	0.022317453337497347	-0.013370686704880674
AMC	-0.02857555049521095	-0.00041176982140835863	-0.0010029228519522182	0.03986030871927736	-0.01593448340147336
FOX	0.142761980247612	0.04330790512568089	0.34436051520755856	0.16413372270883536	-0.31200657199898335
The CW	-0.07260433492776643	0.0035412812105143844	-0.008236603817217894	0.14322620092002722	-0.03990487697662374
ABC	0.09497322612071969	0.055458851174749337	-0.3519320925109839	-0.4906392908403235	-0.4951983635006312
Disney+	0.023060549388927334	0.04499276707876444	-0.010750074305013327	-0.036529799932466744	-0.05584385535683514
CBS	-0.014668046685698531	0.14215652426764752	0.3104157761508531	-0.0965534739331619	0.4780682483331555
Prime Video	-0.008440167735541442	0.00014727225573593533	-0.0003910780432355532	0.009498970923804789	-0.0060098945609633565
Adult Swim	0.0033213445264342442	-0.004330708451296324	-0.0014828783099664928	0.006187790123128373	0.004093739029714726
NBC	-0.013311723675696082	-0.015027293273287376	0.04884169191697482	-0.00045964602653244694	-0.011068712186918591
Hulu	-3.1276772085529655e-05	-0.0032563789288100353	-0.00793260204402068	0.00027947080609099065	-0.0019046965223821176
USA Network	-0.038519911674537624	-0.005213026496190774	-0.00035080659639329104	-0.014678960144064659	0.026734607689233855
Cinemax	-0.0071521232502773785	0.001333713297014362	-0.0017947440239963477	0.005950125105244722	0.004999268463447549
Comedy Central	0.028436770138422825	-0.0438900217151775	0.01889302013474469	0.009789335967593283	0.00552292334965801
Showtime	-0.017759964375281222	-0.05511260049667084	-0.030008311717332058	-0.014075260754689154	0.026310134951712753
Paramount+	-0.02696126225010378	-0.00044860999088117446	0.0478302179557728	-0.03320049221566666	0.04189493085712876
Paramount Network	-0.0011461522543050246	-0.0013752299557036288	-0.000436662627453836	0.00017591163244602752	-0.0016009692336287574
Lifetime	0.00537589974849053	0.020715605679857085	-0.021407992879292317	0.00323282779285875	0.016065707898538107
Starz	-0.01024118085005502	-0.00778929568131736	-0.02341958870892591	0.016181167763082198	0.014135185273389646
Telemundo	-0.0	0.0	0.0	0.0	-0.0
Apple TV+	-0.0	0.0	0.0	0.0	-0.0
HBO Max	0.0053423464814405604	-0.003074622389925401	-0.0037482173276798623	0.001235949873416624	0.009106880552742478
BET	0.00017049089412947465	-0.0019312688359981514	-0.0029345755067353295	0.000229744408251386196	0.0023426886242145363
Hallmark Channel	-0.006073692478853398	-0.005889389095220464	-0.007389649443827644	0.008869812333123615	-0.006758854439973927
Peacock	0.013555458282122188	0.037427592374254304	0.01837719000261744	-0.003310756957098572	0.03861010932521633
Sci-Fi & Fantasy	-0.14664963944203205	0.02035173087440084	-0.16262538437862326	0.603229930712699	-0.12496218329054563
Drama	-0.6769210322126961	-0.1484684498549476	-0.2423741521856367	-0.08824834304449893	-0.09038955896662941
Action & Adventure	-0.1525243984942803	0.02636363795670053	0.049469561191418085	0.36851348051453914	0.09804353037461108
Crime	-0.3322098953127639	-0.021750855903549797	0.3985053419199955	-0.37310996452948136	0.19108271791890247
Mystery	-0.18974990413578405	-0.018089692919775163	0.06537926118630691	-0.17742579210324108	0.06521671733669837
Comedy	0.4244284041995145	-0.7104309363810755	0.016850120085542142	-0.057174850119675914	0.14787122995031748
Animation	0.09790864787258731	-0.11879980496676597	0.1882228687165456	0.08362516551693608	-0.14977402492150219
Family	0.09245358096414742	-0.03251840256231355	0.08663595109433106	-0.03935492770167072	-0.2035147609634516
War & Politics	-0.015152845842884686	0.006282527327671735	-0.0035215434295764294	0.0072621104719798124	0.009445391180270551
Soap	-0.010712274934258819	0.0009675674884162799	-0.017792091290370793	-0.00963151438112319	-0.02971025774560537
Western	-0.005633081224602009	0.0009286065426752754	0.0008345852860502238	0.013278309129217722	-0.002076597924136986
Kids	-0.0	0.0	0.0	0.0	-0.0
Documentary	0.0032654650558157565	0.0061491346555438112	0.0010520641468318103	9.106260280779965e-05	0.017243279132234973
Reality	0.2473149731021309	0.5934105782331124	0.068306541256321	-0.07490192130140276	-0.013631057953475873
Romance	-0.0009410484696034003	-0.00018730655895541392	-0.0006700687900625127	4.7015474392297156e-05	-0.0007312140644447505
Adult	-0.0	0.0	0.0	0.0	-0.0
Biography	-0.0	0.0	0.0	0.0	-0.0

Film-Noir	-0.0	0.0	0.0	0.0	-0.0
Game-Show	-0.0	0.0	0.0	0.0	-0.0
History	-0.0	0.0	0.0	0.0	-0.0
Horror	-0.0	0.0	0.0	0.0	-0.0
Musical	-0.0	0.0	0.0	0.0	-0.0
Music	-0.0	0.0	0.0	0.0	-0.0
News	-0.0	0.0	0.0	0.0	-0.0
Short	-0.0	0.0	0.0	0.0	-0.0
Sport	-0.0	0.0	0.0	0.0	-0.0
Talk-Show	-0.0	0.0	0.0	0.0	-0.0
Thriller	-0.0	0.0	0.0	0.0	-0.0

variable	importance
PC1	0.15319232024221863
PC2	0.10295199500656636
PC3	0.2956294289707757
PC4	0.205635058086157
PC5	0.24259119769428228







Interpretation

General Data Statistics

Viewers (millions)

Mean: 2.8906696475178064 Median: 1.864 Mode: [0.624] Variance: 9.70721064047449 Standard Deviation: 3.1156396839933995 Summary: count 41651.00000 mean 2.89067 std 3.11564 min 0.00600 25% 0.64000 50% 1.86400 75% 3.84000 max 26.97200

18-49 Demo

Mean: 0.7120083203853316 Median: 0.47 Mode: [0.7] Variance: 0.6670111299265791 Standard Deviation: 0.8167074934923636 Summary: count 41651.000000 mean 0.712008 std 0.816707 min 0.000000 25% 0.190000 50% 0.470000 75% 1.000000 max 9.300000

Regression Results for "Viewers (Millions)"

Best Estimator

- Model: XGBRegressor
- learning_rate: 0.2
- max_depth: 7
- n_estimators: 300
- Performance Metrics
- Root Mean Square Error (RMSE): 0.1083
- R²: 0.9823
- Mean Absolute Error (MAE): 0.0582

Feature Importances (Principal Components)

- PC1: 0.1405
- PC2: 0.1464
- PC3: 0.6057
- PC4: 0.0508
- PC5: 0.0565

Loadings (Contribution of Original Features to PCs)

Features like vote_average, Series_Votes, and popularity have high loadings on the principal components, indicating significant contributions to the variability captured by these components.

Interpretation

- The XGBRegressor model is highly accurate, as evidenced by the high R² value (0.9823), meaning it explains 98.23% of the variance in the target variable "Viewers (millions)".
- The low RMSE (0.1083) indicates that the model's predictions are very close to the actual values.
- The high contribution of vote_average and Series_Votes suggests that these features are significant predictors of viewership.

Regression Results for "18-49 Demo"

Best Estimator

- Model: RandomForestRegressor
- max_depth: 30
- max_features: 'sqrt'
- n_estimators: 200
- Performance Metrics
- Root Mean Square Error (RMSE): 0.2201
- R^2 : 0.9948
- Mean Absolute Error (MAE): 0.1116

Feature Importances (Principal Components)

- PC1: 0.1134
- PC2: 0.2522
- PC3: 0.5205
- PC4: 0.0341
- PC5: 0.0798

Loadings (Contribution of Original Features to PCs)

Similar to the "Viewers (millions)" results, features like vote_average, Series_Votes, and popularity have high loadings, indicating their importance.

Interpretation

- The RandomForestRegressor model is extremely accurate, with an R^2 value of 0.9948, meaning it explains 99.48% of the variance in the target variable "18-49 Demo".
- The RMSE (0.2201) indicates a very close fit to the actual values.
- The high contribution of vote_average, Series_Votes, and popularity to the principal components indicates their importance as predictors of the 18-49 demographic ratings.

Summary and Recommendations

Both models demonstrate excellent predictive power with high R^2 values and low RMSEs, suggesting they are well-suited for predicting their respective target variables. The features contributing significantly to the principal components (e.g., vote_average, Series_Votes, and popularity) should be considered key indicators of viewership and demographic ratings.

Do Same Strategy on Seasons of Data

- 2 seasons or more (train on one year)
- 3 seasons or more (train on two years)

```
In [398... def seasonal_set_regressor_model_run(df, target_column, seasons):
    # Ensure target is a pandas Series
    target = df[target_column]
    if not isinstance(target, pd.Series):
        raise ValueError("target must be a pandas Series")

    # get test and train sets
    X_train, X_test, Y_train, Y_test, X, Y = test_train_over_seasons(df, seasons, series_grouping = True,
                                                                    fuzziness=S, genres=True, networks=True, content=True)

    # Select the specific target column
    Y_train = Y_train[target_column]
    Y_test = Y_test[target_column]

    best_estimator, coefs, rmse, r2, mae = regressor_model_run(X_train, X_test, Y_train, Y_test)

    # do predictions
    predictions = best_estimator.predict(X)
    # calculate residuals
    residuals = Y[target_column] - predictions
    return best_estimator, coefs, rmse, r2, mae, X, Y[target_column], predictions, residuals

best_estimator_18_to_49_2s, coefs_18_to_49_2s, rmse_18_to_49_2s, r2_18_to_49_2s, mae_18_to_49_2s, \
    X_18_to_49_2s, Y_18_to_49_2s, predictions_18_to_49_2s, residuals_18_to_49_2s = \
    seasonal_set_regressor_model_run(two_seasons_or_more_df, '_18_to_49_Demo', 1)

best_estimator_viewers_2s, coefs_viewers_2s, rmse_viewers_2s, r2_viewers_2s, mae_viewers_2s, \
    X_viewers_2s, Y_viewers_2s, predictions_viewers_2s, residuals_viewers_2s = \
    seasonal_set_regressor_model_run(two_seasons_or_more_df, 'Viewers_millions', 1)

best_estimator_18_to_49_3s, coefs_18_to_49_3s, rmse_18_to_49_3s, r2_18_to_49_3s, mae_18_to_49_3s, \
    X_18_to_49_3s, Y_18_to_49_3s, predictions_18_to_49_3s, residuals_18_to_49_3s = \
    seasonal_set_regressor_model_run(three_seasons_or_more_df, '_18_to_49_Demo', 2)

best_estimator_viewers_3s, coefs_viewers_3s, rmse_viewers_3s, r2_viewers_3s, mae_viewers_3s, X_viewers_3s, \
    Y_viewers_3s, predictions_viewers_3s, residuals_viewers_3s = \
    seasonal_set_regressor_model_run(three_seasons_or_more_df, 'Viewers_millions', 2)
```

```

Ranking 556 series in 112.0 classification pockets.
Fitting 5 folds for each of 259 candidates, totalling 1295 fits
Best parameters found: {'regressor': RandomForestRegressor(), 'regressor__max_depth': 30, 'regressor__max_features': 'sqrt', 'regressor__min_samples_le
af': 1, 'regressor__min_samples_split': 10, 'regressor__n_estimators': 100}
Best estimator found: Pipeline(steps=[('scaler', MinMaxScaler()),
                                      ('regressor',
                                       RandomForestRegressor(max_depth=30, max_features='sqrt',
                                                             min_samples_split=10))])
RMSE: 0.5986314859503664, R2: 0.5157089855407712, MAE: 0.34912907600476345
variable importance
0      rating      0.030941
1      Episode_Votes 0.091094
2      fuzzy_vote_average 0.121926
3      fuzzy_popularity 0.211771
4      IMDb_Episode_Rating 0.029336
..      ...
60      News      0.000000
61      Short      0.000000
62      Sport      0.000000
63      Talk-Show 0.000000
64      Thriller 0.000000

[65 rows x 2 columns]
Ranking 556 series in 112.0 classification pockets.
Fitting 5 folds for each of 259 candidates, totalling 1295 fits
Best parameters found: {'regressor': RandomForestRegressor(), 'regressor__max_depth': None, 'regressor__max_features': 'sqrt', 'regressor__min_samples_
leaf': 1, 'regressor__min_samples_split': 5, 'regressor__n_estimators': 50}
Best estimator found: Pipeline(steps=[('scaler', MinMaxScaler()),
                                      ('regressor',
                                       RandomForestRegressor(max_features='sqrt', min_samples_split=5,
                                                             n_estimators=50))])
RMSE: 1.745082362575616, R2: 0.711386995942157, MAE: 1.0403642049072674
variable importance
0      rating      0.025127
1      Episode_Votes 0.081259
2      fuzzy_vote_average 0.092192
3      fuzzy_popularity 0.178904
4      IMDb_Episode_Rating 0.025926
..      ...
60      News      0.000000
61      Short      0.000000
62      Sport      0.000000
63      Talk-Show 0.000000
64      Thriller 0.000000

[65 rows x 2 columns]
Ranking 385 series in 78.0 classification pockets.
Fitting 5 folds for each of 259 candidates, totalling 1295 fits
Best parameters found: {'regressor': RandomForestRegressor(), 'regressor__max_depth': None, 'regressor__max_features': 'log2', 'regressor__min_samples_
leaf': 1, 'regressor__min_samples_split': 5, 'regressor__n_estimators': 200}
Best estimator found: Pipeline(steps=[('scaler', MinMaxScaler()),
                                      ('regressor',
                                       RandomForestRegressor(max_features='log2', min_samples_split=5,
                                                             n_estimators=200))])
RMSE: 0.5819548790237785, R2: 0.5783008292026611, MAE: 0.3265465403328334
variable importance
0      rating      0.031973
1      Episode_Votes 0.098261
2      fuzzy_vote_average 0.122313
3      fuzzy_popularity 0.201770
4      IMDb_Episode_Rating 0.032331
..      ...
60      News      0.000000
61      Short      0.000000
62      Sport      0.000000
63      Talk-Show 0.000000
64      Thriller 0.000000

[65 rows x 2 columns]
Ranking 385 series in 78.0 classification pockets.
Fitting 5 folds for each of 259 candidates, totalling 1295 fits
Best parameters found: {'regressor': RandomForestRegressor(), 'regressor__max_depth': 30, 'regressor__max_features': 'sqrt', 'regressor__min_samples_le
af': 1, 'regressor__min_samples_split': 10, 'regressor__n_estimators': 100}
Best estimator found: Pipeline(steps=[('scaler', MinMaxScaler()),
                                      ('regressor',
                                       RandomForestRegressor(max_depth=30, max_features='sqrt',
                                                             min_samples_split=10))])
RMSE: 1.6296137747154205, R2: 0.7659125904502707, MAE: 0.9444540584943772
variable importance
0      rating      0.022940
1      Episode_Votes 0.079283
2      fuzzy_vote_average 0.092699
3      fuzzy_popularity 0.191204
4      IMDb_Episode_Rating 0.021445
..      ...
60      News      0.000000
61      Short      0.000000
62      Sport      0.000000
63      Talk-Show 0.000000
64      Thriller 0.000000

[65 rows x 2 columns]

```

```

In [399... show_regression_results("18 to 49 Demo Estimation for Future Years After One Season", two_seasons_or_more_df, '_18_to_49_Demo', None, \
select_features(), best_estimator_18_to_49_2s, rmse_18_to_49_2s, r2_18_to_49_2s, mae_18_to_49_2s, \
coefs_18_to_49_2s, X_18_to_49_2s, Y_18_to_49_2s, predictions_18_to_49_2s, residuals_18_to_49_2s)

```

18 to 49 Demo Estimation for Future Years After One Season

Viewers (millions) Statistics:

Mean: 3.0117604958522115

Median: 1.99

Mode: [0.624]

Variance: 10.211331092712312

Standard Deviation: 3.195517343516119

Summary: count 38270.000000

mean 3.011760

std 3.195517

min 0.006000

25% 0.671000

50% 1.990000

75% 4.093750

max 26.972000

Name: Viewers_millions, dtype: float64

18-49 Demo Statistics:

Mean: 0.7473094735398338

Median: 0.5

Mode: [0.7]

Variance: 0.7039714481405789

Standard Deviation: 0.8390300639074734

Summary: count 38270.000000

mean 0.747309

std 0.839030

min 0.000000

25% 0.200000

50% 0.500000

75% 1.060000

max 9.300000

Name: _18_to_49_Demo, dtype: float64

Best Estimator for _18_to_49_Demo: Pipeline(steps=[('scaler', MinMaxScaler()),
('regressor',
RandomForestRegressor(max_depth=30, max_features='sqrt',
min_samples_split=10))])

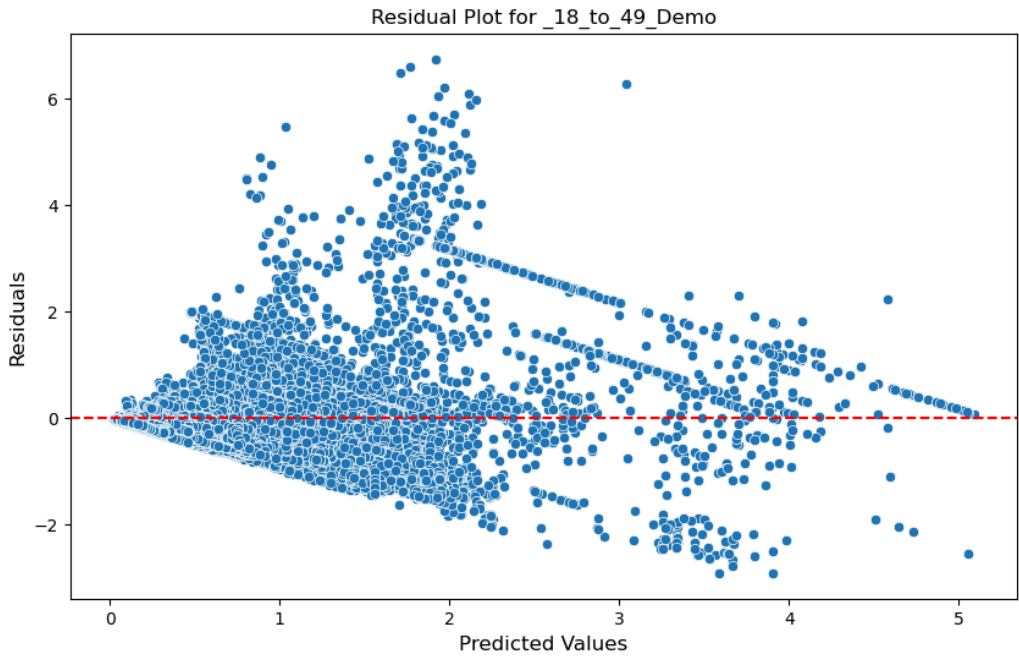
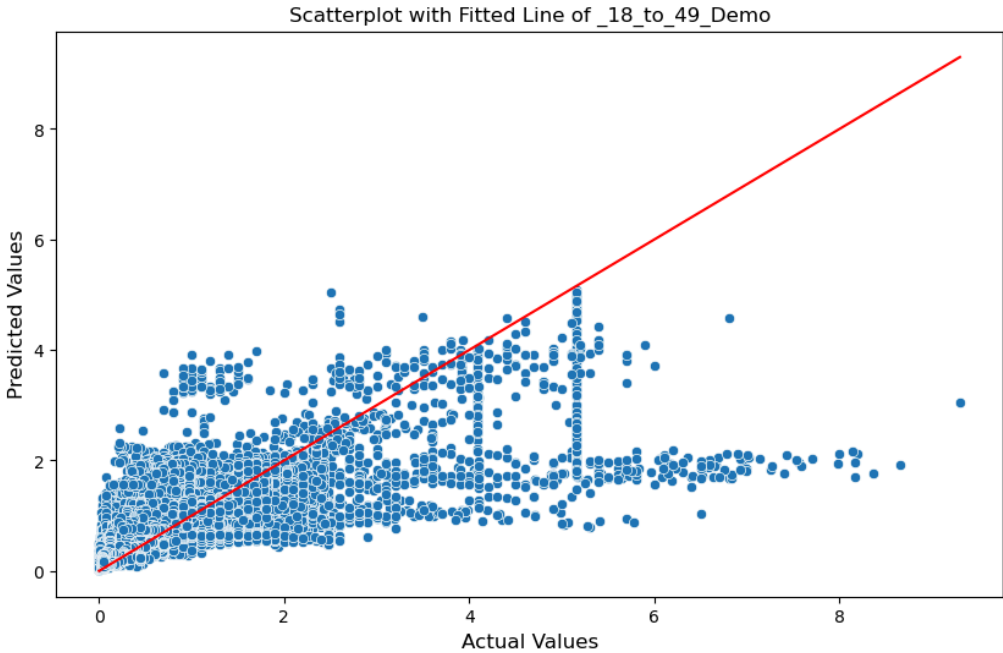
Root mean square error: 0.5986314859503664

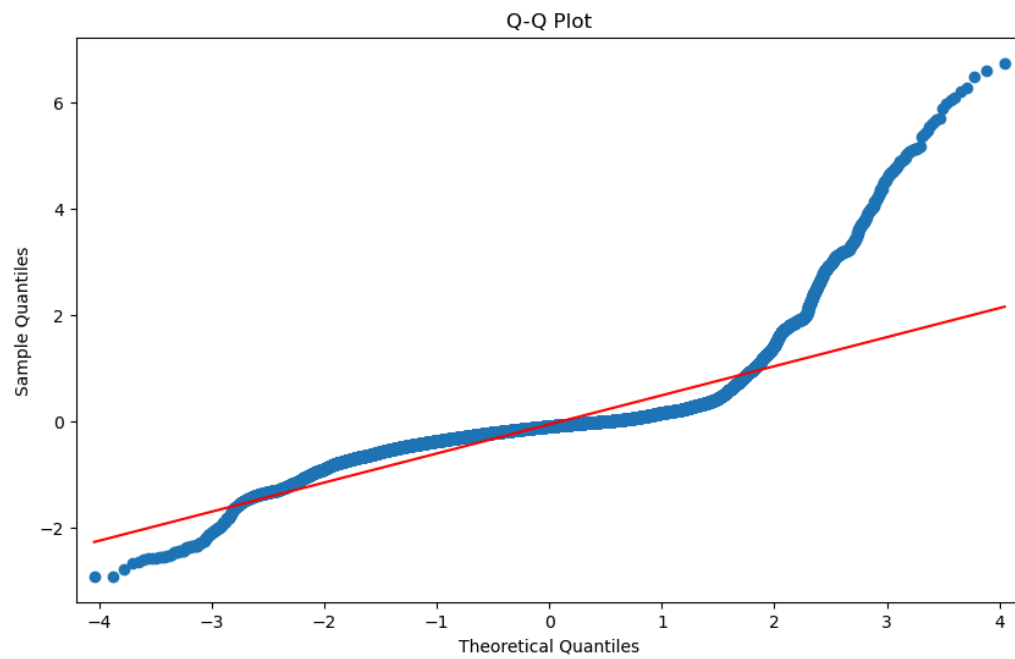
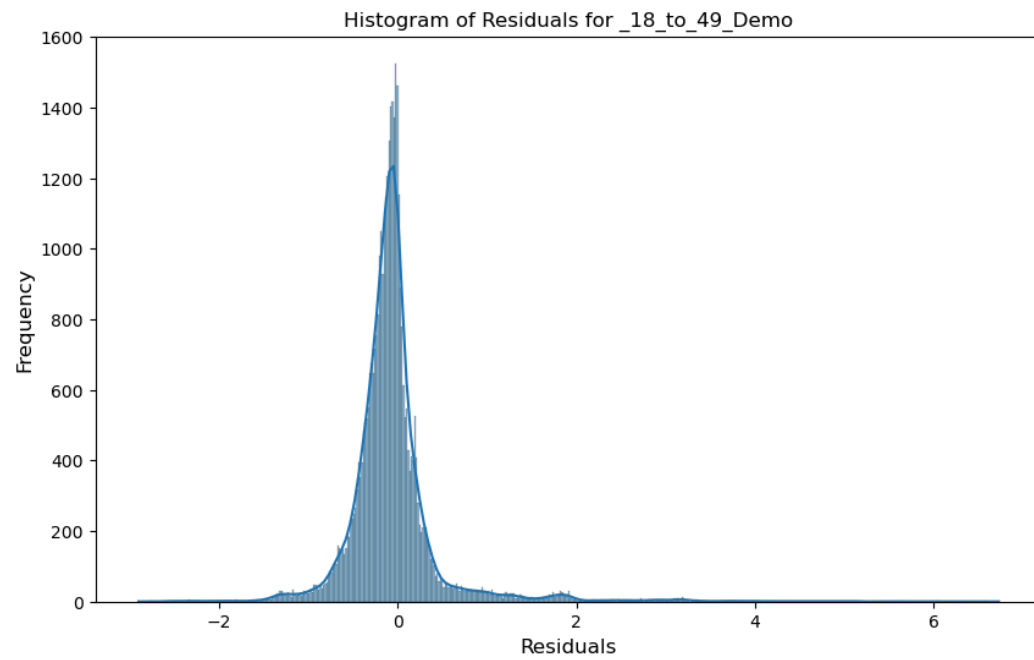
r2: 0.5157089855407712

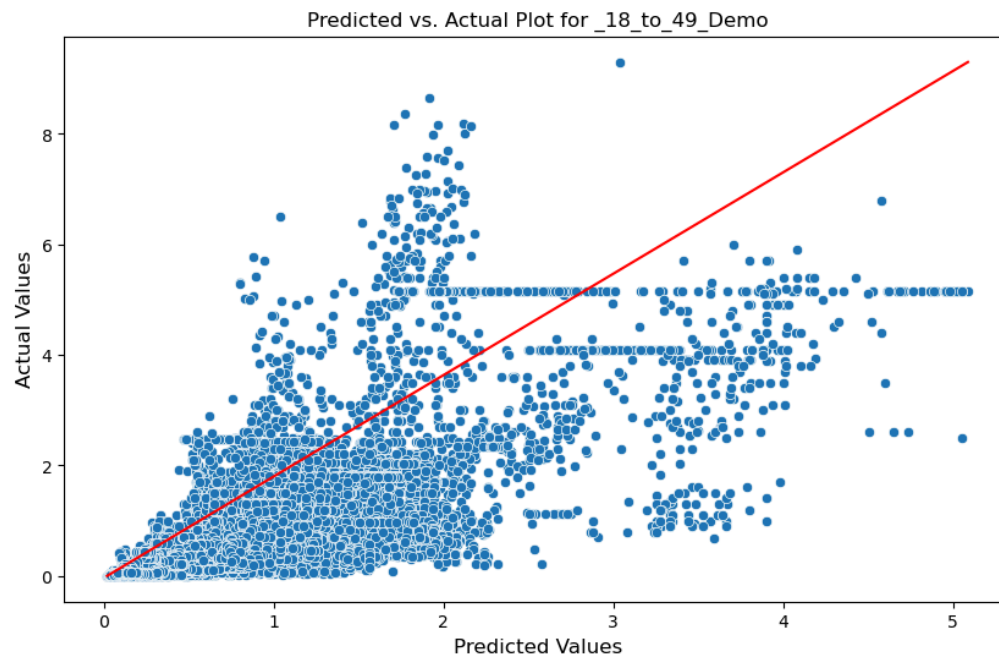
Mean average error: 0.34912907600476345

variable	importance
rating	0.030940574291091386
Episode_Votes	0.0910939876903834
fuzzy_vote_average	0.12192634438191038
fuzzy_popularity	0.21177051906563416
IMDb_Episode_Rating	0.029335993936998
IMDb_Episode_Votes	0.09214110411933679
sex_factor	0.01333268432099557
violence_factor	0.011508030273535032
profanity_factor	0.013608183446947192
drugs_factor	0.009778842161506167
intense_factor	0.008315585221580478
HBO	0.015295416125266139
Netflix	0.002322245492290465
AMC	0.0069049407367327825
FOX	0.015007593442120437
The CW	0.012198468119193957
ABC	0.048169569112650135
Disney+	6.80111975309221e-05
CBS	0.03098500981966255
Prime Video	0.0011012567414449583
Adult Swim	6.631391340947e-05
NBC	0.04401682045085139
Hulu	0.00040273097839651194
USA Network	0.03579402709263696
Cinemax	0.0016483140306228017
Comedy Central	0.001224833228700861
Showtime	0.009167862425474613
Paramount+	0.0008938968374788073
Paramount Network	0.0005847000395802416
Lifetime	0.003891237780041993
Starz	0.008134826792957845
Telemundo	0.0
Apple TV+	0.0
HBO Max	0.0005939070699695098
BET	0.0008589179714224275
Hallmark Channel	0.0006142302917061645
Peacock	0.00023721669583159105
Sci-Fi & Fantasy	0.014530711124008035
Drama	0.02778235593433576
Action & Adventure	0.009859846940854698
Crime	0.013716199050182782
Mystery	0.012893476535097577
Comedy	0.025628566595536962
Animation	0.0014274123221907561
Family	0.005687282135790811
War & Politics	0.000976567470409981
Soap	0.0003522254267636264
Western	0.0036905129039658895
Kids	0.0
Documentary	6.762636681800179e-05
Reality	0.009342076844326197
Romance	0.00011094505382686978
Adult	0.0
Biography	0.0

Film-Noir	0.0
Game-Show	0.0
History	0.0
Horror	0.0
Musical	0.0
Music	0.0
News	0.0
Short	0.0
Sport	0.0
Talk-Show	0.0
Thriller	0.0







```
In [400... show_regression_results("Viewers (Millions) Estimation for Future Years After One Season", two_seasons_or_more_df, 'Viewers_millions', None, \
    select_features(), best_estimator_viewers_2s, rmse_viewers_2s, r2_viewers_2s, mae_viewers_2s, \
    coefs_viewers_2s, X_viewers_2s, Y_viewers_2s, predictions_viewers_2s, residuals_viewers_2s)
```

Viewers (Millions) Estimation for Future Years After One Season

Viewers (millions) Statistics:

Mean: 3.0117604958522115

Median: 1.99

Mode: [0.624]

Variance: 10.211331092712312

Standard Deviation: 3.195517343516119

Summary: count 38270.000000

mean 3.011760

std 3.195517

min 0.006000

25% 0.671000

50% 1.990000

75% 4.093750

max 26.972000

Name: Viewers_millions, dtype: float64

18-49 Demo Statistics:

Mean: 0.7473094735398338

Median: 0.5

Mode: [0.7]

Variance: 0.7039714481405789

Standard Deviation: 0.8390300639074734

Summary: count 38270.000000

mean 0.747309

std 0.839030

min 0.000000

25% 0.200000

50% 0.500000

75% 1.060000

max 9.300000

Name: _18_to_49_Demo, dtype: float64

Best Estimator for Viewers_millions: Pipeline(steps=[('scaler', MinMaxScaler()),
('regressor',
RandomForestRegressor(max_features='sqrt', min_samples_split=5,
n_estimators=50))])

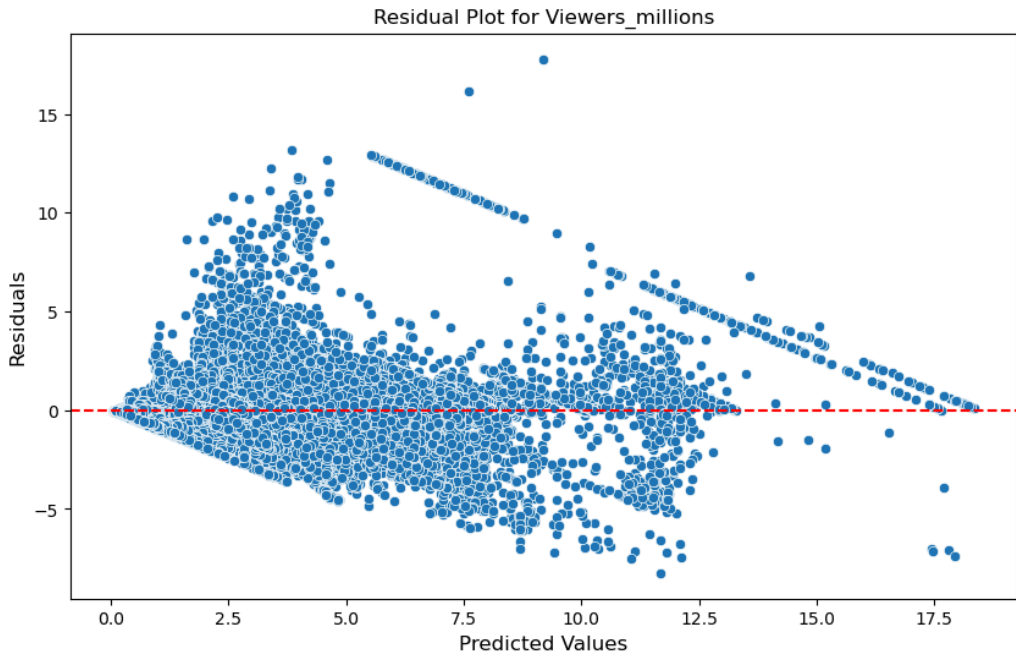
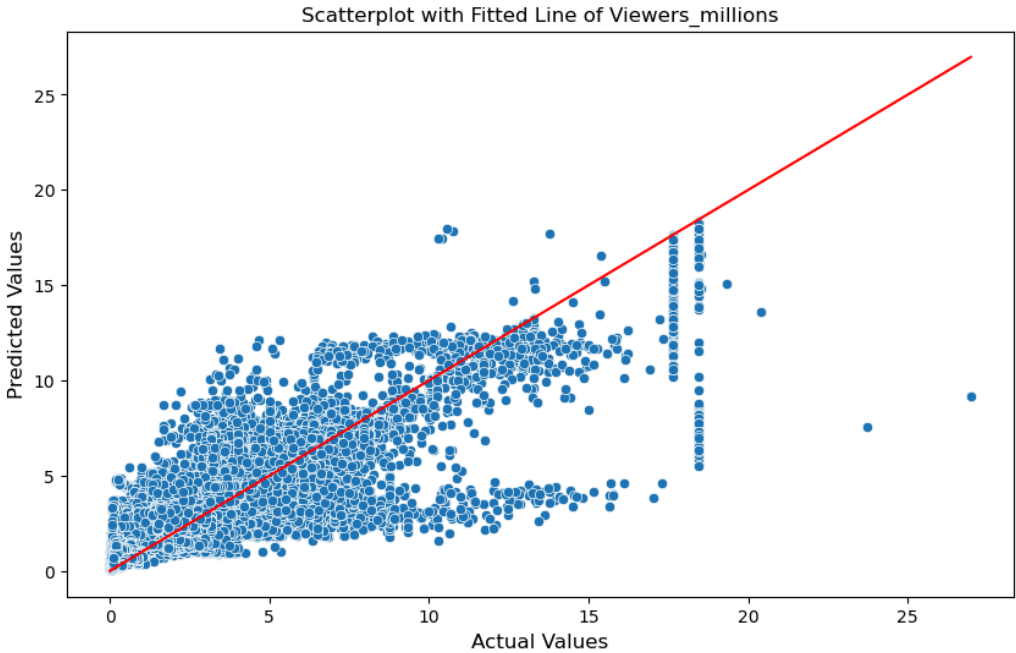
Root mean square error: 1.745082362575616

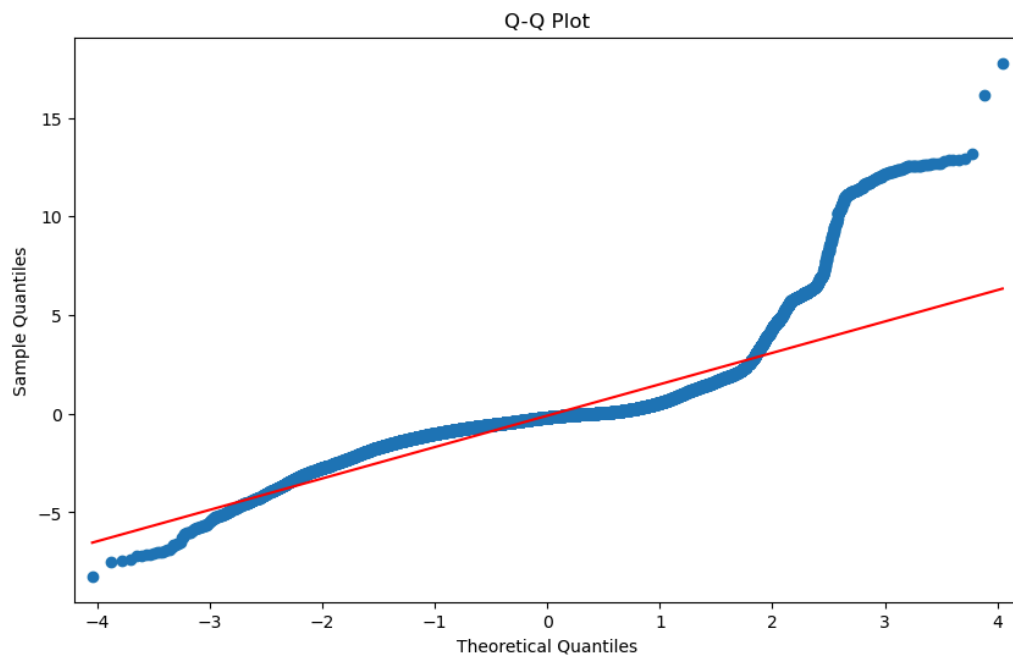
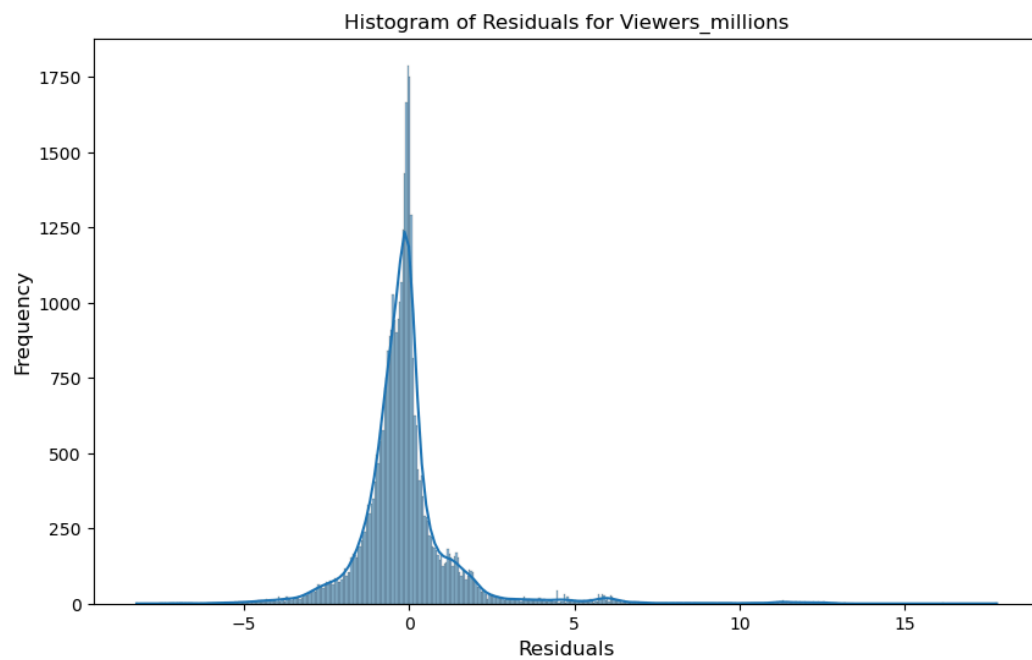
r2: 0.711386995942157

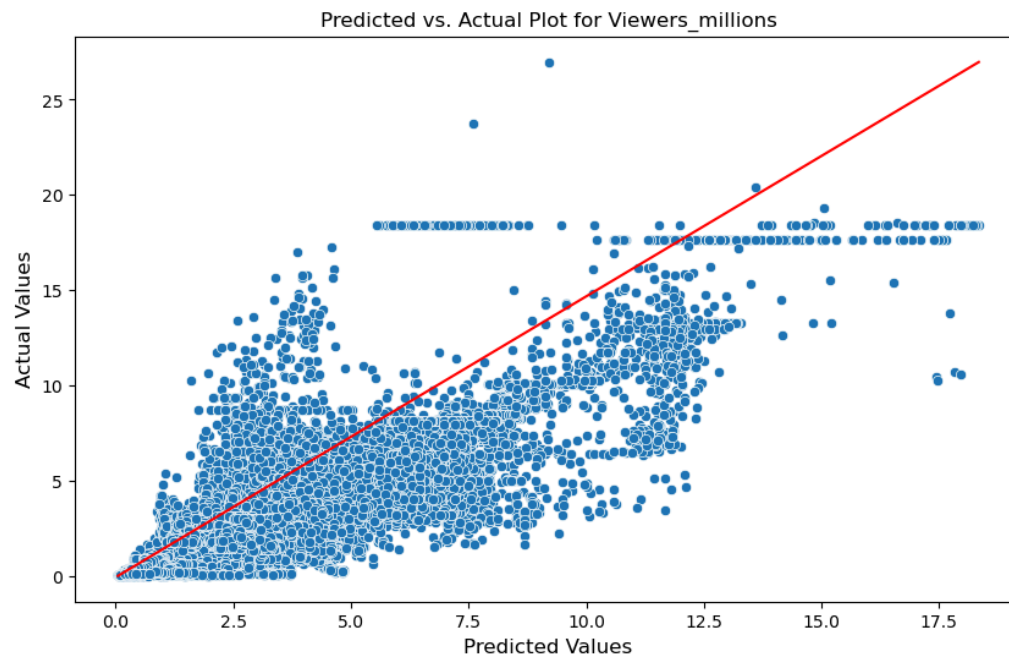
Mean average error: 1.0403642049072674

variable	importance
rating	0.02512651400814183
Episode_Votes	0.08125850667791835
fuzzy_vote_average	0.0921922437438865
fuzzy_popularity	0.17890403113700679
IMDb_Episode_Rating	0.025925930469788642
IMDb_Episode_Votes	0.07002437615920967
sex_factor	0.008781098138826675
violence_factor	0.007776684833286076
profanity_factor	0.0091969522558756
drugs_factor	0.007727659927116154
intense_factor	0.007256081206187736
HBO	0.006289926336334851
Netflix	0.002635731935023565
AMC	0.0009858052213582965
FOX	0.013561570935339592
The CW	0.013350630077031996
ABC	0.0522099319920831
Disney+	0.0006779797598120483
CBS	0.18801272550923329
Prime Video	0.00039417817655070913
Adult Swim	4.9311907954337626e-05
NBC	0.05513867145216937
Hulu	0.00041782000968653544
USA Network	0.014086842079554249
Cinemax	0.0013780504042928488
Comedy Central	0.002047822210434644
Showtime	0.008500524594549953
Paramount+	0.0022018923547795235
Paramount Network	1.9183383449165204e-05
Lifetime	0.0018728594765174298
Starz	0.0035468759801633803
Telemundo	0.0
Apple TV+	0.0
HBO Max	0.00029701154625365615
BET	0.0001559265796259849
Hallmark Channel	0.00043033081161613485
Peacock	0.0010492217710985337
Sci-Fi & Fantasy	0.014767018963875776
Drama	0.02063549326930941
Action & Adventure	0.00948276247934736
Crime	0.01702461262289851
Mystery	0.01098472361782543
Comedy	0.020197921303189162
Animation	0.0010226812533939917
Family	0.008387838577130761
War & Politics	0.0026730432738188816
Soap	0.0003375922553839867
Western	0.00075922678340829
Kids	0.0
Documentary	0.00016391101063291922
Reality	0.009981763962002895
Romance	0.00010050756562543134
Adult	0.0
Biography	0.0

Film-Noir	0.0
Game-Show	0.0
History	0.0
Horror	0.0
Musical	0.0
Music	0.0
News	0.0
Short	0.0
Sport	0.0
Talk-Show	0.0
Thriller	0.0







```
In [401... show_regression_results("18 to 49 Demo Estimation for Future Years After Two Seasons", three_seasons_or_more_df, '_18_to_49_Demo', None, \
select_features(), best_estimator_18_to_49_3s, rmse_18_to_49_3s, r2_18_to_49_3s, mae_18_to_49_3s, \
coefs_18_to_49_3s, X_18_to_49_3s, Y_18_to_49_3s, predictions_18_to_49_3s, residuals_18_to_49_3s)
```

18 to 49 Demo Estimation for Future Years After Two Seasons

Viewers (millions) Statistics:

Mean: 3.2304460719514516

Median: 2.256

Mode: [0.624]

Variance: 10.80646435590437

Standard Deviation: 3.2873187183332817

Summary: count 33835.000000

mean 3.230446

std 3.287319

min 0.008000

25% 0.795000

50% 2.256000

75% 4.540000

max 26.972000

Name: Viewers_millions, dtype: float64

18-49 Demo Statistics:

Mean: 0.8077153240688486

Median: 0.58

Mode: [0.7]

Variance: 0.7502568457920092

Standard Deviation: 0.8661736810778825

Summary: count 33835.000000

mean 0.807715

std 0.866174

min 0.000000

25% 0.240000

50% 0.580000

75% 1.100000

max 9.300000

Name: _18_to_49_Demo, dtype: float64

Best Estimator for _18_to_49_Demo: Pipeline(steps=[('scaler', MinMaxScaler()),
('regressor',
RandomForestRegressor(max_features='log2', min_samples_split=5,
n_estimators=200))])

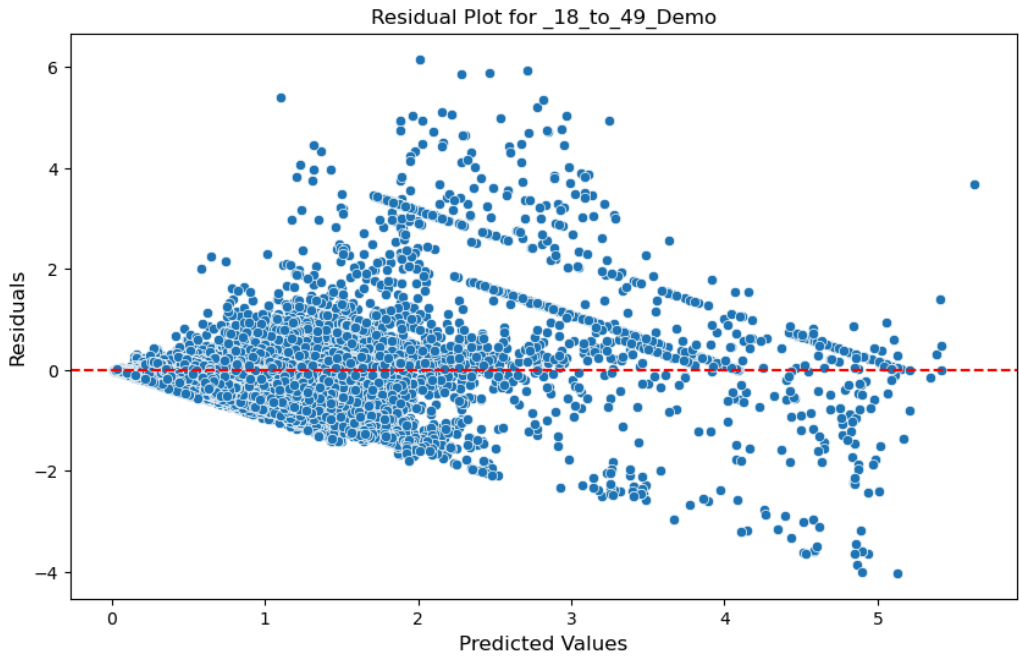
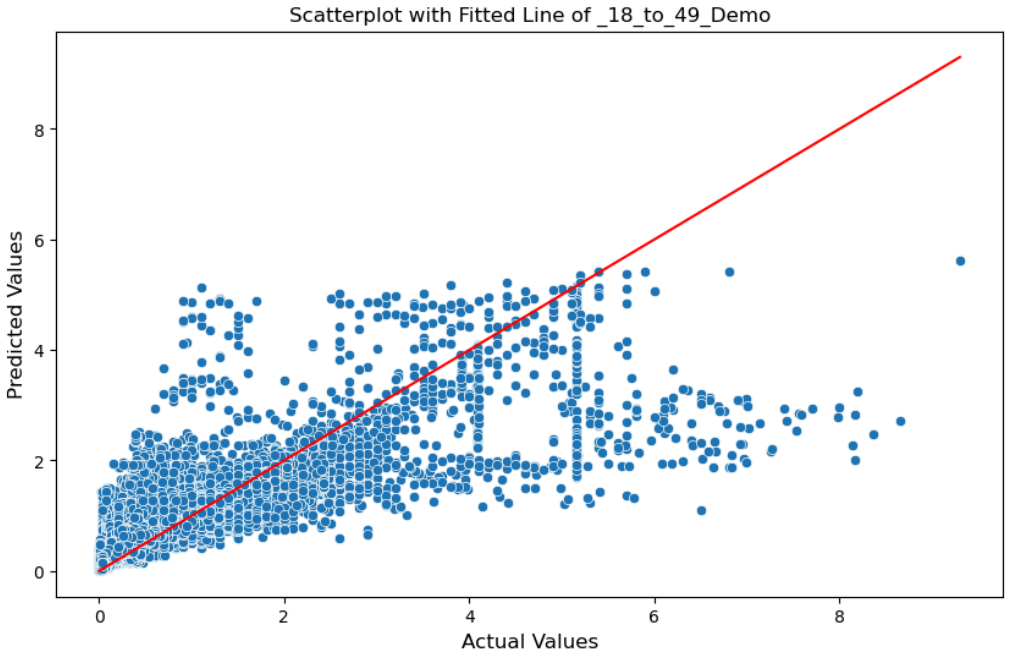
Root mean square error: 0.5819548790237785

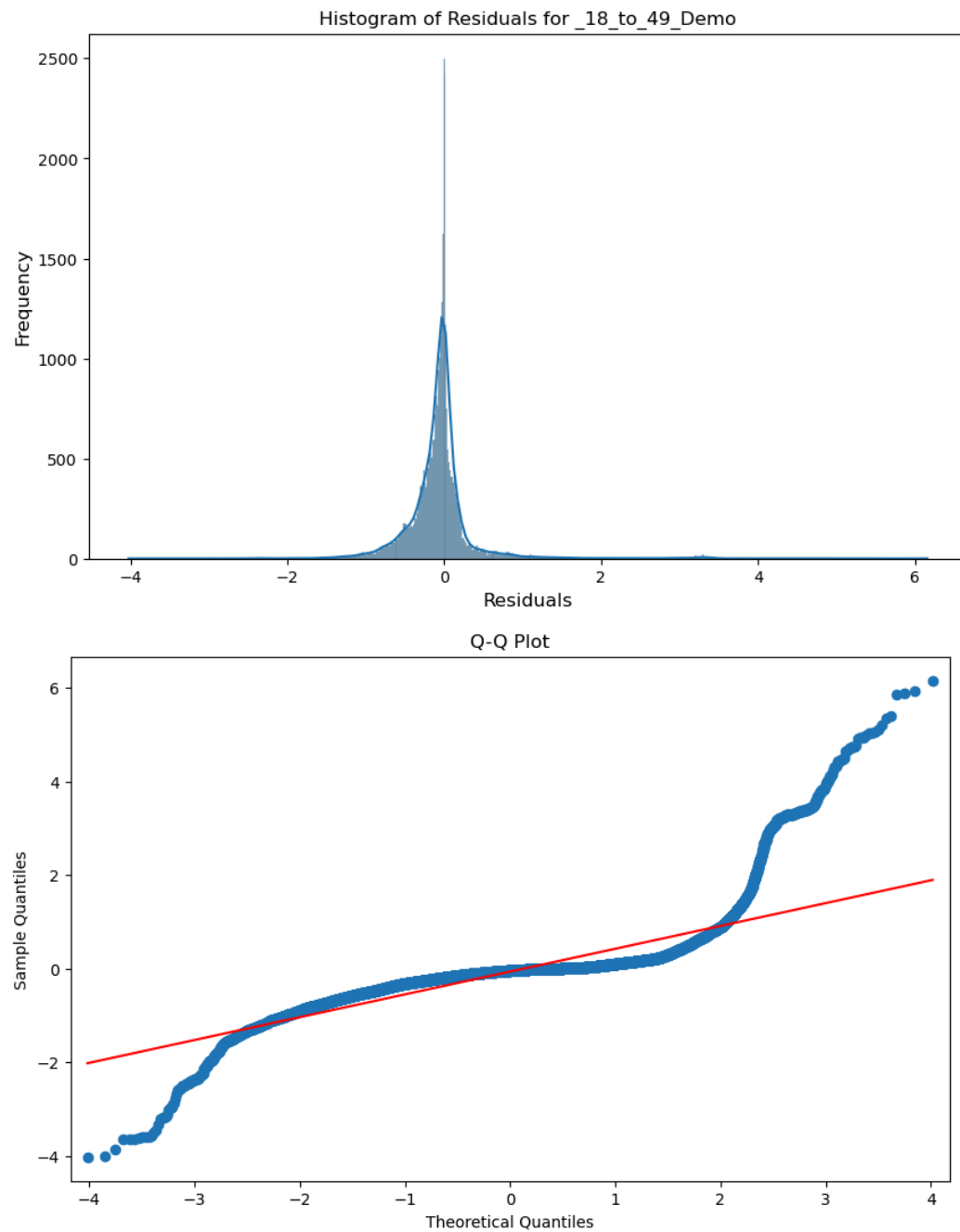
r2: 0.5783008292026611

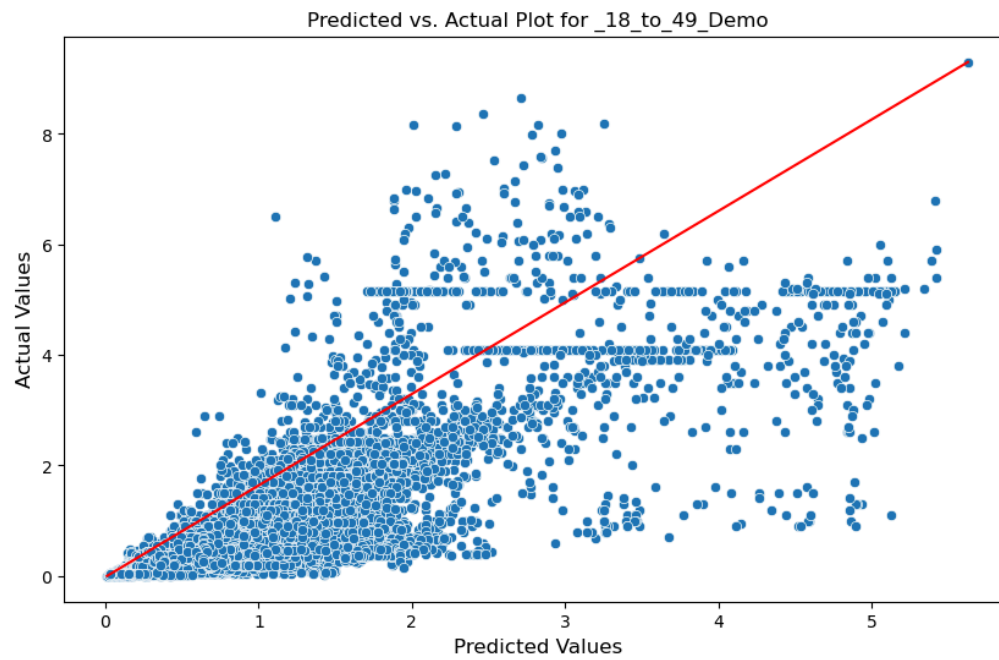
Mean average error: 0.3265465403328334

variable	importance
rating	0.03197293057004449
Episode_Votes	0.09826060932190261
fuzzy_vote_average	0.1223131068552516
fuzzy_popularity	0.2017701059623555
IMDb_Episode_Rating	0.032331384466565805
IMDb_Episode_Votes	0.09816095818372561
sex_factor	0.012760416300093703
violence_factor	0.01336356952289862
profanity_factor	0.014949079325316262
drugs_factor	0.012930182189531642
intense_factor	0.010739212439751237
HBO	0.0209474116613838
Netflix	0.0016249872923893003
AMC	0.0061634211194994066
FOX	0.014270113534182047
The CW	0.017344308790523604
ABC	0.0465009555695204
Disney+	0.0002750922855240049
CBS	0.022983113487069027
Prime Video	0.0010351593090124817
Adult Swim	0.0002410024617664131
NBC	0.03888777453851314
Hulu	0.0004062839775433503
USA Network	0.03540580412533253
Cinemax	0.0023350500022821063
Comedy Central	0.0013897556872663835
Showtime	0.008587115715562322
Paramount+	0.0014421751138774307
Paramount Network	0.0006204893452853664
Lifetime	0.003493850514957985
Starz	0.004455530848526827
Telemundo	0.0
Apple TV+	0.0
HBO Max	0.0013056418278082661
BET	0.0013071161974752682
Hallmark Channel	0.0011086909091882619
Peacock	0.0002592488813586387
Sci-Fi & Fantasy	0.015155097021641062
Drama	0.021485228201658903
Action & Adventure	0.010077648442200104
Crime	0.015663140343986366
Mystery	0.010136658633732842
Comedy	0.028110124621800587
Animation	0.0018631432180168132
Family	0.005310632869608701
War & Politics	0.0018787788414452824
Soap	0.00041975959266520583
Western	0.0018763215609842133
Kids	0.0
Documentary	0.00010766122107003539
Reality	0.005861455922577849
Romance	0.00011270117532668602
Adult	0.0
Biography	0.0

Film-Noir	0.0
Game-Show	0.0
History	0.0
Horror	0.0
Musical	0.0
Music	0.0
News	0.0
Short	0.0
Sport	0.0
Talk-Show	0.0
Thriller	0.0







```
In [402... show_regression_results("Viewers (Millions) Estimation for Future Years After Two Seasons", three_seasons_or_more_df, 'Viewers_millions', None, \
    select_features(), best_estimator_viewers_3s, rmse_viewers_3s, r2_viewers_3s, mae_viewers_3s, \
    coefs_viewers_3s, X_viewers_3s, Y_viewers_3s, predictions_viewers_3s, residuals_viewers_3s)
```

Viewers (Millions) Estimation for Future Years After Two Seasons

Viewers (millions) Statistics:

Mean: 3.2304460719514516

Median: 2.256

Mode: [0.624]

Variance: 10.80646435590437

Standard Deviation: 3.2873187183332817

Summary: count 33835.000000

mean 3.230446

std 3.287319

min 0.008000

25% 0.795000

50% 2.256000

75% 4.540000

max 26.972000

Name: Viewers_millions, dtype: float64

18-49 Demo Statistics:

Mean: 0.8077153240688486

Median: 0.58

Mode: [0.7]

Variance: 0.7502568457920092

Standard Deviation: 0.8661736810778825

Summary: count 33835.000000

mean 0.807715

std 0.866174

min 0.000000

25% 0.240000

50% 0.580000

75% 1.100000

max 9.300000

Name: _18_to_49_Demo, dtype: float64

Best Estimator for Viewers_millions: Pipeline(steps=[('scaler', MinMaxScaler()),
('regressor',
RandomForestRegressor(max_depth=30, max_features='sqrt',
min_samples_split=10))])

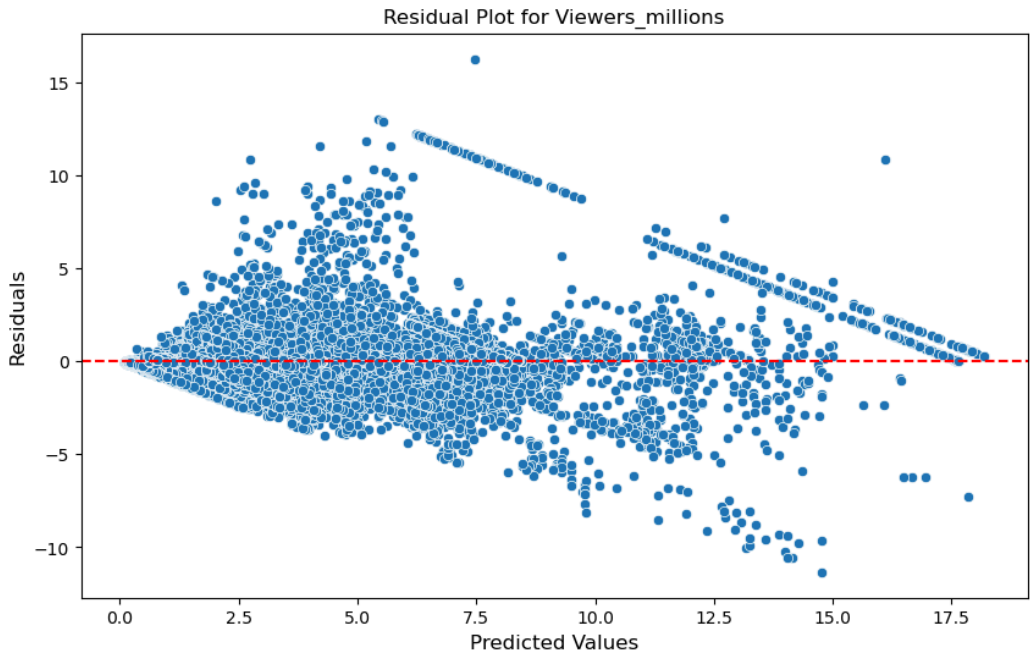
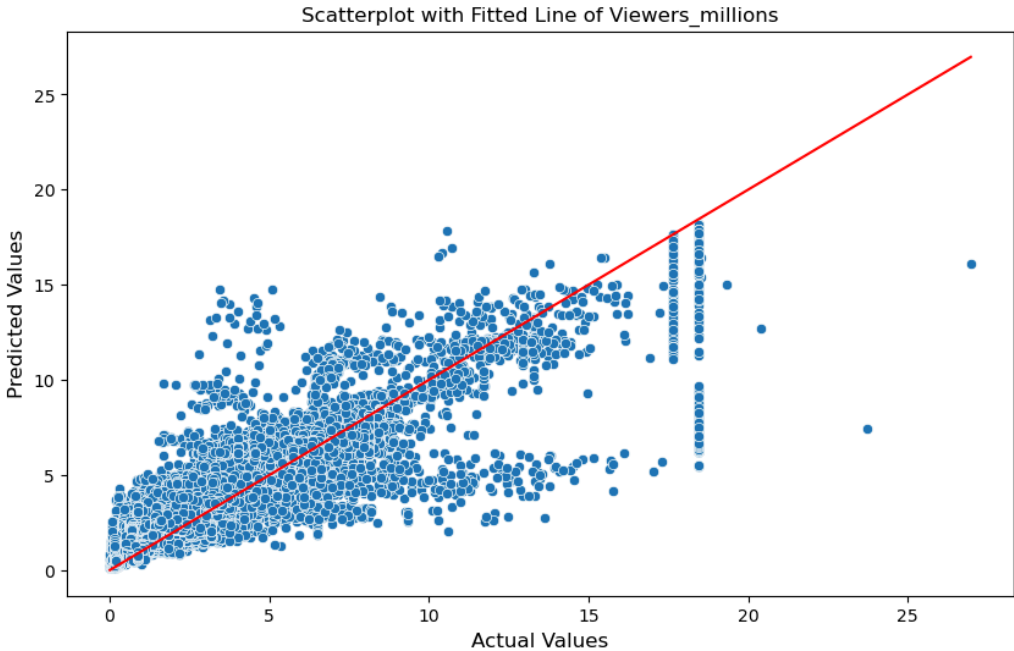
Root mean square error: 1.6296137747154205

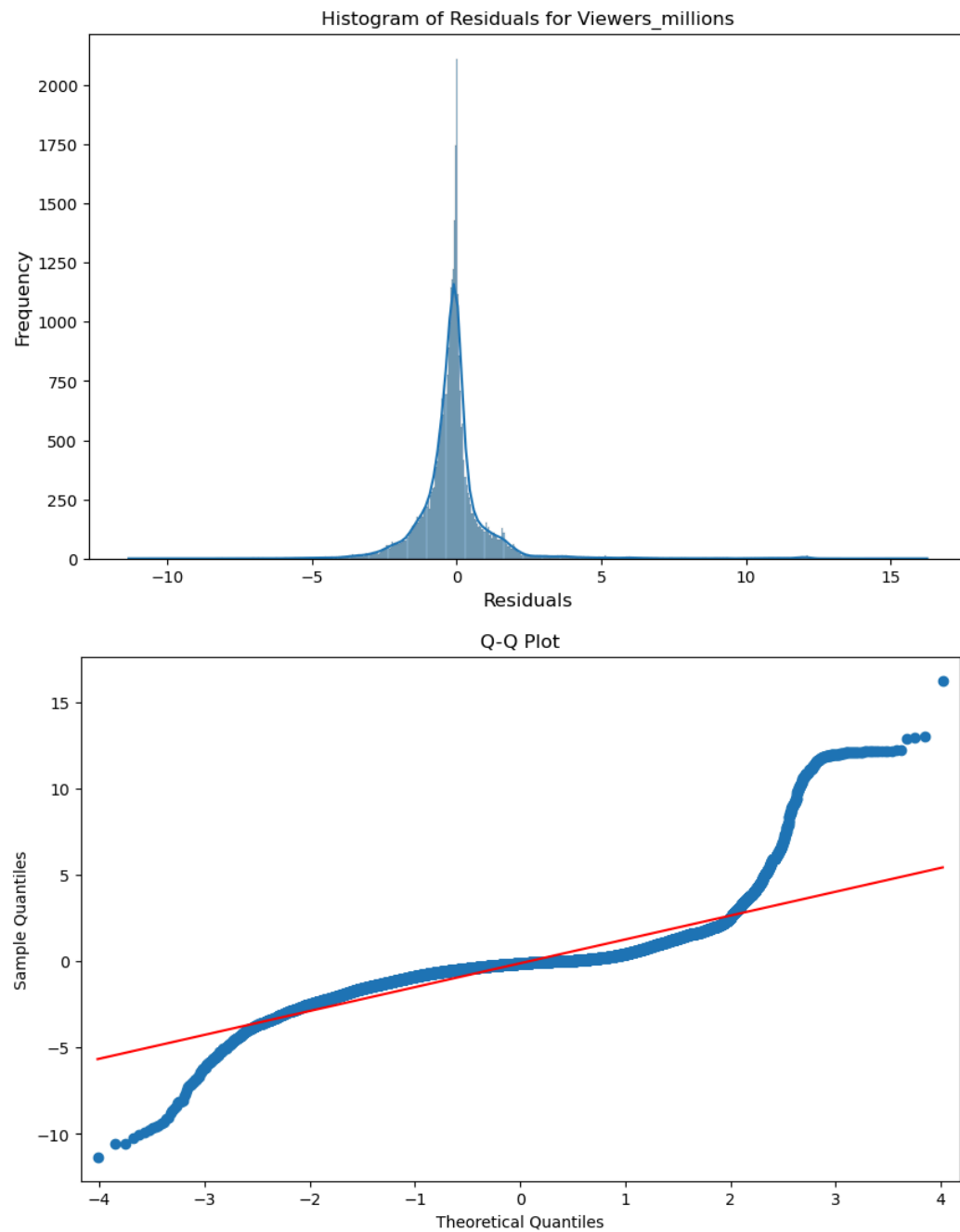
r2: 0.7659125904502707

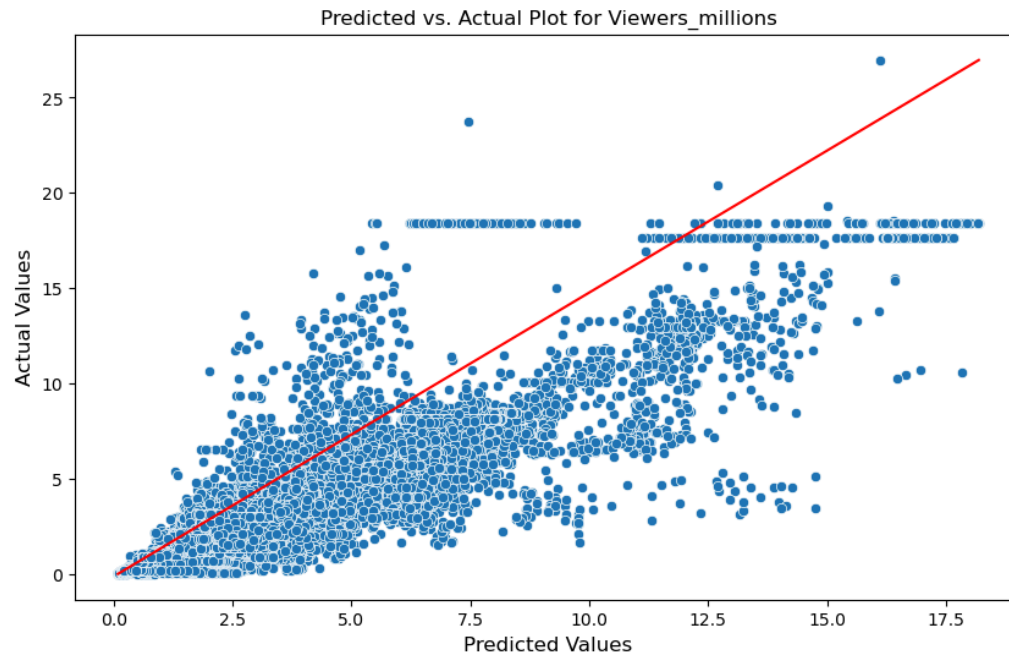
Mean average error: 0.9444540584943772

variable	importance
rating	0.02293994712752943
Episode_Votes	0.0792831323701497
fuzzy_vote_average	0.09269921687001606
fuzzy_popularity	0.19120424386815263
IMDb_Episode_Rating	0.02144484151901728
IMDb_Episode_Votes	0.07452071586229804
sex_factor	0.0077040873862750945
violence_factor	0.007156277813843655
profanity_factor	0.007671860763552159
drugs_factor	0.0059754373221960234
intense_factor	0.004609988200807425
HBO	0.005354546855947735
Netflix	0.0013203739324053197
AMC	0.00139079808852389
FOX	0.01386535197395807
The CW	0.020874761009368185
ABC	0.054373795643156995
Disney+	0.002475504198835308
CBS	0.1730780699408828
Prime Video	0.00036196548564308234
Adult Swim	0.0002934959528905394
NBC	0.04894292200905335
Hulu	0.0002940333371833431
USA Network	0.01961631851472921
Cinemax	0.0012806106594216626
Comedy Central	0.0015472497354979862
Showtime	0.009959757013649253
Paramount+	0.004445123391656448
Paramount Network	0.00016484751901723644
Lifetime	0.0014432704115140149
Starz	0.002971337713866284
Telemundo	0.0
Apple TV+	0.0
HBO Max	0.0007956829480420745
BET	0.000193264041936247
Hallmark Channel	0.000507312868271745
Peacock	0.0011183610395267749
Sci-Fi & Fantasy	0.015785024802395888
Drama	0.019141912377404324
Action & Adventure	0.009314823938944852
Crime	0.01687363135299139
Mystery	0.013485578167319639
Comedy	0.023138956624265565
Animation	0.0017034602945937686
Family	0.007415591920947129
War & Politics	0.0022392686610177687
Soap	0.00041049978393576536
Western	0.0005163857307197474
Kids	0.0
Documentary	0.00033043251551276515
Reality	0.007739764420217005
Romance	2.6166020919314525e-05
Adult	0.0
Biography	0.0

Film-Noir	0.0
Game-Show	0.0
History	0.0
Horror	0.0
Musical	0.0
Music	0.0
News	0.0
Short	0.0
Sport	0.0
Talk-Show	0.0
Thriller	0.0







In []:

Testing

In [403...

```
test_data_df = data_for_tuning_df.copy()
show_df_info(test_data_df, "Test Data")

wider_main_series_df = pd.read_csv(UNFILTERED_MAIN_SERIES_FOR_MODELING_DATA_FILE)
show_df_info(wider_main_series_df, "Series Data")
```

Dataframe information for Test Data:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	IMDb Episode ID	object	0
2	season	int32	0
3	episode	int32	0
4	rating	float64	0
5	Episode_Votes	int32	0
6	number_of_seasons	float64	0
7	number_of_episodes	float64	0
8	Series_Votes	float64	0
9	vote_average	float64	0
10	in_production	bool	0
11	popularity	float64	0
12	Sci-Fi & Fantasy	bool	0
13	Drama	bool	0
14	Action & Adventure	bool	0
15	Crime	bool	0
16	Mystery	bool	0
17	Comedy	bool	0
18	Animation	bool	0
19	Family	bool	0
20	War & Politics	bool	0
21	Soap	bool	0
22	Western	bool	0
23	Kids	bool	0
24	Documentary	bool	0
25	Reality	bool	0
26	Romance	bool	0
27	Adult	bool	0
28	Biography	bool	0
29	Film-Noir	bool	0
30	Game-Show	bool	0
31	History	bool	0
32	Horror	bool	0
33	Musical	bool	0
34	Music	bool	0
35	News	bool	0
36	Short	bool	0
37	Sport	bool	0
38	Talk-Show	bool	0
39	Thriller	bool	0
40	HBO	bool	0
41	Netflix	bool	0
42	AMC	bool	0
43	FOX	bool	0
44	The CW	bool	0
45	ABC	bool	0
46	Disney+	bool	0
47	CBS	bool	0
48	Prime Video	bool	0
49	Adult Swim	bool	0
50	NBC	bool	0
51	Hulu	bool	0
52	USA Network	bool	0
53	Cinemax	bool	0
54	Comedy Central	bool	0
55	Showtime	bool	0
56	Paramount+	bool	0
57	Paramount Network	bool	0
58	Lifetime	bool	0
59	Starz	bool	0
60	Telemundo	bool	0
61	Apple TV+	bool	0
62	HBO Max	bool	0
63	BET	bool	0
64	Hallmark Channel	bool	0
65	Peacock	bool	0
66	_18_to_49_Demo	float64	0
67	Viewers_millions	float64	0
68	sex_factor	int64	0
69	violence_factor	int64	0
70	profanity_factor	int64	0
71	drugs_factor	int64	0
72	intense_factor	int64	0
73	IMDb_Episode_Rating	float64	0
74	IMDb_Episode_Votes	int64	0
75	status_Canceled	bool	0
76	status_Ended	bool	0
77	status_Returning Series	bool	0
78	type_Documentary	bool	0
79	type_Miniseries	bool	0
80	type_News	bool	0
81	type_Reality	bool	0
82	type_Scripted	bool	0
83	type_Talk Show	bool	0
84	ranking_of_Series_Votes	float64	0
85	ranking_of_vote_average	float64	0
86	ranking_of_popularity	float64	0
87	fuzzy_Series_Votes	float64	0
88	fuzzy_vote_average	float64	0
89	fuzzy_popularity	float64	0

There are 41651 rows and 90 columns of information.

Dataframe information for Series Data:

Dataframe Information:

	Column	Data Type	NaN Count
0	id	int64	0
1	Title	object	0
2	number_of_seasons	float64	0
3	number_of_episodes	float64	0
4	original_language	object	0
5	vote_count	float64	0
6	vote_average	float64	0
7	overview	object	506
8	adult	bool	0
9	backdrop_path	object	502
10	first_air_date	object	45
11	last_air_date	object	30
12	homepage	object	0
13	in_production	bool	0
14	original_name	object	0
15	popularity	float64	0
16	poster_path	object	62
17	type	object	0
18	status	object	0
19	tagline	object	10470
20	genres	object	138
21	created_by	object	5589
22	languages	object	65
23	networks	object	183
24	origin_country	object	194
25	spoken_languages	object	69
26	production_companies	object	2533
27	production_countries	object	1995
28	episode_run_time	object	0
29	IMDb Series ID	object	0
30	Title Type	object	0

There are 13038 rows and 31 columns of information.

Testing

- once the model is trained use it to predict the performance and fate of series like this.

In [404...

```
from sklearn.exceptions import NotFittedError

def select_series_episode_data(df, series_id):
    return df[df['IMDb Series ID'] == series_id]

def get_matching_series_record(series_id):
    return wider_main_series_df[wider_main_series_df['IMDb Series ID'] == series_id]

def get_matching_series_record_by_title(title):
    return wider_main_series_df[wider_main_series_df['Title'] == title]

def select_series_records_by_title(df, title):
    print(f"looking up {title}")
    matching_record = get_matching_series_record_by_title(title)
    series_id = matching_record['IMDb Series ID'].unique()[0]
    return select_series_episode_data(df, series_id)

def choose_random_series(df):
    series_ids = df['IMDb Series ID'].unique()
    random_series_id = random.choice(series_ids)
    return random_series_id

def get_all_episodes_of_random_series(df, series_ids):
    # just once per show.
    random_id = choose_random_series(df)
    while random_id in series_ids:
        random_id = choose_random_series(df)

    return select_series_episode_data(df, random_id)

def fit_already_fuzzified_data_for_testing(test_df, genres=False, networks=False, content=False):
    column_names = select_features(genres=genres, networks=networks, content=content)
    features = test_df[column_names]
    features_df = pd.DataFrame(features, columns=features.columns)
    return features_df

def fitted_survival_test(trained_classifier, target_field_name, train_df, test_df, genres=True, networks=True, content=True):
    all_labels = trained_classifier.classes_

    x_train = fit_already_fuzzified_data_for_testing(train_df, genres, networks, content)
    x_test = fit_already_fuzzified_data_for_testing(test_df, genres, networks, content)
    y_train = train_df[target_field_name]
    y_test = test_df[target_field_name]
    trained_classifier.fit(x_train, y_train)

    predictions = trained_classifier.predict(x_test)
```

```

extra_trees_accuracy = accuracy_score(y_test, predictions)
extra_trees_precision = precision_score(y_test, predictions, zero_division=0)
extra_trees_recall = recall_score(y_test, predictions, zero_division=0)
f1_score = 2 * (extra_trees_precision * extra_trees_recall) / (extra_trees_precision + extra_trees_recall) \
    if (extra_trees_precision + extra_trees_recall) != 0 else 0

conf_matrix = confusion_matrix(y_test, predictions, labels=all_labels)
return predictions, conf_matrix

def fitted_regression_test(trained_estimator, target_field_name, train_df, test_df, genres=True, networks=True, content=True):
    x_train = fit_already_fuzzified_data_for_testing(train_df, genres, networks, content)
    x_test = fit_already_fuzzified_data_for_testing(test_df, genres, networks, content)
    y_train = train_df[target_field_name]
    y_test = test_df[target_field_name]
    trained_estimator.fit(x_train, y_train)
    predictions = trained_estimator.predict(x_test)
    residuals = y_test - predictions
    return predictions, residuals

def extract_confusion_matrix_elements(conf_matrix):
    """
    Extracts the TN, FP, FN, TP from a confusion matrix
    Args:
    conf_matrix (numpy.ndarray): Confusion matrix

    Returns:
    tn, fp, fn, tp (tuple): The values of true negatives, false positives, false negatives, and true positives
    """
    tn, fp, fn, tp = conf_matrix.ravel()
    return tn, fp, fn, tp

# Pre-trained models and PCA objects loaded from previous runs
best_estimator_18_to_49 = save_regression_estimator_18_to_49_demo
best_estimator_viewers = save_regression_estimator_viewers

fitted_classifier_canceled = fitted_classifier_canceled_50
fitted_classifier_ended = fitted_classifier_ended_50

list_of_models = [best_estimator_18_to_49, best_estimator_viewers, fitted_classifier_canceled, fitted_classifier_ended,
    save_regression_estimator_18_to_49_demo, save_regression_estimator_viewers,
    fitted_classifier_canceled_50, fitted_classifier_ended_50]

# for model in list_of_models:
#     test_if_trained(model)

# Initialize dataframes to store predictions
_18_to_49_regression_predictions_df = pd.DataFrame(columns=["IMDb Series ID", "Title", "status",
    "18_to_49_demo_prediction", "18_to_49_demo_residual"])
viewers_regression_predictions_df = pd.DataFrame(columns=["IMDb Series ID", "Title", "status", "viewers_prediction", "viewers_residual"])

# Remove any entirely NaN columns from the original DataFrames
_18_to_49_regression_predictions_df = _18_to_49_regression_predictions_df.dropna(axis=1, how='all')
viewers_regression_predictions_df = viewers_regression_predictions_df.dropna(axis=1, how='all')

canceled_tns = []
canceled_fps = []
canceled_fns = []
canceled_tps = []
ended_tns = []
ended_tps = []
ended_fns = []
ended_fps = []
series_ids = []
series_names = []
statuses = []

# start by seeding the test code with some well known series from imdb list
# Last one is Stranger Things (tests using imdb series number directly)
titles = ['Game of Thrones', 'Euphoria', 'Breaking Bad', 'tt4574334']

for title in titles:
    # start with saved data each time
    episode_info_df = data_for_tuning_df.copy()
    # fuzzify at 5 series per box
    fuzzify_series(episode_info_df, 5)

    # Check if the title starts with 'tt'
    if title.startswith('tt'):
        series_id = title # Assign the IMDb ID directly
        matching_series_info = get_matching_series_record(title)
    else:
        # Otherwise, proceed with typical logic
        test_data_df = select_series_records_by_title(episode_info_df, title)
        series_id = test_data_df['IMDb Series ID'].unique()[0]
        matching_series_info = get_matching_series_record_by_title(title)

    series_name = matching_series_info['Title'].values[0] # Ensure series_name is extracted correctly

    train_data_df = pd.concat([episode_info_df, test_data_df]).drop_duplicates(keep=False)

    status = matching_series_info['status'].values[0] # Ensure status is extracted correctly
    seasons = matching_series_info['number_of_seasons'].values[0] # Ensure status is extracted correctly
    episodes = matching_series_info['number_of_episodes'].values[0] # Ensure status is extracted correctly

    train_data_df = pd.concat([episode_info_df, test_data_df]).drop_duplicates(keep=False)

    # reassuring message

```

```

print(f"\n\tTest for Series: {series_id}. Title: {series_name}. Seasons: {seasons}. Episodes: {episodes}.\n")

# Perform predictions
predictions_18_to_49, residuals_18_to_49 = fitted_regression_test(best_estimator_18_to_49, '_18_to_49_Demo', train_data_df, test_data_df)
predictions_viewers, residuals_viewers = fitted_regression_test(best_estimator_viewers, 'Viewers_millions', train_data_df, test_data_df)

survival_predictions_ended, conf_matrix_ended = fitted_survival_test(fitted_classifier_ended, 'status_ended', train_data_df, test_data_df,
                                                                    genres=False, networks=False, content=False)
survival_predictions_canceled, conf_matrix_canceled = fitted_survival_test(fitted_classifier_canceled, 'status_canceled', train_data_df,
                                                                    test_data_df, genres=False, networks=False, content=False)

ended_tn, ended_fp, ended_fn, ended_tp = conf_matrix_ended.ravel()
canceled_tn, canceled_fp, canceled_fn, canceled_tp = conf_matrix_canceled.ravel()

# Create temporary dataframes for the current iteration of regressions
temp_18_to_49_df = pd.DataFrame({
    "IMDb Series ID": [series_id] * len(predictions_18_to_49),
    "Title": [series_name] * len(predictions_18_to_49),
    "status": [status] * len(predictions_18_to_49),
    "_18_to_49_demo_prediction": predictions_18_to_49,
    "_18_to_49_demo_residual": residuals_18_to_49
}).fillna(0).dropna(axis=1, how='all')

temp_viewers_df = pd.DataFrame({
    "IMDb Series ID": [series_id] * len(predictions_viewers),
    "Title": [series_name] * len(predictions_viewers),
    "status": [status] * len(predictions_viewers),
    "viewers_prediction": predictions_viewers,
    "viewers_residual": residuals_viewers
}).fillna(0).dropna(axis=1, how='all')

# survival results are kept in lists (summaries)
ended_tns.append(ended_tn)
ended_tps.append(ended_tp)
ended_fns.append(ended_fn)
ended_fps.append(ended_fp)
canceled_tns.append(canceled_tn)
canceled_tps.append(canceled_tp)
canceled_fns.append(canceled_fn)
canceled_fps.append(canceled_fp)
series_ids.append(series_id)
series_names.append(series_name)
statuses.append(status)

# Remove any entirely NaN columns from the temporary DataFrames
temp_18_to_49_df = temp_18_to_49_df.dropna(axis=1, how='all')
temp_viewers_df = temp_viewers_df.dropna(axis=1, how='all')

# Concatenate the temporary dataframes with the dataframes
_18_to_49_regression_predictions_df = pd.concat([_18_to_49_regression_predictions_df, temp_18_to_49_df], ignore_index=True)
viewers_regression_predictions_df = pd.concat([viewers_regression_predictions_df, temp_viewers_df], ignore_index=True)

# add some randoms
range_val = 8
for _ in range(range_val):
    # start with saved data
    episode_info_df = data_for_tuning_df.copy()
    # fuzzify in boxes of 5
    fuzzify_series(episode_info_df, 5)
    test_data_df = get_all_episodes_of_random_series(episode_info_df, series_ids)
    # remove the matching series from the training data (otherwise classification test is just cheating)
    train_data_df = pd.concat([episode_info_df, test_data_df]).drop_duplicates(keep=False)
    # get row data for output
    series_id = test_data_df['IMDb Series ID'].unique()[0]
    matching_series_info = get_matching_series_record(series_id)
    series_name = matching_series_info['Title'].values[0] # Ensure series_name is extracted correctly
    status = matching_series_info['status'].values[0] # Ensure status is extracted correctly
    seasons = matching_series_info['number_of_seasons'].values[0] # Ensure status is extracted correctly
    episodes = matching_series_info['number_of_episodes'].values[0] # Ensure status is extracted correctly

    # print a reassuring message
    print(f"\n\tTest for Series: {series_id}. Title: {series_name}. Seasons: {seasons}. Episodes: {episodes}.\n")

    # Perform predictions
    predictions_18_to_49, residuals_18_to_49 = fitted_regression_test(best_estimator_18_to_49, '_18_to_49_Demo', train_data_df, test_data_df)
    predictions_viewers, residuals_viewers = fitted_regression_test(best_estimator_viewers, 'Viewers_millions', train_data_df, test_data_df)

    survival_predictions_ended, conf_matrix_ended = fitted_survival_test(fitted_classifier_ended, 'status_ended', train_data_df,
                                                                    test_data_df, genres=False, networks=False, content=False)
    survival_predictions_canceled, conf_matrix_canceled = fitted_survival_test(fitted_classifier_canceled, 'status_canceled', train_data_df,
                                                                    test_data_df, genres=False, networks=False, content=False)

    # prepare output data frames
    ended_tn, ended_fp, ended_fn, ended_tp = conf_matrix_ended.ravel()
    canceled_tn, canceled_fp, canceled_fn, canceled_tp = conf_matrix_canceled.ravel()

    # Create temporary dataframes for the current iteration
    temp_18_to_49_df = pd.DataFrame({
        "IMDb Series ID": [series_id] * len(predictions_18_to_49),
        "Title": [series_name] * len(predictions_18_to_49),
        "status": [status] * len(predictions_18_to_49),
        "_18_to_49_demo_prediction": predictions_18_to_49,
        "_18_to_49_demo_residual": residuals_18_to_49
    }).fillna(0).dropna(axis=1, how='all')

    temp_viewers_df = pd.DataFrame({
        "IMDb Series ID": [series_id] * len(predictions_viewers),
        "Title": [series_name] * len(predictions_viewers),
        "status": [status] * len(predictions_viewers),

```

```

    "viewers_prediction": predictions_viewers,
    "viewers_residual": residuals_viewers
}).fillna(0).dropna(axis=1, how='all')

# prepare fate lists
ended_tns.append(ended_tn)
ended_tps.append(ended_tp)
ended_fns.append(ended_fn)
ended_fps.append(ended_fp)
canceled_tns.append(canceled_tn)
canceled_tps.append(canceled_tp)
canceled_fns.append(canceled_fn)
canceled_fps.append(canceled_fp)
series_ids.append(series_id)
series_names.append(series_name)
statuses.append(status)

# Remove any entirely NaN columns from the temporary DataFrames
temp_18_to_49_df = temp_18_to_49_df.dropna(axis=1, how='all')
temp_viewers_df = temp_viewers_df.dropna(axis=1, how='all')

# Concatenate the temporary dataframes with the main dataframes
_18_to_49_regression_predictions_df = pd.concat([_18_to_49_regression_predictions_df, temp_18_to_49_df], ignore_index=True)
viewers_regression_predictions_df = pd.concat([viewers_regression_predictions_df, temp_viewers_df], ignore_index=True)

# fate data frame
fate_classification_predictions_df = pd.DataFrame({
    "IMDb Series ID": series_ids,
    "Title": series_names,
    "status": statuses,
    "ended_tn": ended_tns,
    "ended_fp": ended_fps,
    "ended_fn": ended_fns,
    "ended_tp": ended_tps,
    "canc_tn": canceled_tns,
    "canc_fp": canceled_fps,
    "canc_fn": canceled_fns,
    "canc_tp": canceled_tps,
}).fillna(False).dropna(axis=1, how='all')

# write it
_18_to_49_regression_predictions_df.to_csv(FINAL_DATA_DIRECTORY+"/test 18-49 demo.csv", index=False)
viewers_regression_predictions_df.to_csv(FINAL_DATA_DIRECTORY+"/test viewers.csv", index=False)
fate_classification_predictions_df.to_csv(FINAL_DATA_DIRECTORY+"/test series fate.csv", index=False)

```

Ranking 743 series in 149.0 classification pockets.
looking up *Game of Thrones*

Test for Series: tt0944947. Title: Game of Thrones. Seasons: 8.0. Episodes: 73.0.

Ranking 743 series in 149.0 classification pockets.
looking up *Euphoria*

Test for Series: tt8772296. Title: Euphoria. Seasons: 2.0. Episodes: 16.0.

Ranking 743 series in 149.0 classification pockets.
looking up *Breaking Bad*

Test for Series: tt0903747. Title: Breaking Bad. Seasons: 5.0. Episodes: 62.0.

Ranking 743 series in 149.0 classification pockets.

Test for Series: tt4574334. Title: Stranger Things. Seasons: 4.0. Episodes: 34.0.

Ranking 743 series in 149.0 classification pockets.

Test for Series: tt10327354. Title: Prodigal Son. Seasons: 2.0. Episodes: 33.0.

Ranking 743 series in 149.0 classification pockets.

Test for Series: tt5511582. Title: Timeless. Seasons: 2.0. Episodes: 26.0.

Ranking 743 series in 149.0 classification pockets.

Test for Series: tt6873926. Title: Two Sentence Horror Stories. Seasons: 3.0. Episodes: 29.0.

Ranking 743 series in 149.0 classification pockets.

Test for Series: tt1429449. Title: Lost Girl. Seasons: 5.0. Episodes: 77.0.

Ranking 743 series in 149.0 classification pockets.

Test for Series: tt1845307. Title: 2 Broke Girls. Seasons: 6.0. Episodes: 137.0.

Ranking 743 series in 149.0 classification pockets.

Test for Series: tt2314792. Title: Low Winter Sun. Seasons: 1.0. Episodes: 10.0.

Ranking 743 series in 149.0 classification pockets.

Test for Series: tt5640060. Title: Chicago Justice. Seasons: 1.0. Episodes: 13.0.

Ranking 743 series in 149.0 classification pockets.

Test for Series: tt1048176. Title: Living Biblically. Seasons: 1.0. Episodes: 13.0.

```

In [405... # Function to add episode number to the DataFrame
def add_episode_number(df):
    df = df.copy() # Make a copy of the DataFrame to avoid modifying the original DataFrame
    df['Episode Number'] = df.groupby('IMDb Series ID').cumcount() + 1
    return df

# Function to plot predicted vs actual values for selected series
def plot_predicted_vs_actual(df, series_ids, predicted_column, residual_column, label):
    # Filter the DataFrame to include only the selected series
    filtered_df = df[df['IMDb Series ID'].isin(series_ids)]

    # Define colors for each series
    colors = plt.cm.tab20.colors

    # Set figure size and adjust layout to create space for the Legend
    plt.figure(figsize=(18, 6)) # Increased size for better clarity
    plt.subplots_adjust(right=0.75) # Adjust right margin to create space for the Legend

    # Plot each series separately
    for i, series_id in enumerate(series_ids):
        # Filter the DataFrame to include only the selected series
        group = filtered_df[filtered_df['IMDb Series ID'] == series_id]

        # Calculate actual values (predicted value + residual)
        actual_values = group[predicted_column] + group[residual_column]
        title = group['Title'].unique()[0]

        # Use the 'Episode Number' column as the episode number
        episode_numbers = group['Episode Number']

        # Plot predicted values with '+'
        plt.scatter(episode_numbers, group[predicted_column], marker='+', color=colors[i % len(colors)], label=f'{title} Predicted')

        # Plot actual values
        plt.scatter(episode_numbers, actual_values, marker='.', color=colors[i % len(colors)], label=f'{title} Actual')

    # Add Legend outside the plot
    plt.legend(bbox_to_anchor=(1.25, 1), loc='upper right', borderaxespad=0.)

    # Set axis Labels
    plt.xlabel('Episode')
    plt.ylabel(label)

    # Show plot
    plt.show()

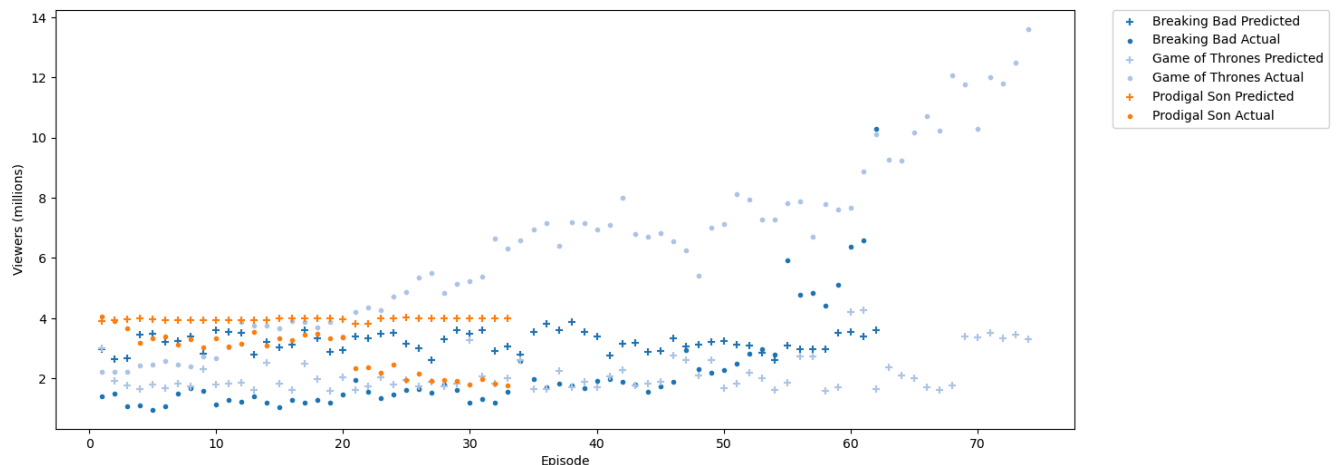
# Add episode number to the DataFrame (x axis)
viewers_regression_predictions_df = add_episode_number(viewers_regression_predictions_df)
_18_to_49_regression_predictions_df = add_episode_number(_18_to_49_regression_predictions_df)

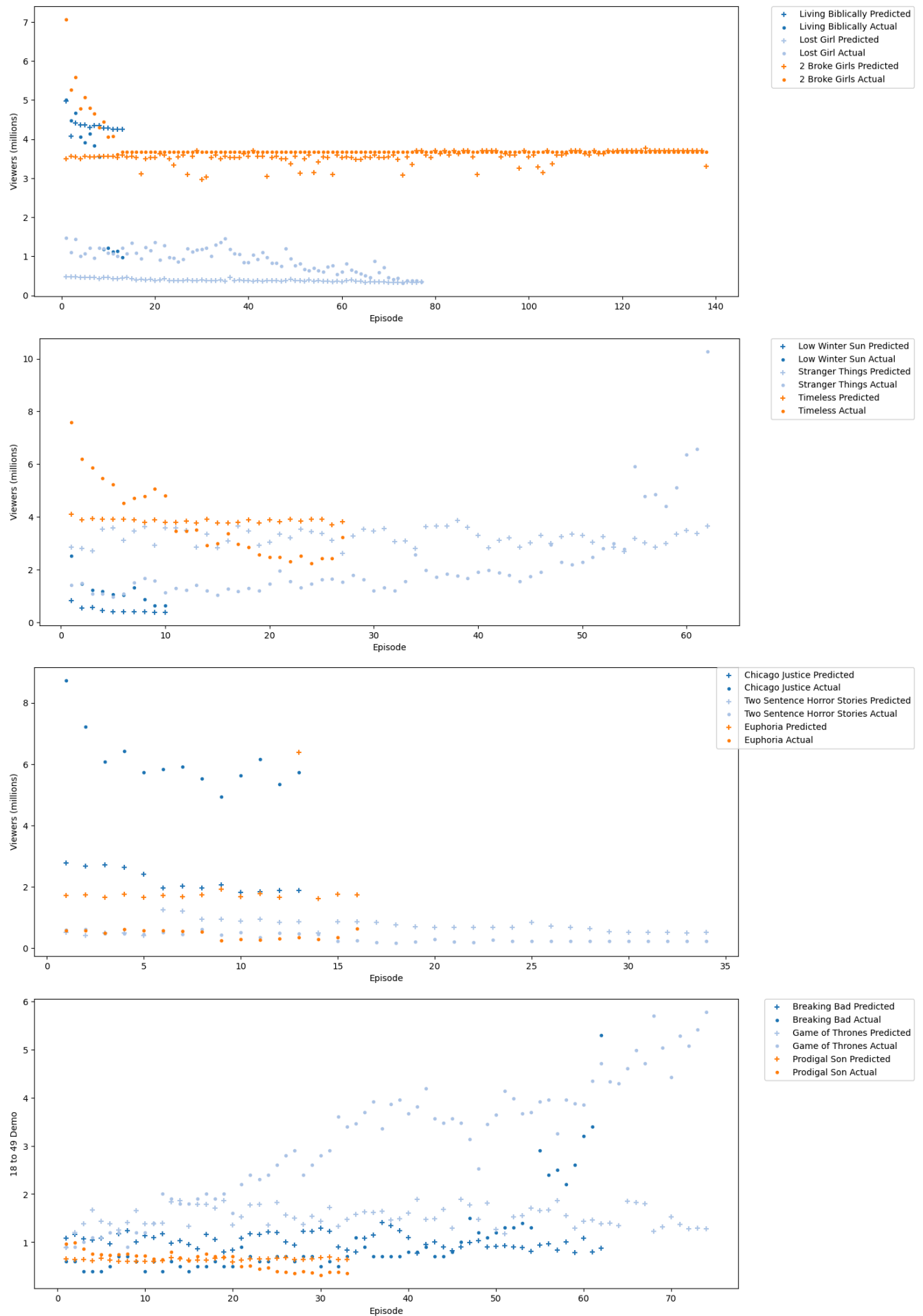
# Ensure the DataFrame is correctly structured
viewers_regression_predictions_df = viewers_regression_predictions_df.sort_values(by=['IMDb Series ID', 'Episode Number'])
_18_to_49_regression_predictions_df = _18_to_49_regression_predictions_df.sort_values(by=['IMDb Series ID', 'Episode Number'])

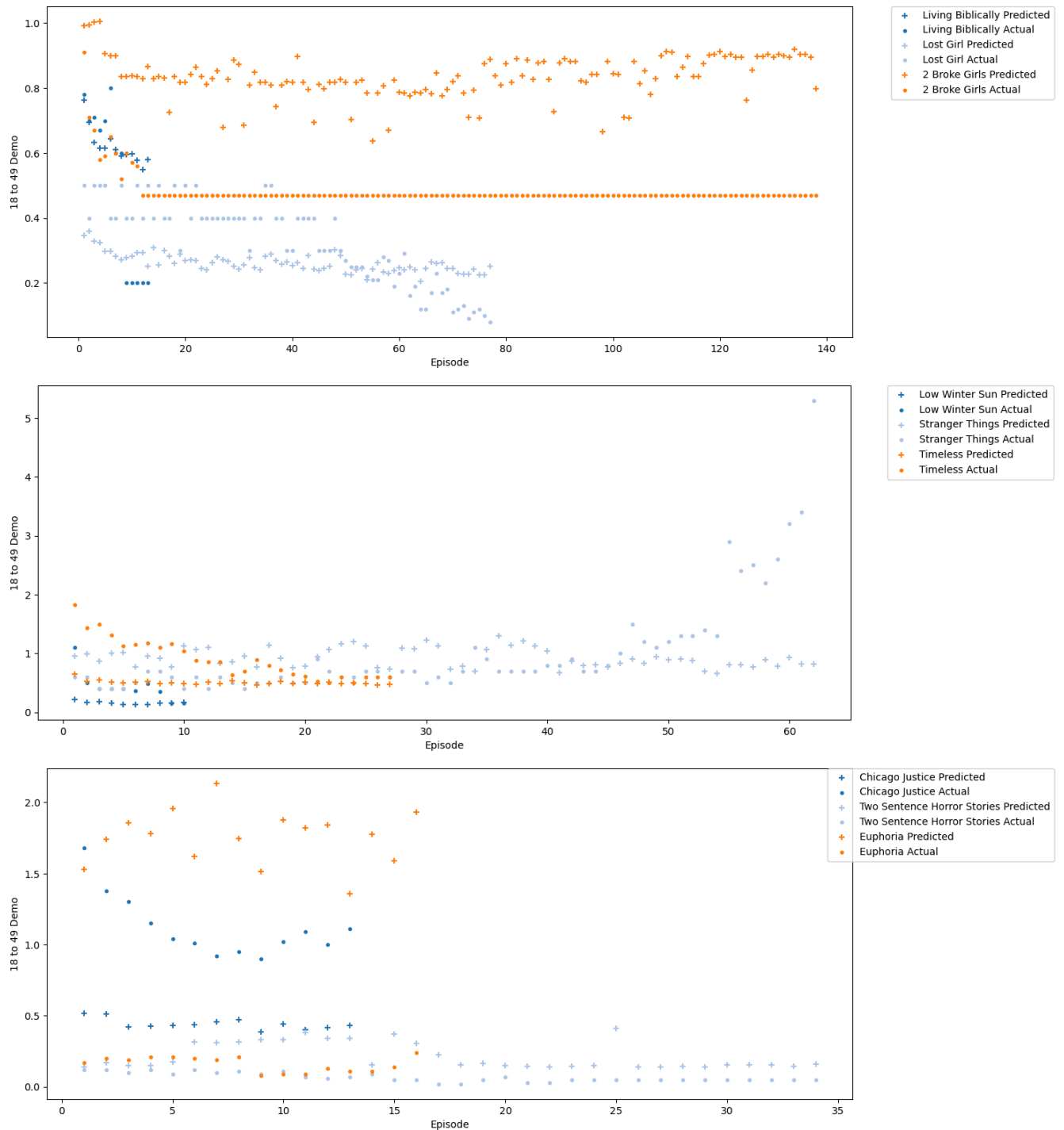
# Call the function with slices of the DataFrame
series_ids_list = viewers_regression_predictions_df['IMDb Series ID'].unique()
for i in range(0, len(series_ids_list), 3):
    batch_series_ids = series_ids_list[i:i+3]
    plot_predicted_vs_actual(viewers_regression_predictions_df, batch_series_ids, 'viewers_prediction', 'viewers_residual', 'Viewers (millions)')

# Call the function with slices of the DataFrame (3 at a time)
series_ids_list = _18_to_49_regression_predictions_df['IMDb Series ID'].unique()
for i in range(0, len(series_ids_list), 3):
    batch_series_ids = series_ids_list[i:i+3]
    plot_predicted_vs_actual(_18_to_49_regression_predictions_df, batch_series_ids, '_18_to_49_demo_prediction',
                            '_18_to_49_demo_residual', '18 to 49 Demo')

```







In [406... show_df_info(fate_classification_predictions_df, "fates")

Dataframe information for fates:

Dataframe Information:

	Column	Data Type	NaN Count
0	IMDb Series ID	object	0
1	Title	object	0
2	status	object	0
3	ended_tn	int64	0
4	ended_fp	int64	0
5	ended_fn	int64	0
6	ended_tp	int64	0
7	canc_tn	int64	0
8	canc_fp	int64	0
9	canc_fn	int64	0
10	canc_tp	int64	0

There are 12 rows and 11 columns of information.

```
In [408... # Display the result
fate_classification_predictions_df["Canceled Predictions"] = fate_classification_predictions_df['canc_tp'] + \
    fate_classification_predictions_df['canc_fp']
fate_classification_predictions_df["Ended Prediction"] = fate_classification_predictions_df['ended_tp'] + \
    fate_classification_predictions_df['ended_fp']
pretty_print_df(fate_classification_predictions_df,
                interesting_columns=['IMDb Series ID', 'Title', 'status', "Canceled Predictions", "Ended Prediction"])
```

IMDb Series ID	Title	status	Canceled Predictions	Ended Prediction
tt0944947	Game of Thrones	Ended	0	72
tt8772296	Euphoria	Returning Series	0	16
tt0903747	Breaking Bad	Ended	0	62
tt4574334	Stranger Things	Returning Series	0	62
tt10327354	Prodigal Son	Canceled	0	31
tt5511582	Timeless	Canceled	0	0
tt6873926	Two Sentence Horror Stories	Returning Series	34	0
tt1429449	Lost Girl	Ended	0	77
tt1845307	2 Broke Girls	Canceled	138	0
tt2314792	Low Winter Sun	Canceled	10	0
tt5640060	Chicago Justice	Canceled	13	0
tt1048176	Living Biblically	Canceled	13	0

Recommendations for Future Enhancements

Google Trends

- Incorporating Trends data was in my original plan. I needed to change course because that API is giving 429 errors even when I use proxy servers and good citizen requests.

Cast Social Media

- TikTok and Instagram would be good sources of trending data.

Cast Public Appearances

- Publicity appearances are another factor that increases show popularity.

Investigate Other Map Ops

- rather than direct correlation, try logarithmic (nat log or log 10)

```
In [ ]:
```